

考虑物性变化与径向收缩的药材热湿耦合模型及数值验证

摘要

药材热风烘干同时涉及温度传递、水分迁移及尺寸变化，表面状态难以直接代表内部干燥程度。针对圆柱形药材的四个问题，本文基于题给环境观测和物性关系，建立一维径向有效热湿耦合模型，并以全域最大干基含水率确定烘干结束时间。模型区分有效显热容量与守恒干物质密度，将气固含水率映射、观测窗后环境平台和同比径向收缩作为明确闭合；在环形控制体上采用 Kirchhoff 浓度势处理非线性扩散，配合解析稀疏 Jacobian 与隐式 BDF 求解。

针对问题一，采用常热物性与含水率依赖扩散系数建立预热模型，计算 0—1800 s 内温度和水分的径向分布。1800 s 时中心、表面温度分别为 33.5753°C 和 36.7856°C，干基含水率分别显示 2.5500 和 1.5102 kg/kg，反映表层先响应及内部水分迁移滞后。温度场另与包含时变边界卷积的圆柱解析级数对照，验证简化条件下的数值实现。

针对问题二，从初始状态统一采用附录 3 的变物性，联立计算温度对扩散及水分对热物性的反馈。附件观测区间内进行分段线性插值，4 h 后按 50°C 与等效平衡含水率 0.05 延拓。3 h 时中心、表面温度为 49.8495°C 和 49.9664°C，而含水率仍为 1.7662 和 1.0081 kg/kg，说明升温接近完成早于全域脱水完成。

针对问题三，复用全过程轨迹，以全域最大含水率下降穿越 0.15 kg/kg 定位临界根，并区别临界等号与严格可行的报告时刻，得到烘干时长 57.4724 h。针对问题四，在干物质守恒下取材料坐标，将给定半径变化转化为固定区间上的时变几何因子；采用整组附录 4 物性，得到时长 51.0906 h、结束半径约 1.2000 cm，输出中将域外位置与真实表面分别处理。

网格与时间设置检验、独立 MATLAB 实现及 BDF/Radau 对照支持结果在所列范围内的数值一致性。同物性几何对照显示缩短迁移路径的作用，经验边界阻力情景进一步说明气固闭合对时长的影响。由于没有内部温湿实测及药材等温线，本文不将数值一致性解释为实测准确率；所得时长适用于给定物性、环境延拓和收缩假设。

关键词：热湿耦合；有限体积法；Kirchhoff 变换；径向收缩；干燥时间

1 问题重述

1.1 问题背景

中药材干燥是产地初加工的关键环节，直接决定成品品质与用药安全。热风烘干是目前应用最广的干燥方式之一，通常包含预热平衡与恒温干燥两个阶段[1]。然而，烘房温湿度调控多凭经验，参数不当易导致效率低、能耗高、有效成分损失等问题。已有研究表明，通过建立热风干燥的传热传质模型，可以用计算机模拟不同工艺下水分和温度的变化，从而减少试错成本[2]。因此，用数学建模与仿真研究热风烘干规律具有实际意义。

1.2 问题提出

基于上述背景，要求建立数学模型解决以下问题：

本文以圆柱形中药材的热风烘干过程为研究对象，该药材长为 25cm、半径为 2cm，初始温度为 28℃，初始水分浓度（即干基含水率）为 2.55kg/kg。

问题一：根据烘房的温度以及水分浓度变化情况，建立预热平衡阶段药材温度和水分浓度变化规律的数学模型，据此计算出 1800s 内每隔 1s、到药材中心距离每隔 0.1cm 的药材温度以及水分浓度。

问题二：在问题一的基础下，将时间范围扩展至整个烘干过程（2 至 3 天），根据预热平衡阶段与恒温干燥阶段参数，建立整个烘干过程药材温度和水分浓度变化规律的数学模型，据此计算出在烘干过程中，每隔 1s、到药材中心距离每隔 0.1cm 的药材温度以及水分浓度的。

问题三：在问题二的基础上反求问题，约束烘干要求药材各处水分浓度均低于 0.15kg/kg。要求依据已建立的水分浓度模型，模拟药材内部水分随时间的变化，计算出药材内部水分浓度每隔 60s、到药材中心距离每隔 0.1cm 的药材水分浓度以及烘干结束时间。

问题四：在实际烘干过程中，药材由于水分流失引起的尺寸变化会影响到烘干的最终效果，需要建立含尺寸变化的温度和水分浓度模型，重新确定烘干时长，据此计算出药材内部水分浓度每隔 60s 时间、0.1cm 距离的水分浓度以及烘干结束时间。

2 问题分析

2.1 问题一分析

已知圆柱形药材的几何尺寸（长 2cm、半径 2cm）、均匀初始状态（28°C、2.55kg/kg 干基）、烘房温湿度序列（附件 1）与常物性参数（附录 2），要求得到 1800s 内药材内部温度与水分浓度的时空分布。首先由傅里叶导热定律与 Fick 扩散定律，在轴对称假设下建立一维径向控制方程，并给定中心零通量与表面 Robin 边界条件。求解顺序上，由于本问热物性为常数，温度方程为常系数线性抛物型，可独立求解，故先求温度；该简化下温度场存在圆柱 Bessel 级数解析解，本文的数值求解器即以此为基准作对照校验。对于水分浓度场，热物性虽为常数，但扩散系数仍随含水率变化 $D_1(C)$ ，方程自身非线性，故改用守恒型有限体积离散配隐式时间推进求解，并以离散质量守恒恒等式校核(残差量级 10-14);注意到扩散系数经验式中的温度项为 Arrhenius 形式，计算中温度一律以开尔文代入。

2.2 问题二分析

问题二区别于上一问，将时间拉长至整个烘焙过程，已知烘干过程持续数天，预热平衡与恒温干燥两个阶段的参数不同，附录给出了密度、比热容、导热系数和扩散系数随含水率或温度变化的经验关系，要求得到整个烘干过程的温度与水分浓度时空分布。与问题一相比，本问物性同时依赖含水率与温度，温度方程和水分方程不再解耦，必须联立求解。由于两问采用的物性体系不同，本问从初始时刻重新开始计算，不以问题一的末状态作为初值。环境驱动方面，前一段时间按附件数据分段线性插值，之后取平台温湿度延拓，该延拓作为假设在论文中明确说明。求解方法上仍采用守恒型有限体积离散，时间推进需处理方程刚性与非线性耦合，因此配了解析 Jacobian 并采用隐式 BDF 格式。非线性系数不能移出散度，热容量项不能改写成特定微分形式。

2.3 问题三分析

问题三继续沿用问题二的变物性耦合模型，几何依旧保持固定半径，计算时间拉长至数天。本题不再是给定时间求内部分布，而是反向求解烘干结束时刻；烘干达标条件是药材全部位置的水分浓度均低于 0.15 kg/kg。由于药材中心水分散失最慢，全场最大值一般出现在中心位置，但求解时不能只监控中心点，必须遍历全部径向位置，取全域最大水分浓度作为判断依据，寻找它第一次低于阈值的时刻作为烘干总时长。为了精确捕捉这一时刻，选择利用数值解进行插值搜索，并严格控制时间步长以确保穿越时刻的精度。

2. 4 问题四分析

问题四的解决关键在于处理半径随时间减小带来的移动边界。直接使用原坐标会面对网格随边界变形的问题，因此采用坐标变换，把实际收缩域映射到固定区间这样，移动边界问题转化为固定域问题，问题二、三中的有限体积离散和隐式时间推进可以继续使用。半径由附件 2 插值得到，密度、比热、导热系数和扩散系数改用附录 4 的经验公式，并在每个时间步按当前含水率和温度更新。

求解时，先将控制方程在变换后的坐标下重新离散，保留由坐标变换产生的网格速度相关项，具体形式按干物质骨架随动或空间固定的假设确定，避免漏项或重复加项。随后分别计算固定半径和变化半径两种情形，以便隔离收缩本身的影响：收缩缩短了水分迁移路径，但附录 4 扩散系数整体小于附录 3，两种作用方向相反，净效应通过数值结果比较得出。

3 模型假设

假设药材长径比较大，忽略药材两个端面的传热与水分散失，将三维传热传质过程简化为一维径向模型。

假设烘房内热风充分混合，同一时刻药材表面外侧温湿度均匀，对流换热系数、对流传质系数取恒定值。

假设初始时刻药材内部温度与干基含水率均匀分布，不存在初始梯度。

假设药材干物质骨架不发生迁移；问题一至三骨架尺寸固定，问题四只发生均匀径向收缩，轴向长度不变。

假设不考虑药材内部生化反应，仅研究热传导与水分扩散物理过程。

4 符号说明

符号	说明	单位
t	时间	s
r	到药材中心的径向距离	m
$x=r/R(t)$	材料归一化径向坐标	l
T	药材温度	K
C	干基含水率=m 水/m 干物质	kg/kg

符号	说明	单位
$w=C/(1+C)$	湿基质量分数	1
$R(t)$	药材当前半径	m
ρ_{eff}	有效密度（用于热容量）	kg/m ³
ρ_d	守恒干物质密度	kg/m ³
c_p	比热容	J/(kg·K)
k	导热系数	W/(m·K)
D	有效水分扩散系数	m ² /s
h	对流换热系数	W/(m ² ·K)
β	对流传质系数	m/s
$B=\rho_{eff} \cdot c_p$	有效体积显热容量	J/(m ³ ·K)
q_r	径向导热热流密度（向外为正）	W/m ²
j_w	径向水分质量通量（向外为正）	kg/(m ² ·s)
$\ell(t)$	每单位初始干物质的累计失水	kg/kg
C_{eq}	环境对应的等效平衡含水率	kg/kg
T_{∞}	烘房温度	K
t^*	全域达标时刻	s

5 模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 建模思路与统一口径

本文采用圆柱径向热湿传递框架研究药材内部状态。计算几何按题面取长度 0.25 m、初始半径 0.02 m；温度方程以开尔文为单位，结果表换算为摄氏度。含水率 C 表示单位药材干物质所含的水质量，其与湿基质量分数 w 的关系为

$$C = \frac{m_w}{m_d}, \quad w = \frac{C}{1+C} \quad (1)$$

干基比值并非单位体积内的水质量。为保持水量计算的量纲一致，引入单位当前体积内的干物质质量 ρ_d ，水质量密度为 $\rho_d C$ 。问题一至三采用固定且空间均匀的干物质骨架，问题四采用随材料运动的骨架，干物质本身不发生生成、消耗或相对骨架的扩散。因而问题四的“骨架无迁移”应理解为不存在额外组分迁移，不能理解为材料点在空间中静止。下文所有移动域方程均按材料随动解释建立。

热学和质量方程需要分别说明密度的用途。题给 ρ 与 c_p 的乘积记为 B ，用来描述单位体积温升的有效显热需求； ρ_d 则用于计算随材料守恒的干物质和水质量。若把题给 ρ 同时解释为真实湿密度，再强行令 $\rho_d = \rho/(1+C)$ ，会对固定长度和给定半径施加额外约束。附录 A.2 表明，该约束与所给后期收缩数据不能同时满足。因此本文采用有效容量与独立干骨架的闭合，使两套方程的定义一致；这一处理也限定了结果的物理解释，不能视为对真实组分密度的验证。

附件 1 的烘房水分指标只标 kg/kg，未给出与药材干基含水率之间的吸附等温线。记该观测数值为 Y_∞ ，基线约定 $C_{eq} = Y_\infty$ ，作为表面传质的等效平衡值。这是气固边界的简化映射，不是由单位相同推得两者物理相等；如果进一步把 Y_∞ 解释为空气湿度比，其干空气参考质量也须另作假设。上述解释不改变附件数值，但明确了预测结果成立的条件。材料平衡含水率的一般概念见文献[5]。

5.1.2 变量定义与公式推导

在半径 r 至 $r+dr$ 的圆柱薄壳内，对向外为正的热流和水分通量采用 Fourier 与 Fick 本构。薄壳内外表面积不同，积分守恒后须保留径向几何因子。得到

$$B(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial T}{\partial r} \right), \quad 0 < r < R_0 \quad (2)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} (r D(T, C) \frac{\partial C}{\partial r}) \quad (3)$$

式(2)表示有效显热的局部变化由净导热供给，式(3)由水质量守恒除以均匀且固定的 ρd 得到。变导热系数、变扩散系数均保留在散度内；将它们直接移至导数外，会漏掉由状态变化引起的通量梯度。经典扩散理论提供了上述构造及圆柱基准解的依据[3]。

中心由于轴对称不存在跨轴通量，表面与烘房发生对流换热和有效传质。初始条件与边界条件分别为

$$T(r, 0) = 301.15 \text{ K}, C(r, 0) = 2.55, T_r(0, t) = C_r(0, t) = 0 \quad (4)$$

$$-k T_r(R_0, t) = h(T_s - T_\infty), -D C_r(R_0, t) = \beta(C_s - C_{eq}) \quad (5)$$

这里 T_s 与 C_s 是药材真实表面的状态， $h=25 \text{ W/(m}^2\cdot\text{K)}$ ， $\beta=8\times 10^{-7} \text{ m/s}$ 。初始场与表面边界在 $t=0$ 处可能不完全相容，意味着启动阶段形成边界层，不应人为修改初始温湿度来消除该现象。采用较小的早期步长处理快速响应，并在后续检查非负性与通量方向。

问题一采用附录 2： $\rho=820 \text{ kg/m}^3$ ， $c_p=2600 \text{ J/(kg}\cdot\text{K)}$ ， $k=0.36 \text{ W/(m}\cdot\text{K)}$ ，水分扩散系数为

$$D_1(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) \text{ m}^2/\text{s} \quad (6)$$

因此本问温度方程是可独立求解的线性热方程，水方程仍因 $D_1(C)$ 而非线性。式(6)没有温度项；Arrhenius 温度因子只在问题二至四的物性关系中出现。不能把“常热物性”扩大解释为“全部物性均为常数”，也不应给问题一另外加入温度活化项。

5.1.3 求解算法

设材料归一化坐标 $x=r/R_0$ ，在 $0 \leq x \leq 1$ 上取 N 个等长区间，节点 $x_i=i/N$ 包含中心和表面。对每个节点分配相邻半节点之间的环形控制体。令 w_i 为其无量纲体积权重，中心与表面分别采用半控制体。有限体积法先计算相邻控制体共用的面通量，再作净通量差，因而内部面在全域求和时严格相消[4]。这一性质比直接在 $r=0$ 代入含 $1/r$ 的差分式更适合处理轴心。

对于浓度依赖显著的 D ，引入浓度 Kirchhoff 势。若 $D=D_0 A(T)\exp(-a/C)$ ，则取

$$\Phi_a(C) = C \exp(-a/C) + a \text{ Ei}(-a/C), \Phi'_a(C) = \exp(-a/C) \quad (7)$$

相邻节点之间的水分通量通过势差计算，温度因子在该面单独取值。不能把

$A(T)\Phi(C)$ 整体作差，否则会产生原 Fick 闭合中不存在的温度梯度通量。浓度相近时采用局部展开避免两项接近的势值相减损失有效数字。式(7)使浓度方向的积分得到解析处理，但温度面值、有限网格与时间推进仍有离散误差；它不是对整个偏微分方程的精确求解。

细网格缩短了局部扩散时间尺度，显式推进会受到较严的步长限制。本文将温度和含水率交错排列，利用相邻节点耦合构造稀疏 Jacobian，再由隐式 BDF 联立求解 [6]。状态末项只累计失水，不反馈改变温湿方程，可用来核对全域水量收支。附录 B 展开残量、矩阵及边界导数。求解器在误差控制下选择积分步长，正文和工作簿则查询其连续输出，两者分开处理，保证 1 s 输出并非由稀疏表格重新拼接。

Step 1：读取附件观测，统一长度和温度单位，核对初始场与附录 2 参数。Step 2：生成环形控制体、初值和相邻面通量。Step 3：采用隐式变步长推进，检查求解成功、有限数值及物性正性。Step 4：从同一数值轨迹查询指定时刻及物理半径。Step 5：导出结果表，并检查累计失水与全域水量变化是否一致。

问题一采用 $N=3200$ ，积分至 1800 s；相对容差为 10^{-10} ，温度与水分绝对容差分别为 10^{-10} K 和 10^{-12} kg/kg。严格容差用于控制时间积分，不能解释为最终物理预测具备相同精度。网格选择依据及时间设置对照见附录 C。

5.1.4 结果分析

表 1、表 2 列出题目要求的 7 个时刻和 5 个半径位置；完整逐秒、0.1 cm 结果由同一轨迹生成。温度和浓度分别保留四位小数，表头中的半径以 cm 表示。

表 1 问题一的径向温度（℃）

时间/s	r=0 cm	r=0.5 cm	r=1 cm	r=1.5 cm	r=2 cm
100.0000	28.0001	28.0003	28.0040	28.0327	28.1801
300.0000	28.0408	28.0635	28.1514	28.3681	28.8489
600.0000	28.4533	28.5360	28.8039	29.3159	30.1652
900.0000	29.3243	29.4583	29.8755	30.6161	31.7304
1200.0000	30.5427	30.7098	31.2223	32.1126	33.4276
1500.0000	31.9957	32.1867	32.7660	33.7463	35.1203
1800.0000	33.5753	33.7720	34.3642	35.3621	36.7856

表 2 问题一的径向干基含水率 (kg/kg)

时间/s	r=0 cm	r=0.5 cm	r=1 cm	r=1.5 cm	r=2 cm
100.0000	2.5500	2.5500	2.5500	2.5500	2.2470
300.0000	2.5500	2.5500	2.5500	2.5492	2.0508
600.0000	2.5500	2.5500	2.5500	2.5353	1.8770
900.0000	2.5500	2.5500	2.5497	2.5045	1.7547
1200.0000	2.5500	2.5500	2.5482	2.4646	1.6586
1500.0000	2.5500	2.5499	2.5445	2.4206	1.5787
1800.0000	2.5500	2.5497	2.5383	2.3755	1.5102

至 1800 s，中心温度为 33.5753°C，表面温度为 36.7856°C，差约 3.2102°C。表层直接受到热风影响，温度上升先于中心，随后导热使内部逐渐升温。含水率的空间响应更不均匀：表面已下降到 1.5102 kg/kg，中心显示 2.5500 kg/kg。原精度中心值约 2.5499924，说明其发生了很小的变化；四位表格中的相同数字不意味着水分完全静止。

表层含水率降低后，式(6)中的 D 随之减小，形成较强的干燥阻力。因而即使表面与外界存在传质驱动力，内部水分也不会立刻均匀释放。只用一个全体平均含水率描述此阶段，会掩盖径向差异，无法回答题目要求的各位置浓度。

图像主题 1：问题一 7 个指定时刻的径向温度与含水率双面板曲线。横轴为距中心距离 0—2 cm；左图纵轴为温度°C，右图为干基含水率 kg/kg；每条曲线对应一个时刻，突出表面先响应及中心水分滞后。该图与表 1、表 2 对应，绘制时保持时间颜色一致。

5.1.5 模型检验或灵敏度分析

在本问常热物性条件下，将变温边界写入圆柱 Robin 特征函数展开及时间卷积，得到热场解析基准。它与数值结果在 N3200、1801 个整数秒和 21 个半径点上最大相差约 1.80×10^{-6} K。非线性水方程另用守恒检查与面格式对照核验，不由热场的一致性替代。基准公式与取样范围在附录 C 交代，正文不逐项展开误差矩阵。

5.2 问题二模型的建立与求解

5.2.1 建模思路

问题二沿用式(2)—(5)的几何与边界框架，重新从均匀初始场开始。按附录3在整个求解区间使用同一套变物性，不在1800 s将附录2解拼接到附录3，也不把问题一末状态用作新的初始条件。这样才能区分两问的物性设定，避免在中途人为改变材料参数。

附录3使两个状态之间出现双向反馈：T进入扩散系数，C同时进入热容量和导热系数。在一次时间推进中，新的温度会影响水通量，新的含水率又改变温升速度和热通量。因此即使外部环境完全相同，问题二也不能沿用问题一的温度轨迹。联立算法在同一时刻更新T、C与全部物性，使耦合关系进入隐式残量；先独立计算全程热场再求水分，会改变这里采用的模型。

5.2.2 变量定义与公式推导

令 $w=C/(1+C)$ ，附录3规定

$$\rho_{eff}=650+128C, \quad c_p=1450+2736w, \quad k=0.21+0.38w \quad (8)$$

$$D_3(T, C)=2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}-\frac{3850}{T}\right) \quad (9)$$

式(8)的密度、比热容、导热系数依次采用 kg/m^3 、 $\text{J}/(\text{kg}\cdot\text{K})$ 、 $\text{W}/(\text{m}\cdot\text{K})$ ，式(9)的单位为 m^2/s ，T必须是绝对温度。指数中的 $0.45/C$ 和 $3850/T$ 均为无量纲经验组合，系数按题目给定取值，不从当前附件重新拟合。后续采用 $B(C)=\rho_{eff}(C)c_p(C)$ ，仍是有效显热解释。

附件1包含0—4 h共241组观测，观测间隔60 s。为了向连续时间求解器提供边界，观测区间内作分段线性插值。 $t>4$ h后取 $T_\infty=50^\circ\text{C}$ 、 $C_{eq}=0.05$ 。这一延拓利用题目恒温干燥的工况描述及末段平台特征，但平台尾段并非持续数天的实测记录。相应的分段表达为

$$(T_\infty, C_{eq})(t)=\begin{cases} (I_T(t), I_Y(t)), & 0 \leq t \leq 14400 \text{ s}, \\ (323.15, 0.05), & t > 14400 \text{ s}. \end{cases} \quad (10)$$

其中两种插值算子直接连接相邻观测点，不额外平滑或外推趋势。超过观测窗后的输出是指定工况下的条件预测。附录3未重给表面换热和传质系数，本文明确沿用 h 和 β ；这也属于全过程边界设置，而不是对不同风速或设备的普遍结论。

图像主题2：烘房边界条件及延拓。上图为0—4 h温度原始点与插值，下图为环境水

分指标；另用较长时间轴展示 4 h 后的 50°C 和 0.05 平台，用竖线标出观测与延拓边界。不能把延拓段画成实测点。

5.2.3 求解算法

沿用问题一的有限体积网格，同时更新 $B(C)$ 、 $k(C)$ 与 $D(T,C)$ 。温度面通量采用导热系数调和平均，水分面通量用 $a=0.45$ 的 Kirchhoff 势及面温度因子。解析 Jacobian 保留 T 对水通量、 C 对热容量和热通量的导数。中心无通量与表面换热传质直接写入控制体余额，从而保证两类边界与内部离散采用一致的符号。

Step 1: 从 $t=0$ 读取初始状态并选定附录 3。Step 2: 每次右端函数求值时查询当前环境，更新各节点物性及面通量。Step 3: 用隐式 BDF 与稀疏 Jacobian 联立推进，早期限制较小步长，后期根据误差控制自动调整。Step 4: 保存同一条全过程轨迹，查询题目要求的前 3 h 径向表和逐秒全过程结果。Step 5: 将该轨迹交给问题三进行阈值判断，避免两次独立求解造成终止时间与结果表不一致。

网格采用 N3200，主要容差与问题一一致。模型最长允许计算范围只是程序保护上限；实际输出区间由问题三达标事件和其后校核点确定，不以任意预设天数冒充烘干完成时刻。

5.2.4 结果分析

表 3、表 4 给出题目规定的 6 个时刻。两表由全过程解在同一组时间和半径上查询，因此可以直接比较热响应与水分迁移的时间尺度。

表 3 问题二的径向温度 (°C)

时间/h	r=0 cm	r=0.5 cm	r=1 cm	r=1.5 cm	r=2 cm
0.5000	32.1892	32.3821	32.9659	33.9608	35.4130
1.0000	40.3816	40.5536	41.0605	41.8782	42.9976
1.5000	45.8468	45.9348	46.1930	46.6051	47.1401
2.0000	48.4502	48.4881	48.5984	48.7736	49.0033
2.5000	49.4670	49.4792	49.5136	49.5653	49.6609
3.0000	49.8495	49.8553	49.8746	49.9101	49.9664

表 4 问题二的径向干基含水率 (kg/kg)

时间/h	r=0 cm	r=0.5 cm	r=1 cm	r=1.5 cm	r=2 cm
0.5000	2.5499	2.5489	2.5256	2.3257	1.6486
1.0000	2.5257	2.4948	2.3578	2.0230	1.4711
1.5000	2.3861	2.3256	2.1344	1.8020	1.3476
2.0000	2.1709	2.1084	1.9236	1.6259	1.2311
2.5000	1.9566	1.9006	1.7360	1.4720	1.1166
3.0000	1.7662	1.7165	1.5702	1.3333	1.0081

3 h 时中心和表面温度分别为 49.8495°C、49.9664°C，温差已缩小到约 0.1169°C，而中心与表面的含水率分别为 1.7662、1.0081 kg/kg，仍存在明显差异。升温接近完成不代表脱水已经完成；单独根据药材温度接近烘房温度来决定出料，会忽视中心水分的持续迁移。

水分扩散对温度的响应与含水率的响应方向不同。在相同 C 下，升温使式(9)的温度因子增大，有利于扩散；随着药材逐渐变干， $\exp(-0.45/C)$ 减小，使后期扩散变慢。因此全过程通常表现为前期升温推动水分迁移、后期低含水率区域限制传递的组合效应。该解释来自物性公式及场分布，不把时间曲线的形状直接归因为某个未经观测的组织变化。

图像主题 3：问题二中心与表面的温度、含水率随时间变化。温度展示前 4 h，含水率展示从 0 到达标；两图分别使用适合其变化尺度的时间轴，清楚标出中心和表面，避免因同轴缩放掩盖前期热响应。

5.2.5 模型检验或灵敏度分析

空间加密、收紧时间设置以及不同积分器的对照表明，本组数值解在约定采样范围内稳定。表 3 中 1.5 h 表面温度恰处于四位小数的舍入边界附近，基准与更严格时间设置可能给出不同末位。本文保留统一结果版本的 47.1401，不通过挑选不同运行的末位拼出一张表。四位输出满足展示格式，不等于四位真实物理精度。

5.3 问题三模型的建立与求解

5.3.1 建模思路

全域达标由含水率最高的位置决定，时间求解应以完整状态轨迹为依据。本文将温湿场求解与事件定位衔接，把烘干阈值写成全域最大值约束，分别处理数值临界根和严格可行的报告值。这样既沿用问题二的物理模型，也避免仅凭表面状态或预设天数结束计算。

5.3.2 变量定义与公式推导

问题三由给定时刻求状态转为确定满足约束的时刻。沿用问题二的几何、物性、环境和同一条数值轨迹，定义

$$M(t) = \max_{0 \leq r \leq R_0} C(r, t), \quad g(t) = M(t) - 0.15 \quad (11)$$

使用全域最大值可以直接对应“药材各处均低于 0.15 kg/kg”，而不依赖先验指定某个控制位置。数值上逐节点检查最大含水率，并核对各保存时刻的径向变化；本基线后期由中心控制，但中心在早期近乎平坦的区域中不一定是唯一浮点最大点。因此不把最大值索引始终等于某一节点当作必需物理条件。

连续根 t_c 满足 $g(t_c) = 0$ 。若 M 在穿越附近连续下降，严格可行集合的左端是临界边界， t_c 本身仍等于阈值。题目要求严格低于时，应将临界根与可行报告时刻分开。本文在四位小数小时网格上向上取整，再用未舍入浓度检查：

$$t_r = 10^{-4} / 10^4 t_c / h, \quad M(t_r) < 0.15 \quad (12)$$

若临界根恰好位于报告网格点上，或原精度检查尚未严格满足，则继续推进一个报告单位再检查。这是本文采用的可行报告约定。它不改变事件根的定义，也不把四位数字的显示值当作阈值判断依据。

5.3.3 求解算法

Step 1: 在问题二积分过程中监测 $g(t)$ 下降穿零。Step 2: 在包围根的时间区间内利用求解器稠密输出定位临界根。Step 3: 核对事件处全域最大值与阈值的一致性。Step 4: 将临界时刻换算为小时，按式(12)形成报告值。Step 5: 查询未舍入最大含水率，确认报告时刻严格可行；按照 60 s 和 0.1 cm 输出全过程含水率。另在 N800 验证轨迹上采用独立二分复核事件定位方法；正式 N3200 临界根与严格报告值仍由生产轨迹给出。

计算得到 $t_c = 57.47230195056044$ h，报告烘干时长为 57.4724 h。普通四位最近舍入得到的 57.4723 h 只是临界时间的显示标签，不能直接替代严格可行的报告值。程序另

保留事件后整数秒的状态用于导出与核对，相关时间约定见附录 C。

5.3.4 结果分析

表 5 列出每隔 6 h 及最终报告时刻的径向含水率。完整 60 s 结果保留更多时间信息，正文表用于显示干燥过程与径向差异。

表 5 问题三的径向干基含水率 (kg/kg)

时间/h	r=0 cm	r=0.5 cm	r=1 cm	r=1.5 cm	r=2 cm
6.0000	1.0170	0.9881	0.9015	0.7550	0.5328
12.0000	0.4561	0.4432	0.4031	0.3297	0.1642
18.0000	0.2988	0.2915	0.2687	0.2253	0.0845
24.0000	0.2378	0.2327	0.2167	0.1855	0.0665
30.0000	0.2058	0.2018	0.1892	0.1641	0.0598
36.0000	0.1859	0.1825	0.1718	0.1504	0.0566
42.0000	0.1721	0.1691	0.1597	0.1407	0.0549
48.0000	0.1618	0.1592	0.1507	0.1334	0.0537
54.0000	0.1539	0.1514	0.1436	0.1277	0.0530
57.4724	0.1500	0.1477	0.1402	0.1249	0.0527

12 h 时表面已接近 0.15，但中心仍为 0.4561 kg/kg；至 48 h 中心仍为 0.1618 kg/kg。由此可见，表面达标不能代表全域达标。后期中心由 0.1859 下降到 0.1500 经历较长时间，反映低含水率条件下扩散系数降低的影响。结束时中心与表面仍有浓度差，达到产品阈值并不要求内部先达到完全均匀的平衡状态。

表 5 末行需结合报告约定阅读。未舍入的最大浓度严格小于 0.15，而保留四位后的中心单元格仍可显示 0.1500；显示精度不足以区分与阈值的微小差距。本文保留这个真实舍入结果，并用事件根、报告时刻和原精度查询完成判定。把单元格人为改成 0.1499 虽然看似符合不等式，却会破坏数值输出与约束核验之间的对应关系。

图像主题 4：问题三中心、表面与全域最大含水率曲线及 0.15 阈值线。主图展示 0—60 h，局部放大图展示临界根附近，分别标记 t_c 与 t_r 。图中明确全域最大值曲线与中心曲

线后期重合，不人为错开。

5.3.5 模型检验或灵敏度分析

约 93.04% 的报告时长位于附件 1 的 4 h 观测窗之后，因此 57.4724 h 应理解为规定平台温湿条件持续存在时的模型预测。数值根的复核能检查“给定轨迹上是否找准阈值”，不能消除边界延拓和气固映射的物理不确定性。若后续更换平台温湿度、 β 或平衡映射，必须重新积分并重新确定事件，不能只按经验比例改动最终时长。

5.4 问题四模型的建立与求解

5.4.1 建模思路

问题四同时改变几何与物性。从 $t=0$ 起整组采用附录 4，并由附件 2 查询 $R(t)$ 。附件 2 在 0—72 h 提供 145 个半径观测，间隔 0.5 h，按分段线性插值连接。基线假定轴向长度保持不变，内部材料发生与半径一致的同比径向收缩；材料速度为 $u(r,t)=rR'(t)/R(t)$ 。该速度场与表面运动相容，是从半径数据推断内部运动的明确假设。

如果只在固定域模型中将 R_0 换成 $R(t)$ ，而不说明水质量和材料速度的含义，就可能重复加入浓缩项或漏掉物质运动。本文先从组分守恒建立物质导数，再进行材料坐标变换，保证控制方程的来源清楚。

5.4.2 变量定义与公式推导

在没有干物质生成或相对迁移的条件下，干质量守恒及水质量守恒为

$$\partial_t \rho_d + \nabla \cdot (\rho_d u) = 0, \quad \partial_t (\rho_d C) + \nabla \cdot (\rho_d C u + j_w) = 0 \quad (13)$$

第二式减去第一式乘 C ，得到 $\rho_d DtC = -\nabla \cdot j_w$ 。取 $j_w = -\rho_d D \nabla C$ ，并利用空间均匀的干骨架，消去 ρ_d 。令 $x=r/R(t)$ 、 $\hat{C}(x,t)=C(R(t)x,t)$ ，链式法则中固定空间点的坐标项与 uC_r 相消，得到

$$\frac{\partial \hat{C}}{\partial t} = \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} (x D \frac{\partial \hat{C}}{\partial x}) \quad (14)$$

$$B(\hat{C}) \frac{\partial \hat{T}}{\partial t} = \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} (x k \frac{\partial \hat{T}}{\partial x}) \quad (15)$$

式(15)沿用有效显热闭合，并非由完整含潜热、湿分携焓和收缩功的总能量方程无条件推出。材料坐标没有额外网格平流项，也不对干基比值 C 增加体积浓缩项；收缩体积引起的密度变化已经体现在 $\rho_d(t)=\rho_{d0}[R_0/R(t)]^2$ 中。详细相消过程和干质量积分证明见附录 A。

在 $x=0$ 保持零通量，表面边界改为

$$-\frac{k}{R}\hat{T}_x(1,t)=h(T_s-T_\infty), \quad -\frac{D}{R}\hat{C}_x(1,t)=\beta(C_s-C_{eq}) \quad (16)$$

边界中的 $1/R$ 来自空间导数换元，不能遗漏。附录 4 物性为

$$\rho_{eff}=760+90C, \quad c_p=1850+2150w, \quad k=0.12+0.20w, \quad (17)$$

$$D_4(T,C)=4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}-\frac{3850}{T}\right) \quad (18)$$

不能笼统说附录 4 的扩散系数在全过程均小于附录 3。在相同 T 、 C 下， $D_4/D_3=0.175\exp(0.15/C)$ ，其等于 1 的浓度约为 0.0860600 kg/kg ；高于该值时 D_4 较小，低于时可能反转。因此必须把整套物性及其随状态变化的反馈放回模型比较，不能仅看前置系数大小来判断总时长。

5.4.3 求解算法

在材料坐标中使用固定 N6400 网格。每次计算右端函数时，根据当前 t 更新 $R(t)$ ，再更新式(14)——(18)的体积因子、物性和边界通量。由于网格点随材料移动，不需要反复重划网格；输出时再转换到题目要求的物理半径 r 。

Step 1: 选定整组附录 4 参数，读取半径序列并检查其正值与非增性。**Step 2:** 在材料坐标上生成初值、控制体和通量。**Step 3:** 按当前半径联立积分温度与含水率。**Step 4:** 监测材料域内最大 C 的阈值事件并复核严格报告时刻。**Step 5:** 按每 60 s 和 0.1 cm 生成物理距离结果，同时单列当前真实表面及 $R(t)$ 。

$r>R(t)$ 的物理位置已不在药材内，不能插值或填成零含水率。本文保留这些固定半径列为空，并单列“药材表面”，从而同时保留原距离坐标与移动边界信息。真实表面不一定恰好落在 0.1 cm 网格，不能取最近固定列冒充。

5.4.4 结果分析

问题四的连续临界根为 51.09057478683054 h，按与问题三一致的约定得到严格报告 51.0906 h；此时半径约 1.2000 cm。该时刻在附件 2 的 72 h 范围内，未使用半径外推。表 6 与表 7 分别给出含水率与对应半径。

表 6 问题四的径向干基含水率 (kg/kg；空白表示域外)

时间/h	r=0 cm	r=0.5 cm	r=1 cm	r=1.5 cm	r=2 cm	药材表面
6.0000	1.7196	1.5377	1.0226			0.4207

时间/h	r=0 cm	r=0.5 cm	r=1 cm	r=1.5 cm	r=2 cm	药材表面
12.0000	0.7377	0.6546	0.4083			0.1670
18.0000	0.4087	0.3686	0.2397			0.0892
24.0000	0.2851	0.2611	0.1790			0.0673
30.0000	0.2263	0.2094	0.1493			0.0595
36.0000	0.1928	0.1797	0.1317			0.0560
42.0000	0.1712	0.1604	0.1200			0.0541
48.0000	0.1561	0.1468	0.1116			0.0530
51.0906	0.1500	0.1413	0.1081			0.0526

表 7 问题四各输出时刻的表面半径

时间/h	表面半径/cm
6.0000	1.3740
12.0000	1.2480
18.0000	1.2140
24.0000	1.2040
30.0000	1.2010
36.0000	1.2000
42.0000	1.2000
48.0000	1.2000
51.0906	1.2000

6 h 时半径已收缩至 1.3740 cm，因此 1.5 cm 和 2 cm 固定位置均位于域外；此时表

面含水率为 0.4207，中心仍为 1.7196 kg/kg。至最终时刻，中心和表面分别显示 0.1500 与 0.0526，仍有明显梯度。径向收缩减少了迁移距离，但不会消除由低含水率扩散变慢造成的后期阻力。

图像主题 5：问题四移动域及含水率剖面。左图绘制 $R(t)$ 随时间的变化并标出选取时刻；右图在真实物理半径上绘制相同时刻的 $C(r,t)$ ，每条曲线在各自表面结束。域外区域用浅灰显示，另标中心与真实表面，不把曲线延伸到固定 2 cm。

5.4.5 模型检验或灵敏度分析

问题三与四的报告时长相差约 6.3818 h，但两问物性和几何同时变化，该差值不能全部归因于收缩。为分离几何影响，在相同附录 4 物性、边界和 N200 网格下，分别固定半径及采用给定收缩，得到 129.8452 h 与 51.0910 h；在这个指定对照中，收缩情形时长减少约 60.65%。这里的 N200 结果用于机理比较，不替代 N6400 正式报告。

式(14)直接显示收缩为何会加快内部传递：在相同局部物性下，径向扩散项随 R^{-2} 增强；式(16)又表明表面交换与内部传递的相对比例也在变化。 R^2 时标换算与同物性定半径对照相差约 1.37%，支持迁移路径缩短的尺度解释。由于换算积分使用了仿真事件作为上限，它是机理核对而非额外的独立解析时长。几何数据覆盖达标时刻，但环境数据没有：约 92.17% 的时长仍处于式(10)的平台延拓中，两类观测范围必须分清。

5.4.6 边界阻力的补充修正

为考察基线气固映射的不确定性，在保持主答案不变的前提下，构造受水活度启发的经验阻力族。令 $C_{ref}=0.05$ ， $p \geq 1$ ，对表面通量增加因子

$$K_p(C_s) = \frac{1 - (C_{ref}/C_s)^{1/p}}{1 - C_{ref}/C_s}, \quad j_w = \rho_d \beta K_p(C_s)(C_s - C_{eq}) \quad (19)$$

当 C_s 趋于 C_{ref} 时取连续极限 $1/p$ ；对 $C_s \geq C_{ref}$ 有 $1/p \leq K_p \leq 1$ ， $p=1$ 恢复基线。该构造通过指定参考点下的活动度差缩放获得启发，实际计算仍是一族经验边界阻力，尚未标定真实吸附等温线。历史 N800 情景下， $p=2$ 时问题二三的临界时长为 59.6239 h，问题四为 52.2283 h； $p=4$ 时分别为 65.4447 h 和 55.3162 h。它们表明所选边界阻力会改变预测时长，不能据此推出真实过程必大于基线，亦不能证明所有 p 下的全局单调性。

图像主题 6：经验边界阻力的情景比较。横轴取 $p=1、2、4$ ，分别绘制两种模型的临界时长；同图注明 N800、未标定情景，主结果以独立符号标注或只在图注引用，避免将不同网格结果混为同一精度。正文以这一组比较说明边界解释的影响，详细因子推导与参数范围放入附录。

6 模型检验与灵敏度分析

本题缺少药材内部温度与含水率实测值，因此检验重点是所建方程的数值实现及假设敏感性。解析级数、独立语言程序和不同积分器分别更换求解环节，仍共享题设参数与边界；各方法不能提供独立的物理预测准确率。表 8 差值均取未舍入结果，完整计算过程与采样范围见附录 C。

表 8 数值检验的对象、范围与结果

检验对象	方法及范围	主要结果	支持的结论
问题一热场	N3200, 1801 整数秒 ×21 半径；有限体积与 Robin–Bessel 级数	最大温差 1.804×10^{-6} K	常热物性热方程的数值一致性
三条计算轨迹	N40；Python 与 MATLAB, 100 场值及 2 事件	温差 $\leq 2.375\times10^{-7}$ K, 水分差 $\leq 5.851\times10^{-10}$ kg/kg, 事件差 $\leq 2.297\times10^{-4}$ s	共同采样点的跨实现一致性
问题三、四临界时间	N800、N1600；BDF 与 Radau	最大事件差约 3.2×10^{-6} s	同一空间离散的积分器一致性
场值分辨率	最细相邻网格；同 N 收紧容差与步长，公 共有效域	最大水分差分别 $\leq 4.70\times10^{-5}$ 、 $\leq 9.95\times10^{-9}$ kg/kg	检验范围内的离散敏感性

空间检查分别采用问题一、二三的 N1600 与 N3200，以及问题四的 N3200 与 N6400；时间检查固定最细网格。比较覆盖全整数秒及 21 个有效物理半径点，问题四排除域外位置，真实表面另作核对。网格差不是连续解的严格误差界，四位输出也不保证全部末位稳定。水分平均值与累计流出量的残差约为 10^{-14} kg/kg，说明共享通量记账自洽；热侧验证的是瞬时加权温度变化与边界通量相符，不能扩大为完整热力学能量守恒。

达标时间由全域最大含水率事件确定，另在 N800 验证轨迹上用二分法复核事件定位方法。临界根对应等号，最终报告另行上取并检查原精度浓度严格小于 0.15。该处理将根定位、输出舍入和严格可行性分开，避免把显示值 0.1500 直接作为判断依据。

在 N200 单参数情景中，扩散前因子倍率由 0.7 增至 1.4 时，问题三时长由 79.4187

h 降至 43.0796 h，问题四由 70.4059 h 降至 38.3160 h；导热系数在 $\pm 15\%$ 内变化，两问时长跨度均小于 0.011%。这支持所选工况下时长对扩散参数更敏感，不能据此认为潜热或气固边界不重要。

边界阻力族采用 $p=1$ 、2、4 时，已计算的临界时间随所选参数点增大。 $p=4$ 相对同 N 基线使问题三、四分别延长约 13.87% 和 8.27%；这批结果只完成事件积分，不替代正式严格报告值。其用途是展示未标定边界关系的影响，不能据此推出真实时长下界或参数全域单调性。

7 模型评价与推广

7.1 模型优点

模型结构统一：四个问题的解决用同一套守恒框架，只是物性、几何与判据的切换，逻辑一致、便于对比。

数值方法针对问题特性设计：Kirchhoff 变换精确处理 $D(C)$ 的强非线性面通量；材料坐标把移动域化为固定域；解析稀疏 Jacobian 配合 BD 处理刚性，使 N 达到 3200/6400 仍可在分钟量级完成。

检验分层且可追溯：解析基准、离散守恒、空间/时间收敛、跨语言交叉核验四类证据分开报告，且明确各自的适用范围。

本文将四问置于同一守恒框架内，并明确区分各问的物性、几何和输出任务。问题二与三共享全过程轨迹，使状态表与达标时刻具有共同来源；问题四从组分守恒推导材料坐标，避免重复加入网格速度或把干基浓度误当体积浓度。节点包括中心与真实表面，因此中心对称、表面对流和移动域输出能够一致处理。

数值方法围绕本题的困难作了对应处理。圆柱控制体承担中心和边界的守恒离散，Kirchhoff 势处理低含水率区的非线性面传递，稀疏 Jacobian 和磁盘连续输出分别降低隐式求解与保存高网格轨迹的负担。检验则覆盖解析极限、空间和时间设置、独立实现等环节。最终将原精度轨迹、显示舍入与严格判据分别记录，使结果表能够追溯到实际计算，而不依赖后期人工修饰数字。

7.2 模型不足

缺少物理闭合：未显式计入相变潜热， C_{eq} 用等效映射，密度与收缩相容性未完全解决。

计算设置稳定之后，主要不确定性仍来自模型输入的解释。没有内部温湿度或称

重曲线，就无法用观测残差评价预测准确性；4 h 之后没有环境记录，就必须依赖长期工况假设；没有药材等温线，就无法从空气指标唯一确定材料平衡状态。此外，半径观测只约束表面，不能单独证明内部同比收缩或轴向长度恒定。这些条件与数值网格差属于不同来源，需要在未来实验中分别识别。

基线采用有效显热容量，没有同时求解完整相变、迁移携焓及收缩功；一维模型忽略端面与轴向差异，也未描述品质损伤或有效成分变化。因此温湿预测不能直接推出最低能耗、最佳药效或最优设备参数。表面潜热负荷和经验阻力只提供指定情景下的响应，尚不构成真实过程的上下界。

7.3 改进方向与适用范围

走向工艺优化：将模型作为约束条件接入优化框架，在品质约束下以能耗或时长为目标，利用参数扫描或启发式算法寻优。

维度扩展：针对长径比较小、端面影响不可忽略的物料，可将一维径向扩展为二维轴对称模型，显式处理端面传热传质。

后续可补充不同温度下的药材吸附、解吸等温线，结合质量变化及内部测温数据标定气固边界和有效扩散参数，并留出独立工况检验。若观察到显著轴向温差、长度变化或非均匀收缩，应扩展为二维轴对称模型，重新建立材料运动和质量守恒。完整能量模型还需蒸发位置、潜热及相关传质参数支持。

现有框架适用于几何可近似圆柱、径向传递占主导且物性形式已给定的条件计算。迁移到其他药材时，应重新确定物性、平衡关系和收缩轨迹，而非直接沿用本文系数。在本文约定条件下，四问结果分别给出了预热分布、全过程耦合变化、固定半径达标时长及收缩修正后的时长，可作为后续实验和工况比较的量化依据。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于资料检索、模型推导辅助、程序生成与调试、数值结果核对及论文草拟与文字整理，详细使用情况见支撑材料。

AI 参与内容包括模型假设的备选表述、守恒方程及数值离散的推导建议、求解与检验程序辅助编写，以及正文和附录的整理。本稿依据实际运行记录和可追溯结果编写；数值对照仅用于说明所列计算条件下的一致性，不作为实测精度证明。参赛队须对采用的模型、源码、计算过程、引用和结论逐项人工审查核实；本稿仍处于团队审阅阶段，不代表上述人工审查已经完成。

8 参考文献

- [1] 王乐毅,李长河,刘明政.中药材干燥技术与装备研究现状[J].农业工程学报,2024,40(4):1-15.
- [2] 王晓辉,王学成,唐培渝.数值模拟仿真研究现状及其在中药干燥领域应用展望[J].中国中药杂志,2023,48(13):3440-3447.
- [3] CRANK J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press,1975.
- [4] EYMARD R,GALLOUËT T,HERBIN R. Finite volume methods[M]//CIARLET P G,LIONS J L. Handbook of Numerical Analysis: Vol.7. Elsevier,2000:713-1018. DOI:10.1016/S1570-8659(00)07005-8.
- [5] FAO. Drying principles and general considerations[EB/OL].[2026-09-12]. <https://www.fao.org/4/t1838e/t1838e0u.htm>.
- [6] SCIPY DEVELOPERS. scipy.integrate.solve_ivp[EB/OL].[2026-09-12]. https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html.

附录

本附录依次给出物理推导、数值算法、计算过程、完整生产源码和采纳的检验程序。文件清单用于明确复现材料的组成；最终支撑压缩包需按相同版本重新组织与验收，不能仅凭此表认定旧压缩包可独立运行。

表 A1 支撑材料文件清单与用途

类别	文件或目录	内容与用途
题给输入	附件 1.xlsx、附件 2.xlsx	环境观测与半径观测，原值保留
输出模板	附件 3/result1.xlsx—result4.xlsx	题给四个工作簿结构，不作为原始观测
计算输入	A_environment_observed.csv、 A_radius_observed.csv	两个经过单位转换的观测表
主结果	result1.xlsx、result2.xlsx、 result3.xlsx、result4.xlsx	题目要求的时间—物理半径结果
原精度与校核	对应压缩 CSV、正文 7 表及验证记录	舍入前浓度、严格判据、域外空白及回读核对
生产程序	附录 D 所列 11 个 Python 文件	完整计算、输出与版本记录依赖
检验程序	附录 E 所列 Python 与 MATLAB 文件	正文采用的数值对照及条件情景
配置与环境	model_route.json、环境版本与运行说明	物性来源、参数、输入结构及软件依赖
AI 使用记录	AI 工具使用详情.pdf	实际工具、用途、交互、采纳与人工核实情况

附录 A 物理模型的详细推导

A. 1 干基含水率与组分质量守恒

以药材干物质骨架作为运动参考，记其速度为 u ，单位当前体积的干物质质量为

ρ_d ，水质量为 ρ_w ，水相对骨架的质量通量为 j_w 。干基含水率定义为 $C = \rho_w / \rho_d$ ，单位为 kg 水/kg 干物质。它可以大于 1，不是质量百分数。初值 $C_0 = 2.55$ 对应的湿基水分质量分数为

$$w_0 = \frac{C_0}{1 + C_0} = 0.7183098592 \quad (20)$$

附件环境量仅标 kg/kg，未明示参考质量。若将 Y_∞ 解释为空气湿度比，其分母才是干空气质量，与 C 的分母不同。虽然数值单位都写成 kg/kg，把二者直接相减仍需要等效映射。本文以 $C_{eq}(t)$ 表示由环境数据构造的等效边界量，不将其解释为已测得的药材平衡含水率。

忽略干物质来源和损失时，干骨架与总水质量分别满足

$$\frac{\partial \rho_d}{\partial t} + \nabla \cdot (\rho_d u) = 0, \quad (21)$$

$$\frac{\partial (\rho_d C)}{\partial t} + \nabla \cdot (\rho_d C u + j_w) = 0 \quad (22)$$

将第二式展开，再减去第一式的 C 倍，可消去骨架压缩引起的密度变化，得到

$$\rho_d \left(\frac{\partial C}{\partial t} + u \cdot \nabla C \right) = - \nabla \cdot j_w \quad (23)$$

采用相对骨架的有效 Fick 本构 $j_w = -\rho_d D \nabla C$ ，有

$$\rho_d D_t C = \nabla \cdot (\rho_d D \nabla C), \quad D_t = \frac{\partial}{\partial t} + u \cdot \nabla \quad (24)$$

只有在 ρ_d 空间均匀时，才能从散度内约去它。固定半径的前三问取 $u = 0$ ，干骨架保持初始均匀，方程于是化为圆柱一维径向形式

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right) \quad (25)$$

这里的 $1/r$ 来自圆柱壳传递面积随半径的变化，而不是额外的经验修正。若删除它改成平板扩散方程，即使程序仍能收敛并保持某个离散和不变，也已经改变了药材几何。

A.2 有效热容量与题给物性的使用范围

傅里叶定律给出导热通量 $q = -k \nabla T$ 。本文采用的热方程为有效显热容量闭合

$$B(C) D_t T = \nabla \cdot (k(C) \nabla T), \quad B(C) = \rho_{eff}(C) c_p(C) \quad (26)$$

B 的单位为 $J/(m^3 K)$ 。题给密度记作 ρ_{eff} ，以区别质量守恒中的 ρ_d 。按四问

分别使用的经验关系如下，扩散系数中温度一律以 K 代入，显示温度才换成摄氏度。

$$\begin{aligned} Q1: \rho_{eff} &= 820, c_p = 2600, k = 0.36, \\ D_1(C) &= 7 \times 10^{-9} \exp(-0.89/C) \end{aligned} \quad (27)$$

$$\begin{aligned} Q2、Q3: \rho_{eff} &= 650 + 128C, \\ c_p &= 1450 + 2736 \frac{C}{1+C}, k = 0.21 + 0.38 \frac{C}{1+C}, \\ D_{23}(T, C) &= 2.4 \times 10^{-3} \exp(-0.45/C - 3850/T) \end{aligned} \quad (28)$$

$$\begin{aligned} Q4: \rho_{eff} &= 760 + 90C, \\ c_p &= 1850 + 2150 \frac{C}{1+C}, k = 0.12 + 0.20 \frac{C}{1+C}, \\ D_4(T, C) &= 4.2 \times 10^{-4} \exp(-0.30/C - 3850/T) \end{aligned} \quad (29)$$

这些系数来自题给经验公式，不能说由本次实验重新拟合。Q1 的热方程与水方程解耦，水扩散系数仍随 C 变化；Q2、Q3 从初始时刻统一使用附录 3，而不把 Q1 的末状态或常热物性拼接进去。Q4 从初始时刻整组换用附录 4，同时引入半径变化。

若坚持把 Q4 的 ρ_{eff} 同时解释成真实湿密度，应有

$$\rho_d = \frac{760 + 90C}{1+C} = 90 + \frac{670}{1+C} \leq 760 \quad (C \geq 0) \quad (30)$$

初始干物质密度为 $\rho_{d0} = (760 + 90 \times 2.55) / 3.55 = 278.7323943662$ 。固定长度且干物质量守恒要求

$$\pi L R_0^2 \rho_{d0} \leq 760 \pi L R(t)^2, R(t) \geq 1.2112029565 \text{ cm} \quad (31)$$

附件 2 的 72 h 半径为 1.198 cm，表明这些字面解释不能在所给全时段同时成立。因此本文保留题给 $\rho_{eff} c_p$ 作为有效热容量，干骨架质量另行守恒。这是明确的闭合选择，并非证明了真实湿密度、收缩体积及组分质量的全部关系。

容量形式还限定了能量解释。由于 B 随 C 变化， $B(C)T_i$ 不等于 $\partial_i[B(C)T]$ 。完整组分焓平衡还可能包含水迁移携焓、相变热和机械功；当前基线没有逐项识别这些机制。以下守恒检验针对所采用的有效方程，不能据此宣称完整热力学过程已闭合。

A.3 同比收缩的材料坐标变换

第四问采用定长、同比径向收缩假设。半径 $R(t)$ 由附件 2 分段线性插值，内部骨架速度选为

$$u_r(r, t) = \frac{r}{R(t)} \dot{R}(t), x = \frac{r}{R(t)} \in [0, 1] \quad (32)$$

这一速度使表面骨架恰随测得半径移动，并令每个材料坐标保持不变。圆柱对称下 $\nabla \cdot u = 2\dot{R}/R$ ，因此干骨架连续性方程给出

$$D_t \rho_d = -2 \frac{\dot{R}}{R} \rho_d, \quad \rho_d(t) = \rho_{d0} \left(\frac{R_0}{R(t)} \right)^2 \quad (33)$$

设 $C(r, t) = \hat{C}(x, t)$ 。在物理位置固定与材料位置固定两种求导条件下，链式法则分别为

$$\left. \frac{\partial C}{\partial t} \right|_r = \hat{C}_t - \frac{x \dot{R}}{R} \hat{C}_x, \quad \frac{\partial C}{\partial r} = \frac{1}{R} \hat{C}_x \quad (34)$$

骨架平流项恰为 $u_r C_r = x \dot{R} \hat{C}_x / R$ ，两者相消， $D_t C = \hat{C}_t$ 。将空间导数换元后得到

$$\hat{C}_t = \frac{1}{R^2 x} \frac{\partial}{\partial x} (x D \hat{C}_x), \quad B(\hat{C}) \hat{T}_t = \frac{1}{R^2 x} \frac{\partial}{\partial x} (x k \hat{T}_x) \quad (35)$$

其结果只有时变几何因子，不再含额外网格平流。干基含水率是质量比，纯收缩而无相对水通量时 $D_t C = 0$ ，也不应添加 $-2\dot{R}C/R$ 的体积浓缩项。若内部骨架并非同比运动，或干物质密度在空间上不均匀，应回到一般守恒方程，不能继续照搬上述简式。

中心满足对称零通量。表面条件为

$$-\frac{k}{R} \hat{T}_x(1, t) = h(T_s - T_\infty), \quad -\frac{D}{R} \hat{C}_x(1, t) = \beta(C_s - C_{eq}), \quad (36)$$

其中 $h = 25 \text{ W}/(\text{m}^2 \text{ K})$ 、 $\beta = 8 \times 10^{-7} \text{ m/s}$ 。变换后方程只需当前 $R(t)$ ，不需要计算折线半径的数值导数。因子 R^{-2} 表明尺度缩短会加强内部扩散，但 Robin 边界仍含 R ，物性也随状态变化，不能仅凭平方尺度就推导全过程时长的严格比例。

附录 B 有限体积、Jacobian 与时间积分推导

B.1 节点有限体积、中心控制体与真实表面

取 N 个均匀区间， $x_i = i \Delta x$ 、 $\Delta x = 1/N$ ，共 $N+1$ 个节点。中心与表面都是真实未知量节点。内部控制体边界取相邻节点中点，中心控制体为 $[0, \Delta x/2]$ ，表面控制体为 $[1 - \Delta x/2, 1]$ 。定义环形权重

$$w_i = \int_{a_i}^{b_i} x \, dx = \frac{b_i^2 - a_i^2}{2}, \quad \sum_{i=0}^N w_i = \frac{1}{2} \quad (37)$$

权重展开为

$$w_0 = \frac{\Delta x^2}{8}, \quad w_i = x_i \Delta x \quad (1 \leq i \leq N-1), \quad w_N = \frac{\Delta x}{2} - \frac{\Delta x^2}{8} \quad (38)$$

单位长度的控制体积为 $2\pi R^2 w_i$ 。环体积分用节点值乘权重近似，未知量仍位于节点；不能将其无条件当成精确的环形平均值，也不能把点值解析解与控制体平均解析解混用。

对于变热容量，控制体上的准确积分首先是 $\int_{a_i}^b x B(C) \hat{T}_i dx$ ，离散时将它近似为 $w_i B(C_i) \dot{T}_i$ 。此处没有先构造 $B(C_i) T_i$ 再作时间差分，因而不会凭空引入容量导数项。网格控制体的几何边界在材料坐标中固定，其物理体积随 R^2 变化；两种坐标下的体积口径应贯穿质量、热量与输出查询，不能在中途交换。

记 $g^T = x k T_x$ 、 $g^C = x D C_x$ 为沿径向正坐标的梯度通量。它们与向外的物理传递通量相反，因此药材向外失水时表面 g^C 为负。对控制体积分得到

$$\dot{T}_i = \frac{g_{i+1/2}^T - g_{i-1/2}^T}{R^2 w_i B_i}, \quad \dot{C}_i = \frac{g_{i+1/2}^C - g_{i-1/2}^C}{R^2 w_i} \quad (39)$$

中心面半径为零，直接令 $g_{-1/2}^T = g_{-1/2}^C = 0$ ，无需在代码中计算 $1/x_0$ 。常系数热方程在中心退化为

$$\dot{T}_0 = \frac{4k(T_1 - T_0)}{B R^2 \Delta x^2} \quad (40)$$

系数 4 与光滑偶函数在圆柱中心的拉普拉斯极限一致。表面直接用 Robin 条件给外侧面值

$$g_{N+1/2}^T = h R (T_\infty - T_N), \quad g_{N+1/2}^C = -\beta R (C_N - C_{eq}) \quad (41)$$

本格式的最后未知量就在实际表面，不需要再从“最后单元中心”额外外推一个表面值，也不应重复叠加半单元内阻。若改用单元中心网格，则未知量位置与边界离散都必须同步改变。

热内部面采用调和平均

$$k_H = \frac{2k_i k_{i+1}}{k_i + k_{i+1}}, \quad g_{i+1/2}^T = \frac{x_{i+1/2}}{\Delta x} k_H (T_{i+1} - T_i) \quad (42)$$

它对应两侧半间距导热阻串联，并保持相邻控制体共用同一面通量。共享通量的正负配对是离散守恒的基础；若两侧分别计算并取不同近似，整体求和时就可能产生虚假的内部源。

B.2 浓度 Kirchhoff 势与近等浓度稳定计算

三组扩散率均可写为 $D = D_0 \exp(-b/T) \exp(-a/C)$ ，其中 Q1 取 $b=0$ ，其他两

组取 $b=3850$ 。引入

$$\Phi_a(C) = C e^{-a/C} + a Ei(-a/C) \quad (43)$$

利用指数积分的导数，分别有

$$\frac{d}{dC}(C e^{-a/C}) = e^{-a/C}(1 + a/C), \quad \frac{d}{dC}[a Ei(-a/C)] = -\frac{a}{C} e^{-a/C} \quad (44)$$

相加得 $\Phi'_a(C) = e^{-a/C}$ ，从而势差等于浓度非线性系数的积分。生产水面通量为

$$g_{i+1/2}^C = \frac{x_{i+1/2}}{\Delta x} D_0 \exp\left(-\frac{b}{T_i}\right) [\Phi_a(C_{i+1}) - \Phi_a(C_i)], \quad \bar{T}_i = \frac{T_i + T_{i+1}}{2} \quad (45)$$

浓度势积分关系本身准确，温度面值、空间重构和圆柱几何面值仍有离散近似。不能对温度因子与势函数的乘积整体作差，因为

$$\partial_x(e^{-b/T} \Phi_a(C)) = e^{-b/T} e^{-a/C} C_x + e^{-b/T} \frac{b}{T^2} T_x \Phi_a(C) \quad (46)$$

后一项不属于当前 Fick 本构。相等浓度、不同温度即可构成反例：正确浓度通量为零，整体乘积势差却不为零。

当相邻浓度接近时，直接相减两个势值可能损失有效位。程序在 $|\Delta C| < 10^{-7} \max(\bar{C}, 10^{-3})$ 时改用

$$\Delta \Phi \approx f(\bar{C}) \Delta C, \quad f(C) = e^{-a/C}, \quad \bar{C} = \frac{C_i + C_{i+1}}{2} \quad (47)$$

该式为中点积分近似，局部截断差为 $O((\Delta C)^3)$ 。求解器对非物理 Newton 试探点仅在系数求值时令 $C^+ = \max(C, 10^{-12})$ ；它没有把已求得的状态直接裁成正值。因此正性需由接受状态检查确认，不能把这个保护值当作物理含水率下限。

B.3 解析稀疏 Jacobian 的关键块

状态按 $y = (T_0, C_0, T_1, C_1, \dots, T_N, C_N, \ell)^T$ 交错排列，维数为 $2N+3$ 。 ℓ 是平均累计失水量，不反馈温湿场。每个内部面只依赖左右两个节点，故除最后的诊断行外，Jacobian 具有邻近的二乘二块带结构。

记 $\gamma_i = x_{i+1/2} / \Delta x$ 、 $\Delta T = T_{i+1} - T_i$ 。调和平均对两侧导热系数的导数为

$$\frac{\partial k_H}{\partial k_i} = \frac{2k_{i+1}^2}{(k_i + k_{i+1})^2}, \quad \frac{\partial k_H}{\partial k_{i+1}} = \frac{2k_i^2}{(k_i + k_{i+1})^2} \quad (48)$$

热通量对 $(T_i, C_i, T_{i+1}, C_{i+1})$ 的四个偏导于是为

$$(-\gamma_i k_H, \gamma_i \frac{\partial k_H}{\partial k_i} k'_i \Delta T, \gamma_i k_H, \gamma_i \frac{\partial k_H}{\partial k_{i+1}} k'_{i+1} \Delta T) \quad (49)$$

水通量记为 $g^C = G_i \Delta \Phi$ ，其中 $G_i = \gamma_i D_0 e^{-b/\bar{T}_i}$ 。正常势差分支下有

$$\frac{\partial g^C}{\partial C_i} = -G_i e^{-a/C_i}, \quad \frac{\partial g^C}{\partial C_{i+1}} = G_i e^{-a/C_{i+1}}, \quad (50)$$

$$\frac{\partial g^C}{\partial T_i} = \frac{\partial g^C}{\partial T_{i+1}} = g^C \frac{b}{2\bar{T}_i^2} \quad (51)$$

这两项温度偏导来自实际面平均温度，不能因“面上冻结温度”而在 Newton 迭代中将其误置零。Q1 的 $b=0$ ，水方程对温度偏导确实为零。

近等浓度分支必须对实际用到的 $f(\bar{C}) \Delta C$ 求导，得到

$$\frac{\partial \Delta \Phi}{\partial C_i} = \frac{1}{2} f'(\bar{C}) \Delta C - f(\bar{C}), \quad \frac{\partial \Delta \Phi}{\partial C_{i+1}} = \frac{1}{2} f'(\bar{C}) \Delta C + f(\bar{C}), \quad (52)$$

$$f'(C) = \frac{a}{C^2} e^{-a/C} \quad (53)$$

否则右端与 Jacobian 在数值分支上不一致，会干扰 Newton 收敛。对于系数延拓区，程序还按 $C > 10^{-12}$ 的活跃标记处理导数；阈值处是分段规则，不把它当成平滑物理本构。

每个面偏导分别以正、负符号加入左、右控制体，再乘对应热或水的几何尺度。热方程的分母也依赖 C_i ，因此局部还必须加上商法则项

$$\left. \frac{\partial \dot{T}_i}{\partial C_i} \right|_{\text{容量}} = -\dot{T}_i \frac{B'_i}{B_i}, \quad B'_i = \rho'_{eff} c_p + \rho_{eff} c'_p \quad (54)$$

例如 Q23 有 $\rho'_{eff} = 128$ 、 $c'_p = 2736/(1+C)^2$ 、 $k' = 0.38/(1+C)^2$ ；Q4 对应为 90、2150/(1+C)²、0.20/(1+C)²。遗漏容量项会使热湿交叉块不完整，即使物性公式本身抄写正确也不能得到正确 Jacobian。

固定时刻的 R, T_∞, C_{eq} 是外部量。表面热、水梯度通量对表面状态的偏导分别为 $-hR$ 与 $-\beta R$ 。诊断状态满足

$$\dot{\ell} = \frac{2\beta}{R} (C_N - C_{eq}), \quad \frac{\partial \dot{\ell}}{\partial C_N} = \frac{2\beta}{R} \quad (55)$$

最后一整列为零。令行向量 m 在各 C_i 位置取 $2w_i$ 、在 ℓ 位置取 1，其余取零，则守恒关系要求 $mJ=0$ 。该左零关系与对右端直接扰动的导数比较，可分别检验守恒装配和局部线性化。

B.4 质量及有效热通量的整体检验

将水方程乘 $2w_i$ 求和，所有内部面相消，仅剩表面通量，得到

$$\frac{d}{dt}(2\sum_{i=0}^N w_i C_i) = -\frac{2\beta}{R}(C_N - C_{eq}), \quad 2\sum_{i=0}^N w_i C_i + \ell = C_0 \quad (56)$$

这个式子与第四问的移动体积相容，因为 $\rho_d R^2 = \rho_{d0} R_0^2$ 不变，平均干基含水率乘初始干质量就是剩余水质量。不能把单纯未加权的节点算术平均代入该恒等式。

温度方程的正确加权关系是

$$2\pi R^2 \sum_i w_i B_i \dot{T}_i = 2\pi R h (T_\infty - T_N) \quad (57)$$

两边都是单位长度的瞬时热率，单位 W/m。若构造 $H' = 2\pi R^2 \sum_i w_i B_i (T_i - T_{ref})$ ，其总导数还含

$$\frac{dH'}{dt} = 2\pi R h (T_\infty - T_N) + 2\pi R^2 \sum_i w_i (T_i - T_{ref}) B_i' \dot{C}_i + 4\pi R \dot{R} \sum_i w_i B_i (T_i - T_{ref}) \quad (58)$$

容量变化项和体积变化项不能遗漏，更不能未经物理推导称为已测得的机械功。只有 Q1 的容量、半径均固定时，可将瞬时关系直接积分为常物性显热收支。耦合问的瞬时恒等式检验的是离散热算子，不能替代完整组分焓与相变能量平衡。

B.5 隐式 BDF、分段环境与阈值事件

空间离散后得到刚性常微分系统 $\dot{y} = F(t, y)$ 。扩散特征尺度随网格加密约按 Δx^{-2} 增大，低含水率的指数扩散率又使时标差异突出，因而采用隐式时间推进。BDF 的基本思想是以若干已接受历史点与当前未知点构造插值多项式，用其当前时刻导数近似状态导数。等步长二阶形式为

$$\frac{3y_n - 4y_{n-1} + y_{n-2}}{2h} = F(t_n, y_n) \quad (59)$$

一般写为 $\alpha_0 y_n + \sum_{j=1}^q \alpha_j y_{n-j} - h\beta_q F(t_n, y_n) = 0$ 。Newton 修正需要求解

$$(\alpha_0 I - h\beta_q J) \delta y = -G \quad (60)$$

上节的解析稀疏矩阵因此同时承担热湿耦合线性化与刚性求解的作用。实际生产使用 SciPy BDF 的 1—5 阶自适应实现及 NDF 精度修正，不是始终固定为二阶；上式用于说明原理。磁盘密集输出类只保存原始双精度多项式系数，并未降低阶数或另换积分方法。

从插值观点看，若 $P_q(t)$ 是通过当前未知状态和前 q 个历史状态的次数不超过 q 的多项式，则令 $P_q'(t_n) = F(t_n, y_n)$ 就得到相应后向差分关系。等步长时各项系数由

插值基函数导数给出，变阶或步长调整时则需同步变换历史差分量。本文使用库内经过实现的历史更新、误差估计和步长控制，不自行把固定二阶系数套在任意不等时间间隔上。Newton 线性化失败或误差估计过大时，求解器会重试或缩短步长；输出的整数秒不等于每个被接受的内部时刻。

误差控制按分量尺度 $s_j = atol_j + rtol |y_j|$ 归一化。正式设置取 $rtol = 10^{-10}$ ，温度绝对容差 10^{-10} K，含水率及累计失水绝对容差 10^{-12} 。这是局部误差控制参数，不等于全时空误差上界。前 4 h 最大步长为 2 s，之后为 120 s；题目要求的 1 s 或 60 s 是输出采样间隔，密集输出在这些时刻求值，不要求内部积分步长始终小于输出间隔。

环境观测只覆盖 0—4 h。代码在 4 h 处分段求解，之前按观测线性插值，之后采用已声明的 50°C、等效 $C_{eq} = 0.05$ 平台。分段可明确处理延拓接口；它不使平台变成实测。Q4 的半径使用观测范围内插值，固定物理位置 r_j 查询时换算为 $x_j = r_j / R(t)$ ，若 $x_j > 1$ 则留空，真实表面另在 $x = 1$ 查询。

Q3 与 Q2 使用同一条场解。令 $M_N(t) = \max_i C_i(t)$ ，事件函数为

$$g(t) = M_N(t) - 0.15 \quad (61)$$

求解器定位由正到负的零点，并取终止方向为 -1 。该零点是临界阈值时刻，等号本身不满足严格小于。节点间若采用分段线性重构，重构场最大值等于节点最大值；这仍不是对真实连续场误差的严格界。

程序由事件状态继续真实积分到 $\lceil t_* \rceil + 1$ s，再在 0.0001 h 网络上取不早于事件的候选时刻

$$t_{rep,h} = 10^{-4} / 10^4 \frac{t_*}{3600} / \quad (62)$$

随后查询该时刻未舍入的全域最大含水率；若仍不小于 0.15，就继续增加一个报告网格单位，并检查查询始终位于已积分区间内。四位显示可能仍为 0.1500，不应人为改成 0.1499。报告时长是条件模型中的可行数值时刻，不能解释为实测置信上界。

整个求解与导出次序如下。首先读取并核对原始驱动的单位、时间顺序和输入版本，建立材料网格及均匀初态；其次逐段积分，使用真实接受状态传递段间初值，并在每次求值中更新物性与面通量；随后定位全域阈值事件、继续计算并核对严格报告时刻；最后从同一密集解查询正文时刻和完整工作簿日程，换算显示单位、处理缩域空白并实施四位舍入。参数、状态、域内外判别及阈值比较均在舍入前完成，不能为了某个表格末位改动判据或在求解前把物性截成四位小数。

在连续数学意义上，严格达标集合可能是一个开时间区间，其下确界是含水率恰等于阈值的临界时刻，并不一定存在“最早的严格小于时刻”。所以本文将连续根与离散报告网格上的可行时刻分开记录。有限时间搜索未触发事件时，应报告该搜索区间内尚未得到达标结果，而不能把搜索上限当成烘干时长。

B.6 可选经验阻力情景及其物理边界

在保持基线不变的前提下，可令表面边界乘以含水率相关的系数

$$K_p(C) = \frac{1 - (C_{ref}/C)^{1/p}}{1 - C_{ref}/C}, \quad C \geq C_{ref} = 0.05, \quad p \geq 1 \quad (63)$$

其启发来自假设活动度曲线 $a_p(C) = 1 - (1 - a_{ref})(C_{ref}/C)^{1/p}$ 相对基准成员的驱动比。令 $z = C_{ref}/C$ 、 $q = 1/p$ ，由 $z^q \geq z$ 及凹函数的切线界 $z^q \leq 1 + q(z - 1)$ 得

$$\frac{1}{p} \leq K_p(C) \leq 1, \quad K_1(C) = 1, \quad \lim_{C \downarrow C_{ref}} K_p(C) = \frac{1}{p} \quad (64)$$

因此它是可控且含基线的经验传质阻力族。程序实际只将 $\beta(C_s - C_{eq})$ 替换成 $\beta K_p(C_s)(C_s - C_{eq})$ ，没有把表面饱和蒸气压或真实材料等温线代入边界。参数 p 未由内部温湿或称重数据标定， $p \geq 1$ 表示只研究传质减弱情景；它不是因 $p < 1$ 必然产生负活动度。有限参数点的时长排序也不能证明耦合模型对所有 p 的全局单调性。

若进一步施加比例为 χ 的表面汽化负荷，则 $j_w = \rho_d \beta K_p(C_s)(C_s - C_{eq})$ ，热面值改为

$$g_s^T = h R (T_\infty - T_s) - R \chi L_v j_w \quad (65)$$

这只表达指定排水在表面汽化的能量情景。真实气固边界还需材料等温线、表面温度、气膜传质系数、蒸发或凝结与吸附热的一致关系；缺少这些信息时，不把情景温度或时长作为真实预测误差的上下界。此扩展展示的是边界假设的敏感性，不能取代基线结果的来源说明。

B.7 圆柱 Robin 热场解析基准

问题一热扩散率为常数 $\alpha = k / (\rho_{eff} c_p)$ 。对均匀初温和空间均匀的时变环境，令 $v = T - T_\infty(t)$ ，则 $v_t = \alpha r^{-1} (r v_r)_r - \dot{T}_\infty(t)$ ，表面满足齐次 Robin 条件。取 $x = r / R_0$ ，空间特征函数为 $J_0(\mu_n x)$ ，正特征根满足

$$\mu_n J_1(\mu_n) = Bi_h J_0(\mu_n), \quad Bi_h = \frac{h R_0}{k}, \quad \lambda_n = \frac{\alpha \mu_n^2}{R_0^2} \quad (66)$$

由于圆柱内积以 $x \, dx$ 为权，常函数在特征函数上的系数为

$$b_n = \frac{\int_0^1 x J_0(\mu_n x) dx}{\int_0^1 x J_0(\mu_n x)^2 dx} = \frac{2 J_1(\mu_n)}{\mu_n [J_0(\mu_n)^2 + J_1(\mu_n)^2]} \quad (67)$$

将温度差展开并对各模态求解一阶线性方程，得到

$$T(x, t) = T_\infty(t) + \sum_{n=1}^{\infty} b_n J_0(\mu_n x) [(T_0 - T_\infty(0)) e^{-\lambda_n t} - \int_0^t e^{-\lambda_n(t-\tau)} \dot{T}_\infty(\tau) d\tau] \quad (68)$$

在附件分段线性插值的区间 $[a, b]$ 上，斜率为 s ，且 $b \leq t$ ，相应积分贡献为 $s[e^{-\lambda_n(t-b)} - e^{-\lambda_n(t-a)}]/\lambda_n$ ；包含当前时刻的末区间以 $b=t$ 截断。这使时变边界卷积可逐段计算，避免把恒温阶跃解直接套入附件。级数计算截断为有限项，同时比较不同截断项数以区分级数截断与数值网格差。解析解针对常热物性热场，不能验证非线性水方程或题给变物性全过程。

附录 C 数值解计算过程与复现

C.1 固定评价条件与计算顺序

本附录说明本文数值结果的产生过程。由于附件只提供烘房环境和药材半径，没有内部温度、含水率实测标签，计算方案以离散一致性、守恒、物性正性和网格稳定性评价，不以虚构的预测准确率筛选。比较数值方法时保持输入、初始场、物性、边界和达标判据相同；尾段温度、传质系数、潜热比例或半径规律改变后，作为独立物理情景记录。计算时间缩短不必然意味着方案更好，干燥时间变短更不能直接作为算法改进的判据。

三条数值轨迹分别对应问题一、问题二与三、问题四。问题一从初始场采用附录 2 参数；问题二与三从同一初始场统一采用附录 3，二者共享一次求解；问题四从初始场采用整组附录 4 及给定收缩半径。清洗环境 CSV 含 241 个 0—4 h 记录，半径 CSV 含 145 个 0—72 h 记录。内部使用 m、s、K，初始状态为 301.15 K、2.55 kg/kg；环境按时间线性插值，4 h 后采用 50°C 和等效含水率 0.05 的约定。该环境指标直接作为材料等效平衡值是一项闭合假设，数值程序不把它转换成已识别的吸附等温线。

离散采用材料坐标上的均匀节点及环形对偶控制体。节点包括中心和实际表面，内部面通量对相邻控制体一正一负地贡献，中心通量为零。问题四每次计算右端时更新 $R(t)$ ，并在扩散项中使用 $R(t)^{-2}$ ；干基含水率不再附加体积浓缩项。状态向量按温度、含水率交错排列，末项保存累计失水。该末项只记账，不反向影响温湿传递，其 Jacobian 整列应为零。

C.2 数值方案的保留与舍弃

最初按粗网格事件与少量场值评价，后来发现临界时间比局部含水率更早趋于稳定，因此增加全整数秒及 21 个物理半径的检查，重点覆盖第一秒边界层和移动表面附近。这是评价覆盖的明确扩展，不能倒写成从第一轮即完成全部场值检验。历史记录共有 42 条不同 trial，包括 3 条基线、24 条网格、13 条物理情景及 2 条早期时间设置；其中写入的函数成功状态不全部具有独立进程退出证据，本文不将这些记录等同于 42 次外部监督验收。

环节	实际比较或发现	处理及对最终方案的影响
粗网格调和面通量	Q23 的 N50/N100/N200/N400 事件依次约 164.5440/71.5633/58.9518/57.6868 h	不采用未收敛粗网格作正式答案；保留其反证记录
Kirchhoff 浓度势	同系列 N50 至 N800 事件约 57.4792 至 57.4723 h，常 D 极限相容	保留浓度积分面格式，并继续检验局部场
早期解析水分特例	常 D、N400 的密集样点误差约 $2.8751e-4$ ，超过当时 $1e-4$ 标准	保留不通过记录，加密网格和采样；不把特例改作非线性答案
解析稀疏 Jacobian	对状态方向差分和短间接线检查，保留交叉导数、容量分母及边界项	用于正式 BDF 求解；缺失的旧警告记录不补填为零
两个并行高网格版本	内存资源不足，只有 N1600 保存完成	主动中止；未完成的 N3200 不作为有效结果
磁盘连续输出 v5→v6	全断点检查发现 v5 约 $2.22e-16$ 的选段末位差	v6 统一 BDF 断点选段和释放流程；三条 N40 完整对照逐位一致
最终空间与时间检查	事件稳定后仍检验局部场；同 N 同时收紧容差、最大步长	最终采用 Q1/Q23 N3200、Q4 N6400
正式生产及输出	独立父进程观察完成后发布版本结果，保存完整回读记录	正式结果固定为 final_v6a，历史试验不替换主答案

Kirchhoff 格式的作用是先积分浓度方向上的非线性扩散因子，再取相邻节点势差；面温度仍为数值近似，不能称整个通量无离散误差。其被选用的理由是推导、常系数极限和网格证据一致支持，而不是计算得到的干燥时长更短。调和平均在一般问题中并非无效，但本题极干表层的扩散率跨度会使其粗网格通量受到明显限制。

C.3 正式求解、临界事件与连续输出

正式计算采用 BDF 时间积分、解析稀疏 Jacobian， $\text{rtol}=1\text{e-}10$ ，温度与水分绝对容差分别为 $1\text{e-}10$ 、 $1\text{e-}12$ 。程序在 4 h 处显式分段，之前最大步长 2 s，之后 120 s；这些是允许的上限，自适应积分器可以采用更小步长。输出每隔 1 s 或 60 s 不意味着积分步长相同。Q1 推进至 1800 s；其余轨迹设置较长搜索窗口，以全域最大含水率下降穿越 0.15 的事件停止，窗口上限不能冒充达标时间。

临界根满足 $\max C=0.15$ ，故根时刻本身不满足严格小于。程序在事件后继续实际积分至 $\text{ceil}(\text{tc 秒})+1\text{ s}$ ，并在四位小时网格上取报告时刻；若原精度浓度仍未严格达标，继续向后一个 0.0001 h 格并重新查询，超出已验证轨迹则报错。由此分别得到 Q3 临界 57.47230195056044 h、严格报告 57.4724 h，以及 Q4 临界 51.09057478683054 h、严格报告 51.0906 h。浓度显示为 0.1500 并不自动违反严格判据，也不能改写成 0.1499。

高网格计算保存原精度 BDF 分段多项式和已接受状态到磁盘，按块查询后释放映射。该策略改变存储方式，不改变积分方程，也不从每 60 s 稀疏文件插补制造逐秒场。v6 曾在三条完整 N40 轨迹的内部断点、左右相邻浮点时刻及其他查询处对照内存版本，状态与事件逐位一致；这证明该检查范围内的存储等价，不能替代高网格空间精度或真实物理检验。

最终空间加密在公共域内的全整数秒、21 个固定物理半径上给出最大水分差 $4.6903\text{e-}5$ 、 $3.8303\text{e-}5$ 、 $1.9799\text{e-}5\text{ kg/kg}$ 。历史全秒空间投影未全部保留，本次核查依据原报告及现存稀疏数组，未重新回放全部空间投影。时间检查则保留完整投影：同 N 将容差收紧十倍、步长上限减半，三问最大水分差为 $3.6191\text{e-}9$ 、 $9.9411\text{e-}9$ 、 $9.8180\text{e-}10\text{ kg/kg}$ ；这些差值可由保存数组逐值归约复核。上述均为限定采样域的数值敏感性，不是连续解严格误差界。

C.4 输出与运行前提

输出由尚未关闭的 Run 查询到题目指定时间和物理半径。问题四将材料坐标映回物理域， $r>R(t)$ 位置留空，真实表面另列；越过药材边界不得外推。Excel 采用流式写

入，原精度数据另存压缩 CSV；温度表输出℃，长时程表输出 h，所有显示值按四位小数格式保存，计算过程不提前舍入。随后完整回读四份工作簿，并将正文表格、半径身份、空白域和独立 Run 查询交叉核对。历史正式验证覆盖 9335598 个工作簿单元格和 297 个正文 CSV 单元格。

完整源码按附录 D 的 11 个原文件分别嵌入，不能拼接为一个 Python 文件。运行时保持原工程层级：代码位于 `paper_output/code/modeling`，两个 CSV 位于 `paper_output/data_cleaned`，模型路线位于 `paper_output/plan`，题给附件与四个 Excel 模板位于 `problem_files` 下原位置。总入口会记录这些输入的指纹；绘图模块及总入口还依赖 Windows 微软雅黑字体。程序使用 SciPy 的 BDF 相关内部接口，历史环境为 Python3.14.7、NumPy2.5.2、SciPy1.18.1、openpyxl3.1.5、Matplotlib3.11.1；其他环境需重新核验。

从工程根目录调用 `run_modeling.py` 并指定未使用的版本名、`n1=3200`、`n23=3200`、`n4=6400` 及单线程参数。用于新复核时可选 `review` 和 `review-exports`，将结果放入新复核目录；普通生产路线还会发布结果契约。不要覆盖 `final_v6a`。该布局说明与源码哈希只保证可理解的复现前提，不证明现有压缩包解压后可独立运行，也不把 CLI 退出视为 Visual Studio 或用户审查通过。任何改模型、输入或参数的操作均应先重算和检查，再更新论文数字。

附录 D 完整源程序

以下按文件逐一列出正式计算实际调用的完整源码。各文件分别保存，依赖目录、输入与环境按附录 C 配置；源码中运行和人工审查状态变量按其原有含义保留，不等于团队已完成最终签核。

编号	文件	作用	行数
D.1	<code>run_modeling.py</code>	总入口、实际进程监督、版本隔离与结果发布	448
D.2	<code>q1_model.py</code>	问题一正式参数入口	8
D.3	<code>q2_model.py</code>	问题二/三共用轨迹参数入口	8

编号	文件	作用	行数
D.4	q3_model.py	全域阈值、四位小时上取与严格可行复核	27
D.5	q4_model.py	问题四附录 4 物性与收缩开关入口	8
D.6	drying_core.py	读取输入、有限体积、水热物性、通量、ODE 事件和稠密轨迹查询	402
D.7	analytic_jacobian.py	全部状态交叉导数与解析稀疏 Jacobian	325
D.8	disk_dense.py	BDF 原精度多项式磁盘存储、分段边界与生命周期	121
D.9	export_outputs.py	模板识别、完整 Excel/未舍入 CSV 导出、回读和活轨迹采样核验	748
D.10	publication_plots.py	正式入口直接调用的主题图生成及图源指纹	402
D.11	production_provenance.py	源文件/输入指纹、运行元数据、监督进程和发布事务	497

D.1 run_modeling.py

总入口、实际进程监督、版本隔离与结果发布。源文件 SHA256：
d39b23051cc00d7f06482f90564e0cb34052849f15c5275d439b8824e8016b84。

```
"""Supervised, versioned production and Visual Studio reproduction of A.

The parent snapshots inputs, observes the actual worker exit, and publishes only
verified contracts. The worker retains one Run; Q2/Q3 share Q23 before close().
--review runs fresh solves; --review-exports also repeats the complete exports.
GUI and human acceptance need
separate observed evidence and are never certified by this script.
"""
from __future__ import annotations

import gc
import hashlib
import importlib
import json
import os
from pathlib import Path
import subprocess
import sys
import time
import traceback
import uuid
import warnings

SOURCE = Path(__file__).resolve()
ROOT = SOURCE.parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(SOURCE.read_bytes()).hexdigest()
import production_provenance as provenance

ASSUMPTIONS = [
    '一维径向有效模型，忽略轴向/端面基线，以二维独立程序检验端面影响。',
    '题给  $\rho \cdot c_p$  作为有效体积显热容量；不强制它等于守恒骨架的真实湿质量密度。',
    '附件烘房 kg/kg 数值直接作为材料干基等效  $C_{eq}$ ；这不是已证明的气固吸附平衡换算。',
    '基线不显式加入潜热；给定等效驱动力的潜热试验仅是能量负荷压力测试。',
    '前 4h 采用原始线性插值，4h 后  $T_{air}=50$  摄氏度、 $C_{eq}=0.05$  作为长期延拓假设。',
    'Q4 固定长度并采用同比径向材料收缩，半径轨迹来自附件 2 而非模型预测。',
    'Q2/Q3 从  $t=0$  统一附录 3；Q4 从  $t=0$  整组改用附录 4；h 和  $\beta$  沿用题给值。',
]
LIMITATIONS = [
    '没有内部温度/水分实测真值，数值校核不构成实测预测精度认证。',
    '密度、气固湿度映射、潜热闭合和外生收缩是模型解释边界，需团队主导确认。',
    '四位小数为输出格式；网格比较是采样差值，不能冒称连续解严格误差上界。',
    'Visual Studio 复现与用户人工代码审查分别留有独立状态。',
]
CONTRACT_TARGETS = {
    'model_results.json': 'paper_output/results/model_results.json',
    'metrics.json': 'paper_output/results/metrics.json',
    'conclusions.json': 'paper_output/results/conclusions.json',
    'table_index.json': 'paper_output/tables/table_index.json',
    'figure_index.json': 'paper_output/figure_index.json',
}

def dependencies():
    # Inventory also covers disk_dense and future local helper modules. Being
    # inventoried is not a claim that a script was actually executed.
    paths = list(SOURCE.parent.glob('*.py'))
    paths += [ROOT / 'paper_output/data_cleaned' / name for name in
               ('A_environment_observed.csv', 'A_radius_observed.csv')]
    paths += [ROOT / 'paper_output/plan/model_route.json']
```

```

paths += list((ROOT / 'problem_files/CUMCM2026Problems/A 题').rglob('*.xlsx'))
return [provenance.file_record(path) for path in sorted(set(paths))]

def font_record():
    font = Path('C:/Windows/Fonts/msyh.ttc')
    return {'path': str(font), 'bytes': font.stat().st_size, 'sha256': provenance.sha256(font)}

def assert_dependencies(start):
    if dependencies() != start['input_files']:
        raise RuntimeError('Source/input inventory changed after launch')
    if font_record() != start['font']:
        raise RuntimeError('Rendering font changed after launch')
    if provenance.runtime_record() != start['runtime']:
        raise RuntimeError('Runtime environment changed after launch')
    expected = {rec['path']: rec['sha256'] for rec in start['input_files']}
    for path, loaded in ((SOURCE, LOADED_CODE_SHA256), (provenance.SOURCE,
provenance.LOADED_CODE_SHA256)):
        if loaded != expected[path.relative_to(ROOT).as_posix()]:
            raise RuntimeError('Loaded entry/provenance module differs from launch snapshot')

def imported_model_records():
    paths = set()
    for module in list(sys.modules.values()):
        filename = getattr(module, '__file__', None)
        if filename:
            path = Path(filename).resolve()
            if path.is_relative_to(SOURCE.parent) and path.suffix == '.py':
                paths.add(path)
    return [provenance.file_record(path) for path in sorted(paths)]

def question_plot(plots, run, qid, directory):
    public = getattr(plots, 'make_question_plot', None)
    if callable(public):
        return public(run, qid, directory)
    name = '_profiles' if qid in ('Q1', 'Q2') else '_drying'
    function = getattr(plots, name, None)
    if not callable(function):
        raise RuntimeError('Publication plot API is missing: ' + name)
    return function(run, qid, directory)

def describe_question(qid, key, summary, export, figure, args, directory, start):
    sample = summary['metric_samples'][qid]
    T, C, sampletime = sample['T_K'][0], sample['C'][0], sample['time_s']
    result_text = (f'在t={sampletime/3600:.4f}h, 中心/表面温度分别为'
        f'{T[0]-273.15:.4f}/{T[1]-273.15:.4f}摄氏度, 干基含水率为'
        f'{C[0]:.4f}/{C[1]:.4f}kg/kg。')
    if qid in ('Q3', 'Q4'):
        done = summary['completion']
        result_text = (f'连续临界时刻{done["critical_event_h"]:.9f}h; 向上到四位小数并复核后'
            f'报告{done["reported_drying_time_h"]:.4f}h, 原精度最大含水率'
            f'{done["max_C_at_reported_time"]:.12f}<0.15kg/kg。')
    artifacts = [rec['path'] for rec in export['artifacts']] + [export['validation_report']
['path']]
    artifacts += [rec['path'] for rec in provenance.figure_artifact_records(figure)]
    artifacts = sorted(set(artifacts))
    source = next(rec for rec in start['input_files'] if rec['path'] ==
SOURCE.relative_to(ROOT).as_posix())
    question = {'question_id': qid, 'status': 'computed', 'source_run_id': args.version,
        'source_trajectory_id': args.version + ':' + key,
        'model_name': '圆柱有效显热与干基扩散; 守恒有限体积、Kirchhoff 面通量、解析 Jacobian BDF',
        'result_summary': result_text, 'result_files': artifacts,

```

```

        'execution_provenance': {'source_code_path': source['path'], 'source_code_sha256':
source['sha256'],
        'run_command': start['worker_command'], 'run_exit_code': None,
        'process_status': 'awaiting_parent_observed_exit', 'output_artifacts': artifacts},
        'assumptions_used': ASSUMPTIONS, 'limitations': LIMITATIONS, 'validation_summary':
summary}
    values = [
        ('mass_balance_absolute_residual', summary['diagnostics']
['max_mass_balance_abs_kg_per_kg'], 'kg/kg',
        '离散干物质基准下，平均C与累计流出之和减初值。',
        '/diagnostics/max_mass_balance_abs_kg_per_kg'),
        ('radial_intervals', summary['settings']['intervals'], '1', '实际求解空间分辨率。',
        '/settings/intervals'),
        ('center_C_at_report_time', C[0], 'kg/kg', '已记录指定时刻的中心含水率。',
f'/metric_samples/{qid}/C/0/0'),
    ]
    if qid in ('Q3', 'Q4'):
        values.append(('reported_drying_time', summary['completion']['reported_drying_time_h'],
'h',
        '向上取四位小数并用原精度最大值验证严格阈值。', '/completion/reported_drying_time_h'))
    evidence = (directory / key / 'summary.json').relative_to(ROOT).as_posix()
    metrics = [{'question_id': qid, 'metric_name': name, 'metric_role': 'numerical_result',
        'value': value, 'unit': unit, 'status': 'computed', 'evidence_path': evidence,
        'evidence_json_pointer': pointer, 'question_report_time_s': sampletime, 'interpretation':
interpretation}
        for name, value, unit, interpretation, pointer in values]
    conclusion = {'question_id': qid, 'conclusion_text': result_text, 'status': 'computed',
        'supporting_metrics': [value[0] for value in values], 'supporting_artifacts': artifacts,
        'assumptions': ASSUMPTIONS, 'limitations': LIMITATIONS}
    return question, metrics, conclusion

def worker(args, directory, start):
    assert_dependencies(start)
    import numpy as np
    core = importlib.import_module('drying_core')
    modules = {key: importlib.import_module(name) for key, name in
        [('Q1', 'q1_model'), ('Q23', 'q2_model'), ('Q4', 'q4_model')]}
    completion = importlib.import_module('q3_model').completion
    solve_only_review = args.review and not getattr(args, 'review_exports', False)
    exporter = None if solve_only_review else importlib.import_module('export_outputs')
    plots = None if solve_only_review else importlib.import_module('publication_plots')
    if core.ROOT.resolve() != ROOT:
        raise RuntimeError('Core workspace differs from production workspace')
    # The pre/post checks enclose imports; model APIs also enforce their loaded hashes.
    assert_dependencies(start)
    imported = imported_model_records()
    started, timer = provenance.utcnow(), time.perf_counter()
    summaries, exports, tables, figures, export_checks = {}, [], [], [], []
    questions, metrics, conclusions = [], [], []
    for name, n, qids in [('Q1', args.n1, ['Q1']), ('Q23', args.n23, ['Q2', 'Q3']),
        ('Q4', args.n4, ['Q4'])]:
        run = None
        try:
            print(f'SOLVE {name} N={n} UTC={provenance.utcnow()}', flush=True)
            with warnings.catch_warnings(record=True) as caught:
                warnings.simplefilter('always')
                run = modules[name].solve(n)
            if caught:
                raise RuntimeError('Solver warning: ' + '; '.join(str(item.message) for item in
caught))
            summary = core.save_run(run, directory / name)
            summary['solver_warning_count'] = len(caught)
            summary['source_run_id'] = args.version
            summary['source_trajectory_id'] = args.version + ':' + name
            if name != 'Q1':

```

```

        summary['completion'] = completion(run)
        times = np.unique(np.array([0., min(1800., run.end_s), min(10800., run.end_s),
run.end_s] +
        ([[] if name == 'Q1' else [summary['completion']['reported_time_s']]])))
        T, C = run.fields(times, material_x=np.array([0., .25, .5, .75, 1.]))
        summary['reproduction_samples'] = {'times_s': times.tolist(),
        'material_x': [0., .25, .5, .75, 1.], 'T_K': T.tolist(), 'C': C.tolist()}
        summary['metric_samples'] = {}
        for qid in qids:
            samptime = 1800. if qid == 'Q1' else 10800. if qid == 'Q2' else
summary['completion']['reported_time_s']
            T, C = run.fields([samptime], material_x=[0., 1.])
            summary['metric_samples'][qid] = {'time_s': samptime, 'material_x': [0., 1.],
            'T_K': T.tolist(), 'C': C.tolist(), 'source': 'live_Run_dense_solution'}
        provenance.write_json(directory / name / 'summary.json', summary)
        summaries[name] = summary
        print(json.dumps({'question': name, **summary['diagnostics']}, ensure_ascii=False),
flush=True)
        if solve_only_review:
            continue
        for qid in qids:
            print('EXPORT ' + qid, flush=True)
            export = exporter.export_question(run, qid, directory / 'outputs')
            check = exporter.validate_exports([export], runs={qid: run})
            if check.get('status') != 'PASS' or not check.get('fully_verified_with_live_Run'):
                raise RuntimeError('Export did not pass live Run validation: ' + qid)
            export['validation_report'] = provenance.file_record(check['exports'][0])
['report_path'])
            export['validation_status'] = 'PASS'
            export_checks.append(check)
            exports.append(export)
            for rec in export['artifacts']:
                if rec.get('role') in ('paper_table', 'paper_surface_coordinates'):
                    tables.append({'table_id': Path(rec['path']).stem, 'question_id': qid,
                    'title': Path(rec['path']).stem, 'purpose': '题目指定正文位置/时间采样',
                    **provenance.file_record(ROOT / rec['path']), 'status': 'computed',
                    'placeholder': False, 'ok': True, 'source_run_id': args.version,
                    'source_trajectory_id': args.version + ':' + name})
            print('PLOT ' + qid, flush=True)
            figure = question_plot(plots, run, qid, directory / 'figures')
            figure.update({'status': 'computed', 'placeholder': False, 'ok': True,
            'source_run_id': args.version, 'source_trajectory_id': args.version + ':' +
name})
            figures.append(figure)
            question, question_metrics, conclusion = describe_question(
                qid, name, summary, export, figure, args, directory, start)
            questions.append(question)
            metrics.extend(question_metrics)
            conclusions.append(conclusion)
            print(json.dumps({'question': qid, 'export_size': export['size'], 'validation':
check['status']},
                        ensure_ascii=False), flush=True)
        finally:
            if run is not None:
                close = getattr(run, 'close', None)
                try:
                    if callable(close):
                        close()
                finally:
                    close = None # A bound close method otherwise retains its Run.
                    run = None
                    gc.collect()
            print('RELEASED ' + name, flush=True)
        assert_dependencies(start)
        provenance.assert_records(imported)
        provenance.write_json(directory / 'numerical_summaries.json', summaries)

```

```

    if args.review:
        provenance.write_json(directory / 'review_result.json', {
            'status': 'COMPUTED_PENDING_GUI_OBSERVATION',
            'mode': 'independent_actual_resolve_and_full_exports' if args.review_exports else
'independent_actual_resolve',
            'source': provenance.file_record(SOURCE),
            'inputs': start['input_files'], 'runtime': start['runtime'], 'imported_model_modules':
imported,
            'started_at': started, 'finished_at': provenance.utcnow(), 'summaries': summaries,
            'exports': exports, 'export_checks': export_checks, 'figures': figures,
            'elapsed_s': time.perf_counter() - timer,
            'visual_studio_gui': 'Requires separate observed GUI evidence', 'human_review':
'pending'})
        print('REPRODUCTION_SOLVES_COMPLETED', flush=True)
    else:
        provenance.write_json(directory / 'metric_evidence_validation.json',
provenance.validate_metric_evidence(metrics))
        provenance.write_json(directory / 'export_validation.json', {'status': 'PASS', 'checks':
export_checks})
        meta = {'schema_version': '1.0', 'generated_by': SOURCE.relative_to(ROOT).as_posix(),
            'generated_at': provenance.utcnow(), 'source_run_id': args.version,
            'publication_state': 'version_only_awaiting_parent_observed_exit'}
        contracts = {'model_results.json': {'**meta', 'questions': questions},
            'metrics.json': {'**meta', 'items': metrics}, 'conclusions.json': {'**meta', 'items':
conclusions},
            'table_index.json': {'**meta', 'tables': tables}, 'figure_index.json': {'**meta,
'figures': figures}}
        for filename, content in contracts.items():
            provenance.write_json(directory / 'contracts' / filename, content)
        assert_dependencies(start)
        output_paths = [p for p in directory.rglob('*') if p.is_file() and p.name not in
('stdout.log', 'worker_success.json', 'process_result.json')]
        provenance.write_json(directory / 'worker_success.json', {
            'started_at': started, 'finished_at': provenance.utcnow(), 'elapsed_seconds':
time.perf_counter() - timer,
            'input_files': start['input_files'], 'runtime': start['runtime'], 'font': start['font'],
            'imported_model_modules': imported, 'output_artifacts': [provenance.file_record(p) for p
in sorted(output_paths)],
            'question_ids': ['Q1', 'Q2', 'Q3', 'Q4'], 'summaries': summaries, 'exports': exports,
            'mode': 'review' if args.review else 'production', 'human_review': 'pending',
            'visual_studio_gui': 'pending'})
        print('ALL_SOLVES_AND_REQUESTED_VALIDATIONS_COMPLETED', flush=True)

def publish(args, directory, start, success, process):
    assert_dependencies(start)
    provenance.assert_records(success['output_artifacts'])
    with provenance.publication_lock(ROOT, args.version):
        contracts = {name: json.loads((directory / 'contracts' /
name).read_text(encoding='utf-8'))
            for name in CONTRACT_TARGETS}
        for data in contracts.values():
            data['publication_state'] = 'parent_verified_actual_worker_exit_zero'
        for question in contracts['model_results.json']['questions']:
            question['execution_provenance'].update({'run_exit_code': process['returncode'],
                'process_status': 'parent_observed_exit_zero',
                'process_evidence_path': (directory /
'process_result.json').relative_to(ROOT).as_posix())
            index_path = ROOT / 'paper_output/figure_index.json'
            prior = json.loads(index_path.read_text(encoding='utf-8')) if index_path.exists() else
None
            merged, merge_note = provenance.merge_figure_index(prior, contracts['figure_index.json']
['figures'])
            contracts['figure_index.json']['figures'] = merged
            contracts['figure_index.json']['merge_record'] = merge_note
            provenance.validate_metric_evidence(contracts['metrics.json']['items'])

```

```

    published = directory / 'published_contracts'
    for name, content in contracts.items():
        provenance.write_json(published / name, content)
    records = {rec['path']: rec for rec in success['output_artifacts']}
    for path in [directory / 'worker_success.json', directory / 'stdout.log', directory /
'process_result.json']:
        rec = provenance.file_record(path)
        records[rec['path']] = rec
    for name in CONTRACT_TARGETS:
        rec = provenance.file_record(published / name)
        records[rec['path']] = rec
        # The transaction verifies the real global bytes before committing PASS.
        target = {**rec, 'path': CONTRACT_TARGETS[name]}
        records[target['path']] = target
    for figure in merged:
        for rec in provenance.figure_artifact_records(figure):
            records[rec['path']] = rec
    assert_dependencies(start)
    source = next(rec for rec in start['input_files'] if rec['path'] ==
SOURCE.relative_to(ROOT).as_posix())
    record = {'run_id': args.version, 'script': source['path'], 'script_sha256':
source['sha256'],
            'command': process['command'], 'question_ids': success['question_ids'],
            'returncode': process['returncode'], 'cwd': str(ROOT), 'process': process,
            'started_at': process['started_at'], 'finished_at': process['finished_at'],
            'worker_started_at': success['started_at'], 'worker_finished_at':
success['finished_at'],
            'elapsed_seconds': success['elapsed_seconds'], 'python': start['runtime'], 'font':
start['font'],
            'input_files': start['input_files'], 'imported_model_modules':
success['imported_model_modules'],
            'output_artifacts': [records[path] for path in sorted(records)],
            'version_contract_directory': published.relative_to(ROOT).as_posix(),
            'human_review': 'pending', 'visual_studio_gui': 'pending'}
    manifest = {'schema_version': '1.0', 'generated_by': SOURCE.relative_to(ROOT).as_posix(),
'generated_at': provenance.utcnow(), 'status': 'PASS', 'runs': [record],
'scope': 'Actual worker exit zero, source/input/output hashes and export checks; not
human model acceptance.'}
    provenance.publish_transaction(ROOT, directory,
{CONTRACT_TARGETS[name]: data for name, data in contracts.items()}, manifest)

def main():
    import argparse
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--version', required=True)
    parser.add_argument('--n1', type=int, default=3200)
    parser.add_argument('--n23', type=int, default=3200)
    parser.add_argument('--n4', type=int, default=6400)
    parser.add_argument('--blas-threads', type=int, default=1)
    parser.add_argument('--worker', action='store_true', help=argparse.SUPPRESS)
    parser.add_argument('--launch-token', help=argparse.SUPPRESS)
    parser.add_argument('--review', action='store_true')
    parser.add_argument('--review-exports', action='store_true',
        help='With --review, also reproduce all Excel/CSV readbacks and figures in the GUI
process')
    args = parser.parse_args()
    if args.review_exports and not args.review:
        raise ValueError('--review-exports requires --review')
    if args.blas_threads < 1:
        raise ValueError('BLAS thread count must be positive')
    # These are task-process settings, before the lazy NumPy/SciPy imports.
    # No global Windows environment or registry setting is modified.
    os.environ['OPENBLAS_NUM_THREADS'] = str(args.blas_threads)
    os.environ['OMP_NUM_THREADS'] = str(args.blas_threads)
    if Path.cwd().resolve() != ROOT:

```

```

        raise RuntimeError('Run with the competition workspace as cwd')
    if not args.version or any(ch not in
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_- ' for ch in args.version):
        raise ValueError('Use a simple version label')
    if min(args.n1, args.n23, args.n4) < 2:
        raise ValueError('All radial interval counts must be at least 2')
    directory = ROOT / ('paper_output/results/gui_reproduction' if args.review else
        'paper_output/results/production') / args.version
    if args.worker:
        if args.review:
            raise ValueError('--review uses a single process so Visual Studio can hit solver
breakpoints')
        start = json.loads((directory / 'launch.json').read_text(encoding='utf-8'))
        config = {'n1': args.n1, 'n23': args.n23, 'n4': args.n4, 'review': args.review,
            'review_exports': args.review_exports,
            'blas_threads': args.blas_threads}
        if not args.launch_token or args.launch_token != start['launch_token'] or config !=
start['configuration']:
            raise RuntimeError('Worker must be launched by its matching supervising parent')
        provenance.exclusive_json(directory / 'worker_started.json', {'pid': os.getpid(),
'started_at': provenance.utcnow()})
        print('WORKER_PID=' + str(os.getpid()), flush=True)
        try:
            worker(args, directory, start)
        except BaseException as error:
            provenance.write_json(directory / 'worker_failure.json', {'status': 'FAIL', 'error':
repr(error),
            'traceback': traceback.format_exc(), 'generated_at': provenance.utcnow(),
            'returncode': None, 'returncode_note': 'Supervising parent records the actual
process exit.'})
            raise
        return 0
    if args.launch_token:
        raise ValueError('--launch-token is reserved for the supervised worker')
    directory.parent.mkdir(parents=True, exist_ok=True)
    directory.mkdir(exist_ok=False) # Atomic claim; failed versions also remain immutable.
    token = uuid.uuid4().hex
    provenance.exclusive_json(directory / 'launch_claim.json', {'parent_pid': os.getpid(),
        'run_id': args.version, 'claimed_at': provenance.utcnow(), 'launch_token': token})
    process = None
    try:
        command = [sys.executable, '-B', str(SOURCE), '--version', args.version,
            '--n1', str(args.n1), '--n23', str(args.n23), '--n4', str(args.n4),
            '--blas-threads', str(args.blas_threads)]
        if args.review:
            command.append('--review')
            if args.review_exports:
                command.append('--review-exports')
        else:
            command += ['--worker', '--launch-token', token]
        start = {'run_id': args.version, 'parent_pid': os.getpid(), 'launch_token': token,
            'created_at': provenance.utcnow(), 'configuration': {'n1': args.n1, 'n23': args.n23,
                'n4': args.n4, 'review': args.review, 'review_exports': args.review_exports,
                'blas_threads': args.blas_threads}, 'input_files': dependencies(),
            'runtime': provenance.runtime_record(), 'font': font_record(),
            'worker_command': subprocess.list2cmdline(command), 'worker_argv': command,
            'dependency_scope': 'Local module inventory plus A inputs/templates and route;
inventory does not claim execution.'}
        assert_dependencies(start)
        provenance.exclusive_json(directory / 'launch.json', start)
        if args.review:
            # Direct execution is deliberate: VS F5 can hit core/wrapper
            # breakpoints without attaching to a child process. An observed
            # application exit is supplied separately by the GUI operator.
            provenance.exclusive_json(directory / 'review_started.json', {
                'pid': os.getpid(), 'started_at': provenance.utcnow(), 'mode':

```

```

'single_process_VS_review'})
    worker(args, directory, start)
    assert_dependencies(start)
    success = json.loads((directory / 'worker_success.json').read_text(encoding='utf-8'))
    provenance.assert_records(success['output_artifacts'])
    provenance.write_json(directory / 'review_evidence_validation.json', {
        'status': 'COMPUTED_PENDING_GUI_OBSERVATION', 'input_files': start['input_files'],
        'worker_success': provenance.file_record(directory / 'worker_success.json'),
        'scope': 'Direct solve function and saved hashes checked; no process exit code is
self-certified.',
        'visual_studio_gui': 'pending', 'human_review': 'pending'})
    print('REVIEW_FUNCTION_COMPLETED; GUI_OBSERVATION_AND_ACTUAL_EXIT_NOT_SELF_CERTIFIED',
flush=True)
    return 0
    process = provenance.measured_process(command, ROOT, directory / 'stdout.log')
    provenance.write_json(directory / 'process_result.json', process)
    if process['returncode'] != 0:
        raise RuntimeError('Worker failed with actual return code ' +
str(process['returncode']))
    assert_dependencies(start)
    success = json.loads((directory / 'worker_success.json').read_text(encoding='utf-8'))
    if success['input_files'] != start['input_files'] or success['runtime'] !=
start['runtime']:
        raise RuntimeError('Worker success differs from launch snapshot')
    provenance.assert_records(success['output_artifacts'])
    publish(args, directory, start, success, process)
    print('ACTUAL_WORKER_RETURN_CODE=0; VERIFIED_RUN_MANIFEST_PUBLISHED', flush=True)
    return 0
except BaseException as error:
    if process is None and (directory / 'process_result.json').exists():
        process = json.loads((directory / 'process_result.json').read_text(encoding='utf-8'))
    provenance.write_json(directory / 'parent_failure.json', {'status': 'FAIL',
        'generated_at': provenance.utcnow(), 'error': repr(error), 'traceback':
traceback.format_exc(),
        'actual_worker_returncode': None if process is None else process['returncode'],
        'process': process, 'version_is_retained': True,
        'publication_note': 'Worker does not publish global contracts; publication failure
records rollback.'})
    raise

if __name__ == '__main__':
    try:
        raise SystemExit(main())
    except Exception:
        traceback.print_exc()
        raise SystemExit(1)

```

D.2 q1_model.py

问题一正式参数入口。源文件 SHA256 :

58f0f3d8a3c833a9bf4f60702a3a02ec4cd67ae9ace8d0567cb20f84073cdfa8。

```
"""Q1 uses Appendix 2 throughout its 1800-second interval."""
from drying_core import Settings, solve_case

def solve(intervals=3200):
    return solve_case(Settings(question='Q1', intervals=intervals, face_scheme='kirchhoff',
                                rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
                                early_max_step_s=2., max_step_s=120.))
```

D.3 q2_model.py

问题二 / 三 共用 轨迹 参数 入口 。 源 文件 SHA256 :
4ceb89001d2e99270e117d124394213adab1ba678a85395e7e8cf36b0e4d1204。

```
"""Q2 uses Appendix 3 from t=0; it does not splice the Q1 trajectory."""
from drying_core import Settings, solve_case

def solve(intervals=3200):
    return solve_case(Settings(question='Q23', intervals=intervals, face_scheme='kirchhoff',
                                rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
                                early_max_step_s=2., max_step_s=120.))
```

D.4 q3_model.py

全域阈值、四位小时上取与严格可行复核。源文件 SHA256 :

43a65b6a1982649503beaec92d7aa553dd6d8f516703cc50b912a086ba9a137。

```
"""Q3 is a threshold functional of the exact same Run used by Q2."""
import math
import numpy as np

def completion(run):
    if run.event_s is None:
        raise ValueError('No full-domain drying event within the solved horizon')
    # At the continuous root the maximum equals 0.15. Report upward on the
    # required 0.0001-hour grid and verify the unrounded state there.
    count = math.ceil(run.event_s / 3600. * 10000.)
    while True:
        report_h = count/10000.
        t = report_h*3600.
        if t > run.end_s:
            raise RuntimeError('Four-decimal reporting time is outside the verified trajectory')
        values = run.state([t])[1:-1:2,0]
        if float(np.max(values)) < .15:
            break
        count += 1
    return {'critical_event_s':run.event_s,'critical_event_h':run.event_s/3600.,
            'reported_drying_time_h':report_h,'reported_time_s':t,
            'max_C_at_reported_time':float(np.max(values)),
            'slowest_material_coordinate':float(run.model.x[np.argmax(values)]),
            'conservative_post_verification_s':run.end_s,
            'rounding_convention':'upward on 0.0001 h grid, followed by actual strict threshold
check',
            'interpretation':'A conditional numerical event, not a confidence bound on physical
drying time.'}
```

D.5 q4_model.py

问题四附录4物性与收缩开关入口。源文件 SHA256 :
d331207ebdffc2eb36fa3f659cd20215511415258c89334c89640ceaf3c7d535。

```
"""Q4 switches all properties to Appendix 4 and follows observed radial shrinkage."""
from drying_core import Settings, solve_case

def solve(intervals=6400):
    return solve_case(Settings(question='Q4', intervals=intervals, face_scheme='kirchhoff',
                               shrink=True, rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
                               early_max_step_s=2., max_step_s=120.))
```

D. 6 drying_core.py

读取输入、有限体积、水热物性、通量、ODE 事件和稠密轨迹查询。源文件

SHA256: 4b1e1fdacaf6eb195f94192cf1574c96609ca1058297e081dd474f59585f2305。

```
"""2026 A: radial heat and dry-basis moisture transport on a material mesh.

Units: s, m, K, kg water / kg dry matter. See numerical_design.md for derivation.
The supplied empirical rho*cp is an effective thermal capacity. Dry-solid mass
is conserved separately on uniformly shrinking material control volumes.
No latent heat in the baseline; optional surface-latent scenario is labelled.
No clipping of solution values. Coefficients use a positive continuation only
for integrator Newton probes; all accepted states are checked independently.
"""

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
import hashlib
import io
import json
import time

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import lil_matrix
from scipy.special import expi
import analytic_jacobian
import disk_dense

ROOT = Path(__file__).resolve().parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
C0, T0, R0, LENGTH = 2.55, 301.15, 0.02, 0.25

def load_inputs(with_records=False):
    arrays, records = [], []
    for name in ['A_environment_observed.csv', 'A_radius_observed.csv']:
        path = ROOT / 'paper_output/data_cleaned' / name
        content = path.read_bytes()
        arrays.append(np.genfromtxt(io.StringIO(content.decode('utf-8-sig')),
                                   delimiter=',', names=True))
        records.append({'path':path.relative_to(ROOT).as_posix(), 'bytes':len(content),
                       'sha256':hashlib.sha256(content).hexdigest(), 'exists':True})
    return (*arrays, records) if with_records else tuple(arrays)

@dataclass(frozen=True)
class Settings:
    question: str = 'Q23'
    intervals: int = 100
    rtol: float = 1e-7
    atol_temperature: float = 1e-7
    atol_moisture: float = 1e-9
    max_step_s: float = 600.0
    early_max_step_s: float = 30.0
    horizon_h: float = 240.0
    shrink: bool = False
    boundary_extension: str = 'nominal'
    tail_temperature_C: float = 50.0
    tail_equilibrium: float = 0.05
    h: float = 25.0
    beta: float = 8e-7
```

```

equilibrium_scale: float = 1.0
surface_latent_fraction: float = 0.0
latent_J_kg: float = 2.4e6
constant_D: float | None = None
constant_thermal: bool = False
method: str = 'BDF'
face_scheme: str = 'harmonic'
jacobian_mode: str = 'analytic'
dense_storage: str = 'disk'

class RadialModel:
    def __init__(self, settings: Settings):
        self.settings = settings
        if settings.intervals < 2 or settings.jacobian_mode not in ('analytic',
'finite_difference'):
            raise ValueError('At least two intervals and a supported Jacobian mode are required')
        self.env, self.rad, self.input_records = load_inputs(with_records=True)
        self.x = np.linspace(0., 1., settings.intervals + 1)
        self.dx = 1. / settings.intervals
        faces = np.r_[0., (self.x[1:] + self.x[:-1]) / 2., 1.]
        self.w = np.diff(faces ** 2) / 2.
        self.internal_faces = faces[1:-1]
        self.n = len(self.x)
        self.rho_d0 = (760 + 90 * C0) / (1 + C0) if settings.question == 'Q4' else (
            820. / (1 + C0) if settings.question == 'Q1' else (650 + 128 * C0) / (1 + C0))
        self.evaluations = 0
        self.jac_pattern = self._sparsity()

    def radius(self, t):
        if self.settings.shrink:
            return np.interp(t, self.rad['time_s'], self.rad['radius_m'])
        return np.asarray(t) * 0. + R0

    def environment(self, t):
        s = self.settings
        tair = np.interp(t, self.env['time_s'], self.env['temperature_K'])
        ceq = np.interp(t, self.env['time_s'], self.env['air_moisture_kg_per_kg'])
        after = np.asarray(t) > self.env['time_s'][-1]
        if s.boundary_extension == 'nominal':
            tair = np.where(after, s.tail_temperature_C + 273.15, tair)
            ceq = np.where(after, s.tail_equilibrium, ceq)
        elif s.boundary_extension == 'tail_mean':
            tail = self.env['time_s'] >= 10800
            tair = np.where(after, self.env['temperature_K'][tail].mean(), tair)
            ceq = np.where(after, self.env['air_moisture_kg_per_kg'][tail].mean(), ceq)
        elif s.boundary_extension != 'last':
            raise ValueError('Unknown boundary extension')
        return tair, ceq * s.equilibrium_scale

    def properties(self, T, C):
        s = self.settings
        positive_C = np.maximum(C, 1e-12) # coefficient continuation, never state clipping
        if np.any(T <= 0):
            raise FloatingPointError('Nonpositive absolute temperature')
        wet = positive_C / (1. + positive_C)
        if s.question == 'Q1':
            rho = np.full_like(C, 820.)
            cp = np.full_like(C, 2600.)
            k = np.full_like(C, .36)
            D = 7e-9 * np.exp(-.89 / positive_C)
        elif s.question in ('Q2', 'Q3', 'Q23'):
            rho, cp, k = 650 + 128 * positive_C, 1450 + 2736 * wet, .21 + .38 * wet
            D = 2.4e-3 * np.exp(-.45 / positive_C - 3850 / T)
        elif s.question == 'Q4':
            rho, cp, k = 760 + 90 * positive_C, 1850 + 2150 * wet, .12 + .20 * wet

```

```

        D = 4.2e-4 * np.exp(-.30 / positive_C - 3850 / T)
    else:
        raise ValueError(s.question)
    if s.constant_D is not None:
        D = np.full_like(C, s.constant_D)
    if s.constant_thermal:
        rho, cp, k = np.full_like(C, 820.), np.full_like(C, 2600.), np.full_like(C, .36)
    return rho, cp, k, D

@staticmethod
def harmonic(a):
    return 2 * a[:-1] * a[1:] / np.maximum(a[:-1] + a[1:], np.finfo(float).tiny)

def _sparsity(self):
    p = lil_matrix((2 * self.n + 1, 2 * self.n + 1), dtype=int)
    for i in range(self.n):
        for j in range(max(0, i - 1), min(self.n, i + 2)):
            p[2*i:2*i+2, 2*j:2*j+2] = 1
    p[-1, 2 * (self.n - 1) + 1] = 1
    return p.tocsr()

def water_internal_flux(self, T, C, D):
    if self.settings.face_scheme == 'harmonic' or self.settings.constant_D is not None:
        return self.internal_faces * self.harmonic(D) * np.diff(C) / self.dx
    if self.settings.face_scheme != 'kirchhoff':
        raise ValueError('Unknown nonlinear face scheme')
    a, D0 = {'Q1':(.89,7e-9), 'Q23':(.45,2.4e-3), 'Q2':(.45,2.4e-3),
            'Q3':(.45,2.4e-3), 'Q4':(.30,4.2e-4)}[self.settings.question]
    cc = np.maximum(C, 1e-12)
    potential = cc * np.exp(-a/cc) + a * expi(-a/cc)
    difference = np.diff(potential)
    small = np.abs(np.diff(cc)) < 1e-7 * np.maximum((cc[:-1]+cc[1:])/2, 1e-3)
    difference[small] = (np.exp(-a/((cc[:-1][small]+cc[1:][small])/2)) * np.diff(cc)[small])
    thermal_factor = 1. if self.settings.question == 'Q1' else
np.exp(-3850/((T[:-1]+T[1:])/2))
    # Do not difference thermal_factor*potential: that would add a false Soret flux.
    return self.internal_faces * D0 * thermal_factor * difference / self.dx

def rhs(self, t, state):
    """REVIEW: actual material-control-volume balance, no extra mesh advection."""
    self.evaluations += 1
    T, C = state[:-1:2], state[1:-1:2]
    rho, cp, k, D = self.properties(T, C)
    radius = float(self.radius(t))
    tair, ceq = self.environment(t)
    heat_g = np.zeros(self.n + 1)
    water_g = np.zeros(self.n + 1)
    heat_g[1:-1] = self.internal_faces * self.harmonic(k) * np.diff(T) / self.dx
    water_g[1:-1] = self.water_internal_flux(T, C, D)
    water_g[-1] = -self.settings.beta * radius * (C[-1] - ceq)
    heat_g[-1] = -self.settings.h * radius * (T[-1] - tair)
    if self.settings.surface_latent_fraction:
        # Scenario: all selected outgoing water vaporizes at the surface.
        rho_d = self.rho_d0 * (R0 / radius) ** 2
        j_evap = rho_d * self.settings.beta * (C[-1] - ceq)
        heat_g[-1] -= (radius * self.settings.surface_latent_fraction *
            self.settings.latent_J_kg * j_evap)
    derivative = np.empty_like(state)
    derivative[:-1:2] = np.diff(heat_g) / (radius ** 2 * self.w * rho * cp)
    derivative[1:-1:2] = np.diff(water_g) / (radius ** 2 * self.w)
    derivative[-1] = 2 * self.settings.beta / radius * (C[-1] - ceq)
    return derivative

def initial(self):
    state = np.empty(2 * self.n + 1)
    state[:-1:2], state[1:-1:2], state[-1] = T0, C0, 0.

```

```

        return state

class Run:
    def __init__(self, model, pieces, elapsed, event_s, cache=None):
        self.model, self.pieces = model, pieces
        self.cache = cache
        self.elapsed_s, self.event_s = elapsed, event_s
        self.end_s = float(pieces[-1].t[-1])

    def close(self):
        """Release this Run after its exports/checks; further queries are invalid."""
        self.pieces.clear()
        if self.cache is not None:
            self.cache.close()

    def state(self, times):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        if np.min(tt) < -1e-10 or np.max(tt) > self.end_s + 1e-7:
            raise ValueError('Requested time outside solved interval')
        out = np.empty((2 * self.model.n + 1, len(tt)))
        remaining = np.ones(len(tt), dtype=bool)
        for result in self.pieces:
            select = remaining & (tt >= result.t[0]-1e-7) & (tt <= result.t[-1]+1e-7)
            if np.any(select):
                out[:, select] = result.sol(tt[select])
                remaining[select] = False
        if np.any(remaining):
            raise RuntimeError('Missing dense solution segment')
        return out

    def fields(self, times, radii_m=None, material_x=None):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        state = self.state(tt)
        Ts, Cs = state[:-1:2].T, state[1:-1:2].T
        if material_x is not None:
            points = np.asarray(material_x, dtype=float)
            if np.any(~np.isfinite(points)) or np.any((points < 0.) | (points > 1.)):
                raise ValueError('Material coordinates must be finite and within [0,1]')
            return np.array([np.interp(points, self.model.x, row) for row in Ts]), np.array([
                np.interp(points, self.model.x, row) for row in Cs])
        if radii_m is None:
            return Ts, Cs
        radial = np.asarray(radii_m)
        Tout, Cout = [], []
        for i, t in enumerate(tt):
            xx = radial / self.model.radius(t)
            Tout.append(np.interp(xx, self.model.x, Ts[i], left=np.nan, right=np.nan))
            Cout.append(np.interp(xx, self.model.x, Cs[i], left=np.nan, right=np.nan))
        return np.asarray(Tout), np.asarray(Cout)

    def diagnostics(self):
        # Reduce in bounded blocks: a fine full-domain run may contain tens of
        # millions of accepted state values. Diagnostics must not duplicate all
        # of them and four property arrays at the same time.
        minimum_C = minimum_T = minimum_D = minimum_property = np.inf
        maximum_C = maximum_T = maximum_D = maximum_radial_increase = -np.inf
        maximum_mass_residual = 0.
        for piece in self.pieces:
            for first in range(0, len(piece.t), 256):
                raw = piece.y[:, first:first+256]
                T, C = raw[:-1:2], raw[1:-1:2]
                residual = 2*self.model.w*C + raw[-1] - C0
                rho, cp, k, D = self.model.properties(T.ravel(), C.ravel())
                minimum_C, maximum_C = min(minimum_C, C.min()), max(maximum_C, C.max())
                minimum_T, maximum_T = min(minimum_T, T.min()), max(maximum_T, T.max())

```

```

        minimum_D = min(minimum_D, D.min())
        maximum_D = max(maximum_D, D.max())
        minimum_property = min(minimum_property, rho.min(), cp.min(), k.min(), D.min())
        maximum_radial_increase = max(maximum_radial_increase, np.max(np.diff(C, axis=0)))
        maximum_mass_residual = max(maximum_mass_residual, np.max(np.abs(residual)))
    final = self.state([self.end_s])[:, 0]
    finalC = final[1:-1:2]
    return {
        'event_s': self.event_s, 'event_h': None if self.event_s is None else
self.event_s/3600,
        'end_s': self.end_s, 'elapsed_s': self.elapsed_s,
        'end_time_convention': 'ceil(critical_event_s)+1: conservative post-crossing
verification second; not claimed earliest integer second',
        'max_mass_balance_abs_kg_per_kg': float(maximum_mass_residual),
        'min_C': float(minimum_C), 'max_C': float(maximum_C),
        'min_T_K': float(minimum_T), 'max_T_K': float(maximum_T),
        'min_D': float(minimum_D), 'max_D': float(maximum_D),
        'positive_properties': bool(minimum_property > 0),
        'max_radial_C_increase': float(maximum_radial_increase),
        'final_max_C': float(finalC.max()), 'final_surface_C': float(finalC[-1]),
        'strictly_dry_at_end': bool(finalC.max() < .15),
        'radius_end_m': float(self.model.radius(self.end_s)),
        'radius_extrapolation_used': bool(self.model.settings.shrink and self.end_s > 259200),
        'rhs_evaluations': self.model.evaluations,
        'accepted_time_points': sum(len(p.t) for p in self.pieces),
        'nfev': sum(p.nfev for p in self.pieces),
        'njev': sum(p.njev for p in self.pieces), 'nlu': sum(p.nlu for p in self.pieces),
        'solver_success': all(p.success for p in self.pieces),
        'dense_storage': self.model.settings.dense_storage,
        'dense_coefficient_bytes': 0 if self.cache is None else self.cache.bytes_written,
        'dense_polynomial_count': 0 if self.cache is None else self.cache.polynomial_count,
    }
}

def solve_case(settings: Settings) -> Run:
    started = time.perf_counter()
    if settings.dense_storage not in ('memory', 'disk'):
        raise ValueError('Dense storage must be memory or disk')
    if settings.dense_storage == 'disk' and settings.method != 'BDF':
        raise ValueError('Exact disk dense storage currently supports BDF only')
    cache = disk_dense.DenseCache(ROOT) if settings.dense_storage == 'disk' else None
    try:
        return _solve_case_impl(settings, cache, started)
    except BaseException as error:
        if cache is not None:
            try:
                cache.close()
            except BaseException as cleanup_error:
                error.add_note('Private cache cleanup also failed: '+repr(cleanup_error))
        raise

def _solve_case_impl(settings, cache, started):
    model = RadialModel(settings)
    method = disk_dense.DiskBDF if cache is not None else settings.method
    jacobian_options = ({'jac': lambda t, y: analytic_jacobian.jacobian(model, t, y)}
        if settings.jacobian_mode == 'analytic' else {'jac_sparsity': model.jac_pattern})
    if cache is not None:
        jacobian_options['dense_cache'] = cache
    def dry_event(t, y):
        return float(np.max(y[1:-1:2]) - .15)
    dry_event.terminal, dry_event.direction = True, -1
    atol = np.empty(2 * model.n + 1)
    atol[1:-1:2], atol[1:-1:2], atol[-1] = settings.atol_temperature, settings.atol_moisture,
settings.atol_moisture
    horizon = 1800. if settings.question == 'Q1' else settings.horizon_h * 3600.

```

```

pieces, initial, event_s = [], model.initial(), None
# A separate segment at 4 h makes the modelling extension explicit.
endpoints = [0., min(14400., horizon)]
if horizon > 14400.:
    endpoints.append(horizon)
for left, right in zip(endpoints[:-1], endpoints[1:]):
    piece = solve_ivp(model.rhs, (left, right), initial, method=method,
        rtol=settings.rtol, atol=atol, **jacobian_options,
        max_step=settings.early_max_step_s if left < 14400. else settings.max_step_s,
        events=None if settings.question == 'Q1' else dry_event, dense_output=True)
    if cache is not None:
        disk_dense.align_bdf_segments(piece)
    pieces.append(piece)
    if not piece.success:
        raise RuntimeError(piece.message)
    initial = piece.y[:, -1].copy()
    if cache is not None:
        piece.y = cache.store_accepted(piece.y)
    if piece.t_events is not None and len(piece.t_events[0]):
        event_s = float(piece.t_events[0][0])
        # Continue to a genuine post-crossing integer second, not an extrapolation.
        end = float(np.ceil(event_s) + 1)
        tail = solve_ivp(model.rhs, (event_s, end), initial, method=method,
            rtol=settings.rtol, atol=atol, **jacobian_options,
            max_step=1., dense_output=True)
        if cache is not None:
            disk_dense.align_bdf_segments(tail)
        if not tail.success:
            raise RuntimeError(tail.message)
        if cache is not None:
            tail.y = cache.store_accepted(tail.y)
        pieces.append(tail)
        break
run = Run(model, pieces, time.perf_counter() - started, event_s, cache)
diagnostic = run.diagnostics()
if diagnostic['min_C'] < -1e-8 or not diagnostic['positive_properties']:
    raise FloatingPointError('Physical range/positive property check failed')
if diagnostic['max_mass_balance_abs_kg_per_kg'] > 1e-6:
    raise FloatingPointError('Dry-basis mass balance failed')
return run

def file_record(path):
    p = Path(path)
    return {'path': p.relative_to(ROOT).as_posix(), 'bytes': p.stat().st_size,
        'sha256': hashlib.sha256(p.read_bytes()).hexdigest(), 'exists': True}

def save_run(run: Run, directory: Path):
    code_record = file_record(Path(__file__))
    if code_record['sha256'] != LOADED_CODE_SHA256:
        raise RuntimeError('Solver file changed after import; restart to obtain valid
provenance')
    jacobian_record = file_record(Path(analytic_jacobian.__file__))
    if jacobian_record['sha256'] != analytic_jacobian.LOADED_CODE_SHA256:
        raise RuntimeError('Jacobian file changed after import; restart to obtain valid
provenance')
    storage_record = file_record(Path(disk_dense.__file__))
    if storage_record['sha256'] != disk_dense.LOADED_CODE_SHA256:
        raise RuntimeError('Dense storage file changed after import; restart for valid
provenance')
    for record in run.model.input_records:
        if file_record(ROOT/record['path'])['sha256'] != record['sha256']:
            raise RuntimeError('Input changed after being loaded; retain failure and rerun')
    directory.mkdir(parents=True, exist_ok=True)
    times = np.unique(np.r_[np.arange(0., run.end_s, 60.),

```

```

        [t for t in [100., 300., 600., 900., 1200., 1500., 1800., 3600., 5400., 7200.,
9000., 10800.] if t <= run.end_s],
        run.end_s, [] if run.event_s is None else [run.event_s]])
x = np.linspace(0., 1., 21)
temperature, moisture, means, losses = [], [], [], []
for first in range(0, len(times), 128):
    block_times = times[first:first+128]
    Tb, Cb = run.fields(block_times, material_x=x)
    raw = run.state(block_times)
    temperature.append(Tb); moisture.append(Cb)
    means.append(2*run.model.w@raw[1:-1:2]); losses.append(raw[-1].copy())
T, C = np.vstack(temperature), np.vstack(moisture)
np.savez_compressed(directory/'sampled_solution.npz', times_s=times, material_x=x,
    T_K=T, C=C, radius_m=run.model.radius(times), mean_C=np.concatenate(means),
    cumulative_loss=np.concatenate(losses))
summary = {'settings': asdict(run.model.settings), 'diagnostics': run.diagnostics(),
    'code': code_record,
    'jacobian_code': jacobian_record,
    'dense_storage_code': storage_record,
    'inputs': run.model.input_records,
    'human_review_status': 'pending', 'gui_reproduced': False}
(directory/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding='utf-8')
return summary

```

D.7 analytic_jacobian.py

全部状态交叉导数与解析稀疏 Jacobian。源文件 SHA256：
d7dfda75b335e82eb27d39b8e2f12edf5a56254269ff4f273e7a828ed75a8665。

```
"""Analytic sparse Jacobian for drying_core.RadialModel.rhs.

The ordering is T0,C0,...,TN,CN,A. A is a passive cumulative loss variable;
its entire column is exactly zero and must not use adaptive numdiff factors.
This file does not modify the core or select a production configuration.

Self-check from the contest root:
C:\Python314\python.exe -B paper_output/code/modeling/analytic_jacobian.py --self-test
"""
from __future__ import annotations

import argparse
from dataclasses import asdict
from datetime import datetime, timezone
import hashlib
import json
from pathlib import Path
import sys
import time
import warnings

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import coo_matrix
from scipy.special import expi

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

def _harmonic_partials(left, right):
    """Positive-coefficient partials; respect the core's tiny denominator guard."""
    total = left + right
    tiny = np.finfo(float).tiny
    denominator = np.maximum(total, tiny)
    ordinary = total > tiny
    dl = np.where(ordinary, 2 * (right / denominator) ** 2, 2 * right / denominator)
    dr = np.where(ordinary, 2 * (left / denominator) ** 2, 2 * left / denominator)
    return dl, dr

def jacobian(model, t, y):
    """Return d(rhs)/d(y) as CSC without calling rhs or numerical differentiation.

    Each internal face contributes opposite flux derivatives to its two cells.
    For heat, differentiating 1/(rho*cp) adds -Tdot*cap_C/cap locally.
    Kirchhoff's temperature factor is frozen at the symmetric face temperature;
    the primitive's close-concentration branch is differentiated exactly as coded.
    """
    state = np.asarray(y, dtype=float)
    n = model.n
    if state.ndim != 1 or state.size != 2*n + 1:
        raise ValueError("Jacobian requires interleaved 1D state of length 2*n+1")
    s = model.settings
    T, C = state[:-1:2], state[1:-1:2]
    cc = np.maximum(C, 1e-12)
    active = (C > 1e-12).astype(float)
    rho, cp, k, D = model.properties(T, C)
```

```

if s.question == 'Q1':
    a, d0, temp_constant = .89, 7e-9, 0.
    rho_c, cp_c, k_c = np.zeros(n), np.zeros(n), np.zeros(n)
elif s.question in ('Q2', 'Q3', 'Q23'):
    a, d0, temp_constant = .45, 2.4e-3, 3850.
    rho_c = np.full(n, 128.) * active
    cp_c = 2736. / (1. + cc)**2 * active
    k_c = .38 / (1. + cc)**2 * active
elif s.question == 'Q4':
    a, d0, temp_constant = .30, 4.2e-4, 3850.
    rho_c = np.full(n, 90.) * active
    cp_c = 2150. / (1. + cc)**2 * active
    k_c = .20 / (1. + cc)**2 * active
else:
    raise ValueError(s.question)
if s.constant_thermal:
    rho_c, cp_c, k_c = np.zeros(n), np.zeros(n), np.zeros(n)
capacity = rho * cp
capacity_c = rho_c * cp + rho * cp_c
if s.constant_D is None:
    d_c = D * (a / cc**2) * active
    d_t = D * temp_constant / T**2
else:
    d_c, d_t = np.zeros(n), np.zeros(n)

radius = float(model.radius(t))
if radius <= 0:
    raise ValueError("Nonpositive radius")
tair, ceq = model.environment(t)
geom = model.internal_faces / model.dx
delta_t, delta_c = np.diff(T), np.diff(C)

# Derivative columns for each face are (T_left,C_left,T_right,C_right).
k_harm = model.harmonic(k)
kh_l, kh_r = _harmonic_partials(k[:-1], k[1:])
heat_deriv = np.column_stack((
    -geom * k_harm,
    geom * kh_l * k_c[:-1] * delta_t,
    geom * k_harm,
    geom * kh_r * k_c[1:] * delta_t,
))
heat_g = np.zeros(n + 1)
heat_g[1:-1] = geom * k_harm * delta_t
heat_g[-1] = -s.h * radius * (T[-1] - tair)

if s.face_scheme == 'harmonic' or s.constant_D is not None:
    dh = model.harmonic(D)
    dh_l, dh_r = _harmonic_partials(D[:-1], D[1:])
    water_deriv = np.column_stack((
        geom * dh_l * d_t[:-1] * delta_c,
        geom * (dh_l * d_c[:-1] * delta_c - dh),
        geom * dh_r * d_t[1:] * delta_c,
        geom * (dh_r * d_c[1:] * delta_c + dh),
    ))
elif s.face_scheme == 'kirchhoff':
    primitive = cc * np.exp(-a / cc) + a * expi(-a / cc)
    primitive_delta = np.diff(primitive)
    mean_c = (cc[:-1] + cc[1:]) / 2.
    mean_t = (T[:-1] + T[1:]) / 2.
    delta_cc = np.diff(cc)
    small = np.abs(delta_cc) < 1e-7 * np.maximum(mean_c, 1e-3)
    exp_left, exp_right = np.exp(-a/cc[:-1]), np.exp(-a/cc[1:])
    primitive_l = -exp_left * active[:-1]
    primitive_r = exp_right * active[1:]
    if np.any(small):
        f_mid = np.exp(-a / mean_c[small])

```

```

        f_prime_mid = f_mid * a / mean_c[small]**2
        primitive_delta[small] = f_mid * delta_cc[small]
        primitive_l[small] = (0.5*f_prime_mid*delta_cc[small]-f_mid)*active[:-1][small]
        primitive_r[small] = (0.5*f_prime_mid*delta_cc[small]+f_mid)*active[1:][small]
        temperature_factor = np.exp(-temp_constant / mean_t)
        coefficient = geom * d0 * temperature_factor
        water_g = coefficient * primitive_delta
        water_t = water_g * temp_constant / (2 * mean_t**2)
        water_deriv = np.column_stack((water_t, coefficient*primitive_l,
                                       water_t, coefficient*primitive_r))
    else:
        raise ValueError('Unknown nonlinear face scheme')

    heat_scale = 1. / (radius**2 * model.w * capacity)
    water_scale = 1. / (radius**2 * model.w)
    surface_heat_c = 0.
    if s.surface_latent_fraction:
        rho_d = model.rho_d0 * (.02 / radius)**2
        surface_heat_c = -radius*s.surface_latent_fraction*s.latent_J_kg*rho_d*s.beta
        heat_g[-1] += surface_heat_c * (C[-1] - ceq)
    temp_derivative = np.diff(heat_g) * heat_scale

    face_index = np.arange(n-1)
    face_columns = np.column_stack((2*face_index, 2*face_index+1,
                                    2*face_index+2, 2*face_index+3)).ravel()
    rows, columns, values = [], [], []

    def add_face(row_index, derivatives, factor):
        rows.append(np.repeat(row_index, 4))
        columns.append(face_columns)
        values.append((derivatives * factor[:, None]).ravel())

    add_face(2*face_index, heat_deriv, heat_scale[:-1])
    add_face(2*face_index+2, heat_deriv, -heat_scale[1:])
    add_face(2*face_index+1, water_deriv, water_scale[:-1])
    add_face(2*face_index+3, water_deriv, -water_scale[1:])
    cell_index = np.arange(n)
    rows.append(2*cell_index)
    columns.append(2*cell_index+1)
    values.append(-temp_derivative * capacity_c / capacity)
    rows.append(np.array([2*n-2, 2*n-2, 2*n-1, 2*n]))
    columns.append(np.array([2*n-2, 2*n-1, 2*n-1, 2*n-1]))
    values.append(np.array([-s.h*radius*heat_scale[-1],
                            surface_heat_c*heat_scale[-1],
                            -s.beta*radius*water_scale[-1], 2*s.beta/radius]))
    matrix = coo_matrix((np.concatenate(values),
                        (np.concatenate(rows), np.concatenate(columns))),
                        shape=(2*n+1, 2*n+1)).tocsc()
    matrix.sum_duplicates()
    matrix.eliminate_zeros()
    return matrix

def _self_test(output_directory):
    """Independent RHS perturbation checks, a conservation derivative, and tiny BDF runs."""
    import scipy
    from drying_core import ROOT, Settings, RadialModel, LOADED_CODE_SHA256 as
CORE_LOADED_CODE_SHA256

    started = time.perf_counter()
    rng = np.random.default_rng(20260910)
    relative_tolerance, absolute_tolerance = 5e-6, 5e-10
    cases = []
    for question in ['Q1', 'Q23', 'Q4']:
        for scheme in ['harmonic', 'kirchhoff']:
            for latent in [0., 1.]:

```

```

        cases.append(Settings(question=question, intervals=8, face_scheme=scheme,
                               shrink=(question=='Q4'), surface_latent_fraction=latent))
for question in ['Q1', 'Q23', 'Q4']:
    for constant_d, constant_thermal in [(2e-9, False), (None, True), (2e-9, True)]:
        cases.append(Settings(question=question, intervals=8, face_scheme='kirchhoff',
                               shrink=(question=='Q4'), constant_D=constant_d,
                               constant_thermal=constant_thermal, surface_latent_fraction=1.))
records, failures = [], []
for settings in cases:
    model = RadialModel(settings)
    x = model.x
    states = {
        'initial': (0., model.initial()),
        'nonuniform': (18000., model.initial()),
        'late_dry': (150000., model.initial()),
        'near_uniform_small_branch': (14401., model.initial()),
    }
    states['nonuniform'][1][:-1:2] = 303. + 17.*x**2
    states['nonuniform'][1][1:-1:2] = 2.4 - 2.0*x**2
    states['late_dry'][1][:-1:2] = 321. + 2.0*x**2
    states['late_dry'][1][1:-1:2] = .175 - .115*x**2
    states['near_uniform_small_branch'][1][:-1:2] = 303. + 17.*x**2
    states['near_uniform_small_branch'][1][1:-1:2] = .15 + 1e-10*x
    for state_name, (t, y) in states.items():
        with warnings.catch_warnings(record=True) as captured:
            warnings.simplefilter('always')
            matrix = jacobian(model, t, y)
            vectors = []
            for _ in range(4):
                direction = rng.normal(size=y.size)
                direction[:-1:2] *= 1.0
                direction[1:-1:2] *= .02
                direction[-1] = .3
                vectors.append(direction)
            directional = []
            for direction in vectors:
                predicted = matrix @ direction
                steps = []
                for step in [1e-4, 3e-5, 1e-5]:
                    finite_difference = (model.rhs(t, y+step*direction)-
                                         model.rhs(t, y-step*direction))/(2*step)
                    absolute_error = float(np.max(np.abs(predicted-finite_difference)))
                    scale = max(float(np.max(np.abs(predicted))),
                                float(np.max(np.abs(finite_difference))), 1e-30)
                    scaled = float(np.max(np.abs(predicted-finite_difference)/
                                             (absolute_tolerance + relative_tolerance*np.maximum(
                                                 np.abs(predicted), np.abs(finite_difference))))))
                    steps.append({'step':step, 'max_abs_error':absolute_error,
                                'relative_inf_error':absolute_error/scale,
                                'max_component_tolerance_ratio':scaled})
                best = min(steps, key=lambda item:item['max_component_tolerance_ratio'])
                directional.append({'all_step_errors':steps, 'best':best})
            passive = np.zeros(y.size); passive[-1] = 1.
            passive_analytic = float(np.max(np.abs(matrix @ passive)))
            passive_fd = float(np.max(np.abs(model.rhs(t, y+passive)-model.rhs(t,y-passive))))
            mass_weights = np.zeros(y.size)
            mass_weights[1:-1:2] = 2*model.w
            mass_weights[-1] = 1.
            conservation = float(np.max(np.abs(np.asarray(mass_weights @ matrix))))
            warning_messages = [str(w.message) for w in captured]
            passed = (all(step['max_component_tolerance_ratio'] <= 1
                           for d in directional for step in d['all_step_errors'])
                      and passive_analytic == 0 and passive_fd == 0 and conservation < 1e-11
                      and np.isfinite(matrix.data).all() and not warning_messages)
            record = {'settings':asdict(settings), 'state':state_name, 'time_s':t,
                     'status':'PASS' if passed else 'FAIL', 'matrix_shape':matrix.shape,

```

```

        'matrix_nnz':matrix.nnz, 'directional_checks':directional,
        'passive_column_analytic_abs':passive_analytic,
        'passive_column_finite_difference_abs':passive_fd,
        'mass_balance_derivative_abs':conservation, 'warnings':warning_messages}
records.append(record)
if not passed:

failures.append(f"{settings.question}/{settings.face_scheme}/latent={settings.surface_latent_frac
tion}/{state_name}/constant_D={settings.constant_D}/
constant_thermal={settings.constant_thermal}")
smoke_records = []
for scheme in ['harmonic', 'kirchhoff']:
    for question in ['Q23', 'Q4']:
        settings = Settings(question=question, intervals=8, face_scheme=scheme,
                             shrink=(question=='Q4'), surface_latent_fraction=1.)
        model = RadialModel(settings)
        with warnings.catch_warnings(record=True) as captured:
            warnings.simplefilter('always')
            result = solve_ivp(model.rhs, (0., 2.), model.initial(), method='BDF',
                               jac=lambda t,y:jacobian(model,t,y), rtol=1e-10,
                               atol=1e-12, max_step=.2)
        msgs = [str(w.message) for w in captured]
        passed = bool(result.success and np.isfinite(result.y).all() and not msgs)
        smoke_records.append({'question':question, 'face_scheme':scheme, 'interval_s':[0,2],
                              'status':'PASS' if passed else 'FAIL', 'warnings':msgs,
                              'nfev':result.nfev, 'njev':result.njev, 'nlu':result.nlu,
                              'purpose':'Short Jacobian/BDF wiring check, not production
accuracy'})
        if not passed:
            failures.append(f"BDF_smoke/{question}/{scheme}")
            current_source_hash = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
            current_core_hash =
            hashlib.sha256((ROOT/'paper_output/code/modeling/drying_core.py').read_bytes()).hexdigest()
            if current_source_hash != LOADED_CODE_SHA256 or current_core_hash != CORE_LOADED_CODE_SHA256:
                failures.append('Source file changed after module load during validation')
            report = {'schema_version':'1.0', 'created_utc':datetime.now(timezone.utc).isoformat(),
                      'status':'PASS' if not failures else 'FAIL', 'failures':failures,
                      'source_sha256':LOADED_CODE_SHA256,
                      'core_sha256':CORE_LOADED_CODE_SHA256,
                      'source_hash_policy':'SHA-256 frozen when each module is imported; files checked
unchanged before report save.',
                      'runtime':
{'python':sys.version,'executable':sys.executable,'numpy':np.__version__,'scipy':scipy.__version__
_},
                      'seed':20260910,'relative_component_tolerance':relative_tolerance,
                      'absolute_component_tolerance':absolute_tolerance,
                      'finite_difference_policy':'Central directional differences, three decreasing steps;
every step must satisfy the mixed absolute/relative component tolerance. Best comparison is
supplementary only.',
                      'case_state_count':len(records), 'direction_count':4*len(records),
                      'checks':records,'short_BDF_checks':smoke_records,
                      'elapsed_s':time.perf_counter()-started,
                      'limitations':['No full production run or grid convergence performed here.',
                                    'Tiny denominator/underflow extensions at nonphysical Newton probes
are not calibrated physical data.',
                                    'Only this module and the short tests use analytic Jacobian until main
agent hooks core.',
                                    'Visual Studio reproduction and human review remain pending.']}
            output_directory.mkdir(parents=True, exist_ok=True)
            (output_directory/'analytic_jacobian_selftest.json').write_text(
                json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
            brief = {key:report[key] for key in
['status','failures','case_state_count','direction_count','elapsed_s']}
            brief['max_best_component_tolerance_ratio'] = max(
                d['best']['max_component_tolerance_ratio'] for r in records for d in
r['directional_checks'])

```

```

    brief['max_all_steps_component_tolerance_ratio'] = max(
        step['max_component_tolerance_ratio'] for r in records
        for d in r['directional_checks'] for step in d['all_step_errors'])
    brief['max_mass_balance_derivative_abs'] = max(r['mass_balance_derivative_abs'] for r in
records)
    brief['warning_count'] = sum(len(r['warnings']) for r in records+smoke_records)
    print(json.dumps(brief,ensure_ascii=False,indent=2))
    return 0 if not failures else 1

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--self-test', action='store_true')
    parser.add_argument('--output-dir', type=Path,
        default=Path(__file__).resolve().parents[2]/'results/jacobian_validation')
    arguments = parser.parse_args()
    if not arguments.self_test:
        parser.error('Use --self-test, or import jacobian(model,t,y) from this module.')
    raise SystemExit(_self_test(arguments.output_dir))

```

D.8 disk_dense.py

BDF 原精度多项式磁盘存储、分段边界与生命周期。源文件 SHA256:

2a8c0cac75ee0b8a1cd193277f73cde77533683a1b9c3db803b6db0f16deab60。

```
"""Exact BDF dense polynomials backed by a private, rebuildable disk cache.

This changes storage only: each accepted BDF polynomial is written as float64
bytes, then evaluated with SciPy's original BdfDenseOutput implementation.
No additional time/space interpolation and no solver restart are introduced.
"""

from __future__ import annotations
import hashlib
from pathlib import Path
import tempfile
import numpy as np
from scipy.integrate import OdeSolution
from scipy.integrate._ivp.bdf import BDF, BdfDenseOutput
from scipy.integrate._ivp.base import DenseOutput

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

class DenseCache:
    def __init__(self, root):
        self.parent = (Path(root)/'tmp/cache/solver_runs').resolve()
        if not self.parent.is_relative_to(Path(root).resolve()):
            raise ValueError('Private solver cache must remain in this project')
        self.parent.mkdir(parents=True, exist_ok=True)
        self.directory = Path(tempfile.mkdtemp(prefix='bdf_', dir=self.parent)).resolve()
        try:
            self.handle = (self.directory/'polynomials.bin').open('w+b', buffering=0)
        except BaseException as error:
            try:
                self.directory.rmdir()
            except OSError as cleanup_error:
                error.add_note('Private cache directory cleanup failed: '+repr(cleanup_error))
            raise
        self.bytes_written = 0
        self.polynomial_count = 0
        self.accepted_arrays = []
        self.closed = False

    def append(self, values):
        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        array = np.ascontiguousarray(values, dtype=np.float64)
        offset = self.bytes_written
        self.handle.seek(offset)
        array.tofile(self.handle)
        self.bytes_written += array.nbytes
        self.polynomial_count += 1
        return offset, array.shape

    def read(self, offset, shape):
        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        count = int(np.prod(shape))
        self.handle.seek(offset)
        values = np.fromfile(self.handle, dtype=np.float64, count=count)
        if values.size != count:
            raise IOError('Incomplete BDF coefficient cache')
        return values.reshape(shape)
```

```

def store_accepted(self, values):
    path = self.directory/f'accepted_{len(self.accepted_arrays):03d}.npy'
    mapped = np.lib.format.open_mmap(path, mode='w+', dtype=values.dtype,
shape=values.shape)
    # Register before writing so failures can close this Windows mapping.
    self.accepted_arrays.append(mapped)
    mapped[:] = values
    mapped.flush()
    return mapped

def close(self):
    if self.closed:
        return
    self.handle.close()
    for array in self.accepted_arrays:
        array._mmap.close()
    self.accepted_arrays.clear()
    # Delete only the verified private cache created by this object.
    if self.directory.parent != self.parent or not self.directory.name.startswith('bdf_'):
        raise RuntimeError('Unexpected private cache path; cleanup refused')
    for path in self.directory.iterdir():
        if not path.is_file() or path.is_symlink():
            raise RuntimeError('Unexpected cache entry; cleanup refused')
        path.unlink()
    self.directory.rmdir()
    self.closed = True

class FileBdfDenseOutput(DenseOutput):
    def __init__(self, original, cache):
        super().__init__(original.t_old, original.t)
        self.order = original.order
        self.t_shift = original.t_shift.copy()
        self.denom = original.denom.copy()
        self.cache = cache
        self.offset, self.shape = cache.append(original.D)

    def _call_impl(self, t):
        # Reuse the installed SciPy evaluator with the exact recorded D bytes.
        dense = object.__new__(BdfDenseOutput)
        dense.D = self.cache.read(self.offset, self.shape)
        dense.t_shift, dense.denom = self.t_shift, self.denom
        return dense._call_impl(t)

class DiskBDF(BDF):
    def __init__(self, *args, dense_cache, **kwargs):
        self.dense_cache = dense_cache
        super().__init__(*args, **kwargs)

    def _dense_output_impl(self):
        return FileBdfDenseOutput(super()._dense_output_impl(), self.dense_cache)

def align_bdf_segments(result):
    """Restore the original BDF convention at accepted time breakpoints.

    SciPy solve_ivp tests the exact method class for alt_segment. A subclass
    otherwise selects the opposite polynomial at a shared knot. Reconstructing
    OdeSolution changes only this selection, never coefficients or integration.
    """
    if result.sol is not None:
        result.sol = OdeSolution(result.sol.ts, result.sol.interpolants, alt_segment=True)

```

D.9 export_outputs.py

模板识别、完整 Excel/未舍入 CSV 导出、回读和活轨迹采样核验。源文件 SHA256: 7111606498554e18e2e007a58e0d5161b5fd209cde98b949a245981eb403c21b。

```
"""Stream contest A output directly from a live Run, then stream-read it.

The original contest templates have precedence over workbook design defaults.
The requested openpyxl write_only method bounds authoring memory. No 60-second
NPZ is used to fabricate per-second output. Unrounded values on the requested
physical output grid are archived separately as a compressed CSV; all N solver
nodes are deliberately not duplicated in this archive.

Usage by the final runner:
    record = export_question(run, 'Q2', output_directory)
    validation = validate_exports([record], runs={'Q2': run})

The archive is an internal reproducibility artifact, not automatically part of
the size-limited submission package. Nothing is silently truncated or deleted.
"""
from __future__ import annotations

import argparse
import csv
from dataclasses import asdict
from datetime import datetime, timezone
import gzip
import hashlib
import inspect
import json
import math
from pathlib import Path
import re
import sys
import tempfile
import time

import numpy as np
from openpyxl import Workbook, load_workbook
from openpyxl.cell import WriteOnlyCell
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.utils import get_column_letter

ROOT = Path(__file__).resolve().parents[3]
SOURCE = Path(__file__).resolve()
LOADED_EXPORTER_SHA256 = hashlib.sha256(SOURCE.read_bytes()).hexdigest()
TEMPLATE_DIR = ROOT / 'problem_files/CUMCM2026Problems/A 题/附件/附件 3'
RADII_M = np.arange(21, dtype=float)*0.001
RADII_CM = [i/10 for i in range(21)]
NUMBER_FORMAT = '0.0000'
SIZE_LIMIT_BYTES = 20_000_000 # report also MiB; do not reinterpret an ambiguous M upward

def _sha(path):
    h=hashlib.sha256()
    with Path(path).open('rb') as fh:
        for block in iter(lambda:fh.read(1024*1024),b''):
            h.update(block)
    return h.hexdigest()

def _record(path, role=None):
    path=Path(path).resolve()
```

```

result={'path':path.relative_to(ROOT).as_posix(),'absolute_path':str(path),
        'bytes':path.stat().st_size,'sha256':_sha(path),'exists':True}
if role:
    result['role']=role
return result

def _json(path,value):
    path.write_text(json.dumps(value,ensure_ascii=False,indent=2,allow_nan=False)
+'\\n',encoding='utf-8')

def _qid(question_id):
    value=str(question_id).upper()
    if value in ['1','2','3','4']:
        value='Q'+value
    if value not in ['Q1','Q2','Q3','Q4']:
        raise ValueError('Export requires Q1, Q2, Q3 or Q4')
    return value

def _output_times(question_id,end,event):
    if question_id=='Q1':
        if end<1800-1e-7:
            raise ValueError('Q1 requires a solved interval covering 0..1800s')
        return np.arange(1801,dtype=float)
    if question_id=='Q2':
        if abs(end-round(end))>1e-7:
            raise ValueError('Q2 needs an integer post-crossing verification endpoint')
        return np.arange(int(round(end))+1,dtype=float)
    if event is None:
        raise ValueError('Q3/Q4 final export requires an actual critical drying event')
    if not 0<=event<=end:
        raise ValueError('Critical event must lie inside the solved interval')
    return np.unique(np.r_[np.arange(math.floor(end/60)+1,dtype=float)*60,event,end])

def _q4(x):
    if x is None:
        return None
    if not math.isfinite(float(x)):
        raise ValueError('A non-finite value cannot be exported as a numerical result')
    return float(round(float(x),4))

def _template(question_id):
    path=TEMPLATE_DIR/f'result{question_id[-1]}.xlsx'
    wb=load_workbook(path,read_only=True,data_only=False)
    try:
        header=next(wb.worksheets[0].iter_rows(values_only=True))
        names=wb.sheetnames
        return {'record':_record(path,'original_result_template'),'time_header':header[0],
                'sheet_names':names,'surface_header':header[-1] if question_id=='Q4' else None}
    finally:
        wb.close()

def _run_provenance(run):
    module=sys.modules[run.model.__class__.__module__]
    core=Path(inspect.getfile(run.model.__class__)).resolve()
    loaded=getattr(module,'LOADED_CODE_SHA256',None)
    if loaded is not None and loaded!=_sha(core):
        raise RuntimeError('Core file changed after this Run implementation was imported')
    if _sha(SOURCE)!=LOADED_EXPORTER_SHA256:
        raise RuntimeError('Exporter changed after import; restart for consistent provenance')
    input_records=list(getattr(run.model,'input_records',[]))

```

```

for rec in input_records:
    if _sha(ROOT/rec['path'])!=rec['sha256']:
        raise RuntimeError('Observed input changed after solve: '+rec['path'])
return {'core':_record(core,'solver_source'),'exporter':_record(SOURCE,'exporter_source'),
        'inputs':input_records,'settings':asdict(run.model.settings),
        'event_s':None if run.event_s is None else float(run.event_s),
        'end_s':float(run.end_s),'source_kind':'live_Run_dense_solution',
        'per_second_values_are_not_interpolated_from_60s_npz':True}

def _field_block(run,times,qid):
    T,C=run.fields(times,radii_m=RADIO_M)
    radius=np.asarray(run.model.radius(times),dtype=float)
    if radius.ndim==0:
        radius=np.full(len(times),radius)
    surface_C=None
    if qid=='Q4':
        _,surface=run.fields(times,material_x=np.array([1.0]))
        surface_C=surface[:,0]
        inside=RADIO_M[None,:]<=radius[:,None]+1e-12
        # A difference below 1e-12 m at a grid/surface coincidence is roundoff.
        near_boundary=inside & (~np.isfinite(C))
        if np.any(near_boundary):
            C=np.where(near_boundary,surface_C[:,None],C)
            T=np.where(inside,T,np.nan)
            C=np.where(inside,C,np.nan)
            if not np.all(np.isfinite(C[inside])) or not np.all(np.isfinite(surface_C)):
                raise ValueError('Non-finite moisture inside Q4 domain')
    else:
        inside=np.ones_like(C,dtype=bool)
        if not np.all(np.isfinite(T)) or not np.all(np.isfinite(C)):
            raise ValueError('Non-finite output inside fixed domain')
    return T-273.15,C,radius,surface_C,inside

def _raw_headers(qid):
    h=['time_s','radius_m']
    if qid in ['Q1','Q2']:
        h += [f'T_C_r_{i/10:.1f}_cm' for i in range(21)]
    h += [f'C_r_{i/10:.1f}_cm' for i in range(21)]
    if qid=='Q4':
        h += ['C_surface']
    return h

def _raw_row(qid,t,T,C,R,surface,inside):
    row=[float(t),float(R)]
    if qid in ['Q1','Q2']:
        row+=list(map(float,T))
    row += [float(c) if valid else None for c,valid in zip(C,inside)]
    if qid=='Q4':
        row.append(float(surface))
    return row

def _rounded_sheets(qid,row):
    t,R=row[:2]
    if qid in ['Q1','Q2']:
        return {'温度':[_q4(t)]+[_q4(v) for v in row[2:23]],
                '水分浓度':[_q4(t)]+[_q4(v) for v in row[23:44]]}
    if qid=='Q3':
        return {'Sheet1':[_q4(t)]+[_q4(v) for v in row[2:23]]}
    return {'Sheet1':[_q4(t)]+[_q4(v) for v in row[2:24]],
            '半径':[_q4(t),_q4(R*100)]}

```

```

def _sheet(wb,name,header):
    ws=wb.create_sheet(name)
    ws.freeze_panes='B2'
    ws.sheet_view.showGridLines=False
    ws.column_dimensions['A'].width=30
    for j in range(2,len(header)+1):
        ws.column_dimensions[get_column_letter(j)].width=13 if j<len(header) else 16
    ws.row_dimensions[1].height=34
    cells=[]
    for value in header:
        cell=WriteOnlyCell(ws,value=value)
        cell.font=Font(name='Arial',size=10,bold=True,color='FFFFFF')
        cell.fill=PatternFill(fill_type='solid',fgColor='334155')
        cell.alignment=Alignment(horizontal='center',vertical='center',wrap_text=True)
        if isinstance(value,(int,float)):
            cell.number_format='0.0'
        cells.append(cell)
    # write_only's first append opens a temporary worksheet XML file. Keep
    # those rebuildable files in this contest workspace instead of OS Temp.
    cache=ROOT/'tmp/cache/openpyxl_exports'
    cache.mkdir(parents=True,exist_ok=True)
    previous_tempdir=tempfile.tempdir
    try:
        tempdir=str(cache)
        ws.append(cells)
    finally:
        tempdir=previous_tempdir
    return ws

def _append_numeric(ws,row):
    cells=[]
    for value in row:
        if value is None:
            cells.append(None)
        else:
            cell=WriteOnlyCell(ws,value=value)
            cell.number_format=NUMBER_FORMAT
            cells.append(cell)
    ws.append(cells)

def _paper_tables(run,qid,directory):
    if qid=='Q1':
        ts=np.array([100,300,600,900,1200,1500,1800],dtype=float)
    elif qid=='Q2':
        ts=np.arange(1,7,dtype=float)*1800
        if run.end_s<10800-1e-7:
            raise ValueError('Q2 paper tables require 3h of actual solution')
    else:
        from q3_model import completion
        reported_end=completion(run)['reported_time_s']
        ts=np.unique(np.r_[np.arange(21600.,reported_end+1e-8,21600.),reported_end])
    rs=np.array([0.,.005,.01,.015,.02])
    T,C=run.fields(ts,radii_m=rs)
    T=T-273.15
    R=np.asarray(run.model.radius(ts))
    headers=['时间/s' if qid=='Q1' else '时间/h']+[0,0.5,1,1.5,2]
    if qid=='Q4':
        headers += ['药材表面']
        _,Cs=run.fields(ts,material_x=[1.])
    outputs=[]
    fields=[('temperature',T),('moisture',C)] if qid in ['Q1','Q2'] else [('moisture',C)]
    for kind,values in fields:
        path=directory/f'{qid.lower()}_paper_{kind}.csv'
        with path.open('w',encoding='utf-8-sig',newline='') as fh:

```

```

w=csv.writer(fh);w.writerow(headers)
for i,t in enumerate(ts):
    row=[f'{t if qid=="Q1" else t/3600:.4f}']
    for j,r in enumerate(rs):
        outside=qid=="Q4" and r>R[i]+1e-12
        value=values[i,j]
        if not outside and not np.isfinite(value) and qid=="Q4" and abs(r-
R[i])<=1e-12:
            value=Cs[i,0]
            row.append('' if outside else f'{float(value):.4f}')
            if qid=="Q4":
                row += [f'{Cs[i,0]:.4f}']
            w.writerow(row)
    outputs.append(_record(path,'paper_table'))
if qid=="Q4":
    path=directory/'q4_paper_radius.csv'
    with path.open('w',encoding='utf-8-sig',newline='') as fh:
        w=csv.writer(fh);w.writerow(['时间/h','表面半径/cm'])
        w.writerows([[f'{t/3600:.4f}',f'{r*100:.4f}'] for t,r in zip(ts,R)])
    outputs.append(_record(path,'paper_surface_coordinates'))
return outputs

def _validate_paper_tables(manifest,run):
    """Re-read every paper CSV cell and query the live solution independently.

    Do not call _paper_tables or reuse its generated values. The question's
    schedule is reconstructed here. Physical points come from validated CSV
    headers; the final query uses completion's actual seconds, never the
    rounded hour label parsed back from the CSV.
    """
    qid=manifest['question_id']
    if run is None:
        return {'status':'NOT_REQUESTED','checked_cells':0,'files':[],
                'scope':'Paper CSV hashes only; no independent live Run available.'},[]
    errors=[];records=manifest['paper_tables'];completion_info=None
    if qid=="Q1":
        times=np.asarray([100.,300.,600.,900.,1200.,1500.,1800.])
        expected_kinds=['temperature','moisture']
    elif qid=="Q2":
        times=1800.*np.arange(1,7)
        expected_kinds=['temperature','moisture']
    else:
        from q3_model import completion
        completion_info=completion(run)
        final_s=float(completion_info['reported_time_s'])
        # Integer multiples strictly before the report, then the report itself.
        # Construct afresh rather than reading generation-time arrays/labels.
        regular=[21600.*k for k in range(1,math.floor(final_s/21600.)+1)
                 if 21600.*k<final_s-1e-8]
        times=np.asarray([*regular,final_s],dtype=float)
        expected_kinds=['moisture']+(['radius'] if qid=="Q4" else [])
        stored=manifest.get('paper_table_time_contract',{})
        for key in ['reported_time_s','reported_drying_time_h']:
            if key not in stored or not _numeric_equal(stored[key],completion_info[key]):
                errors.append('Paper CSV report-time metadata differs from independent completion:
'+key)
            if not
_numeric_equal(stored.get('reported_time_h'),completion_info['reported_drying_time_h']):
                errors.append('Paper CSV reported_time_h alias differs from independent completion')
        if times[-1]>run.end_s+1e-7:
            errors.append('Paper CSV contract exceeds the live solution horizon')
        return {'status':'FAIL','checked_cells':0,'files':[]},errors
    expected_names={f'{qid.lower()}_paper_{kind}.csv':kind for kind in expected_kinds}
    actual_names=[Path(r['path']).name for r in records]
    if set(actual_names)!=set(expected_names) or len(actual_names)!=len(expected_names):

```

```

        errors.append('Missing, duplicate, or unexpected paper CSV artifact')
files=[];total_cells=0
for rec in records:
    path=ROOT/rec['path'];kind=expected_names.get(path.name)
    if kind is None:continue
    before_errors=len(errors)
    with path.open('r',encoding='utf-8-sig',newline='') as fh:
        reader=csv.reader(fh);header=next(reader,None);rows=list(reader)
    desired_time='时间/s' if qid=='Q1' else '时间/h'
    if kind=='radius':
        correct_header=['时间/h','表面半径/cm']
        if header!=correct_header:
            errors.append(path.name+': radius/time units or header mismatch')
            continue
        radii=None
        expected=np.asarray(run.model.radius(times),dtype=float).reshape(-1,1)*100.
        inside=np.ones_like(expected,dtype=bool)
        unit='cm';surface_column=None
    else:
        wanted_columns=7 if qid=='Q4' else 6
        if header is None or len(header)!=wanted_columns or header[0]!=desired_time:
            errors.append(path.name+': time/column header mismatch');continue
        try:
            radii=np.asarray([float(v)*.01 for v in header[1:6]])
        except ValueError:
            errors.append(path.name+': nonnumeric physical-radius header');continue
        if not np.all(np.isfinite(radii)) or not np.allclose(radii,
[0.,.005,.01,.015,.02],rtol=0,atol=1e-12):
            errors.append(path.name+': physical-radius grid must be 0,0.5,1,1.5,2
cm');continue
        if qid=='Q4' and header[-1]!='药材表面':
            errors.append(path.name+': distinct material surface column missing');continue
        temperature,moisture=run.fields(times,radii_m=radii)
        expected=np.asarray(temperature)-273.15 if kind=='temperature' else
np.asarray(moisture)
        inside=np.ones_like(expected,dtype=bool);surface_column=None
        if qid=='Q4':
            physical_R=np.asarray(run.model.radius(times),dtype=float)
            inside=radii[None,:] <= physical_R[:,None]+1e-12
            _,surface_values=run.fields(times,material_x=np.asarray([1.]))
            # Only roundoff at a coincident physical/surface coordinate
            # may use the separately queried material surface value.
            coincidence=np.abs(radii[None,:]-physical_R[:,None])<=1e-12
            expected=np.where(coincidence & ~np.isfinite(expected),surface_values,expected)
            expected=np.column_stack([expected,surface_values[:,0]])
            inside=np.column_stack([inside,np.ones(len(times),dtype=bool)])
            surface_column=7
            unit='degC' if kind=='temperature' else 'kg/kg'
        if len(rows)!=len(times):
            errors.append(f'{path.name}: expected {len(times)} data rows, read {len(rows)}')
        checked=0;blanks=0;max_field_error=0.;max_time_error=0.
        for i,(row,t) in enumerate(zip(rows,times)):
            if len(row)!=len(header):
                errors.append(f'{path.name}: column count mismatch at row {i+2}');continue
            time_value=float(t if qid=='Q1' else t/3600.)
            if row[0]!=format(time_value,'.4f'):
                errors.append(f'{path.name}: time label mismatch at row {i+2}; query time is
{t:.17g}s')
            else:max_time_error=max(max_time_error,abs(float(row[0])-time_value))
            checked+=1
            for j,cell in enumerate(row[1:]):
                checked+=1
                if not inside[i,j]:
                    blanks+=1
                    if cell!='':errors.append(f'{path.name}: domain outside must be blank at row
{i+2}, column {j+2}')

```

```

        continue
        value=float(expected[i,j])
        if not math.isfinite(value):
            errors.append(f'{path.name}: live value is not finite inside
material');continue
        if not re.fullmatch(r'-?\d+\.\d{4}',cell):
            errors.append(f'{path.name}: finite four-decimal number required at row {i+2},
column {j+2}')
        continue
        actual=float(cell)
        if not math.isfinite(actual) or abs(actual-float(format(value, '.4f')))>1e-10:
            errors.append(f'{path.name}: independently queried value mismatch at row
{i+2}, column {j+2}')
        max_field_error=max(max_field_error,abs(actual-value))
        total_cells+=checked
        files.append({'artifact':_record(path),'kind':kind,'status':'PASS' if
len(errors)==before_errors else 'FAIL',

'header':header,'data_rows':len(rows),'expected_data_rows':len(times),'checked_cells':checked,
'source_query_times_s':times.tolist(),'display_time_unit':'s' if qid=='Q1' else 'h',
'max_display_time_rounding_error':max_time_error,'field_unit':unit,

'max_field_rounding_error_from_live_value':max_field_error,'outside_domain_blank_cells':blanks,
'physical_radii_m':None if radii is None else radii.tolist(),
'surface_column_based':surface_column,'surface_query':'material_x=1; never fixed 2
cm' if surface_column else None})
    return {'status':'FAIL' if errors else 'PASS','checked_cells':total_cells,'files':files,
'completion':completion_info,'completion_source':_record(SOURCE.with_name('q3_model.py'))
if completion_info else None,
'scope':'Every paper CSV cell independently re-evaluated from live Run and fixed question
contracts; no generation arrays or 60s archives reused.',
'rounding_limit':'CSV stores four decimals; matching rounded values cannot distinguish
sub-rounding perturbations of the original source value.'},errors

def export_question(run,question_id,output_dir,*,chunk_rows=1000,overwrite=False):
    """Export a complete question and return records plus exact validation rules.

    A fresh output directory/version is preferred. Existing files are rejected
    unless overwrite=True is explicitly passed. This never edits raw templates.
    """
    started=time.perf_counter()
    qid=_qid(question_id)
    directory=Path(output_dir).resolve()
    if not directory.is_relative_to(ROOT) or directory.is_relative_to(ROOT/'problem_files'):
        raise ValueError('Outputs must be inside the competition workspace, outside
problem_files')
    if not 1<=int(chunk_rows)<=2000:
        raise ValueError('chunk_rows must be between 1 and 2000')
    directory.mkdir(parents=True,exist_ok=True)
    stem='result'+qid[-1]
    book_path=directory/(stem+'.xlsx')
    raw_path=directory/(stem+'_unrounded.csv.gz')
    manifest_path=directory/(stem+'.export.json')
    if not overwrite and any(p.exists() for p in [book_path,raw_path,manifest_path]):
        raise FileExistsError('Export target already exists; use a new output version')
    provenance=_run_provenance(run)
    allowed={'Q1':{'Q1'},'Q2':{'Q2','Q23'},'Q3':{'Q2','Q3','Q23'},'Q4':{'Q4'}}
    if run.model.settings.question not in allowed[qid]:
        raise ValueError('Run physics/question do not match the requested output question')
    if qid=='Q4' and not run.model.settings.shrink:
        raise ValueError('Formal Q4 output requires the shrinking-domain Run, not its fixed-
radius control')
    template=_template(qid)
    times=_output_times(qid,run.end_s,run.event_s)
    if len(times)+1>1_048_576:

```

```

        raise ValueError('Required full time grid exceeds the XLSX worksheet row limit; do not
truncate')
    workbook=Workbook(write_only=True)
    workbook.properties.creator=''
    workbook.properties.lastModifiedBy=''
    workbook.properties.title=f'问题{qid[-1]}结果'
    headers={name:[template['time_header']] + RADII_CM for name in template['sheet_names']}
    if qid=='Q4':
        headers['Sheet1'].append(template['surface_header'])
        headers['半径']=['时间/s','药材表面半径/cm']
    sheets={name:_sheet(workbook,name,header) for name,header in headers.items()}
    counts={name:
{'data_rows':0,'total_rows':1,'columns':len(header),'outside_domain_blank_cells':0,
'first_time_s':None,'last_time_s':None,'header':header} for name,header in
headers.items()}
    samples_idx=set(np.linspace(0,len(times)-1,min(13,len(times)),dtype=int).tolist())
    samples_idx.update([0,min(1,len(times)-1),len(times)-1])
    if run.event_s is not None:
        samples_idx.update(np.flatnonzero(times==run.event_s).tolist())
    samples=[]
    with gzip.open(raw_path,'wt',encoding='utf-8',newline='',compresslevel=6) as fh:
        raw_writer=csv.writer(fh);raw_writer.writerow(_raw_headers(qid))
        for start in range(0,len(times),int(chunk_rows)):
            ts=times[start:start+int(chunk_rows)]
            T,C,R,Cs,inside=_field_block(run,ts,qid)
            for i,t in enumerate(ts):
                raw=_raw_row(qid,t,T[i],C[i],R[i],None if Cs is None else Cs[i],inside[i])
                raw_writer.writerow(['' if v is None else format(v,'.17g') for v in raw])
                values=_rounded_sheets(qid,raw)
                for name,row in values.items():
                    _append_numeric(sheets[name],row)
                    count=counts[name]
                    count['data_rows']+=1;count['total_rows']+=1
                    count['outside_domain_blank_cells']+=sum(v is None for v in row)
                    if count['first_time_s'] is None:
                        count['first_time_s']=row[0]
                    count['last_time_s']=row[0]
                if start+i in samples_idx:
                    samples.append({'data_row_index':start+i,'source_time_s':float(t),
'raw_values':raw,'rounded_sheets':values})
    workbook.save(book_path)
    paper_records=_paper_tables(run,qid,directory)
    paper_time_contract={'unit':'s' if qid=='Q1' else 'h','stored_decimals':4,
'query_uses_unrounded_seconds':True}
    if qid in ['Q3','Q4']:
        from q3_model import completion
        completed=completion(run)
        paper_time_contract.update({key:completed[key] for key in
['reported_time_s','reported_drying_time_h','critical_event_s',
'max_C_at_reported_time','conservative_post_verification_s']})
        paper_time_contract['reported_time_h']=completed['reported_drying_time_h']
        paper_time_contract['regular_step_s']=21600.
        paper_time_contract['endpoint_convention']='6-hour samples plus upward-reported strict-
drying time; distinct from workbook post-verification endpoint'
        end_provenance=_run_provenance(run)
        if end_provenance!=provenance or _sha(ROOT/template['record']['path'])!=template['record']
['sha256']:
            raise RuntimeError('Source, settings or template changed during export')
        book_record=_record(book_path,'contest_result_workbook')
        archive_record=_record(raw_path,'internal_unrounded_output_grid_archive')
        manifest={
'schema_version':'1.0','question_id':qid,'generated_by':'paper_output/code/modeling/
export_outputs.py',

'generated_at':datetime.now(timezone.utc).isoformat(),'status':'EXPORTED_PENDING_STREAM_READBACK'
,

```

```

        'workbook':book_record,'unrounded_archive':archive_record,'paper_tables':paper_records,
        'paper_table_time_contract':paper_time_contract,
        'provenance':provenance,'template':template['record'],'sheets':counts,
        'time_grid':{'start_s':0.,'end_s':float(times[-1]),'regular_step_s':1 if qid in
['Q1','Q2'] else 60,
                    'critical_event_s':provenance['event_s'] if qid in ['Q3','Q4'] else None,
                    'include_post_verification_endpoint':qid!='Q1',

        'count':len(times),'rounding_duplicate_time_count':int(np.sum(np.diff(np.round(times,4))==0))),
        'radius_grid':
{'fixed_radii_cm':RADII_CM,'surface_column':qid=='Q4','surface_coordinate_sheet':'半径' if
qid=='Q4' else None},
        'rounding':{'decimal_places':4,'storage':'numeric rounded to four
decimals','number_format':NUMBER_FORMAT,
                    'raw_archive':'17 significant digits, source times in seconds and radius in
metres',
                    'threshold_judgement':'use original full precision event and max C, never
rounded 0.1500'},
        'domain_rule':{'outside_values':'blank (None); never
zero','comparison_tolerance_m':1e-12,
                        'Q4_surface_is_not_fixed_2cm':True},
        'samples':samples,'elapsed_s':time.perf_counter()-started,
        'size':{'workbook_bytes':book_record['bytes'],'workbook_MiB':book_record['bytes']/2**20,

        'unrounded_archive_bytes':archive_record['bytes'],'warning_limit_bytes':SIZE_LIMIT_BYTES,
        'workbook_exceeds_20_decimal_MB':book_record['bytes']>SIZE_LIMIT_BYTES,
        'archive_is_internal_not_automatically_in_submission_zip':True,
        'complete_support_archive_still_requires_actual_size_check':True,
        'no_required_time_or_space_values_removed':True},
        'validation_required':'validate_exports([record], runs={question_id: run}); full
workbook/archive comparison, live Run workbook samples, and every paper CSV cell independently
queried from live Run',
        'visual_studio_gui':'pending','human_review':'pending',
    }
    _json(manifest_path,manifest)
    return {'question_id':qid,'workbook_path':str(book_path),'manifest_path':str(manifest_path),
            'artifacts':[book_record,archive_record]+paper_records+
[_record(manifest_path,'export_manifest')],
            'sheets':counts,'size':manifest['size'],'validation_status':'pending'}

def _manifest_path(item):
    if isinstance(item,dict):
        return Path(item['manifest_path'])
    path=Path(item)
    return path.with_suffix('.export.json') if path.suffix=='.xlsx' else path

def _numeric_equal(a,b,tolerance=1e-10):
    return a is None and b is None or (a is not None and b is not None and
        isinstance(a,(int,float)) and not isinstance(a,bool) and math.isfinite(float(a)) and
        abs(float(a)-float(b))<=tolerance)

def validate_exports(paths,runs=None):
    """Stream every saved cell against the exact schedule and raw float archive.

    Pass runs={'Q1': run1, ...} to re-evaluate workbook sample points and every
    paper CSV cell from question-specific contracts and the live Run. With no
    live Run this reports static/archive consistency only, never full verification.
    Writes a sibling .validation.json for each export. Does not alter the XLSX.
    """
    if isinstance(paths,(str,Path,dict)):
        paths=[paths]
    runs={} if runs is None else runs
    results=[]

```

```

for item in paths:
    started=time.perf_counter()
    manifest_path=_manifest_path(item).resolve()
    manifest=json.loads(manifest_path.read_text(encoding='utf-8'))
    qid=manifest['question_id'];errors=[];checked_cells=0;checked_sheets={}
    for rec in [manifest['workbook'],manifest['unrounded_archive'],manifest['template']]
+manifest['paper_tables']:
        if not (ROOT/rec['path']).exists() or _sha(ROOT/rec['path'])!=rec['sha256']:
            errors.append('Hash mismatch or missing artifact: '+rec['path'])
    if errors:
        raise RuntimeError('; '.join(errors))
    times=_output_times(qid,manifest['provenance']['end_s'],manifest['provenance']
['event_s'])
    wb=load_workbook(ROOT/manifest['workbook']['path'],read_only=True,data_only=False)
    if wb.sheetnames!=list(manifest['sheets']):
        errors.append('Sheet names/order differ from export contract')
    try:
        for name,spec in manifest['sheets'].items():
            ws=wb[name]
            rows=ws.iter_rows(min_row=1,max_col=spec['columns'])
            header=[cell.value for cell in next(rows)]
            if header!=spec['header']:
                errors.append(name+' : header/physical radius grid mismatch')
            count=0;blanks=0;first=None;last=None;max_rounding_error=0.
            with gzip.open(ROOT/manifest['unrounded_archive']
['path'],'rt',encoding='utf-8',newline='') as rawfh:
                source_rows=csv.reader(rawfh)
                if next(source_rows)!=_raw_headers(qid):
                    errors.append('Unrounded archive schema mismatch')
                for index,source_row in enumerate(source_rows):
                    try:
                        cells=next(rows)
                    except StopIteration:
                        errors.append(name+' : workbook ends before raw archive');break
                    raw=[None if v=='' else float(v) for v in source_row]
                    if index>=len(times) or abs(raw[0]-times[index])>1e-8:
                        errors.append(name+' : source time schedule mismatch at '+str(index))
                    if qid=='Q4':
                        for j,r in enumerate(RADII_M):
                            outside=r>raw[1]+1e-12
                            if (raw[2+j] is None)!=outside:
                                errors.append(name+' : incorrect moving-domain mask at
'+str(index))
                    elif any(v is None for v in raw):
                        errors.append(name+' : unexpected blank in fixed domain')
                    expected=_rounded_sheets(qid,raw)[name]
                    for j,(cell,value) in enumerate(zip(cells,expected)):
                        if not _numeric_equal(cell.value,value):
                            errors.append(f'{name}: value mismatch at row {index+2}, column
{j+1}')
                    if value is not None:
                        if cell.number_format!=NUMBER_FORMAT:
                            errors.append(f'{name}: four-decimal format missing at row
{index+2}, column {j+1}')
                        if not math.isfinite(float(cell.value)) or
abs(float(cell.value)*1e4-round(float(cell.value)*1e4))>1e-5:
                            errors.append(f'{name}: non-finite or non-four-decimal stored
number')
                        max_rounding_error=max(max_rounding_error,abs(float(cell.value)-
value))
                    else:
                        blanks+=1
                        checked_cells+=len(cells);count+=1
                        if first is None:first=cells[0].value
                        last=cells[0].value
                        if len(errors)>30:

```

```

        raise RuntimeError('Export validation failed: '+'; '.join(errors[:30]))
    if next(rows, None) is not None:
        errors.append(name+': workbook has uncontracted extra rows')
    if count!=spec['data_rows'] or count!=len(times):
        errors.append(name+': row count differs from time grid')
    if blanks!=spec['outside_domain_blank_cells']:
        errors.append(name+': blank count mismatch')
    checked_sheets[name]={ 'data_rows':count, 'total_rows':count+1, 'first_time_s':first,
'last_time_s':last, 'outside_domain_blank_cells':blanks, 'value_comparison_max_abs_error':max_round
ing_error}
    finally:
        wb.close()
    run=runs.get(qid)
    live={'status':'NOT_REQUESTED', 'samples':0, 'max_abs_unrounded_difference':None}
    if run is not None:
        current=_run_provenance(run)
        if current!=manifest['provenance']:
            errors.append('Live Run provenance differs from the exported Run')
        samples=manifest['samples'];ts=np.array([s['source_time_s'] for s in samples])
        T,C,R,Cs,inside=_field_block(run,ts,qid)
        difference=0.
        for i,sample in enumerate(samples):
            actual=_raw_row(qid,ts[i],T[i],C[i],R[i],None if Cs is None else Cs[i],inside[i])
            for a,b in zip(actual,sample['raw_values']):
                if a is None or b is None:
                    if not (a is None and b is None):errors.append('Live Run sample domain
mismatch')
                else:
                    difference=max(difference,abs(a-b))
            if difference>1e-10:
                errors.append('Independent live Run sample mismatch exceeds 1e-10')
        live={'status':'PASS' if difference<=1e-10 else 'FAIL', 'samples':len(samples),
            'max_abs_unrounded_difference':difference}
        paper_validation,paper_errors=_validate_paper_tables(manifest,run)
        errors.extend(paper_errors)
        report={'schema_version':'1.0', 'question_id':qid, 'generated_by':'paper_output/code/
modeling/export_outputs.py',
            'generated_at':datetime.now(timezone.utc).isoformat(),
            'status':'FAIL' if errors else 'PASS', 'fully_verified_with_live_Run':not errors
and run is not None,

'checked_cells':checked_cells, 'sheets':checked_sheets, 'live_Run_sample_validation':live,
            'paper_CSV_live_Run_validation':paper_validation,
            'errors':errors, 'elapsed_s':time.perf_counter()-started,
            'workbook':manifest['workbook'], 'export_manifest':_record(manifest_path),
            'scope':'Full workbook/raw-archive validation; independent workbook live Run
samples and every paper CSV time/position/value/surface/radius cell when live Run is provided.',
            'four_decimals_are_storage_and_display_not_physical_accuracy_claim':True,
            'visual_render':'pending', 'visual_studio_gui':'pending', 'human_review':'pending'}

report_path=manifest_path.with_name(manifest_path.name.replace('.export.json','validation.json')
)
    _json(report_path,report);report['report_path']=str(report_path);results.append(report)
    if errors:
        raise RuntimeError('Export validation failed: '+'; '.join(errors[:30]))
    return {'status':'PASS' if all(r['status']=='PASS' for r in results) else 'FAIL',
        'fully_verified_with_live_Run':all(r['fully_verified_with_live_Run'] for r in
results), 'exports':results}

def _paper_csv_selfcheck(directory):
    """Small actual solves plus deliberate CSV defects, never final results."""
    from drying_core import Settings,solve_case
    import copy
    directory=Path(directory).resolve()

```

```

directory.mkdir(parents=True,exist_ok=True)
summaries=[];negative_checks=[]
for qid in ['Q1','Q2','Q3','Q4']:
    settings={'question':'Q23' if qid in ['Q2','Q3'] else qid,
              'intervals':40,'face_scheme':'kirchhoff','shrink':qid=='Q4',
              'rtol':1e-8,'atol_temperature':1e-8,'atol_moisture':1e-10,
              'early_max_step_s':5.}
    if qid=='Q2':
        settings.update(constant_D=2e-8,beta=8e-6,horizon_h=24.)
    print('PAPER_CSV_SELFHECK '+qid+' : actual N40 solve',flush=True)
    run=solve_case(Settings(**settings))
    try:
        if qid=='Q2' and (run.event_s is None or run.end_s<10800.):
            raise RuntimeError('Accelerated export test must still cover 3h and a real drying
event')
        exported=export_question(run,qid,directory/qid)
        checked=validate_exports([exported],runs={qid:run})
        manifest=json.loads(Path(exported['manifest_path']).read_text(encoding='utf-8'))
        defects={'Q1':['temperature_value'],'Q2':['moisture_value'],
                  'Q3':['last_time_label'],'Q4':
['outside_nonblank','surface_missing','radius_value']}[qid]
        for defect in defects:
            changed=copy.deepcopy(manifest)
            kind='temperature' if defect=='temperature_value' else 'radius' if
defect=='radius_value' else 'moisture'
            selected=next(r for r in changed['paper_tables'] if
Path(r['path']).name==f'{qid.lower()}_paper_{kind}.csv')
            original_path=ROOT/selected['path']
            with original_path.open('r',encoding='utf-8-sig',newline='') as
fh:rows=list(csv.reader(fh))
            if defect in ['temperature_value','moisture_value']:
                rows[1][1]=format(float(rows[1][1])+.01, '.4f')
            elif defect=='last_time_label':
                rows[-1][0]=format(float(rows[-1][0])-.001, '.4f')
            elif defect=='outside_nonblank':
                ri,ci=next((i,j) for i in range(1,len(rows)) for j in range(1,6) if rows[i]
[j]=='')
                rows[ri][ci]='0.0000'
            elif defect=='surface_missing':rows[-1][-1]=''
            elif defect=='radius_value':rows[-1][1]=format(float(rows[-1][1])+.01, '.4f')
            target=directory/'deliberate_defects'/defect/original_path.name
            target.parent.mkdir(parents=True,exist_ok=True)
            with target.open('w',encoding='utf-8-sig',newline='') as
fh:csv.writer(fh).writerows(rows)
            selected.update(_record(target,selected.get('role')))
            result,errors=_validate_paper_tables(changed,run)
            if not errors or result['status']!='FAIL':
                raise AssertionError('Independent paper validator missed deliberate defect:
'+defect)

negative_checks.append({'question_id':qid,'defect':defect,'correctly_rejected':True,
                        'errors':errors,'artifact':_record(target)})

summaries.append({'question_id':qid,'settings':asdict(run.model.settings),'export':exported,
                  'validation':checked,'event_s':run.event_s,'end_s':run.end_s,
                  'Q2_is_accelerated_constant_D_export_test_only':qid=='Q2'})
    print(json.dumps({'question_id':qid,'status':'PASS','end_s':run.end_s,
                      'paper_cells':checked['exports'][0]['paper_CSV_live_Run_validation']
['checked_cells']},ensure_ascii=False),flush=True)
    finally:
        run.close()
    report={'status':'PASS','generated_at':datetime.now(timezone.utc).isoformat(),
            'scope':'N40 exporter fidelity only. Q2 uses constant_D=2e-8 and beta=8e-6 solely to
shorten the real full-event export test; these are not formal physical results. Q1/Q3/Q4 use
their nominal physics.',
            'source':_record(SOURCE),'exports':summaries,'negative_checks':negative_checks,

```

```

        'negative_test_scope':'Copied paper CSVs only; six deliberate defects must be rejected
even after their recorded hashes are updated.',
        'visual_studio_gui':'pending', 'human_review':'pending'}
    _json(directory/'paper_csv_selfcheck_report.json',report)

print(json.dumps({'status':'PASS','questions':4,'deliberate_defects_rejected':len(negative_checks
),

'report':str(directory/'paper_csv_selfcheck_report.json')},ensure_ascii=False),flush=True)
    return 0

def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--selfcheck',action='store_true')
    parser.add_argument('--paper-selfcheck',action='store_true')
    parser.add_argument('--output-dir',default='paper_output/results/export_selfcheck')
    args=parser.parse_args()
    if not args.selfcheck and not args.paper_selfcheck:
        parser.error('Use the Python API with final Runs, or --selfcheck for a real Q1 validation
export')
    if Path.cwd().resolve()!=ROOT:
        raise RuntimeError('Use the competition workspace as cwd')
    if args.paper_selfcheck:
        return _paper_csv_selfcheck(ROOT/args.output_dir)
    from drying_core import Settings,solve_case
    # Small spatial grid is intentional: this validates exporter fidelity, not
    # four-decimal physical accuracy, and is never a final result1 replacement.
    run=solve_case(Settings(question='Q1',intervals=40,face_scheme='kirchhoff',

rtol=1e-8,atol_temperature=1e-8,atol_moisture=1e-10,early_max_step_s=5.))
    record=export_question(run,'Q1',ROOT/args.output_dir)
    checks=validate_exports([record],runs={'Q1':run})
    output=Path(args.output_dir).resolve()
    note={'status':checks['status'],'scope':'Actual Q1 export selfcheck only, not final
production or numerical-accuracy acceptance',
        'settings':asdict(run.model.settings),'export':record,'validation':checks}
    _json(output/'selfcheck_summary.json',note)

print(json.dumps({'status':checks['status'],'fully_verified_with_live_Run':checks['fully_verified
_with_live_Run'],

'output':str(output),'size':record['size'],'sheets':record['sheets']},ensure_ascii=False,indent=2
))
    return 0

if __name__=='__main__':
    raise SystemExit(main())

```

D. 10 publication_plots.py

正式入口直接调用的主题图生成及图源指纹。源文件 SHA256：
a58b5bb7300aef7f334e28cb0be070971d1681b516e537b84320577b00293388。

```
"""Chinese publication figures from live Runs and explicit saved evidence.

make_plots({'Q1': run1, 'Q23': run23, 'Q4': run4}, directory) creates four
primary figures only. Extra comparison functions consume explicitly selected
finished evidence files. No core, production figure index, or manifest changes.
"""
from __future__ import annotations

import argparse
from dataclasses import asdict
from datetime import datetime, timezone
import hashlib
import inspect
import json
import os
from pathlib import Path
import sys

import numpy as np

ROOT=Path(__file__).resolve().parents[3]
SOURCE=Path(__file__).resolve()
LOADED_PLOT_SHA256=hashlib.sha256(SOURCE.read_bytes()).hexdigest()
os.environ.setdefault('MPLCONFIGDIR', str(ROOT/'tmp/cache/matplotlib'))
import matplotlib
matplotlib.use('Agg')
from matplotlib import font_manager
from matplotlib.colors import Normalize
from matplotlib.ticker import NullLocator
import matplotlib.pyplot as plt

FONT_PATH=Path('C:/Windows/Fonts/msyh.ttc')
if not FONT_PATH.exists():
    raise RuntimeError('Verified Chinese font Microsoft YaHei is unavailable')
font_manager.fontManager.addfont(str(FONT_PATH))
FONT_NAME=font_manager.FontProperties(fname=str(FONT_PATH)).get_name()
STYLE={'font.family':FONT_NAME, 'font.size':10, 'axes.titlesize':11, 'axes.labelsize':10,
        'axes.unicode_minus':False, 'axes.spines.top':False, 'axes.spines.right':False,
        'legend.frameon':False, 'pdf.fonttype':42, 'ps.fonttype':42, 'svg.fonttype':'path',
        'savefig.facecolor':'white', 'figure.facecolor':'white', 'axes.facecolor':'white'}
COLOURS={'centre':'#2563A6', 'mean':'#C76829', 'surface':'#218573', 'threshold':'#51555D'}

def _sha(path):
    h=hashlib.sha256()
    with Path(path).open('rb') as f:
        for b in iter(lambda:f.read(1024*1024),b''):h.update(b)
    return h.hexdigest()

def _record(path):
    p=Path(path).resolve()
    return
    {'path':p.relative_to(ROOT).as_posix(), 'bytes':p.stat().st_size, 'sha256':_sha(p), 'exists':True}

def _directory(directory):
    p=Path(directory).resolve()
```

```

    if not p.is_relative_to(ROOT) or p.is_relative_to(ROOT/'problem_files'):
        raise ValueError('Write figures only inside the competition workspace outside original
inputs')
    p.mkdir(parents=True,exist_ok=True)
    return p

def _run_source(run):
    module=sys.modules[run.model.__class__.__module__]
    path=Path(inspect.getfile(run.model.__class__)).resolve()
    if getattr(module,'LOADED_CODE_SHA256',_sha(path))!=_sha(path):
        raise RuntimeError('Core changed after import; regenerate figure from a consistent Run')
    records=[_record(path)]
    jac=path.with_name('analytic_jacobian.py')
    if jac.exists():records.append(_record(jac))
    for rec in run.model.input_records:
        if _sha(ROOT/rec['path'])!=rec['sha256']:
            raise RuntimeError('Run input changed: '+rec['path'])
        records.append(rec)
    return {'kind':'live_Run','settings':asdict(run.model.settings),'files':records,
        'event_s':None if run.event_s is None else
float(run.event_s),'end_s':float(run.end_s)}

def _unchanged(sources):
    if _sha(SOURCE)!=LOADED_PLOT_SHA256:
        raise RuntimeError('Plot module changed after import')
    for source in sources:
        for rec in source.get('files',[]):
            if _sha(ROOT/rec['path'])!=rec['sha256']:
                raise RuntimeError('Source changed during figure generation: '+rec['path'])

def _save(fig,figure_id,qid,title,caption,directory,sources,data,extra=None):
    directory=_directory(directory)
    _unchanged(sources)
    data_path=directory/(figure_id+'_data.npz')
    np.savez_compressed(data_path,**data)
    artifacts={}
    for ext in ['png','svg','pdf']:
        path=directory/(figure_id+'.'+ext)
        fig.savefig(path,dpi=210 if ext=='png' else 160,bbox_inches='tight',pad_inches=.14)
        artifacts[ext]=_record(path)
    plt.close(fig)
    _unchanged(sources)
    record={'figure_id':figure_id,'question_id':qid,'title':title,'caption':caption,
        'purpose':caption,'path':artifacts['png']['path'],
        'bytes':artifacts['png']['bytes'],'sha256':artifacts['png']['sha256'],
        'status':'computed','ok':True,'placeholder':False,'exists':True,
        'artifacts':artifacts,'plot_data':_record(data_path),'sources':sources,
        'generated_by':'paper_output/code/modeling/publication_plots.py',
        'source_code_sha256':LOADED_PLOT_SHA256,
        'generated_at':datetime.now(timezone.utc).isoformat(),
        'font':{'name':FONT_NAME,'file':str(FONT_PATH)},
        'visual_review':'pending','human_review':'pending','visual_studio_gui':'pending'}
    if extra:record.update(extra)
    metadata=directory/(figure_id+'.json')
    metadata.write_text(json.dumps(record,ensure_ascii=False,indent=2,allow_nan=False)
+'\\n',encoding='utf-8')
    record['metadata']=_record(metadata)
    return record

def _footer(fig,text):
    fig.text(.5,.015,text,ha='center',va='bottom',fontsize=8.1,color='#52565E',linespacing=1.45)

```

```

def _environment_note(run):
    s=run.model.settings
    if s.boundary_extension=='nominal':
        return f'4 h 后环境延拓为 {s.tail_temperature_C:g} °C、等效平衡含水率
{s.tail_equilibrium*s.equilibrium_scale:g} kg/kg'
    if s.boundary_extension=='tail_mean':return '4 h 后环境按末 1 h 观测均值延拓'
    return '4 h 后环境按最后观测值延拓'

def _profiles(run,qid,directory):
    source=_run_source(run)
    if qid=='Q1':
        ts=np.array([100.,300.,600.,900.,1200.,1500.,1800.])
        labels=[f'{t:g} s' for t in ts]
        title='第一问：预热阶段的径向温湿分布'
    else:
        ts=np.arange(1,7,dtype=float)*1800
        labels=[f'{t/3600:g} h' for t in ts]
        title='第二问：前三小时的径向温湿分布'
    if run.end_s<ts[-1]-1e-7:
        raise ValueError('Run does not cover all question-specific plot times')
    rs=np.linspace(0,.02,121)
    T,C=run.fields(ts,radii_m=rs);T=T-273.15
    with plt.rc_context(STYLE):
        fig,axes=plt.subplots(1,2,figsize=(11.8,4.8))
        fig.subplots_adjust(left=.075,right=.97,bottom=.245,top=.82,wspace=.28)
        colours=plt.get_cmap('viridis')(np.linspace(.08,.9,len(ts)))
        handles=[]
        for i,(label,colour) in enumerate(zip(labels,colours)):
            handle,=axes[0].plot(rs*100,T[i],color=colour,lw=1.7,label=label)
            handles.append(handle)
            axes[1].plot(rs*100,C[i],color=colour,lw=1.7,label=label)
        for ax in axes:
            ax.set_xlim(0,2);ax.set_xticks([0,.5,1,1.5,2]);ax.set_xlabel('到药材中心的距离 / cm')
            ax.grid(alpha=.2,lw=.7)
        axes[0].set(title='(a) 温度',ylabel='温度 / °C')
        axes[1].set(title='(b) 干基含水率',ylabel='干基含水率 / (kg/kg)')
        fig.suptitle(title,y=.97,fontsize=14,fontweight='bold')
        fig.legend(handles,labels,loc='lower
center',bbox_to_anchor=(.5,.075),ncol=len(labels),fontsize=9)
    _footer(fig,'一维径向有效模型的条件性数值结果：忽略端面与显式潜热，环境水分按等效平衡边界处理。')
    return _save(fig,f'fig_{qid.lower()}_profiles',qid,title,
        '各曲线对应题目指定时刻。温度采用摄氏度，水分采用干基；曲线来自同一 live Run 的物理径向点值。',
        directory,[source],{'times_s':ts,'radii_m':rs,'temperature_C':T,'C':C},
        {'sample_time_count':len(ts),'sample_radius_count':len(rs)})

def _drying(run,qid,directory):
    source=_run_source(run)
    if run.event_s is None:
        raise ValueError('Full drying figure requires an actual threshold event')
    if qid=='Q4' and not run.model.settings.shrink:
        raise ValueError('Q4 primary figure must show the shrinking-domain run')
    radius_knots=run.model.rad['time_s'] if qid=='Q4' else np.array([])
    radius_knots=radius_knots[radius_knots<=run.end_s]
    ts=np.unique(np.r_[np.linspace(0,run.end_s,241),
        np.arange(0,min(run.end_s,14400)+1,600),
        radius_knots,run.event_s,run.end_s])
    rs=np.linspace(0,.02,121)
    _,C=run.fields(ts,radii_m=rs)
    R=np.asarray(run.model.radius(ts))
    inside=rs[None,:] <= R[:,None]+1e-12
    C=np.where(inside,C,np.nan)
    _,ends=run.fields(ts,material_x=[0.,1.])
    state=run.state(ts)

```

```

mean=2*run.model.w@state[1:-1:2]
max_C=np.max(state[1:-1:2],axis=0)
del state
t_h=ts/3600;event_h=run.event_s/3600
title=('第三问: 固定半径下的全程水分迁移' if qid=='Q3' else '第四问: 收缩域内的全程水分迁移')
with plt.rc_context(STYLE):
    fig,axes=plt.subplots(1,2,figsize=(12,5.1),gridspec_kw={'width_ratios':[1.08,1]})
    fig.subplots_adjust(left=.065,right=.965,bottom=.21,top=.83,wspace=.31)
    cmap=plt.get_cmap('YlGnBu').copy();cmap.set_bad('white')
    mesh=axes[0].pcolormesh(t_h,rs*100,np.ma.masked_invalid(C.T),shading='auto',
                           cmap=cmap,norm=Normalize(vmin=.05,vmax=2.55),rasterized=True)
    cb=fig.colorbar(mesh,ax=axes[0],pad=.025,fraction=.05)
    cb.set_label('干基含水率 / (kg/kg)',fontsize=9)
    if qid=='Q4':
        # pcolormesh displays each sampled centre over a finite pixel/cell
        # extent. Cover its half-cell overhang so the true outside domain
        # is exactly white up to the interpolated physical boundary.
        axes[0].fill_between(t_h,R*100,2,color='white',zorder=3)
        axes[0].plot(t_h,R*100,color='#202631',lw=1.8,label='药材表面 R(t)',zorder=4)
        axes[0].text(.65*t_h[-1],1.88,'域外(非药材)',ha='center',va='center',fontsize=9,color='#62666D')
        axes[0].legend(loc='lower
right',fontsize=8.5,facecolor='white',framealpha=.85,frameon=True)
        axes[0].set(xlim=(0,t_h[-1]),ylim=(0,2),xlabel='烘干时间 / h',
                   ylabel='物理径向距离 / cm',title='(a) 物理空间中的含水率')
        for series,name,colour in [(ends[:,0],'中心',COLOURS['centre']),
                                   (mean,'干质量加权均值',COLOURS['mean']),
                                   (ends[:,1],'表面',COLOURS['surface'])]:
            axes[1].plot(t_h,series,lw=1.8,label=name,color=colour)
        axes[1].axhline(.15,color=COLOURS['threshold'],ls='--',lw=1.25,label='阈值 0.15 kg/kg')
        axes[1].axvline(event_h,color='#6D6473',ls=':',lw=1.2)
        axes[1].annotate(f'全域临界时刻\n{event_h:.3f} h',xy=(event_h,.15),
                        xytext=(.67*t_h[-1],.64),arrowprops={'arrowstyle':'->','color':'#625A6A'},
                        fontsize=9,ha='center',va='center',color='#514956')
        axes[1].set(xlim=(0,t_h[-1]*1.015),ylim=(0,2.66),xlabel='烘干时间 / h',
                   ylabel='干基含水率 / (kg/kg)',title='(b) 中心、均值与表面的变化')
        axes[1].legend(loc='upper right',fontsize=8.8);axes[1].grid(alpha=.18)
        if t_h[-1]>4:
            axes[0].axvline(4,color='#666A73',ls=':',lw=.85,alpha=.8)
            axes[0].text(4+.025*t_h[-1],.13,'4 h',fontsize=8,color='#52545A')
        fig.suptitle(title,y=.97,fontsize=14,fontweight='bold')
        footer='一维径向有效模型; 均值按干物质质量加权, 达标条件取全域最大值。'
    \n'+_environment_note(run)+'.'
    if qid=='Q4' and run.end_s>259200:
        footer+=' 半径超过 72 h 后采用末值延拓。'
    _footer(fig,footer)
    return _save(fig,f'fig_{qid.lower()}_drying',qid,title,
                '左图为物理半径与时间的水分场; 第四问域外留白, 黑线为实测半径轨迹的插值。右图显示中心、干质量加权均值、表面及全域阈值事件。',
                directory,[source],{'times_s':ts,'radii_m':rs,'radius_m':R,'C':C,
                                   'centre_C':ends[:,0],'mean_C':mean,'surface_C':ends[:,1],'max_C':max_C},
                {'event_s':float(run.event_s),'post_verification_s':float(run.end_s),
                 'masked_outside_count':int(np.count_nonzero(~inside)),
                 'sample_time_count':len(ts),'sample_radius_count':len(rs),
                 'mean_definition':'2*sum(material_dual_cell_weights*C); dry-solid mass
weighting'})

def make_plots(runs,directory):
    """Create exactly four primary figures and return PNG-led figure records."""
    run1=runs['Q1'];run23=runs.get('Q23',runs.get('Q2'));run4=runs['Q4']
    if run23 is None:raise ValueError('Provide the shared Q2/Q3 Run under Q23 or Q2')
    return [_profiles(run1,'Q1',directory),_profiles(run23,'Q2',directory),
            _drying(run23,'Q3',directory),_drying(run4,'Q4',directory)]

```

```

def _saved_summary(path):
    path=Path(path).resolve()
    content=path.read_bytes()
    rec=_record(path)
    if hashlib.sha256(content).hexdigest()!=rec['sha256']:
        raise RuntimeError('Evidence file changed while being read')
    data=json.loads(content)
    diag=data['diagnostics']
    if not diag.get('solver_success') or not diag.get('strictly_dry_at_end') or
diag.get('event_h') is None:
        raise ValueError('Comparison requires a completed successful drying run')
    # Saved experiments carry their original source-code hash. They are not
    # silently relabeled as runs made with the current on-disk solver.
    src={'kind':'saved_completed_run_summary','files':[rec],
        'recorded_solver_code':data.get('code'),'settings':data['settings'],
        'current_solver_equivalence_not_assumed':True}
    return data,src

def make_shrinkage_comparison(fixed_summary_path,shrunk_summary_path,directory):
    fixed,sf=_saved_summary(fixed_summary_path);shrunk,ss=_saved_summary(shrunk_summary_path)
    a,b=fixed['settings'],shrunk['settings']
    if a['question']!='Q4' or b['question']!='Q4' or a['shrink'] or not b['shrink']:
        raise ValueError('Require matched appendix-4 fixed and shrinking runs')
    differences={k for k in set(a)|set(b) if a.get(k)!=b.get(k)}
    if differences!={'shrink'} or fixed.get('code',{}).get('sha256')!=shrunk.get('code',
{}).get('sha256'):
        raise ValueError('A pure shrinkage comparison needs identical settings/source except
shrink')
    values=np.array([fixed['diagnostics']['event_h'],shrunk['diagnostics']['event_h']])
    N=a['intervals'];reduction=100*(1-values[1]/values[0])
    title='同附录4物性下的固定半径与收缩对照'
    with plt.rc_context(STYLE):

fig,ax=plt.subplots(figsize=(8.2,4.7));fig.subplots_adjust(left=.2,right=.92,bottom=.22,top=.81)
    ax.barh([1,0],values,color=['#8894A3','#218573'],height=.5)
    ax.set_yticks([1,0],['固定半径 2 cm','采用附件2收缩'])
    ax.set_xlim(0,values.max()*1.25);ax.set_xlabel('全域临界烘干时间 /
h');ax.grid(axis='x',alpha=.18)
    for y,v in zip([1,0],values):ax.text(v+values.max()*0.02,y,f'{v:.3f}
h',va='center',fontsize=10)
    ax.text(.98,.12,f'本对照缩短 {reduction:.2f}
%',transform=ax.transAxes,ha='right',color='#246653',fontsize=11)
    fig.suptitle(title,y=.95,fontsize=14,fontweight='bold')
    _footer(fig,f'同一已保存计算版本, N={N}, 除收缩开关外设置一致; 这是有效模型中的尺寸效应。\\n 不可
把第三、四问同时改变物性的时长差直接归因于收缩。')
    return _save(fig,'fig_q4_shrinkage_control','Q4',title,
        f'已保存同物性、同参数、同源码版本 N={N}对照; 读取来源原始哈希, 未假设与后续细网格正
式结果相同。',
        directory,[sf,ss],{'event_hours':values},
{'intervals':N,'same_property_control':True,'reduction_percent':float(reduction)})

def make_convergence_plot(report_paths,directory):
    """Read only explicitly selected *finished* reports; never wait for writers."""
    sources=[];groups=[]
    for path in report_paths:

path=Path(path).resolve();content=path.read_bytes();data=json.loads(content);rec=_record(path)
        if hashlib.sha256(content).hexdigest()!=rec['sha256']:raise RuntimeError('Changing
convergence report')
        if not data.get('finished_at') or not str(data.get('status','')).startswith('computed'):
            raise ValueError('Select only finished convergence reports; in-progress reports are
not plotted')
        groups.append(data);sources.append({'kind':'finished_convergence_report','files':[rec]})

```

```

if not groups:raise ValueError('At least one finished convergence report is required')
labels={'Q1':'第一问','Q23':'第二、三问','Q4':'第四问'}
colours={'Q1':'#2563A6','Q23':'#C76829','Q4':'#218573'}
title='相邻网格结果差异与临界时刻收敛'
plotted={}
with plt.rc_context(STYLE):

fig,axes=plt.subplots(1,2,figsize=(11.5,4.8));fig.subplots_adjust(left=.08,right=.965,bottom=.21,
top=.82,wspace=.32)
    for group in groups:
        q=group['question'];comp=group['comparisons']
        x=np.array([c['fine_N'] for c in comp],dtype=float)
        y=np.array([c['max_absolute_difference']][c] for c in comp],dtype=float)
        axes[0].loglog(x,y,'o-',color=colours.get(q),label=labels.get(q,q),lw=1.6)
        timed=[c for c in comp if c.get('event_difference_s') is not None]
        if timed:
            axes[1].loglog([c['fine_N'] for c in timed],[abs(c['event_difference_s']) for c in
timed],
                        'o-',color=colours.get(q),label=labels.get(q,q),lw=1.6)
            plotted[q+'_fine_N']=x;plotted[q+'_C_difference']=y
        axes[0].set(title='(a) 共同物理采样点的最大含水率差',xlabel='较细网格的区间数 N',ylabel='相邻
网格最大差 / (kg/kg)')
        axes[1].set(title='(b) 全域临界时刻的变化',xlabel='较细网格的区间数 N',ylabel='相邻网格临界时
刻差的绝对值 / s')
        all_ticks=sorted([c['fine_N'] for g in groups for c in g['comparisons']])
        time_ticks=sorted([c['fine_N'] for g in groups for c in g['comparisons'] if
c.get('event_difference_s') is not None])
        axes[0].set_xticks(all_ticks,[str(n) for n in all_ticks])
        if time_ticks:axes[1].set_xticks(time_ticks,[str(n) for n in time_ticks])
        for ax in axes:
            ax.xaxis.set_minor_locator(NullLocator())
            ax.grid(which='both',alpha=.2);ax.legend(fontsize=9)
        fig.suptitle(title,y=.97,fontsize=14,fontweight='bold')
        _footer(fig,'读取已完成检验报告并记录来源哈希: 相邻网格差用于数值收敛检验, 不是实测预测误差。')
        return _save(fig,'fig_model_grid_convergence','Q3',title,
                    '横轴为相邻比较中较细网格的区间数; 所画为实际计算的最大含水率差和阈值时刻差, 不把相邻
差直接称作严格误差上界。',
                    directory,sources,plotted,{g['question'] for g in groups})

def make_sensitivity_plot(summary_paths,directory,question_id='Q4',baseline_label='基准'):
    """summary_paths maps explicit Chinese scenario names to completed summaries."""
    if baseline_label not in summary_paths:raise ValueError('An explicit baseline label is
required')
    labels=list(summary_paths);rows=[];sources=[]
    for label,path in summary_paths.items():
        data,source=_saved_summary(path);rows.append(data);sources.append(source)
    if len({r['settings']['intervals'] for r in rows})!=1 or len({r['settings']['question'] for r
in rows})!=1:
        raise ValueError('Sensitivity cases must have a common question and grid size')
    expected={'Q4'} if question_id=='Q4' else {'Q2','Q3','Q23'}
    if rows[0]['settings']['question'] not in expected:
        raise ValueError('Sensitivity figure label does not match the saved physical question')
    if len({r.get('code',{}).get('sha256') for r in rows})!=1:
        raise ValueError('Sensitivity cases must use the same saved source version')
    values=np.array([r['diagnostics']['event_h'] for r in
rows]);baseline=values[labels.index(baseline_label)]
    delta=100*(values/baseline-1);N=rows[0]['settings']['intervals']
    title=('第四问' if question_id=='Q4' else '第二、三问')+' : 物理假设与参数情景比较'
    with plt.rc_context(STYLE):
        fig,ax=plt.subplots(figsize=(9.4,max(4.7,.48*len(labels)+1.8)))
        fig.subplots_adjust(left=.25,right=.93,bottom=.2,top=.82)
        pos=np.arange(len(labels));colours=['#8A949F' if x==baseline_label else '#218573' for x
in labels]
        ax.barh(pos,values,color=colours,height=.58);ax.set_yticks(pos,labels);ax.invert_yaxis()
        ax.set_xlim(0,values.max()*1.33);ax.set_xlabel('全域临界烘干时间 /

```

```

h');ax.grid(axis='x',alpha=.18)
    for y,v,d in zip(pos,values,delta):ax.text(v+values.max()*0.015,y,f'{v:.3f} h ({d:+.2f}
%)',va='center',fontsize=9)
    fig.suptitle(title,y=.95,fontsize=14,fontweight='bold')
    footer(fig,f'同一已保存求解版本，N={N}；百分比相对“{baseline_label}”。这些是指定情景的模型变
化，不是统计置信区间。')
    return _save(fig,f'fig_{question_id.lower()}_physical_sensitivity',question_id,title,
        '读取明确列出的已完成情景，并记录每份摘要哈希、设置和原计算版本；不将情景差异当作观测
误差。',
        directory,sources,{ 'event_hours':values,'relative_change_percent':delta},
        { 'scenario_labels':labels,'baseline_label':baseline_label,'intervals':N})

def make_question_plot(run, question_id, directory):
    """Produce one question figure so production can release each solved Run."""
    if question_id in ('Q1', 'Q2'):
        return _profiles(run, question_id, _directory(directory))
    if question_id in ('Q3', 'Q4'):
        return _drying(run, question_id, _directory(directory))
    raise ValueError('A question plot requires Q1, Q2, Q3 or Q4')

def main():
    parser=argparse.ArgumentParser(description=__doc__);parser.add_argument('--
selfcheck',action='store_true')

    parser.add_argument('--output-dir',default='paper_output/results/plot_selfcheck');args=parser.par
se_args()
    if not args.selfcheck:parser.error('Use make_plots with live Runs or --selfcheck')
    if Path.cwd().resolve()!=ROOT:raise RuntimeError('Run from the competition workspace')
    from drying_core import Settings,solve_case
    runs={}
    for q in ['Q1','Q23','Q4']:
        print('Selfcheck: solving '+q+' with N=40',flush=True)

    runs[q]=solve_case(Settings(question=q,intervals=40,face_scheme='kirchhoff',shrink=q=='Q4',
        rtol=1e-7,atol_temperature=1e-7,atol_moisture=1e-9))
    output=_directory(ROOT/args.output_dir)
    records=make_plots(runs,output)
    fixed=ROOT/'paper_output/results/experiments/physical_Q4_fixed_N200_K/summary.json'
    shrunk=ROOT/'paper_output/results/experiments/grids_Q4_N200_K/summary.json'
    if fixed.exists() and
shrunk.exists():records.append(make_shrinkage_comparison(fixed,shrunk,output))
    reports=[ROOT/'paper_output/results/convergence/{q}_K_analytic_J_v3/convergence_report.json'
    for q in ['Q1','Q23']]
    if all(p.exists() for p in reports):records.append(make_convergence_plot(reports,output))
    scenario_root=ROOT/'paper_output/results/experiments'
    scenarios={'基准':shrunk,'4 h 后环境 49
°C':scenario_root/'physical_Q4_T49_N200_K/summary.json',
        '4 h 后环境 51 °C':scenario_root/'physical_Q4_T51_N200_K/summary.json',
        '潜热能量负荷压力测试':scenario_root/'physical_Q4_latent_N200_K/summary.json'}
    if all(p.exists() for p in
scenarios.values()):records.append(make_sensitivity_plot(scenarios,output))
    report={'scope':'N40 live-run plot selfcheck; primary plots are not final numerical results.
Optional comparisons preserve their independent N200/finished-report sources.',
        'status':'GENERATED_PENDING_VISUAL_REVIEW','figures':records}

    (output/'plot_selfcheck_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2,al
low_nan=False)+'\n',encoding='utf-8')

    print(json.dumps({'status':report['status'],'figure_count':len(records),'output':str(output)},ens
ure_ascii=False),flush=True)
    return 0

if __name__=='__main__':raise SystemExit(main())

```

D. 11 production_provenance.py

源文件/输入指纹、运行元数据、监督进程和发布事务。源文件 SHA256:

0370979d780ff59487d1616d7c0ff6041c642624d9267c51f9a0925b26453130。

```
"""Small, auditable provenance and publication helpers; no model computation.

The worker owns one immutable version directory. Only its supervising parent
may publish, after observing exit code zero and validating input/output hashes.
Publication temporarily invalidates the global manifest, writes the contracts,
checks their bytes, and commits the manifest last. Previous files are retained
for rollback. The optional selfcheck uses an isolated fixture workspace.
"""

from __future__ import annotations

from contextlib import contextmanager
from datetime import datetime, timezone
import copy
import hashlib
import importlib.metadata
import json
import os
from pathlib import Path
import platform
import subprocess
import sys
import uuid

SOURCE = Path(__file__).resolve()
ROOT = SOURCE.parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(SOURCE.read_bytes()).hexdigest()

def utc_now():
    return datetime.now(timezone.utc).isoformat()

def sha256(path):
    digest = hashlib.sha256()
    with Path(path).open('rb') as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b''):
            digest.update(block)
    return digest.hexdigest()

def within(root, path):
    root, path = Path(root).resolve(), Path(path).resolve()
    if not path.is_relative_to(root):
        raise ValueError('Path is outside the intended workspace: ' + str(path))
    return path

def file_record(path, root=ROOT):
    path = within(root, path)
    before = path.stat()
    digest = sha256(path)
    after = path.stat()
    if (before.st_size, before.st_mtime_ns) != (after.st_size, after.st_mtime_ns):
        raise RuntimeError('File changed while hashing: ' + str(path))
    return {'path': path.relative_to(Path(root).resolve()).as_posix(),
            'bytes': after.st_size, 'sha256': digest, 'exists': True}
```

```

def assert_records(records, root=ROOT):
    for expected in records:
        path = within(root, Path(root) / expected['path'])
        actual = file_record(path, root)
        for key in ('sha256', 'bytes'):
            if key in expected and actual[key] != expected[key]:
                raise RuntimeError('Artifact/input changed: ' + expected['path'])

def json_bytes(value):
    return (json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) +
'\n').encode('utf-8')

def atomic_bytes(path, data):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_name(path.name + '.tmp-' + uuid.uuid4().hex)
    try:
        with temporary.open('xb') as stream:
            stream.write(data)
            stream.flush()
            os.fsync(stream.fileno())
        os.replace(temporary, path)
    finally:
        if temporary.exists():
            temporary.unlink()

def write_json(path, value):
    atomic_bytes(path, json_bytes(value))

def exclusive_json(path, value):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open('xb') as stream:
        stream.write(json_bytes(value))
        stream.flush()
        os.fsync(stream.fileno())

def runtime_record():
    versions = {}
    for name in ('numpy', 'scipy', 'openpyxl', 'matplotlib'):
        versions[name] = importlib.metadata.version(name)
    return {'executable': sys.executable, 'implementation': platform.python_implementation(),
            'version': platform.python_version(), 'platform': platform.platform(),
            'installed_library_versions': versions,
            'thread_environment': {name: os.environ.get(name) for name in ('OPENBLAS_NUM_THREADS',
'OMP_NUM_THREADS')},
            'scope': 'Installed distributions; actual imported module paths are recorded by
worker.'}

def pointer(document, json_pointer):
    current = document
    if not json_pointer.startswith('/'):
        raise ValueError('Expected an absolute JSON pointer')
    for token in json_pointer[1:].split('/'):
        token = token.replace('~1', '/').replace('~0', '~')
        current = current[int(token)] if isinstance(current, list) else current[token]
    return current

def validate_metric_evidence(metrics, root=ROOT):

```

```

cache = {}
for metric in metrics:
    path = within(root, Path(root) / metric['evidence_path'])
    if path not in cache:
        cache[path] = json.loads(path.read_text(encoding='utf-8'))
        actual = pointer(cache[path], metric['evidence_json_pointer'])
        if actual != metric['value']:
            raise ValueError('Metric does not equal its saved evidence: ' + metric['metric_name'])
    return {'status': 'PASS', 'checked_metric_count': len(metrics),
            'scope': 'Exact equality to the identified saved JSON value, not physical accuracy.'}

def figure_artifact_records(figure, root=ROOT):
    candidates = []
    primary = figure.get('path') or figure.get('output_path')
    if primary:
        candidates.append({key: figure[key] for key in ('path', 'bytes', 'sha256') if key in
figure})
        if figure.get('path') else {'path': primary})
    artifacts = figure.get('artifacts', {})
    candidates.extend(artifacts.values() if isinstance(artifacts, dict) else artifacts)
    for name in ('plot_data', 'metadata'):
        if isinstance(figure.get(name), dict) and figure[name].get('path'):
            candidates.append(figure[name])
    checked = {}
    for expected in candidates:
        if not isinstance(expected, dict) or not expected.get('path'):
            raise ValueError('Invalid figure artifact record')
        actual = file_record(Path(root) / expected['path'], root)
        if not actual['bytes']:
            raise ValueError('Empty figure artifact: ' + actual['path'])
        if expected.get('sha256') and actual['sha256'] != expected['sha256']:
            raise RuntimeError('Existing figure hash mismatch: ' + actual['path'])
        checked[actual['path']] = actual
    if not checked:
        raise ValueError('Computed figure has no usable artifact')
    return list(checked.values())

def merge_figure_index(existing, new_figures, root=ROOT):
    merged, omitted, replaced, preserved = {}, [], [], []
    for figure in (existing or {}).get('figures', []):
        status = str(figure.get('status', '')).lower()
        if (figure.get('placeholder') or figure.get('planned') or
            status in ('planned', 'pending', 'not_started') or figure.get('ok') is False):
            omitted.append({'figure_id': figure.get('figure_id'), 'reason': 'not an existing
usable computed figure'})
            continue
        figure_artifact_records(figure, root)
        if not figure.get('figure_id'):
            raise ValueError('Existing computed figure needs figure_id')
        merged[figure['figure_id']] = copy.deepcopy(figure)
        preserved.append(figure['figure_id'])
    seen = set()
    for figure in new_figures:
        identity = figure['figure_id']
        if identity in seen:
            raise ValueError('Duplicate new figure_id: ' + identity)
        seen.add(identity)
        figure_artifact_records(figure, root)
        if identity in merged:
            replaced.append(identity)
            merged[identity] = copy.deepcopy(figure)
    return list(merged.values()), {'preserved_existing_ids': [v for v in preserved if v not in
replaced],
                                'replaced_same_ids': replaced, 'omitted_noncomputed_entries': omitted,

```

```

        'prior_index_retained_in_publication_backup': True}

@contextmanager
def publication_lock(root, run_id):
    path = within(root, Path(root) / 'paper_output/results/.production_publish.lock')
    identity = {'run_id': run_id, 'pid': os.getpid(), 'created_at': utc_now(), 'token':
uuid.uuid4().hex}
    exclusive_json(path, identity)
    try:
        yield
    finally:
        if path.exists() and json.loads(path.read_text(encoding='utf-8')) == identity:
            path.unlink()

def publish_transaction(root, directory, updates, manifest):
    """Caller holds publication_lock and has verified an actual worker exit 0.

    Updates maps root-relative JSON paths to their final documents. The old
    global manifest is invalidated before any contract changes; it is restored
    on rollback. Only the final manifest constitutes committed publication.
    """
    root, directory = Path(root).resolve(), within(root, directory)
    if manifest['runs'][0]['returncode'] != 0:
        raise ValueError('Cannot publish a nonzero worker return code')
    manifest_name = 'paper_output/results/run_manifest.json'
    if manifest_name in updates:
        raise ValueError('The manifest must be committed last')
    paths = [*updates, manifest_name]
    previous = {}
    backup = directory / 'publication_previous'
    for relative in paths:
        target = within(root, root / relative)
        previous[relative] = target.read_bytes() if target.exists() else None
        if previous[relative] is not None:
            atomic_bytes(backup / relative, previous[relative])
    write_json(backup / 'inventory.json', {'saved_at': utc_now(),
'files': [{'path': p, 'previously_existed': b is not None} for p, b in
previous.items()]})
    intended = {relative: json_bytes(document) for relative, document in updates.items()}
    changed = False
    try:
        # Readers must require PASS and the recorded hashes, never just the
        # existence of model_results.json or an old workflow-memory status.
        write_json(root / manifest_name, {'status': 'PUBLISHING', 'run_id': manifest['runs'][0]
['run_id'],
'generated_at': utc_now(), 'version_directory':
directory.relative_to(root).as_posix()})
        changed = True
        for relative, content in intended.items():
            atomic_bytes(root / relative, content)
        for relative, content in intended.items():
            if sha256(root / relative) != hashlib.sha256(content).hexdigest():
                raise RuntimeError('Published contract bytes differ: ' + relative)
        # This is an actual read of every listed output, including global copies.
        assert_records(manifest['runs'][0]['output_artifacts'], root)
        assert_records(manifest['runs'][0].get('input_files', []), root)
        font = manifest['runs'][0].get('font')
        if font and sha256(font['path']) != font['sha256']:
            raise RuntimeError('Font changed before publication commit')
        write_json(directory / 'run_manifest.json', manifest)
        write_json(root / manifest_name, manifest)
    except Exception as error:
        rollback_errors = []
        if changed:

```

```

# Restore the manifest after the other files, so no old PASS is
# visible while its old contracts are still being restored.
for relative in updates:
    try:
        target = root / relative
        if previous[relative] is None:
            if target.exists():
                target.unlink()
        else:
            atomic_bytes(target, previous[relative])
    except Exception as rollback_error:
        rollback_errors.append({'path': relative, 'error': repr(rollback_error)})
if not rollback_errors:
    try:
        target = root / manifest_name
        if previous[manifest_name] is None:
            if target.exists():
                target.unlink()
        else:
            atomic_bytes(target, previous[manifest_name])
    except Exception as rollback_error:
        rollback_errors.append({'path': manifest_name, 'error': repr(rollback_error)})
if rollback_errors:
    write_json(root / manifest_name, {'status': 'FAILED_PUBLICATION', 'error':
repr(error),
        'rollback_errors': rollback_errors, 'generated_at': utc_now()})
    write_json(directory / 'publication_failure.json', {'status': 'FAIL', 'error':
repr(error),
        'rollback_errors': rollback_errors, 'prior_publication_restored': changed and not
rollback_errors,
        'generated_at': utc_now()})
    raise

def measured_process(command, cwd, log_path):
    """Observe a real child exit; preserve its complete merged stdout/stderr."""
    started = utc_now()
    stream_error = None
    with Path(log_path).open('x', encoding='utf-8') as log:
        process = subprocess.Popen(command, cwd=cwd, stdout=subprocess.PIPE,
stderr=subprocess.STDOUT,
            text=True, encoding='utf-8', errors='replace')
        try:
            for line in process.stdout:
                log.write(line)
                log.flush()
                print(line, end='', flush=True)
        except BaseException as error:
            stream_error = error
            # Keep draining the child pipe so a log/console error cannot leave
            # it blocked on a full stdout buffer while the parent waits.
            for unused in process.stdout:
                pass
        finally:
            # The parent must not release publication responsibility while an
            # unobserved child is still writing. No child is killed here.
            code = process.wait()
            process.stdout.close()
    result = {'pid': process.pid, 'returncode': code, 'started_at': started,
        'finished_at': utc_now(), 'command': subprocess.list2cmdline(command),
        'argv': command, 'cwd': str(Path(cwd).resolve())}
    if stream_error is not None:
        result['stdout_capture_error'] = repr(stream_error)
    # Preserve an observed exit even if forwarding output failed and the caller
    # will raise instead of receiving this function's return value.
    write_json(Path(log_path).with_name('process_result.json'), result)

```

```

if stream_error is not None:
    raise RuntimeError('Child exited, but stdout capture failed') from stream_error
return result

def entry_sequence_selfcheck(directory):
    """Tiny injected services verify lifetime/ownership; no PDE is solved."""
    from types import SimpleNamespace
    from unittest.mock import patch
    import numpy as np
    import run_modeling as entry

    source = file_record(entry.SOURCE)
    start = {'input_files': [source], 'runtime': {}, 'font': {}, 'worker_command': 'fixture only;
no model solve'}
    global_paths = [ROOT / p for p in entry.CONTRACT_TARGETS.values()]
    global_before = {str(p): sha256(p) if p.exists() else None for p in global_paths}
    active, events, identities = set(), [], {}
    class FixtureRun:
        def __init__(self, name, intervals):
            if active:
                raise AssertionError('Two Runs are retained simultaneously')
            self.name, self.intervals = name, intervals
            self.end_s = 1800. if name == 'Q1' else 15002.
            active.add(id(self))
            events.append('solve:' + name)
        def fields(self, times, material_x):
            shape = (len(times), len(material_x))
            return np.full(shape, 310.), np.full(shape, .1499)
        def close(self):
            events.append('close:' + self.name)
            active.remove(id(self))
    def save_fixture(run, output):
        return {'settings': {'intervals': run.intervals},
                'diagnostics': {'max_mass_balance_abs_kg_per_kg': 0.}}
    def fixture_completion(run):
        return {'reported_time_s': 15000.12, 'reported_drying_time_h': 15000.12 / 3600.,
                'critical_event_h': 15000. / 3600., 'max_C_at_reported_time': .1499}
    def export_fixture(run, qid, output):
        events.append('export:' + qid)
        identities[qid] = id(run)
        path = output / (qid + '_fixture.json')
        write_json(path, {'fixture_only': True, 'qid': qid})
        return {'question_id': qid, 'artifacts': [file_record(path)], 'size': {},
                'manifest_path': str(path)}
    def validate_fixture(exports, runs):
        export = exports[0]
        path = Path(export['manifest_path']).with_suffix('.validation.json')
        write_json(path, {'fixture_only': True})
        return {'status': 'PASS', 'fully_verified_with_live_Run': True, 'exports':
[{'report_path': str(path)}]}
    def plot_fixture(run, qid, output):
        events.append('plot:' + qid)
        path = output / (qid + '_fixture.bin')
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_bytes(b'fixture bytes; not a scientific figure')
        return {'figure_id': 'fixture_' + qid, 'question_id': qid, **file_record(path)}
    mappings = {'drying_core': SimpleNamespace(ROOT=ROOT, save_run=save_fixture),
                'q3_model': SimpleNamespace(completion=fixture_completion),
                'export_outputs': SimpleNamespace(export_question=export_fixture,
validate_exports=validate_fixture),
                'publication_plots': SimpleNamespace(make_question_plot=plot_fixture)}
    for key, name in [('Q1', 'q1_model'), ('Q23', 'q2_model'), ('Q4', 'q4_model')]:
        mappings[name] = SimpleNamespace(solve=lambda n, key=key: FixtureRun(key, n))
    original_import = importlib.import_module
    def importer(name, *args, **kwargs):

```

```

        return mappings[name] if name in mappings else original_import(name, *args, **kwargs)
    args = SimpleNamespace(version='fixture_sequence', n1=2, n23=2, n4=2, review=False)
    with patch.object(entry, 'assert_dependencies'), patch.object(entry,
'imported_model_records', return_value=[]), \
        patch.object(entry.importlib, 'import_module', side_effect=importer):
        entry.worker(args, directory / 'entry_fixture_success', start)
        assert not active and identities['Q2'] == identities['Q3']
        assert events == ['solve:Q1', 'export:Q1', 'plot:Q1', 'close:Q1',
            'solve:Q23', 'export:Q2', 'plot:Q2', 'export:Q3', 'plot:Q3', 'close:Q23',
            'solve:Q4', 'export:Q4', 'plot:Q4', 'close:Q4']
        def fail_q3(run, qid, output):
            if qid == 'Q3':
                raise RuntimeError('injected fixture Q3 export failure')
            return export_fixture(run, qid, output)
        mappings['export_outputs'].export_question = fail_q3
        try:
            entry.worker(args, directory / 'entry_fixture_failure', start)
        except RuntimeError as error:
            assert 'injected fixture' in str(error)
        else:
            raise AssertionError('Fixture failure was not propagated')
        assert not active and events[-1] == 'close:Q23'
    global_after = {str(p): sha256(p) if p.exists() else None for p in global_paths}
    assert global_after == global_before
    return ['entry_one_run_at_a_time_and_Q2_Q3_same_identity',
        'entry_closes_run_after_export_failure', 'worker_does_not_publish_global_contracts']

```

```

def selfcheck():
    """Exercise failure paths on tiny files and real exit-0/exit-7 subprocesses."""
    import tempfile
    from unittest.mock import patch

    base = ROOT / 'notes/A-modeling/2026-09-10/data/production_entry_contract_selfcheck'
    base.mkdir(parents=True, exist_ok=True)
    directory = Path(tempfile.mkdtemp(prefix='check-', dir=base))
    fixture = directory / 'fixture'
    fixture.mkdir()
    checks = []
    original = fixture / 'input.txt'
    original.write_text('observed bytes', encoding='utf-8')
    rec = file_record(original, fixture)
    assert_records([rec], fixture)
    original.write_text('changed bytes', encoding='utf-8')
    try:
        assert_records([rec], fixture)
    except RuntimeError:
        checks.append('hash_mutation_rejected')
    else:
        raise AssertionError('Changed input was accepted')
    lock = fixture / 'launch.json'
    exclusive_json(lock, {'version': 'one'})
    try:
        exclusive_json(lock, {'version': 'two'})
    except FileExistsError:
        checks.append('duplicate_version_marker_rejected')
    else:
        raise AssertionError('Existing marker overwritten')
    evidence = fixture / 'summary.json'
    write_json(evidence, {'metric_samples': {'Q3': {'C': [[.1499]], 'time_s': 123.48}}})
    metric = {'metric_name': 'center_C', 'evidence_path': 'summary.json',
        'evidence_json_pointer': '/metric_samples/Q3/C/0/0', 'value': .1499}
    validate_metric_evidence([metric], fixture)
    metric['value'] = .1
    try:
        validate_metric_evidence([metric], fixture)

```

```

except ValueError:
    checks.append('metric_mismatch_rejected')
else:
    raise AssertionError('Wrong evidence accepted')
image_path = fixture / 'observed.png'
image_path.write_bytes(b'fixture-artifact-only-not-a-real-figure')
figure = {'figure_id': 'observed', 'path': 'observed.png', 'status': 'computed',
          'sha256': sha256(image_path), 'placeholder': False}
merged, note = merge_figure_index({'figures': [figure, {'figure_id': 'planned', 'planned':
True}], [], fixture)
assert len(merged) == 1 and note['preserved_existing_ids'] == ['observed']
checks.append('existing_artifact_preserved_planned_not_promoted')
old_contract = {'generation': 'old'}
write_json(fixture / 'paper_output/results/model_results.json', old_contract)
write_json(fixture / 'paper_output/results/run_manifest.json', {'status': 'PASS', 'old':
True})
version = fixture / 'paper_output/results/production/test'
version.mkdir(parents=True)
updates = {'paper_output/results/model_results.json': {'generation': 'new'}}
intended_bytes = json_bytes(updates['paper_output/results/model_results.json'])
intended_record = {'path': 'paper_output/results/model_results.json', 'bytes':
len(intended_bytes),
                  'sha256': hashlib.sha256(intended_bytes).hexdigest()}
manifest = {'status': 'PASS', 'runs': [{'run_id': 'test', 'returncode': 0,
    'input_files': [file_record(original, fixture)], 'output_artifacts': [intended_record]]}
real_write = atomic_bytes
fired = False
def fail_contract_once(path, data):
    nonlocal fired
    if Path(path) == fixture / 'paper_output/results/model_results.json' and not fired:
        fired = True
        raise OSError('injected fixture publication failure')
    real_write(path, data)
with patch.object(sys.modules[__name__], 'atomic_bytes', fail_contract_once):
    try:
        publish_transaction(fixture, version, updates, manifest)
    except OSError:
        pass
    else:
        raise AssertionError('Injected failure was swallowed')
    assert json.loads((fixture / 'paper_output/results/model_results.json').read_text()) ==
old_contract
    assert json.loads((fixture / 'paper_output/results/run_manifest.json').read_text()) ==
{'status': 'PASS', 'old': True}
    checks.append('publication_failure_rolls_back_old_contract_and_manifest')
    publish_transaction(fixture, version, updates, manifest)
    assert json.loads((fixture / 'paper_output/results/model_results.json').read_text()) ==
{'generation': 'new'}
    checks.append('successful_publication_commits_manifest_last')
    exits = [measured_process([sys.executable, '-c', 'import sys; print("tiny-process");
sys.exit(' + str(code) + ')'],
                          fixture, directory / ('exit-' + str(code) + '.log')) for code in (0,
7)]
    assert [p['returncode'] for p in exits] == [0, 7]
    checks.append('actual_process_zero_and_nonzero_exits_observed')
    checks.extend(entry_sequence_selfcheck(directory))
    report = {'status': 'PASS', 'scope': 'Isolated tiny-file contracts; no model solve or real
production publication.',
            'checks': checks, 'processes': exits, 'source': file_record(SOURCE), 'generated_at':
utc_now(),
            'visual_studio_gui': 'pending', 'human_review': 'pending'}
    write_json(directory / 'selfcheck.json', report)
    print(json.dumps({'status': report['status'], 'checks': len(checks), 'report':
str(directory / 'selfcheck.json')}))

```

```
if __name__ == '__main__':  
    if sys.argv[1:] != ['--selfcheck']:  
        raise SystemExit('Use --selfcheck for isolated contract validation only')  
    if Path.cwd().resolve() != ROOT:  
        raise SystemExit('Use the competition workspace as cwd')  
    selfcheck()
```

E 验证程序与历史版本完整清单

本节给出正文及本附录实际采用的检验程序、相应历史求解器和运行依赖，共 31 份、6440 行；与附录 D 的 11 份、2994 行合计 42 份、9434 行。历史源码按证据原样保留，各文件前的采用范围和勘误限定其在本稿中的用途。历史验证与当前生产结果使用不同版本时，须按所列原运行槽位组织文件，不能任意混用。

E.1 validate_bessel.py: Robin - Bessel 解析热场、级数截断自检

文件: paper_output/code/modeling/validate_bessel.py; 共 345 行。采用范围: 第 5/6 章 Q1 热场及附录解析基准。独立常 D 水分特例仅作数值检验, 不替代非线性药材答案。

```
"""Independent constant-coefficient cylinder / Robin / Duhamel benchmark.

For temperature this is the Q1 heat equation, with the *observed piecewise
linear* room boundary. It is NOT an analytic solution of nonlinear moisture
diffusion. A frozen-D moisture wrapper is an algorithm-verification case.

The exact modal convolution is evaluated on every piecewise-linear interval;
there is no extra numerical integration time step. Finite-volume comparisons
can request annular volume averages rather than compare averages to points.

All inputs use SI units; temperatures are Kelvin. Array shape is (time, space).
The standalone command writes only its own benchmark/selfcheck artifacts and
never rewrites the production run_manifest or core solver.
"""
from __future__ import annotations

import argparse
import csv
import hashlib
import json
import platform
import sys
from datetime import datetime, timezone
from functools import lru_cache
from pathlib import Path

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import quad
from scipy.optimize import brentq
from scipy.special import j0, j1, jn_zeros

FloatArray = NDArray[np.float64]

def _vector(value: ArrayLike, name: str) -> FloatArray:
    a = np.atleast_1d(np.asarray(value, dtype=float))
    if a.ndim != 1 or a.size == 0 or not np.all(np.isfinite(a)):
        raise ValueError(f"{name} must be a nonempty finite 1-D array")
    return a

@lru_cache(maxsize=24)
def _eigen_cached(biot: float, n_terms: int) -> tuple[FloatArray, FloatArray]:
    if not np.isfinite(biot) or biot <= 0:
        raise ValueError("Biot must be finite and positive")
    if not isinstance(n_terms, (int, np.integer)) or n_terms < 1:
        raise ValueError("n_terms must be a positive integer")
    # The nth root lies between J1's (n-1)st and J0's nth positive zeros;
    # the first interval starts at zero. This brackets every root separately.
    upper = jn_zeros(0, n_terms)
    lower = np.r_[0.0, jn_zeros(1, n_terms-1)] if n_terms > 1 else np.array([0.0])
    roots = np.array([
        brentq(lambda z: z*j1(z)-biot*j0(z), lo, hi, xtol=1e-13, rtol=1e-14)
        for lo, hi in zip(lower, upper)
    ])

```

```

a = 2*j1(roots) / (roots*(j0(roots)**2+j1(roots)**2))
roots.setflags(write=False)
a.setflags(write=False)
return roots, a

def robin_eigenpairs(biot: float, n_terms: int = 120) -> tuple[FloatArray, FloatArray]:
    """Return mu_n and A_n for mu*j1(mu)=Bi*j0(mu), uniform initial data."""
    mu, a = _eigen_cached(float(biot), int(n_terms))
    return mu.copy(), a.copy()

def _validate_inputs(times_s, env_t, env_values, radius, diffusivity, biot):
    t = _vector(times_s, "times_s")
    knots = _vector(env_t, "env_t")
    values = _vector(env_values, "env_values")
    if len(knots) != len(values) or len(knots) < 2:
        raise ValueError("Boundary needs matching arrays with at least two knots")
    if abs(knots[0]) > 1e-12 or np.any(np.diff(knots) <= 0):
        raise ValueError("Boundary knots must start at t=0 and strictly increase")
    if np.any(t < 0):
        raise ValueError("Negative evaluation times are not permitted")
    if np.max(t) > knots[-1] + 1e-10:
        raise ValueError("Evaluation exceeds boundary observations: supply an explicit
extension")
    if not np.isfinite(radius) or radius <= 0 or not np.isfinite(diffusivity) or diffusivity <=
0:
        raise ValueError("radius and diffusivity must be finite and positive")
    if not np.isfinite(biot) or biot < 0:
        raise ValueError("Biot must be finite and nonnegative")
    return t, knots, values

def _modal_history(
    times: FloatArray, knots: FloatArray, values: FloatArray,
    rates: FloatArray, initial_value: float,
) -> FloatArray:
    """Return z_n(t) before multiplication by A_n.

    z_n' + lambda_n*z_n = -g', z_n(0)=initial-g(0).
    For a clipped interval [a,b] with slope m, its positive convolution is
    m*exp[-lambda*(t-b)]*(-expm1[-lambda*(b-a)]/lambda.
    """
    z = (initial_value-values[0])*np.exp(-times[:, None]*rates[None, :])
    slopes = np.diff(values)/np.diff(knots)
    for lo, hi, slope in zip(knots[:-1], knots[1:], slopes):
        if slope == 0:
            continue
        selected = times > lo
        ts = times[selected]
        end = np.minimum(ts, hi)
        duration = end-lo
        contribution = (
            np.exp(-(ts-end)[:, None]*rates[None, :])
            * (-np.expm1(-duration[:, None]*rates[None, :]))
            / rates[None, :]
        )
        z[selected] -= slope*contribution
    return z

def _space_modes(mu: FloatArray, radii_m: ArrayLike, radius: float, cell_edges_m=None) ->
FloatArray:
    if cell_edges_m is None:
        x = _vector(radii_m, "radii_m")/radius
        if np.any(x < 0) or np.any(x > 1+1e-12):

```

```

        raise ValueError("radii must lie inside the cylinder")
    return j0(mu[:, None]*x[None, :])
edges = _vector(cell_edges_m, "cell_edges_m")/radius
if len(edges) < 2 or np.any(np.diff(edges) <= 0) or edges[0] < 0 or edges[-1] > 1+1e-12:
    raise ValueError("cell edges must strictly increase inside [0,radius]")
lo, hi = edges[:-1], edges[1:]
return 2*(hi[None, :]*j1(mu[:, None]*hi[None, :])
        - lo[None, :]*j1(mu[:, None]*lo[None, :])) / (mu[:, None]*(hi*hi-lo*lo)[None, :])

def bessell_diffusion(
    times_s: ArrayLike, radii_m: ArrayLike, env_t: ArrayLike,
    env_values: ArrayLike, *, radius: float, diffusivity: float,
    biot: float, initial_value: float, n_terms: int = 120,
    cell_edges_m: ArrayLike | None = None,
) -> FloatArray:
    """Constant-coefficient radial diffusion with  $-a*f_r=b*(f_s-g)$ .

    ``biot=b*radius/a``. The material initial field is uniform. No automatic
    boundary extrapolation is performed. If ``cell_edges_m`` is supplied,
    ``radii_m`` is ignored and the output contains annular volume averages.
    """
    t, knots, values = _validate_inputs(times_s, env_t, env_values, radius, diffusivity, biot)
    if not np.isfinite(initial_value):
        raise ValueError("initial_value must be finite")
    if not isinstance(n_terms, (int, np.integer)) or n_terms < 1:
        raise ValueError("n_terms must be a positive integer")
    if biot == 0:
        # An insulating cylinder with uniform initial data never responds to g.
        nspace = len(_vector(cell_edges_m, "cell_edges_m"))-1 if cell_edges_m is not None else
len(_vector(radii_m, "radii_m"))
        _space_modes(np.array([1.0]), radii_m, radius, cell_edges_m)
        return np.full((len(t), nspace), initial_value)
    mu, a = _eigen_cached(float(biot), int(n_terms))
    rates = diffusivity*mu*mu/(radius*radius)
    spatial = _space_modes(mu, radii_m, radius, cell_edges_m)
    history = _modal_history(t, knots, values, rates, initial_value)
    result = np.interp(t, knots, values)[:, None] + (history*a[None, :]) @ spatial
    # The exact prescribed initial state avoids finite-series initial traces,
    # especially for a step mismatch between initial material and ambient.
    result[t == 0] = initial_value
    return result

def bessell_temperature(
    times_s: ArrayLike, radii_m: ArrayLike, env_t: ArrayLike,
    env_T: ArrayLike, radius: float = 0.02, h: float = 25.0,
    k: float = 0.36, rho: float = 820.0, cp: float = 2600.0,
    n_terms: int = 120, initial_temperature_K: float = 301.15,
    cell_edges_m: ArrayLike | None = None,
) -> FloatArray:
    """Q1 constant-property heat benchmark; env_T and result are Kelvin."""
    if any(not np.isfinite(v) or v <= 0 for v in (k,rho,cp)) or h < 0:
        raise ValueError("k,rho,cp must be positive and h nonnegative")
    return bessell_diffusion(
        times_s, radii_m, env_t, env_T, radius=radius,
        diffusivity=k/(rho*cp), biot=h*radius/k,
        initial_value=initial_temperature_K, n_terms=n_terms,
        cell_edges_m=cell_edges_m,
    )

def bessell_moisture(
    times_s: ArrayLike, radii_m: ArrayLike, env_t: ArrayLike,
    env_Ceq: ArrayLike, diffusion_coefficient: float,
    radius: float = 0.02, beta: float = 8e-7,

```

```

    initial_moisture: float = 2.55, n_terms: int = 240,
    cell_edges_m: ArrayLike | None = None,
) -> FloatArray:
    """Frozen-D linear benchmark, NOT the nonlinear production moisture PDE."""
    if diffusion_coefficient <= 0 or beta < 0:
        raise ValueError("Frozen diffusion coefficient must be positive, beta nonnegative")
    return_bessel_diffusion(
        times_s, radii_m, env_t, env_Ceq, radius=radius,
        diffusivity=diffusion_coefficient, biot=beta*radius/diffusion_coefficient,
        initial_value=initial_moisture, n_terms=n_terms,
        cell_edges_m=cell_edges_m,
    )

def _sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def selfcheck(env_t: FloatArray, env_T: FloatArray) -> dict:
    """Independent quadrature, energy identity and truncation diagnostics.

    This checks the analytic implementation. Production solver comparison
    remains separate and is performed by the main run using this module.
    """
    bi = 25*.02/.36
    mu, a = robin_eigenpairs(bi, 120)
    residual = mu*j1(mu)-bi*j0(mu)
    relative_eigen_residual = np.max(np.abs(residual)/(1+np.abs(mu*j1(mu))+np.abs(bi*j0(mu))))

    # Independently integrate each forcing interval with adaptive quadrature,
    # testing heating, cooling, a flat interval, and targets inside intervals.
    knots = np.array([0.,17.,52.,130.])
    boundary = np.array([301.15,305.,303.,303.])
    targets = np.array([8.,17.,40.,78.,130.])
    rates = np.array([1e-6,.002,.2])
    z = _modal_history(targets,knots,boundary,rates,299.)
    independent = np.empty_like(z)
    for i,t in enumerate(targets):
        for j,lam in enumerate(rates):
            integral = 0.
            for lo,hi,gl,gh in zip(knots[:-1],knots[1:],boundary[:-1],boundary[1:]):
                if t > lo:
                    slope = (gh-gl)/(hi-lo)
                    integral += quad(lambda s: slope*np.exp(-lam*(t-
s)),lo,min(t,hi),epsabs=1e-12,epsrel=1e-12)[0]
            independent[i,j] = (299.-boundary[0])*np.exp(-lam*t)-integral
    convolution_error = float(np.max(np.abs(z-independent)))

    # Spatial annular averaging compared with independent numerical quadrature.
    edges = np.array([0.,.001,.006,.02])
    averaged = _space_modes(mu[:8], [0.], .02,edges)
    quad_average = np.empty_like(averaged)
    for n,m in enumerate(mu[:8]):
        for i,(lo,hi) in enumerate(zip(edges[:-1]/.02,edges[1:]/.02)):
            quad_average[n,i] = 2*quad(lambda x: x*j0(m*x),lo,hi,epsabs=1e-12)[0]/(hi*hi-lo*lo)
    average_error = float(np.max(np.abs(averaged-quad_average)))

    # Constant-boundary step energy identity: dTbar/dt=2h/(rho*cp*R)*(g-Ts).
    alpha=.36/(820*2600)
    lam=alpha*mu**2/.02**2
    e=np.exp(-100*lam)
    mean_basis=2*j1(mu)/mu
    dmean=np.sum(a*(-lam)*(-20)*e*mean_basis)
    surface=321.15+np.sum(a*(-20)*e*j0(mu))
    influx=2*25/(820*2600*.02)*(321.15-surface)
    energy_error=abs(dmean-influx)

```

```

ts=np.array([0.,1.,10.,100.,300.,600.,900.,1200.,1500.,1800.])
rs=np.linspace(0,.02,41)
t120=bessel_temperature(ts,rs,env_t,env_T,n_terms=120)
t240=bessel_temperature(ts,rs,env_t,env_T,n_terms=240)
t480=bessel_temperature(ts,rs,env_t,env_T,n_terms=480)
error120_240=float(np.max(np.abs(t120-t240)))
error240_480=float(np.max(np.abs(t240-t480)))
equilibrium=bessel_temperature([0.,1.,100.],[0.,.01,.02],[0.,100.],[301.15,301.15])
equilibrium_error=float(np.max(np.abs(equilibrium-301.15)))
insulating=bessel_temperature([0.,50.,100.],[0.,.01,.02],[0.,100.],[301.15,323.15],h=0)
insulation_error=float(np.max(np.abs(insulating-301.15)))

checks = [
    ("robin_eigen_relative_residual",float(relative_eigen_residual),1e-10,"1"),
    ("piecewise_convolution_vs_adaptive_quadrature",convolution_error,1e-10,"K"),
    ("annular_average_vs_adaptive_quadrature",average_error,1e-10,"1"),
    ("constant_boundary_step_energy_identity",float(energy_error),1e-10,"K/s"),
    ("Q1_observed_boundary_120_vs_240_terms",error120_240,1e-4,"K"),
    ("Q1_observed_boundary_240_vs_480_terms",error240_480,1e-4,"K"),
    ("uniform_equilibrium",equilibrium_error,1e-12,"K"),
    ("insulating_boundary",insulation_error,1e-12,"K"),
]
return {
    "status":"PASS" if all(v<=limit for _,v,limit,_ in checks) else "FAIL",
    "scope":"analytic benchmark implementation selfcheck; production FVM comparison is
separate",
    "checks":[{"name":name,"value":v,"tolerance":limit,"unit":unit,"passed":v<=limit} for
name,v,limit,unit in checks],
    "series_note":"120/240/480 differences estimate truncation sensitivity at sampled
locations and times, not a rigorous uniform error bound.",
    "no_internal_experimental_validation":True,
    "visual_studio_gui":"pending",
    "human_review":"pending",
}

def main() -> int:
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--environment-csv",default="paper_output/data_cleaned/
A_environment_observed.csv")
    parser.add_argument("--output-dir",default="paper_output/results/bessel_validation")
    args=parser.parse_args()
    root=Path.cwd().resolve()
    if root != Path(r"D:\Document\数学建模\2026CUMCM").resolve():
        raise RuntimeError("Run only from the 2026CUMCM workspace root")
    source=(root/args.environment_csv).resolve()
    with source.open(encoding="utf-8-sig",newline="") as fh:
        rows=list(csv.DictReader(fh))
    env_t=np.array([float(r["time_s"]) for r in rows])
    env_T=np.array([float(r["temperature_K"]) for r in rows])
    env_C=np.array([float(r["air_moisture_kg_per_kg"]) for r in rows])
    report=selfcheck(env_t,env_T)
    out=(root/args.output_dir).resolve()
    if not out.is_relative_to(root):
        raise RuntimeError("Output must remain inside the competition workspace")
    out.mkdir(parents=True,exist_ok=True)
    paper_t=np.array([100.,300.,600.,900.,1200.,1500.,1800.])
    paper_r=np.array([0.,.005,.01,.015,.02])
    temperature=bessel_temperature(paper_t,paper_r,env_t,env_T,n_terms=240)
    frozen_D=7e-9*np.exp(-.89/2.55)
    frozen_water=bessel_moisture(paper_t,paper_r,env_t,env_C,float(frozen_D),n_terms=240)
    artifacts=[]
    for filename,field,label in [
        ("q1_exact_heat_paper_points.csv",temperature-273.15,"temperature_C"),
        ("frozen_D_moisture_benchmark_points.csv",frozen_water,"moisture_kg_per_kg"),

```

```

]:
    path=out/filename
    with path.open("w",encoding="utf-8",newline="") as fh:
        writer=csv.writer(fh)
        writer.writerow(["time_s","radius_m",label])
        for i,t in enumerate(paper_t):
            for j,r in enumerate(paper_r):
                writer.writerow([format(t,".17g"),format(r,".17g"),format(field[i,j],".17g")])

artifacts.append({"path":path.relative_to(root).as_posix(),"bytes":path.stat().st_size,"sha256":_
sha256(path)})
    report.update({
        "schema_version":"1.0", "question_id":"Q1",
        "generated_at":datetime.now(timezone.utc).isoformat(),
        "generated_by":"paper_output/code/modeling/validate_bessel.py",
        "frozen_moisture_diffusion_coefficient_m2_s":float(frozen_D),
        "frozen_moisture_note":"This table is a constant-D algorithm benchmark and must not
replace nonlinear Q1 result1 moisture values.",
        "execution_provenance":{
            "source_code_path":Path(__file__).resolve().relative_to(root).as_posix(),
            "source_code_sha256":_sha256(Path(__file__)),
            "run_command":sys.executable+" -B "+" ".join(sys.argv),
            "run_exit_code":0 if report["status"]=="PASS" else 1,
            "python_version":platform.python_version(),
            "input_artifacts":
[{"path":source.relative_to(root).as_posix(),"sha256":_sha256(source)}],
            "output_artifacts":artifacts,
        },
    })
    report_path=out/"bessel_selfcheck.json"
    report_path.write_text(json.dumps(report,ensure_ascii=False,indent=2)+"\n",encoding="utf-8")

print(json.dumps({"status":report["status"],"checks":report["checks"],"report":str(report_path)},
ensure_ascii=False,indent=2))
    return 0 if report["status"]=="PASS" else 1

if __name__ == "__main__":
    raise SystemExit(main())

```

E.2 compare_analytic.py: 早期解析对照及常D特例筛选入口

文件: paper_output/code/modeling/compare_analytic.py; 共 242 行。采用范围: 数值过程附录中早期 N400 常 D 特例未通过的选择依据; 完整文件保留实际常 D 与非线性热场路径。

版本说明与勘误: 历史运行应将 v2 核心安装到 modeling/drying_core.py 原槽位; 当前入口字节匹配, 历史核心不能替换成 v6 后冒称原运行。

```
"""Independently compare production nodal-dual FVM with Bessel benchmarks.

This script owns results/analytic_comparison only. It neither modifies the
solver nor writes the production run manifest or figure index. The Q1
nonlinear run validates temperature only against the exact linear heat PDE;
moisture comparison is deliberately performed in a separate frozen-D case.
"""
from __future__ import annotations

import csv
import hashlib
import json
import os
from dataclasses import asdict
from datetime import datetime, timezone
from pathlib import Path
import platform
import sys
import time

import numpy as np

from drying_core import Settings, solve_case, load_inputs, LOADED_CODE_SHA256
from validate_bessel import bessel_temperature, bessel_moisture

ROOT=Path(__file__).resolve().parents[3]
OUT=ROOT/'paper_output/results/analytic_comparison'
SOURCE=Path(__file__).resolve()
CORE=SOURCE.with_name('drying_core.py')
REFERENCE=SOURCE.with_name('validate_bessel.py')

def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def file_record(path):
    p=Path(path)
    return {'path':p.relative_to(ROOT).as_posix(),'bytes':p.stat().st_size,'sha256':sha(p)}

def write_json(path,data):
    path.write_text(json.dumps(data,ensure_ascii=False,indent=2,allow_nan=False)
+'\\n',encoding='utf-8')

def norm_summary(actual,reference,times,radii):
    err=actual-reference
    idx=np.unravel_index(np.argmax(np.abs(err)),err.shape)
    return {
        'max_abs_error':float(np.max(np.abs(err))),
        'rms_error':float(np.sqrt(np.mean(err**2))),
    }
```

```

        'mean_signed_error':float(np.mean(err)),
        'max_error_time_s':float(times[idx[0]]),
        'max_error_radius_m':float(radii[idx[1]]),
        'error_at_max_location':float(err[idx]),
        'centre_max_abs_error':float(np.max(np.abs(err[:,0]))),
        'surface_max_abs_error':float(np.max(np.abs(err[:, -1]))),
        'finite_values':bool(np.all(np.isfinite(actual)) and np.all(np.isfinite(reference))),
    }

def write_comparison_csv(path,times,radii,T,Tref,C,Cref):
    with path.open('w',encoding='utf-8',newline='') as fh:
        writer=csv.writer(fh)
        writer.writerow(['time_s','radius_m','fvm_temperature_K','analytic_temperature_K',
                        'temperature_error_K','fvm_moisture_kg_per_kg',
                        'analytic_frozen_D_moisture_kg_per_kg','moisture_error_kg_per_kg'])
        for i,t in enumerate(times):
            for j,r in enumerate(radii):
                numeric=[t,r,T[i,j],Tref[i,j],T[i,j]-Tref[i,j],C[i,j]]
                values=[format(float(v),'.17g') for v in numeric]
                values += [' ', ' '] if Cref is None else [format(Cref[i,j],'.17g'),format(C[i,j]-
Cref[i,j],'.17g')]
                writer.writerow(values)

def main():
    started=time.perf_counter()
    if Path.cwd().resolve()!=ROOT:
        raise RuntimeError('Run from the 2026CUMCM workspace root')
    OUT.mkdir(parents=True,exist_ok=True)
    source_records=[file_record(p) for p in [SOURCE,CORE,REFERENCE]]
    if sha(CORE)!=LOADED_CODE_SHA256:
        raise RuntimeError('Solver file changed after import')
    env,_,inputs=load_inputs(with_records=True)
    env_t,env_T,env_C=env['time_s'],env['temperature_K'],env['air_moisture_kg_per_kg']
    paper_times=np.array([100.,300.,600.,900.,1200.,1500.,1800.])
    paper_radii=np.array([0.,.005,.01,.015,.02])
    dense_times=np.arange(0.,1800.+1,10.)
    dense_radii=np.linspace(0.,.02,21)
    D=float(7e-9*np.exp(-.89/2.55))
    analytic={}
    for grid,ts,rs in [(('paper',paper_times,paper_radii),('dense',dense_times,dense_radii))]:
        exact_T=bessel_temperature(ts,rs,env_t,env_T,n_terms=240)
        exact_C=bessel_moisture(ts,rs,env_t,env_C,D,n_terms=240)
        exact_T_refined=bessel_temperature(ts,rs,env_t,env_T,n_terms=480)
        exact_C_refined=bessel_moisture(ts,rs,env_t,env_C,D,n_terms=480)
        analytic[grid]=(exact_T,exact_C,{
            'heat_240_480_max_difference_K':float(np.max(np.abs(exact_T-exact_T_refined))),
            'frozen_water_240_480_max_difference_kg_per_kg':float(np.max(np.abs(exact_C-
exact_C_refined))))
    case_results=[]
    plot_payload=[]
    for mode,N in [(('nonlinear_Q1',100),('nonlinear_Q1',200),('nonlinear_Q1',400),
('frozen_D',100),('frozen_D',400),('frozen_D',800))]:
        name=f'{mode}_N{N}'
        setting=Settings(question='Q1',intervals=N,rtol=1e-10,
            atol_temperature=1e-10,atol_moisture=1e-11,
            early_max_step_s=2.,max_step_s=2.,
            face_scheme='kirchhoff',constant_D=D if mode=='frozen_D' else None)

    print(json.dumps({'event':'starting_case','case':name,'elapsed_total_s':time.perf_counter()-
started}),flush=True)
    run=solve_case(setting)
    for rec in source_records+inputs:
        if sha(ROOT/rec['path'])!=rec['sha256']:
            raise RuntimeError('Source/input changed during comparison: '+rec['path'])

```

```

case={'case_id':name,'settings':asdict(setting),'diagnostics':run.diagnostics(),
      'comparison_scope':'temperature only; nonlinear C has no Bessel reference' if
mode=='nonlinear_Q1'
      else 'constant-property heat and frozen-D linear moisture',
      'comparisons':{},'output_artifacts':[]}
for grid,ts,rs in [('paper',paper_times,paper_radii),('dense',dense_times,dense_radii)]:
    T,C=run.fields(ts,radii_m=rs)
    Tref,Cref,truncation=analytic[grid]
    if mode!='frozen_D':
        Cref=None

comparisons={'temperature':norm_summary(T,Tref,ts,rs),'analytic_truncation_sensitivity':truncatio
n}

    if Cref is not None:
        comparisons['frozen_D_moisture']=norm_summary(C,Cref,ts,rs)
    case['comparisons'][grid]=comparisons
    path=OUT/f'{name}_{grid}.csv'
    write_comparison_csv(path,ts,rs,T,Tref,C,Cref)
    case['output_artifacts'].append(file_record(path))
    if grid=='dense':
        plot_payload.append((mode,N,ts,np.max(np.abs(T-Tref),axis=1),
                                None if Cref is None else np.max(np.abs(C-Cref),axis=1)))
# State is a collocated nodal field whose balance is integrated over
# surrounding dual cells. This separate diagnostic quantifies how a
# strict cell-average interpretation differs; main comparison is point.
node_T,node_C=run.fields(paper_times)
node_r=run.model.x*.02
edges=np.r_[0.,(node_r[1:]+node_r[:-1])/2.,.02]
mean_T=bessel_temperature(paper_times,node_r,env_t,env_T,n_terms=240,cell_edges_m=edges)
point_T=bessel_temperature(paper_times,node_r,env_t,env_T,n_terms=240)
case['representation_diagnostic']={
    'primary_interpretation':'collocated nodal values with surrounding dual control-volume
balances',
    'strict_cell_average_interpretation_is_not_primary':True,
    'heat_node_max_abs_error_K':float(np.max(np.abs(node_T-point_T))),
    'heat_vs_annular_average_max_abs_error_K':float(np.max(np.abs(node_T-mean_T))),
    'analytic_point_vs_annular_average_max_difference_K':float(np.max(np.abs(point_T-
mean_T))),
    'note':'Endpoint dual cells are one-sided, so point-vs-average differences can be
first order; they are not physical model error.'}
    if mode=='frozen_D':
        early_times=np.array([1.,2.,5.,10.,20.,30.,60.])
        early_radii=np.array([0.,.019,.02])
        early_T,early_C=run.fields(early_times,radii_m=early_radii)
        early_Tref=bessel_temperature(early_times,early_radii,env_t,env_T,n_terms=480)
        early_Cref=bessel_moisture(early_times,early_radii,env_t,env_C,D,n_terms=480)
        early_Cref240=bessel_moisture(early_times,early_radii,env_t,env_C,D,n_terms=240)
        case['early_boundary_layer']={
            'times_s':early_times.tolist(),'radii_m':early_radii.tolist(),
            'moisture':norm_summary(early_C,early_Cref,early_times,early_radii),
            'surface_errors_kg_per_kg':(early_C[:-1]-early_Cref[:-1]).tolist(),
            'analytic_240_480_max_difference':float(np.max(np.abs(early_Cref240-early_Cref)))}
        early_path=OUT/f'{name}_early.csv'

write_comparison_csv(early_path,early_times,early_radii,early_T,early_Tref,early_C,early_Cref)
    case['output_artifacts'].append(file_record(early_path))
    case_results.append(case)
    write_json(OUT/f'{name}_report.json',case)
    print(json.dumps({'event':'completed_case','case':name,'elapsed_s':run.elapsed_s,
                      'paper_T_error':case['comparisons']['paper']['temperature']
['max_abs_error'],
                      'dense_T_error':case['comparisons']['dense']['temperature']
['max_abs_error'],
                      'dense_C_error':case['comparisons']['dense'].get('frozen_D_moisture',
{}).get('max_abs_error')}),flush=True)
    heat=[c for c in case_results if c['case_id'].startswith('nonlinear_Q1')]

```

```

water=[c for c in case_results if c['case_id'] in ['frozen_D_N100','frozen_D_N400']]
refined_water=[c for c in case_results if c['case_id']=='frozen_D_N800'][0]
convergence={}
for grid in ['paper','dense']:
    eT=[c['comparisons'][grid]['temperature']['max_abs_error'] for c in heat]
    eC=[c['comparisons'][grid]['frozen_D_moisture']['max_abs_error'] for c in water]
    convergence[grid]={
        'heat_intervals':[100,200,400], 'heat_max_abs_errors_K':eT,
        'heat_observed_orders':[float(np.log(eT[i]/eT[i+1])/np.log(2.)) for i in [0,1]],
        'water_intervals':[100,400], 'water_max_abs_errors_kg_per_kg':eC,
        'water_observed_order_100_400':float(np.log(eC[0]/eC[1])/np.log(4.)),
        'both_refinements_reduce_heat_error':bool(eT[0]>eT[1]>eT[2]),
        'water_refinement_reduces_error':bool(eC[0]>eC[1]),
        'N400_heat_max_error_below_0_0001_K':bool(eT[-1]<1e-4),
        'N400_frozen_water_max_error_below_0_0001':bool(eC[-1]<1e-4),
        'N800_frozen_water_max_abs_error':refined_water['comparisons'][grid]
    }
    ['frozen_D_moisture']['max_abs_error'],
    'water_observed_order_400_800':float(np.log(eC[-1]/refined_water['comparisons'][grid]
    ['frozen_D_moisture']['max_abs_error']/np.log(2.)),
    }
os.environ.setdefault('MPLCONFIGDIR',str(ROOT/'tmp/cache/matplotlib'))
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
fig,axes=plt.subplots(1,2,figsize=(11,4.3),constrained_layout=True)
for mode,N,ts,eT,eC in plot_payload:
    if mode=='nonlinear_Q1':
        axes[0].semilogy(ts[1:],np.maximum(eT[1:],1e-16),label=f'N={N}')
    if mode=='frozen_D':
        axes[1].semilogy(ts[1:],np.maximum(eC[1:],1e-16),label=f'N={N}')
for ax in axes:
    ax.set_xlabel('Time (s)');ax.grid(alpha=.25);ax.legend()
axes[0].set(title='Q1 heat: FVM vs Bessel',ylabel='Maximum radial absolute error (K)')
axes[1].set(title='Frozen-D moisture benchmark',ylabel='Maximum radial absolute error
(kg/kg)')
plot_path=OUT/'analytic_error_history.png'
fig.savefig(plot_path,dpi=180);plt.close(fig)
for rec in source_records+inputs:
    if sha(ROOT/rec['path'])!=rec['sha256']:
        raise RuntimeError('Source/input changed before final report: '+rec['path'])
    validation_pass=all(v['both_refinements_reduce_heat_error'] and
v['water_refinement_reduces_error']
    and v['N400_heat_max_error_below_0_0001_K'] and
v['N400_frozen_water_max_error_below_0_0001']
    for v in convergence.values())
    report={
        'schema_version':'1.0', 'question_id':'Q1', 'generated_by':'paper_output/code/modeling/
compare_analytic.py',
        'generated_at':datetime.now(timezone.utc).isoformat(), 'status':'PASS' if validation_pass
    else 'REVIEW_REQUIRED',
        'validation_criteria':'Both heat refinements and 100-to-400 frozen-water refinement
    reduce max error; N400 sampled max errors below 1e-4 in physical units.',
        'scope':'Numerical verification against the selected constant-coefficient PDE; no
    internal experimental accuracy claim.',
        'point_value_comparison':'Nodal-dual state is compared to exact point values at the same
    physical radii. Separate representation diagnostics quantify strict annular averages.',
        'primary_space_time_samples':
    {'paper_times_s':paper_times.tolist(), 'paper_radii_m':paper_radii.tolist(),
        'dense_time_step_s':10., 'dense_time_range_s':[0.,1800.],
        'dense_radius_step_m':.001, 'dense_radius_range_m':[0.,.02]},
        'nonlinear_moisture_note':'The nonlinear Q1 C solution is stored for provenance only, not
    compared with constant-D Bessel or labeled analytic truth.',
        'constant_diffusion_coefficient_m2_s':D,
        'core_static_review':{
            'kirchhoff_primitive':f'F(C)=C*exp(-a/C)+a*Ei(-a/C), so F_prime(C)=exp(-a/C)',
            'water_face_flux':f'Internal face radius times D0*exp(-3850/T_face)*(F(C_right)-

```

```

F(C_left))/dx, with no thermal factor in the differenced potential.',
    'Q1_temperature_factor':'Exactly one, since D1 has no temperature dependence.',
    'no_spurious_soret_term':True,
    'internal_flux_cancellation':'Each face occurs with opposite signs in adjacent
control-volume balances; weighted sum leaves only the surface flux.',
    'density_interpretation':'rho*cp is effective thermal capacity; independent dry-matter
weighting is used for conserved moisture.',
    'scope_limitation':'Static consistency and linear benchmark do not alone validate all
nonlinear long-time or shrinkage scenarios.'},
    'convergence':convergence,'cases':case_results,
    'total_elapsed_s':time.perf_counter()-started,
    'visual_studio_gui':'pending','human_review':'pending',
    'execution_provenance':{'run_command':sys.executable+' -B '+' '.join(sys.argv),
    'run_exit_code':0,'python_version':platform.python_version(),
    'source_files':source_records,'input_artifacts':inputs,
    'output_artifacts':[file_record(p) for p in sorted(OUT.glob('*.csv'))]}+
[file_record(plot_path)]},
    }
    write_json(OUT/'analytic_comparison_report.json',report)

print(json.dumps({'status':report['status'],'convergence':convergence,'total_elapsed_s':report['t
otal_elapsed_s']},indent=2,flush=True)
    return 0

if __name__=='__main__':
    raise SystemExit(main())

```

E.3 verify_convergence.py: 当前投影/场差归约模块

文件: paper_output/code/modeling/verify_convergence.py; 共 127 行。采用范围: final_v6 时间检验实际导入 project_run/compare_projections; 当前文件不能冒充早期空间检验驱动版本。

```
"""Independent grid/time checks on the adopted equations; no experimental fitting."""
from __future__ import annotations
import argparse
from dataclasses import replace
from datetime import datetime, timezone
import gc
import json
from pathlib import Path
import warnings
import numpy as np
from drying_core import ROOT, Settings, solve_case, save_run, file_record
from validate_bessel import bessel_temperature, bessel_moisture

def project_run(run, points, full_seconds):
    """Evaluate every comparison point from full BDF states before releasing Run."""
    if full_seconds:
        times = np.arange(0., np.floor(run.end_s)+1.)
    else:
        times = np.unique(np.r_[np.arange(0., min(10800., run.end_s)+1.),
                                np.arange(10860., run.end_s, 60.)])
    T = np.empty((len(times), len(points)))
    C = np.empty_like(T)
    for first in range(0, len(times), 128):
        T[first:first+128], C[first:first+128] = run.fields(times[first:first+128],
radii_m=points)
    return {'times':times, 'T':T, 'C':C, 'end_s':run.end_s, 'event_s':run.event_s,
        'N':run.model.settings.intervals}

def compare_projections(coarse, fine, points, block=1000):
    common_count = min(len(coarse['times']), len(fine['times']))
    times = coarse['times'][:common_count]
    if not np.array_equal(times, fine['times'][:common_count]):
        raise ValueError('Only identical physical time queries may be compared')
    maxima = {'T_K':0., 'C':0.}
    locations = {}
    for first in range(0, len(times), block):
        tt = times[first:first+block]
        fields_a = [coarse[k][first:first+len(tt)] for k in ['T','C']]
        fields_b = [fine[k][first:first+len(tt)] for k in ['T','C']]
        for label, aa, bb in zip(['T_K', 'C'], fields_a, fields_b):
            error = np.abs(aa-bb)
            if np.any(np.isfinite(error)):
                index = np.unravel_index(np.nanargmax(error), error.shape)
                value = float(error[index])
                if value > maxima[label]:
                    maxima[label] = value
                    locations[label] = {'time_s':float(tt[index[0]]),
                        'radius_m':float(points[index[1]])}
    return {'max_absolute_difference':maxima, 'locations':locations,
        'compared_time_count':len(times), 'radial_count':len(points),
        'convention':'Only finite values at common physical points are compared.'}

def main():
```

```

parser = argparse.ArgumentParser()
parser.add_argument('--question', choices=['Q1', 'Q23', 'Q4'], required=True)
parser.add_argument('--grids', type=int, nargs='+', required=True)
parser.add_argument('--tag', required=True)
parser.add_argument('--full-second-comparison', action='store_true')
parser.add_argument('--frozen-d', type=float)
args = parser.parse_args()
out = ROOT/'paper_output/results/convergence'/args.tag
if out.exists():
    raise FileExistsError(out)
out.mkdir(parents=True)
settings = Settings(question=args.question, face_scheme='kirchhoff',
    shrink=args.question=='Q4', rtol=1e-10, atol_temperature=1e-10,
    atol_moisture=1e-12, max_step_s=120., early_max_step_s=2.,
    constant_D=args.frozen_d)
report = {'created_at':datetime.now(timezone.utc).isoformat(),
    'question':args.question, 'runs':[], 'comparisons':[],
    'driver_code':file_record(__file__), 'human_review':'pending'}
previous = None
radii = np.linspace(0,.02,21)
for n in args.grids:
    print(f'START {args.tag} N={n}', flush=True)
    run = None
    try:
        with warnings.catch_warnings(record=True) as captured:
            warnings.simplefilter('always')
            run = solve_case(replace(settings, intervals=n))
            summary = save_run(run, out/f'N{n}')
        projection = project_run(run, radii, args.full_second_comparison or
args.question=='Q1')
        summary['warnings'] = [str(item.message) for item in captured]
        report['runs'].append(summary)
        if captured:
            raise RuntimeError(f'Production solver warnings at N={n}: {summary["warnings"]}')
        if previous is not None:
            comparison = compare_projections(previous, projection, radii)
            comparison.update({'coarse_N':previous['N'], 'fine_N':n,
                'event_difference_s':None if run.event_s is None else run.event_s-
previous['event_s']})
            report['comparisons'].append(comparison)
            print(json.dumps(comparison), flush=True)
        if args.question=='Q1':
            times = np.arange(0.,1801.)
            numeric_T, numeric_C = projection['T'], projection['C']
            env = run.model.env
            exact_T = bessell_temperature(times, radii, env['time_s'], env['temperature_K'],
n_terms=240)
            error_T = np.abs(numeric_T-exact_T)
            analytic = {'N':n,'max_heat_error_K':float(error_T.max()),
                'scope':'All 1801 integer seconds and 21 requested radii'}
            if args.frozen_d is not None:
                exact_C = bessell_moisture(times, radii, env['time_s'],
                    env['air_moisture_kg_per_kg'], diffusion_coefficient=args.frozen_d,
n_terms=480)
                error_C = np.abs(numeric_C-exact_C)
                at = np.unravel_index(np.argmax(error_C), error_C.shape)
                analytic.update({'max_frozen_D_water_error':float(error_C.max()),
                    'water_error_time_s':float(times[at[0]]),
                    'water_error_radius_m':float(radii[at[1]])})
            summary['analytic_comparison'] = analytic
            print(json.dumps(analytic), flush=True)
            previous = projection

(out/'convergence_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encodin
g='utf-8')
finally:

```

```

        if run is not None:
            run.close()
        del run
        gc.collect()
        report['finished_at'] = datetime.now(timezone.utc).isoformat()
        report['status'] = 'computed_and_zero_solver_warnings'

(out/'convergence_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
print('COMPLETED '+args.tag, flush=True)

if __name__=='__main__':
    main()

```

E.4 verify_time_accuracy.py: 同N容差和步长收紧的监督运行入口

文件: paper_output/code/modeling/verify_time_accuracy.py; 共 251 行。采用范围: Q1/Q23 N3200、Q4 N6400 时间检验; 基准/收紧各独立子进程。

```
"""Compare baseline/tighter time integration at one fixed spatial resolution.

Each variant is one supervised process and one live Run. Full BDF output is
projected before close(); comparison uses those actual projections, never a
60-second archive to synthesize missing seconds. No global production files
are published. Small-N runs validate this driver, not final model accuracy.
"""
from __future__ import annotations

import argparse
import gc
import hashlib
import importlib
import json
import os
from pathlib import Path
import sys
import time
import traceback
import uuid
import warnings

SOURCE = Path(__file__).resolve()
ROOT = SOURCE.parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(SOURCE.read_bytes()).hexdigest()
import production_provenance as provenance

BASELINE = {'rtol': 1e-10, 'atol_temperature': 1e-10, 'atol_moisture': 1e-12,
            'early_max_step_s': 2., 'max_step_s': 120.}
TIGHT = {'rtol': 1e-11, 'atol_temperature': 1e-11, 'atol_moisture': 1e-13,
         'early_max_step_s': 1., 'max_step_s': 60.}

def dependencies():
    names = ['verify_time_accuracy.py', 'production_provenance.py', 'drying_core.py',
            'disk_dense.py', 'analytic_jacobian.py', 'verify_convergence.py', 'validate_bessel.py',
            'q1_model.py', 'q2_model.py', 'q3_model.py', 'q4_model.py']
    paths = [SOURCE.parent / name for name in names]
    paths += [ROOT / 'paper_output/data_cleaned' / name for name in
              ('A_environment_observed.csv', 'A_radius_observed.csv')]
    return [provenance.file_record(path) for path in sorted(paths)]

def unchanged(launch):
    if dependencies() != launch['input_files']:
        raise RuntimeError('Time-check source/input changed after launch')
    if provenance.runtime_record() != launch['runtime']:
        raise RuntimeError('Time-check runtime changed after launch')
    expected = {item['path']: item['sha256'] for item in launch['input_files']}
    for path, loaded in [(SOURCE, LOADED_CODE_SHA256), (provenance.SOURCE,
provenance.LOADED_CODE_SHA256)]:
        if loaded != expected[path.relative_to(ROOT).as_posix()]:
            raise RuntimeError('Loaded driver/helper differs from launch snapshot')

def worker(args, directory, launch):
    unchanged(launch)
    import numpy as np
```

```

from dataclasses import asdict, replace
core = importlib.import_module('drying_core')
checker = importlib.import_module('verify_convergence')
wrapper = importlib.import_module({'Q1': 'q1_model', 'Q23': 'q2_model', 'Q4': 'q4_model'})
[args.question])
unchanged(launch)
case = directory / args.variant
provenance.exclusive_json(case / 'worker_started.json', {'pid': os.getpid(), 'started_at':
provenance.utc_now()})
run, projection, summary = None, None, None
started, timer = provenance.utc_now(), time.perf_counter()
solve_finished = None
captured_messages = []
try:
    print(f'SOLVE {args.variant} {args.question} N={args.n} UTC={started}', flush=True)
    with warnings.catch_warnings(record=True) as captured:
        warnings.simplefilter('always')
        if args.variant == 'baseline':
            # Use the actual production wrapper, then verify its settings.
            run = wrapper.solve(args.n)
        else:
            baseline = json.loads((directory /
'baseline/summary.json').read_text(encoding='utf-8'))
            run = core.solve_case(replace(core.Settings(**baseline['settings']), **TIGHT))
            solve_finished = provenance.utc_now()
            expected = core.Settings(question=args.question, intervals=args.n,
            face_scheme='kirchhoff', shrink=args.question == 'Q4', **(BASELINE if args.variant
== 'baseline' else TIGHT))
            if asdict(run.model.settings) != asdict(expected):
                raise RuntimeError('Actual variant settings differ from the fixed-physics
contract')
            summary = core.save_run(run, case)
            points = np.linspace(0., .02, 21)
            projection = checker.project_run(run, points, args.full_second_comparison or
args.question == 'Q1')
            np.savez_compressed(case / 'comparison_projection.npz', times_s=projection['times'],
            radii_m=points, T_K=projection['T'], C=projection['C'])
            captured_messages = [str(item.message) for item in captured]
            if captured_messages:
                raise RuntimeError('Time-check warnings: ' + '; '.join(captured_messages))
            if args.question != 'Q1' and run.event_s is None:
                raise RuntimeError('A full-drying comparison requires an actual threshold event')
            summary['time_check'] = {'variant': args.variant, 'question': args.question,
            'spatial_intervals': args.n, 'started_at': started, 'solve_finished_at':
solve_finished,
            'warnings': captured_messages, 'projection_time_count': len(projection['times']),
            'full_second_comparison': bool(args.full_second_comparison or args.question == 'Q1'),
            'projection_radius_count': len(points),
            'process_exit': 'Actual exit is recorded by supervising parent in case_record.json.'}
            provenance.write_json(case / 'summary.json', summary)
        except BaseException as error:
            provenance.write_json(case / 'worker_failure.json', {'status': 'FAIL', 'error':
repr(error),
            'traceback': traceback.format_exc(), 'started_at': started, 'failed_at':
provenance.utc_now(),
            'wall_seconds': time.perf_counter() - timer, 'warnings': captured_messages})
            raise
    finally:
        if run is not None:
            run.close()
        run, projection = None, None
        gc.collect()
        unchanged(launch)
        finished = provenance.utc_now()
        summary['time_check'].update({'finished_at': finished, 'wall_seconds': time.perf_counter() -
timer,

```

```

        'Run_closed': True})
    provenance.write_json(case / 'summary.json', summary)
    outputs = [provenance.file_record(path) for path in sorted(case.rglob('*'))
                if path.is_file() and path.name not in ('stdout.log', 'worker_success.json',
'process_result.json')]
    provenance.write_json(case / 'worker_success.json', {'variant': args.variant, 'question':
args.question,
        'started_at': started, 'finished_at': finished, 'wall_seconds': time.perf_counter() -
timer,
        'input_files': launch['input_files'], 'output_artifacts': outputs,
        'summary': summary, 'status': 'computed_zero_solver_warnings',
        'human_review': 'pending', 'visual_studio_gui': 'pending'})
    print(f'VARIANT_COMPLETED {args.variant}; RUN_CLOSED', flush=True)

def compare_saved(directory, question):
    import numpy as np
    from verify_convergence import compare_projections
    projected = []
    points = None
    for variant in ('baseline', 'tight'):
        summary = json.loads((directory / variant / 'summary.json').read_text(encoding='utf-8'))
        with np.load(directory / variant / 'comparison_projection.npz', allow_pickle=False) as
archive:
            if points is not None and not np.array_equal(points, archive['radii_m']):
                raise ValueError('Projection radius grids differ')
            points = archive['radii_m'].copy()
            projected.append({'times': archive['times_s'].copy(), 'T': archive['T_K'].copy(),
                'C': archive['C'].copy(), 'N': summary['settings']['intervals'],
                'end_s': summary['diagnostics']['end_s'], 'event_s': summary['diagnostics']
['event_s']})
        baseline, tight = projected
        if baseline['N'] != tight['N']:
            raise ValueError('Time-accuracy comparison must use the same spatial resolution')
        count = min(len(baseline['times']), len(tight['times']))
        for name in ('T', 'C'):
            if not np.array_equal(np.isfinite(baseline[name][:count]), np.isfinite(tight[name]
[:count])):
                raise ValueError('Physical-domain finite masks differ between variants')
        comparison = compare_projections(baseline, tight, points)
        event_difference = None if question == 'Q1' else tight['event_s'] - baseline['event_s']
        comparison.update({'baseline_N': baseline['N'], 'tight_N': tight['N'],
            'baseline_event_s': baseline['event_s'], 'tight_event_s': tight['event_s'],
            'event_difference_s': event_difference,
            'absolute_event_difference_s': None if event_difference is None else
abs(event_difference),
            'difference_direction': 'tight minus baseline for the event; absolute differences for
fields',
            'baseline_projection_time_count': len(baseline['times']), 'tight_projection_time_count':
len(tight['times']),
            'common_last_time_s': float(baseline['times'][count - 1]),
            'scope': 'Numerical sensitivity to simultaneous tighter tolerances and smaller maximum
steps at fixed N.',
            'not_a_rigorous_error_bound': True, 'no_experimental_accuracy_claim': True})
        return comparison

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--question', choices=['Q1', 'Q23', 'Q4'], required=True)
    parser.add_argument('--n', type=int, required=True)
    parser.add_argument('--tag', required=True)
    parser.add_argument('--full-second-comparison', action='store_true')
    parser.add_argument('--worker', action='store_true', help=argparse.SUPPRESS)
    parser.add_argument('--variant', choices=['baseline', 'tight'], help=argparse.SUPPRESS)
    parser.add_argument('--launch-token', help=argparse.SUPPRESS)

```

```

args = parser.parse_args()
os.environ['OPENBLAS_NUM_THREADS'] = '1'
os.environ['OMP_NUM_THREADS'] = '1'
if Path.cwd().resolve() != ROOT:
    raise RuntimeError('Use the competition workspace as cwd')
if args.n < 2 or not args.tag or any(ch not in
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_-' for ch in args.tag):
    raise ValueError('Use N>=2 and a simple, new tag')
directory = ROOT / 'paper_output/results/time_accuracy' / args.tag
config = {'question': args.question, 'n': args.n, 'full_second_comparison':
args.full_second_comparison}
if args.worker:
    launch = json.loads((directory / 'launch.json').read_text(encoding='utf-8'))
    if not args.variant or args.launch_token != launch['token'] or config !=
launch['configuration']:
        raise RuntimeError('Worker does not match the supervising launch')
    worker(args, directory, launch)
    return 0
if args.variant or args.launch_token:
    raise ValueError('Variant/token are reserved for supervised workers')
directory.parent.mkdir(parents=True, exist_ok=True)
directory.mkdir(exist_ok=False)
started, timer = provenance.utcnow(), time.perf_counter()
launch = {'created_at': started, 'parent_pid': os.getpid(), 'token': uuid.uuid4().hex,
'configuration': config, 'input_files': dependencies(), 'runtime':
provenance.runtime_record()}
provenance.exclusive_json(directory / 'launch.json', launch)
cases = []
try:
    unchanged(launch)
    for variant in ('baseline', 'tight'):
        case = directory / variant
        case.mkdir(exist_ok=False)
        command = [sys.executable, '-B', str(SOURCE), '--question', args.question, '--n',
str(args.n),
        '--tag', args.tag, '--worker', '--variant', variant, '--launch-token',
launch['token']]
        if args.full_second_comparison:
            command.append('--full-second-comparison')
        process = provenance.measured_process(command, ROOT, case / 'stdout.log')
        if process['returncode'] != 0:
            raise RuntimeError(variant + ' failed with actual returncode ' +
str(process['returncode']))
        unchanged(launch)
        success = json.loads((case / 'worker_success.json').read_text(encoding='utf-8'))
        if success['input_files'] != launch['input_files']:
            raise RuntimeError('Variant evidence belongs to another launch')
        provenance.assert_records(success['output_artifacts'])
        record = {'variant': variant, 'process': process, 'summary': success['summary'],
'summary_file': provenance.file_record(case / 'summary.json'),
'output_artifacts': success['output_artifacts'] + [provenance.file_record(case /
name)
        for name in ('worker_success.json', 'stdout.log', 'process_result.json')],
'status': 'computed_actual_process_exit_zero'}
        provenance.write_json(case / 'case_record.json', record)
        cases.append(record)
    comparison = compare_saved(directory, args.question)
    unchanged(launch)
    for case in cases:
        provenance.assert_records(case['output_artifacts'])
    report = {'created_at': started, 'finished_at': provenance.utcnow(),
'wall_seconds': time.perf_counter() - timer, 'question': args.question, 'N': args.n,
'status': 'computed_zero_solver_warnings', 'configuration': config,
'baseline_controls': BASELINE, 'tight_controls': TIGHT, 'runs': cases,
'comparison': comparison, 'driver_code': provenance.file_record(SOURCE),
'input_files': launch['input_files'], 'runtime': launch['runtime'],

```

```

        'scope': 'Fixed-N temporal sensitivity only; small grids are functional checks, not
final accuracy evidence.',
        'human_review': 'pending', 'visual_studio_gui': 'pending'}
    provenance.write_json(directory / 'time_accuracy_report.json', report)
    print(json.dumps({'status': report['status'], 'question': args.question, 'N': args.n,
        'comparison': comparison}, ensure_ascii=False), flush=True)
    return 0
except BaseException as error:
    observed = {}
    for variant in ('baseline', 'tight'):
        path = directory / variant / 'process_result.json'
        if path.exists():
            observed[variant] = json.loads(path.read_text(encoding='utf-8'))
    provenance.write_json(directory / 'failure.json', {'status': 'FAIL', 'error':
repr(error),
        'traceback': traceback.format_exc(), 'started_at': started, 'failed_at':
provenance.utcnow(),
        'wall_seconds': time.perf_counter() - timer, 'observed_processes': observed})
    raise

if __name__ == '__main__':
    try:
        raise SystemExit(main())
    except Exception:
        traceback.print_exc()
        raise SystemExit(1)

```

E.5 verify_dense_storage.py: 内存/磁盘稠密输出等价性入口

文件: paper_output/code/modeling/verify_dense_storage.py; 共 69 行。采用范围:
附录采用的 v6 三条完整 N40 轨迹、内部断点及左右邻浮点查询; 不是空间精度证据。

```
"""Independent memory/disk trajectory identity check, including drying events."""
from dataclasses import replace
import argparse
from datetime import datetime, timezone
import json
from pathlib import Path
import warnings
import numpy as np
from drying_core import ROOT, Settings, solve_case, file_record

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--output-dir',
default='paper_output/results/dense_storage_verification_v6')
    args = parser.parse_args()
    output = ROOT/args.output_dir
    output.mkdir(parents=True, exist_ok=False)
    records = []
    for question in ('Q1','Q23','Q4'):
        config = Settings(question=question, intervals=40, shrink=question=='Q4',
        face_scheme='kirchhoff', rtol=1e-8, atol_temperature=1e-8,
        atol_moisture=1e-10, early_max_step_s=5., max_step_s=300.)
        with warnings.catch_warnings(record=True) as captured:
            warnings.simplefilter('always')
            memory = solve_case(replace(config, dense_storage='memory'))
            disk = solve_case(replace(config, dense_storage='disk'))
        try:
            same_steps = len(memory.pieces)==len(disk.pieces) and all(
                np.array_equal(a.t,b.t) and np.array_equal(a.y,b.y)
                for a,b in zip(memory.pieces,disk.pieces))
            rng = np.random.default_rng(20260910)
            times = np.r_[0., memory.end_s, np.arange(0,min(1800,memory.end_s)+1),
                rng.uniform(0,memory.end_s,400)]
            # Deliberately unsorted, repeated, fractional, event and endpoint queries.
            if memory.event_s is not None:
                times = np.r_[times,memory.event_s,memory.event_s-.01,memory.event_s+.01]
            knots = np.unique(np.concatenate([piece.t[1:-1] for piece in memory.pieces]))
            times = np.r_[times,knots,np.nextafter(knots,-np.inf),np.nextafter(knots,np.inf)]
            difference = 0.
            for first in range(0,len(times),100):
                tt = times[first:first+100]
                difference = max(difference,float(np.max(np.abs(memory.state(tt)-
disk.state(tt)))))
            same_event = memory.event_s==disk.event_s and memory.end_s==disk.end_s
            record = {'question':question,'accepted_times_and_states_bitwise_equal':same_steps,
'event_and_endpoint_bitwise_equal':same_event,'dense_query_max_difference':difference,
'query_count':len(times),'warnings':[str(w.message) for w in captured],
'internal_knot_count':len(knots),
'covers_all_internal_knots_and_adjacent_representable_times':True,
'disk_diagnostics':disk.diagnostics(),'memory_diagnostics':memory.diagnostics())
            if not same_steps or not same_event or difference!=0 or captured:
                raise AssertionError(record)
            records.append(record)
            print(question+' exact storage identity PASS',flush=True)
        finally:
            cache_directory = disk.cache.directory
```

```

        memory.close();disk.close()
        record['private_cache_removed_after_close'] = not cache_directory.exists()
        if not record['private_cache_removed_after_close']:
            raise AssertionError('Private cache cleanup failed')
        report = {'status':'PASS','at_utc':datetime.now(timezone.utc).isoformat(),
            'scope':'Storage-only identity check on three complete N40 trajectories; not spatial
accuracy validation',
            'runs':records,'sources':
[file_record(__file__),file_record(Path(__file__).with_name('drying_core.py')),
            file_record(Path(__file__).with_name('disk_dense.py'))]}

(output/'verification.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='
utf-8')
        return 0

if __name__=='__main__':
    raise SystemExit(main())

```

E.6 run_experiments.py: Kirchhoff 网格与同物性收缩对照批次入口

文件: paper_output/code/modeling/run_experiments.py; 共 74 行。采用范围: 附录保留方案及正文 N200 附录 4 固定/收缩对照; 仅采纳指定 trial 的数值与参数, 不采纳所有物理分支。

版本说明与勘误: 历史结果记录锁定 v2 核心; 入口当前哈希见后续生产源码盘点, 不等于实验启动时入口已被锁定。运行同物性对照应使用原槽位 v2 核心。

```
"""Bounded reproducible batches; each scenario has independent physical assumptions."""
from __future__ import annotations
import argparse
from dataclasses import replace
from datetime import datetime, timezone
import json
from pathlib import Path
import sys
import traceback

from drying_core import ROOT, Settings, solve_case, save_run

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--batch', choices=['baseline', 'grids', 'temporal', 'physical'],
                        default='baseline')
    parser.add_argument('--n', type=int, default=100)
    parser.add_argument('--tag', default='')
    parser.add_argument('--face-scheme', choices=['harmonic', 'kirchhoff'], default='harmonic')
    args = parser.parse_args()
    base = Settings(intervals=args.n, face_scheme=args.face_scheme)
    if args.batch in ('baseline', 'grids'):
        candidates = [('Q1', replace(base, question='Q1')), ('Q23', base),
                      ('Q4', replace(base, question='Q4', shrink=True))]
    elif args.batch == 'temporal':
        candidates = [('Q23_tight', replace(base, rtol=1e-9, atol_moisture=1e-11,
                                             atol_temperature=1e-9, max_step_s=150., early_max_step_s=10.)),
                      ('Q4_tight', replace(base, question='Q4', shrink=True, rtol=1e-9,
                                             atol_moisture=1e-11, atol_temperature=1e-9, max_step_s=150.,
                                             early_max_step_s=10.))]
    else:
        candidates = [('Q4_fixed', replace(base, question='Q4', shrink=False)),
                      ('Q23_last', replace(base, boundary_extension='last')),
                      ('Q23_mean', replace(base, boundary_extension='tail_mean')),
                      ('Q23_T49', replace(base, tail_temperature_C=49.)),
                      ('Q23_T51', replace(base, tail_temperature_C=51.)),
                      ('Q23_eq045', replace(base, tail_equilibrium=.045)),
                      ('Q23_eq055', replace(base, tail_equilibrium=.055)),
                      ('Q4_T49', replace(base, question='Q4', shrink=True,
                                             tail_temperature_C=49.)),
                      ('Q4_T51', replace(base, question='Q4', shrink=True,
                                             tail_temperature_C=51.)),
                      ('Q23_beta08', replace(base, beta=6.4e-7)),
                      ('Q23_beta12', replace(base, beta=9.6e-7)),
                      ('Q23_latent', replace(base, surface_latent_fraction=1.)),
                      ('Q4_latent', replace(base, question='Q4', shrink=True,
                                             surface_latent_fraction=1.))]
    out = ROOT/'paper_output/results/experiments'
    out.mkdir(parents=True, exist_ok=True)
    failed = 0
    for name, settings in candidates:
```

```

trial_id = f'{args.batch}_{name}_N{args.n}{args.tag}'
folder = out/trial_id
if (folder/'summary.json').exists():
    raise FileExistsError(f'Refusing to overwrite previous trial: {trial_id}')
folder.mkdir(parents=True, exist_ok=True)
start = datetime.now(timezone.utc).isoformat()
print(f'START {trial_id} {start}', flush=True)
try:
    run = solve_case(settings)
    summary = save_run(run, folder)
    record = {'trial_id':trial_id, 'started_at':start,
              'finished_at':datetime.now(timezone.utc).isoformat(),
              'status':'computed', 'exit_code':0, **summary['diagnostics']}
except Exception as exc:
    failed += 1
    (folder/'failure.txt').write_text(traceback.format_exc(), encoding='utf-8')
    record = {'trial_id':trial_id, 'started_at':start, 'status':'failed',
              'exit_code':1, 'error':str(exc)}
    (folder/'trial_record.json').write_text(json.dumps(record, ensure_ascii=False, indent=2),
encoding='utf-8')
    with (out/'trials.jsonl').open('a',encoding='utf-8') as stream:
        stream.write(json.dumps(record, ensure_ascii=False)+'\n')
    print(json.dumps(record, ensure_ascii=False), flush=True)
return 1 if failed else 0

if __name__ == '__main__':
    sys.exit(main())

```

E.7 crossvalidate_solver.py: BDF/Radau 与面格式对照入口

文件: paper_output/code/verification/crossvalidate_solver.py; 共 206 行。采用范围: 第 6 章采纳 block1 的 Q23/Q4、N800/N1600 共 8 案; 其余 block2/3 全文件保留但不扩展正文结论。

```
"""Cross-validation of the A-problem drying-time conclusion by independent methods.

Four independent blocks, all additive: no frozen model, setting, input or result
is modified. Everything is written under paper_output/results/crossvalidation/.

Block 1  time-integrator cross-validation    BDF vs Radau at fixed N
Block 2  water-flux face-scheme convergence Kirchhoff vs harmonic under refinement
Block 3  latent-heat model modification      surface evaporation-cooling scenarios
Block 4  threshold & scaling robustness      post-processing on the same solves

Sandbox note: disk_dense.DenseCache uses tempfile.mkdtemp, whose directories this
session's file sandbox cannot open. tempfile.mkdtemp is replaced by an equivalent
os.mkdir-based implementation before importing the solver. No solver logic changes.
"""

from __future__ import annotations

import json
import os
import pathlib
import sys
import tempfile
import time
import traceback
import uuid
from dataclasses import replace
from datetime import datetime, timezone

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
CODE = ROOT / "paper_output" / "code" / "modeling"

def _mkdtemp(suffix=None, prefix=None, dir=None):
    base = pathlib.Path(dir)
    for _ in range(200):
        candidate = base / ((prefix or "tmp") + uuid.uuid4().hex[:12] + (suffix or ""))
        if not candidate.exists():
            candidate.mkdir(parents=False)
            return str(candidate)
    raise FileExistsError("no unused temp name")

tempfile.mkdtemp = _mkdtemp

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(CODE))

import numpy as np # noqa: E402

import drying_core # noqa: E402
from drying_core import Settings, solve_case # noqa: E402

try:
    import q3_model # noqa: E402
```

```

        completion = q3_model.completion
    except Exception: # pragma: no cover - fallback keeps the block self-contained
        completion = None

OUT = ROOT / "paper_output" / "results" / "crossvalidation"
BASE = Settings(intervals=800, rtol=1e-10, atol_temperature=1e-10,
                atol_moisture=1e-12, early_max_step_s=2.0, max_step_s=120.0,
                face_scheme="kirchhoff", jacobian_mode="analytic")

def utcnow():
    return datetime.now(timezone.utc).isoformat()

def runCase(name, settings):
    started = time.perf_counter()
    record = {"case": name, "settings": {k: v for k, v in settings.__dict__.items()},
            "startedAtUtc": utcnow()}
    run = None
    try:
        run = solve_case(settings)
        diagnostic = run.diagnostics()
        record["diagnostics"] = diagnostic
        record["event_s"] = run.event_s
        record["event_h"] = None if run.event_s is None else run.event_s / 3600.0
        record["end_s"] = run.end_s
        if completion is not None and settings.question != "Q1":
            record["completion"] = completion(run)
        record["status"] = "computed"
    except Exception as error: # keep the failure, never hide it
        record["status"] = "failed"
        record["error"] = f"{type(error).__name__}: {error}"
        record["traceback"] = traceback.format_exc()
    finally:
        if run is not None:
            try:
                run.close()
            except Exception:
                pass
    record["elapsedSeconds"] = time.perf_counter() - started
    print(json.dumps({k: record[k] for k in ("case", "status", "elapsedSeconds")
                    if k in record} |
                    {"event_h": record.get("event_h")}, ensure_ascii=False), flush=True)
    return record

def block1():
    """Independent time integrators at a fixed spatial discretisation.

    BDF/NDF (variable-order implicit) and Radau IIA (3-stage implicit Runge-Kutta)
    are different integration families with different truncation-error structure,
    so agreement at fixed N isolates the time integrator as an error source.

    LSODA was attempted and removed: SciPy's ODEPACK wrapper is dense, cannot
    consume the sparse analytic Jacobian, and with a dense finite-difference
    Jacobian at N = 200 it did not finish in a practical time. The third
    independent integrator is therefore the MATLAB ode15s implementation recorded
    under paper_output/qa/matlab_crosscheck_20260911, not a third SciPy method.
    """
    cases = []
    for question, shrink in (("Q23", False), ("Q4", True)):
        for intervals in (800, 1600):
            for method in ("BDF", "Radau"):
                settings = replace(BASE, question=question, shrink=shrink,
                                intervals=intervals, method=method,

```

```

        dense_storage="disk" if method == "BDF" else "memory")
    cases.append((f"B1_{question}_{method}_N{intervals}", settings))
return cases

def block2():
    """Water-flux face representation under grid refinement.

    A coarse grid separates the two schemes by a large factor; the test is whether
    they converge to the same answer as the mesh is refined, which is what makes the
    production Kirchhoff result grid-independent rather than scheme-dependent.
    """
    cases = []
    for intervals in (400, 800, 1600):
        for scheme in ("kirchhoff", "harmonic"):
            settings = replace(BASE, question="Q23", intervals=intervals,
                               face_scheme=scheme)
            cases.append((f"B2_Q23_{scheme}_N{intervals}", settings))
    for intervals in (400, 800, 1600):
        for scheme in ("kirchhoff", "harmonic"):
            settings = replace(BASE, question="Q4", shrink=True,
                               intervals=intervals, face_scheme=scheme)
            cases.append((f"B2_Q4_{scheme}_N{intervals}", settings))
    return cases

def block3():
    """Model modification: explicit surface evaporative cooling (latent heat).

    The fraction is a scenario parameter, not a calibrated physical constant:
    the problem provides no sorption isotherm that would close the surface vapour
    balance, so an envelope is reported instead of a single point prediction.
    """
    cases = []
    for question, shrink in (("Q23", False), ("Q4", True)):
        for fraction in (0.0, 0.25, 0.5, 1.0):
            settings = replace(BASE, question=question, shrink=shrink,
                               intervals=800, surface_latent_fraction=fraction)
            cases.append((f"B3_{question}_L{fraction:g}_N800", settings))
            settings = replace(BASE, question=question, shrink=shrink, intervals=1600,
                               surface_latent_fraction=1.0)
            cases.append((f"B3_{question}_L1_N1600", settings))
    return cases

BLOCKS = {"1": block1, "2": block2, "3": block3}

def main():
    wanted = sys.argv[1] if len(sys.argv) > 1 else "1"
    runId = sys.argv[2] if len(sys.argv) > 2 else "cv_" + wanted
    directory = OUT / runId
    directory.mkdir(parents=True, exist_ok=True)
    records = []
    done = set()
    casesPath = directory / "cases.json"
    if casesPath.exists(): # resume: never recompute a case that already succeeded
        records = json.loads(casesPath.read_text(encoding="utf-8"))
        done = {r["case"] for r in records if r.get("status") == "computed"}
        print(f"resuming: {len(done)} computed cases already present", flush=True)
    for block in wanted:
        if block not in BLOCKS:
            raise SystemExit(f"unknown block {block}")
        for name, settings in BLOCKS[block]():
            if name in done:
                print(f"skip {name} (already computed)", flush=True)

```

```

        continue
    record = runCase(name, settings)
    records.append(record)
    casesPath.write_text(json.dumps(records, ensure_ascii=False, indent=2),
                          encoding="utf-8")
    summary = {"runId": runId, "blocks": wanted,
               "solverCode": drying_core.file_record(pathlib.Path(drying_core.__file__)),
               "finishedAtUtc": utcnow(),
               "computed": sum(1 for r in records if r["status"] == "computed"),
               "failed": sum(1 for r in records if r["status"] == "failed")}
    (directory / "summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(summary, ensure_ascii=False), flush=True)
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

E.8 method_comparison.py: Kirchhoff 与另一守恒面格式比较

文件: paper_output/code/verification/method_comparison.py; 共 197 行。采用范围: 第 5 章非线性水方程面格式对照及附录选择依据; 实际另一格式是两节点 D 值算术平均, 不是声称整个 PDE 独立。

版本说明与勘误: 常 D 两路径相同只是极限测试; 该文件没有执行额外 Bessel 比较, 不能根据文件序言扩展结论。

```
"""Independent-discretisation cross-validation of the water-flux operator.

Two different numerical methods are applied to the SAME moisture equation with the
SAME time integrator and the SAME tolerances, so any disagreement is a spatial
discretisation effect and nothing else:

S1 production Kirchhoff potential   q_f = D0 e^{-B/T_f} [Phi(C_R) - Phi(C_L)]/dx
S2 conservative finite difference   q_f = D(C_f) [C_R - C_L]/dx , D(C_f) midpoint

S2 is the textbook flux form; it shares no code path with the Kirchhoff primitive
(no `expi`, no primitive difference, no small-difference branch). A third check
ties both to an exact solution: with a constant D the Kirchhoff scheme collapses
algebraically to S2, and both must reproduce the cylindrical Robin Bessel series
to the same error the frozen production benchmark reports.

Triple cross-validation:
(a) limiting case: constant D -> S1 and S2 identical to round-off, and both
    agree with the Bessel series;
(b) variable D(C): S1 vs S2 differences must shrink under grid refinement;
(c) observed order from the scheme-pair difference (EXPECTED_ORDER declared
    before the numbers are inspected).

Run from the contest root:
C:\Python314\python.exe -B paper_output/code/verification/method_comparison.py [--quick]

Read-only with respect to given data, frozen results, model and settings; the
frozen solver file is not modified on disk. The alternative flux is injected at
run time into a private instance of the model.
"""
from __future__ import annotations

import json
import os
import pathlib
import sys
import time
from dataclasses import replace
from datetime import datetime, timezone

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
CODE = ROOT / "paper_output" / "code" / "modeling"
OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "method_v1"

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(CODE))

import numpy as np # noqa: E402

import drying_core as core # noqa: E402
```

```

FROZEN_BENCHMARK_D = 4.937655094173937e-09      # from results/bessel_validation
D_REFERENCE = 4.938e-09
EXPECTED_ORDER = 1.0                          # declared before inspecting numbers
DECLARED_TOLERANCE_KG_PER_KG = 1.0e-4          # S1 vs S2 after the finest refinement
TIMES_S = np.array([0., 1., 10., 60., 300., 600., 900., 1200., 1500., 1800.])
RADII_M = np.linspace(0.0, 0.02, 21)

ACTIVE = {"scheme": "kirchhoff"}

def midpointDiffusionFlux(model, T, C, D):
    """S2: conservative second-order finite-difference flux on the same faces.

    ACTIVE records which flux path actually ran, so a silently ineffective
    override can never be mistaken for agreement between two methods.
    """
    Dface = D[:-1] + (D[1:] - D[:-1]) * 0.5
    ACTIVE["scheme"] = "finiteDifference"
    return model.internal_faces * Dface * np.diff(C) / model.dx

def runScheme(scheme, intervals, kind):
    """Solve Q1 moisture with the chosen face operator; return sampled fields + event.

    solve_case() constructs its own RadialModel, so the alternative operator must be
    injected through the class, exactly as the other scenario islands do. The guard
    below fails loudly if the injection ever stops taking effect, because a silent
    no-op would otherwise be reported as perfect agreement between two methods.
    """
    settings = core.Settings(question="Q1", intervals=intervals, rtol=1e-10,
                              atol_temperature=1e-10, atol_moisture=1e-12,
                              max_step_s=2.0, early_max_step_s=2.0,
                              face_scheme="kirchhoff", jacobian_mode="finite_difference",
                              dense_storage="memory",
                              constant_D=(None if kind == "variable" else FROZEN_BENCHMARK_D))
    originalClass = core.RadialModel

    class FluxModel(originalClass):
        def water_internal_flux(self, T, C, D): # type: ignore[override]
            return midpointDiffusionFlux(self, T, C, D)

    ACTIVE["scheme"] = "kirchhoff"
    if scheme == "finiteDifference":
        core.RadialModel = FluxModel
    started = time.perf_counter()
    run = None
    try:
        run = core.solve_case(settings)
        T, C = run.fields(TIMES_S, radii_m=RADII_M)
        record = {"scheme": scheme, "intervals": intervals, "kind": kind,
                  "constantD": settings.constant_D,
                  "fluxPathUsed": ACTIVE["scheme"],
                  "temperatureK": T.tolist(), "moisture": C.tolist(),
                  "moistureFinite": bool(np.all(np.isfinite(C))),
                  "surfaceMoistureAt1800": float(C[-1, -1]),
                  "maxMoistureAt1800": float(np.max(C[-1])),
                  "massResidual": run.diagnostics()["max_mass_balance_abs_kg_per_kg"],
                  "elapsedSeconds": time.perf_counter() - started}
    finally:
        core.RadialModel = originalClass
        if run is not None:
            run.close()
    if record["fluxPathUsed"] != scheme:
        raise RuntimeError("Flux-path guard failed: scheme '" + scheme +

```

```

        "" but the run used "" + record["fluxPathUsed"] + "")
    return record

def compareFiniteDifferenceToKirchhoff(records):
    """Max |S1 - S2| over the common sampled grid, per refinement level."""
    rows = []
    levels = sorted({r["intervals"] for r in records})
    for n in levels:
        a = next(r for r in records if r["intervals"] == n and r["scheme"] == "kirchhoff")
        b = next(r for r in records if r["intervals"] == n and r["scheme"] == "finiteDifference")
        difference = np.abs(np.asarray(a["moisture"]) - np.asarray(b["moisture"]))
        rows.append({"intervals": n, "maxAbsDifference": float(np.nanmax(difference)),
                    "maxDifferenceTimeS": float(TIMES_S[int(np.nanargmax(difference) //
difference.shape[1]))],
                    "locationOfMax": float(RADII_M[int(np.nanargmax(difference) %
difference.shape[1]))])})
    order = None
    if len(rows) >= 2:
        coarse, fine = rows[-2]["maxAbsDifference"], rows[-1]["maxAbsDifference"]
        if coarse > 0 and fine > 0:
            order = float(np.log2(coarse / fine))
    passed = rows[-1]["maxAbsDifference"] <= DECLARED_TOLERANCE_KG_PER_KG
    return {"rows": rows, "observedOrder": order, "expectedOrder": EXPECTED_ORDER,
            "declaredToleranceKgPerKg": DECLARED_TOLERANCE_KG_PER_KG,
            "passedDeclaredTolerance": bool(passed)}

def main():
    quick = "--quick" in sys.argv
    OUT.mkdir(parents=True, exist_ok=True)
    levels = (100, 200) if quick else (200, 400, 800, 1600)
    records = []
    for kind in ("constant", "variable"):
        for n in levels:
            for scheme in ("kirchhoff", "finiteDifference"):
                record = runScheme(scheme, n, kind)
                records.append(record)
                print(json.dumps({"kind": kind, "scheme": scheme, "N": n,
                                "surfaceC1800": round(record["surfaceMoistureAt1800"], 10),
                                "elapsedS": round(record["elapsedSeconds"], 1)},
                                ensure_ascii=False, flush=True))

    constantRows = [r for r in records if r["kind"] == "constant"]
    # (a) limiting case: with constant D the Kirchhoff path and the finite-difference
    # path are the same discrete operator, so their samples must agree exactly.
    limitChecks = []
    for n in levels:
        a = next(r for r in constantRows if r["intervals"] == n and r["scheme"] == "kirchhoff")
        b = next(r for r in constantRows if r["intervals"] == n and r["scheme"] ==
"finiteDifference")
        limitChecks.append({"intervals": n,
                            "maxAbsDifference": float(np.max(np.abs(
                                np.asarray(a["moisture"]) - np.asarray(b["moisture"]))))})

    report = {
        "generatedAtUtc": datetime.now(timezone.utc).isoformat(),
        "note": ("Same moisture equation, same time integrator, same tolerances; only the "
                "face-flux discretisation differs. Any gap is a spatial discretisation "
                "effect. Frozen baseline and production results are untouched."),
        "levels": list(levels),
        "frozenBenchmarkD_m2_s": FROZEN_BENCHMARK_D,
        "referenceD_m2_s": D_REFERENCE,
        "declaredToleranceKgPerKg": DECLARED_TOLERANCE_KG_PER_KG,
        "limitingCaseConstantD": {
            "note": ("Kirchhoff collapses to the midpoint finite difference when D is constant, "

```

```

        "because  $\Phi'(C) = \exp(-a/C)$  makes the primitive difference proportional to "
        "D dC; the two must agree to round-off."),
    "rows": limitChecks},
    "variableD": compareFiniteDifferenceToKirchhoff(
        [r for r in records if r["kind"] == "variable"]),
    "records": records,
}
(OUT / "method_comparison.json").write_text(
    json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
print(json.dumps({k: report[k] for k in ("limitingCaseConstantD", "variableD")},
    ensure_ascii=False, indent=2)[:2000], flush=True)
return 0

if __name__ == "__main__":
    sys.exit(main())

```

E.9 threshold_and_scaling_checks.py: N800 阈值二分及 R 平方时标归约

文件: paper_output/code/verification/threshold_and_scaling_checks.py; 共 175 行。
采用范围: 正文 N800 同轨迹事件复核; N200 同物性控制时刻和 $R(t)$ 时标积分。

版本说明与勘误: 二分共享数值轨迹且初始括区依赖事件; R 平方积分上限也取仿真时刻。文件序言中 “all sampled centre” 不是严格逐节点唯一极值, 实际 JSON 披露浮点并列; 不将这些旧措辞作为正文结论。

```
"""Threshold-definition robustness and an analytical cross-check of the shrinkage effect.
```

```
Part A (instant, no solver): the pure  $R^2$  diffusion-scaling test.
```

```
If the only effect of shrinkage were the shrinking diffusion length, then the equivalent fixed-radius time
```

```
tau_eq = integral_0^{t*_shrink} (R0 / R(t))^2 dt
should equal the computed fixed-radius drying time t*_fixed. The gap between tau_eq and t*_fixed measures how much of the difference the scaling argument explains. Both t* values come from the same-property (attachment 4) control runs.
```

```
Part B (solver): independent threshold root and max-location certificate.
```

```
The production code finds the crossing with the ODE event mechanism. Here the same conclusion is re-derived by bisection on  $M(t) = \max_x C(x, t)$  evaluated from the dense output on an independent time grid, so the reported time no longer depends on the event solver. Also certifies that the maximum is at the centre ( $x = 0$ ) at all sampled times, and re-checks the strict inequality with unrounded values.
```

```
Read-only with respect to given data, frozen results, model and settings.
```

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
import os
```

```
import pathlib
```

```
import sys
```

```
import tempfile
```

```
import time
```

```
import uuid
```

```
from dataclasses import replace
```

```
ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
```

```
def _mkdtemp(suffix=None, prefix=None, dir=None):
```

```
    base = pathlib.Path(dir)
```

```
    for _ in range(200):
```

```
        candidate = base / ((prefix or "tmp") + uuid.uuid4().hex[:12] + (suffix or ""))
```

```
        if not candidate.exists():
```

```
            candidate.mkdir(parents=False)
```

```
            return str(candidate)
```

```
    raise FileExistsError("no unused temp name")
```

```
tempfile.mkdtemp = _mkdtemp
```

```
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
```

```
os.chdir(ROOT)
```

```
sys.dont_write_bytecode = True
```

```
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))
```

```
import io # noqa: E402
```

```

import numpy as np # noqa: E402

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "threshold_scaling_v1"
R0 = 0.02

def read_radius():
    text = (ROOT / "paper_output/data_cleaned/A_radius_observed.csv").read_bytes().decode("utf-8-sig")
    return np.genfromtxt(io.StringIO(text), delimiter=",", names=True)

def scaling_check():
    rad = read_radius()
    t = rad["time_s"].astype(float)
    r = rad["radius_m"].astype(float)
    t_fixed_s = 129.84522834701946 * 3600.0 # N200, attachment-4 properties, fixed radius
    t_shrink_s = 51.090973419826916 * 3600.0 # N200, attachment-4 properties, shrinking
    grid = np.linspace(0.0, t_shrink_s, 200001)
    r_of_t = np.interp(grid, t, r)
    integrand = (R0 / r_of_t) ** 2
    tau_eq = float(np.trapezoid(integrand, grid))
    # trapezoid is available as np.trapezoid on NumPy >= 2.0; fall back if needed
    return {
        "fixedRadiusTimeH": t_fixed_s / 3600.0,
        "shrinkingTimeH": t_shrink_s / 3600.0,
        "reductionPercent": float(100.0 * (1.0 - t_shrink_s / t_fixed_s)),
        "equivalentFixedRadiusTimeH": tau_eq / 3600.0,
        "ratioEquivalentToFixed": float((tau_eq / 3600.0) / (t_fixed_s / 3600.0)),
        "scalingResidualPercent": float(
            100.0 * ((tau_eq / 3600.0) / (t_fixed_s / 3600.0) - 1.0)),
        "interpretation": ("Stretching the shrinking timeline by (R0/R(t))**2 gives an "
                           "equivalent fixed-radius time within about 1.4 percent of the "
                           "directly simulated fixed-radius drying time. The numerically "
                           "computed shortening is therefore corroborated by an independent "
                           "analytical length-scale argument; the small residual comes from "
                           "the coupled changes in the moisture and temperature profiles."),
    }

def solver_checks(intervals=800):
    from drying_core import Settings, solve_case

    base = Settings(intervals=intervals, rtol=1e-10, atol_temperature=1e-10,
                    atol_moisture=1e-12, early_max_step_s=2.0, max_step_s=120.0,
                    face_scheme="kirchhoff", jacobian_mode="analytic")
    records = {}
    for question, shrink in (("Q23", False), ("Q4", True)):
        started = time.perf_counter()
        run = solve_case(replace(base, question=question, shrink=shrink))
        try:
            t_event = float(run.event_s)

            def max_c(t):
                state = run.state([t])
                return float(np.max(state[1:-1:2, 0]))

            # independent bisection on M(t) - 0.15, bracketed one hour around the event
            lo, hi = t_event - 3600.0, t_event + 3600.0
            for _ in range(80):
                mid = 0.5 * (lo + hi)
                if max_c(mid) - 0.15 > 0.0:
                    lo = mid
                else:
                    hi = mid
            t_root = 0.5 * (lo + hi)

```

```

# max-location certificate and monotonicity of the profile
grid = np.linspace(0.0, t_event, 2001)
# run.state returns (2*n+1, len(times)); concatenate along TIME (axis=1).
# np.vstack was wrong here and also broke on the final short chunk.
states = np.hstack([run.state(grid[i:i + 200]) for i in range(0, len(grid), 200)])
cs = states[1:-1:2, :]
argmax_x = run.model.x[np.argmax(cs, axis=0)]
profile_increase = float(np.max(np.diff(cs, axis=0)))
# The argmax of a radially flat profile is numerically degenerate: several
# nodes can agree to round-off. Record the margin so the certificate says
# "the maximum is at the centre up to X" instead of a bare boolean.
centre_margin = np.max(cs, axis=0) - cs[0, :]

strict_time = float(np.ceil(t_event) + 1.0)
strict_max_c = max_c(strict_time)
records[question] = {
    "intervals": intervals,
    "solverEventTimes": t_event,
    "solverEventTimeH": t_event / 3600.0,
    "independentBisectionTimeS": t_root,
    "independentBisectionTimeH": t_root / 3600.0,
    "eventVsBisectionDifferenceS": t_event - t_root,
    "maxLocationUniqueX": sorted(set(np.round(argmax_x, 12).tolist())),
    "maxLocationAlwaysCentre": bool(np.all(np.abs(argmax_x) < 1e-12)),
    "maxExcessOverCentreMaxKgPerKg": float(np.max(centre_margin)),
    "maxExcessOverCentreMedianKgPerKg": float(np.median(centre_margin)),
    "centreWithinStrictThresholdMarginKgPerKg": float(0.15 - float(np.max(cs[0, :]))),
    "centreIsMaximumAtEvent": bool(np.argmax(cs[:, -1]) == 0),
    "maxLocationNote": ("Off-centre argmax appears only where the radial profile is
flat "
                        "to round-off; the excess over the centre value is reported "
                        "above and is many orders below any physically meaningful "
                        "moisture difference."),
    "maxRadialMoistureIncrease": profile_increase,
    "strictCheckTimeS": strict_time,
    "unroundedMaxCATStrictTime": strict_max_c,
    "strictlyBelowThreshold": bool(strict_max_c < 0.15),
}
finally:
    run.close()
    records[question]["elapsedSeconds"] = time.perf_counter() - started
    print(json.dumps({question: records[question]}, ensure_ascii=False), flush=True)
return records

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    result = {"scalingCheck": scaling_check()}
    if "--with-solver" in sys.argv:
        result["thresholdChecks"] = solver_checks()
    (OUT / "threshold_and_scaling_checks.json").write_text(
        json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(result["scalingCheck"], ensure_ascii=False, indent=2), flush=True)
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

E. 10 energy_balance_check.py: 有效热方程瞬时加权通量检验

文件: paper_output/code/verification/energy_balance_check.py; 共 245 行。采用范围: 第 6 章与推导附录仅采纳 rateIdentity() 的实际加权 $B \cdot T_{\text{dot}}$ 运算。

版本说明与勘误: 必须完整附源码并在文件前说明: 序言 /JSON 中 $d/dt \sum(2wBR^2)$ 写法错误, 实际是 $\sum(2wBR^2 \cdot T_{\text{dot}})$ 。CHECK B 不构成变容量/收缩总能量守恒; 不采纳累计残差已 100% 解释等旧主张。

```
"""Energy-conservation certificate for the thermal side of the frozen model.

Why this layer was missing
Mass has a discrete conservation identity ( $2 \cdot \sum(w_i C_i) + A = C_0$ ) and it is
checked on every run. The thermal side had only grid refinement and an
analytic benchmark; it had no independent invariant. This script derives and
tests one.

Derivation, and an honest statement of what the model conserves
The frozen temperature equation is the EFFECTIVE-CAPACITY form
 $B(C) T_t = (1/r) (r k T_r)_r$ ,  $B = \rho_{\text{eff}} c_p$ ,
not a strict enthalpy equation: B depends on the moisture field, so  $d(B T)/dt$  is
not  $B dT/dt$  and integral  $B T dV$  is not the conserved quantity of this model.

Summing the discrete equations with the weights  $2 w_i R(t)^2$  cancels the
 $1/(R^2 w_i)$  prefactor exactly; every interior face then appears twice with
opposite signs and cancels pairwise, leaving only the surface face. In this
code the surface heat face carries the radius itself,


$$g_{\{N+1/2\}}^T = -h R (T_s - T_{\text{inf}}),$$


so after the weighting the exchange rate is  $2 h R (T_{\text{inf}} - T_s)$ . Two separate
statements follow, and they are NOT equally strong:

CHECK A rate identity (the certificate). For the capacity actually used at the
evaluation instant,

$$d/dt \sum_i 2 w_i B_i T_i R^2 = 2 h R (T_{\text{inf}} - T_s)$$
 exactly
which is verified below by comparing the model's own right-hand side with the
surface face, with no quadrature and no trajectory differencing involved.

CHECK B cumulative balance. Because the rate identity involves B at the
evaluation instant while the solver advances  $B(C) T_t$ , the cumulative form is
exact only for a capacity frozen in time:

$$H_0(t) = \sum_i 2 w_i B_i(0) T_i(t) R(t)^2,$$


$$H_0(t) - E(t) = H_0(0), \quad dE/dt = 2 h R (T_{\text{inf}} - T_s).$$

This is reported per question, and its residual is dominated by the quadrature
of E rather than by the spatial operator.

History, kept because each step was a real defect and each was caught only by
the residual refusing to shrink under refinement: the first version used the
instantaneous B but dropped the radius factor from the surface term (a
factor-of-fifty error at  $R = 0.02$  m); the second used the correct radius but
instantaneous B in the cumulative form (Q23 residual 0.428 relative). Both
grid-independent failures meant the identity, not the solver, was wrong.

Read-only with respect to given data, frozen results, model and settings. Writes
only its own result directory.
"""
from __future__ import annotations

import json
import os
```

```

import pathlib
import sys
import time
from dataclasses import replace
from datetime import datetime, timezone

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import numpy as np # noqa: E402
from scipy.integrate import solve_ivp # noqa: E402

import drying_core as core # noqa: E402

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "energy_balance_v1"
DECLARED_TOLERANCE_J_PER_M = 1.0e-3 # declared before the numbers are inspected

def energyResidual(model, times, states, boundaryExchange, kernel):
    """CHECK B:  $|H_0(t) - E(t) - H_0(0)|$  with the capacity frozen at  $t = 0$ ."""
    residuals = []
    for index, t in enumerate(times):
        y = states[:, index]
        T = y[:-1:2]
        radius = float(model.radius(t))
        stored = float(2.0 * (model.w * kernel * T).sum() * radius ** 2)
        residuals.append(abs(stored - boundaryExchange[index] - INITIAL[0]))
    return np.asarray(residuals)

def rateIdentity(model, times, states, settings):
    """CHECK A: the model's own  $dH/dt$  compared with the surface face, no quadrature."""
    rows = []
    for index, t in enumerate(times):
        y = states[:, index]
        T = y[:-1:2]
        rho, cp, _k, _D = model.properties(T, y[1:-1:2])
        radius = float(model.radius(t))
        tair, _ceq = model.environment(t)
        derivative = model.rhs(t, y)
        lhs = float(2.0 * (model.w * rho * cp * derivative[:-1:2]).sum() * radius ** 2)
        rhs = 2.0 * settings.h * radius * (float(tair) - float(T[-1]))
        rows.append({"timeS": float(t), "modelRate": lhs, "surfaceFaceRate": rhs,
                    "ratio": (lhs / rhs) if rhs != 0.0 else None})
    finite = [row for row in rows if row["ratio"] is not None]
    worst = max((abs(row["ratio"] - 1.0) for row in finite), default=None)
    return {"rows": rows, "maxRelativeDeviationFromOne": worst}

INITIAL = [0.0]

def runCase(question, intervals, shrink, sampleCount=41):
    settings = core.Settings(question=question, intervals=intervals, shrink=shrink,
                             rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
                             early_max_step_s=2.0, max_step_s=120.0, horizon_h=6.0,
                             face_scheme="kirchhoff", jacobian_mode="analytic",
                             dense_storage="memory")
    record = {"question": question, "intervals": intervals, "shrink": shrink,
             "horizonH": settings.horizon_h}
    started = time.perf_counter()
    run = core.solve_case(settings)

```

```

try:
    model = run.model
    end = float(run.end_s)
    times = np.linspace(0.0, end, sampleCount)
    states = run.state(times)
    # CHECK A needs no capacity choice at all: it compares the right-hand side
    # with the surface face at the same instant.
    rate = rateIdentity(model, times, states, settings)

    # CHECK B uses the capacity frozen at t = 0, which is the only form in which
    # the cumulative balance is exact for this effective-capacity model.
    radius0 = float(model.radius(times[0]))
    T0, C0 = states[:-1:2, 0], states[1:-1:2, 0]
    rho0, cp0, _k0, _D0 = model.properties(T0, C0)
    kernel = rho0 * cp0
    INITIAL[0] = float(2.0 * (model.w * kernel * T0).sum() * radius0 ** 2)

    def exchangeRate(t, y):
        T = y[:-1:2]
        tair, _ceq = model.environment(t)
        radius = float(model.radius(t))
        # The frozen surface heat face is -h R (T_s - T_inf); after the R^2
        # weighting the exchange rate is +2 h R (T_inf - T_s). Omitting R is a
        # factor-of-fifty error at R = 0.02 m and was the first real defect
        # found in this check.
        return 2.0 * settings.h * radius * (float(tair) - float(T[-1]))

    grid = np.linspace(0.0, end, 4001)
    rates = np.array([exchangeRate(t, run.state([t])[:, 0]) for t in grid])
    cumulative = np.concatenate([[0.0], np.cumsum(
        0.5 * (rates[1:] + rates[:-1]) * np.diff(grid))])
    boundaryExchange = np.interp(times, grid, cumulative)

    residuals = energyResidual(model, times, states, boundaryExchange, kernel)

    # Quantify the energy the effective-capacity closure discards. CHECK B is
    # exact only for a capacity frozen in time; the missing piece is the work
    # done against the changing capacity,  $\sum_i \frac{1}{2} w_i (dB_i/dt) T_i R^2$ , which
    # this effective-capacity model does not carry. Reporting it turns the
    # CHECK B residual into a physical statement instead of an unexplained gap.
    omitted = []
    for index in range(1, len(times)):
        t0, t1 = float(times[index - 1]), float(times[index])
        y1 = states[:, index]
        y0 = states[:, index - 1]
        r1 = float(model.radius(t1))
        r0 = float(model.radius(t0))
        rho1, cp1, _k1, _D1 = model.properties(y1[:-1:2], y1[1:-1:2])
        rho0s, cp0s, _k0s, _D0s = model.properties(y0[:-1:2], y0[1:-1:2])
        b1 = 2.0 * (model.w * rho1 * cp1 * y1[:-1:2]).sum() * r1 ** 2
        b0 = 2.0 * (model.w * rho0s * cp0s * y0[:-1:2]).sum() * r0 ** 2
        omitted.append(float(b1 - b0) - float(
            2.0 * (model.w * kernel * (y1[:-1:2] - y0[:-1:2]).sum() * r1 ** 2))
    omittedTotal = float(np.sum(omitted))

    record.update({
        "initialStoredPerMetre": INITIAL[0],
        "rateIdentityMaxRelativeDeviation": rate["maxRelativeDeviationFromOne"],
        "rateIdentityRows": rate["rows"],
        "cumulativeMaxAbsoluteResidualJPerM": float(residuals.max()),
        "cumulativeRelativeResidual": float(residuals.max() / abs(INITIAL[0])),
        "discardedCapacityWorkJPerM": omittedTotal,
        "discardedRelativeToStored": float(omittedTotal / abs(INITIAL[0])),
        "unexplainedResidualJPerM": float(residuals[-1] - omittedTotal),
        "declaredToleranceJPerM": DECLARED_TOLERANCE_J_PER_M,
        "cumulativePassed": bool(residuals.max() <= DECLARED_TOLERANCE_J_PER_M),

```

```

        "sampleTimesS": times.tolist(),
        "cumulativeResidualsJPerM": residuals.tolist(),
        "maxMassResidualKgPerKg": run.diagnostics()["max_mass_balance_abs_kg_per_kg"],
    })
    finally:
        run.close()
    record["elapsedSeconds"] = time.perf_counter() - started
    print(json.dumps({k: record[k] for k in
        ("question", "intervals", "rateIdentityMaxRelativeDeviation",
         "cumulativeRelativeResidual", "discardedCapacityWorkJPerM",
         "unexplainedResidualJPerM")},
        ensure_ascii=False), flush=True)
    return record

def refinementCheck(question, shrink, levels=(100, 200, 400)):
    """The cumulative residual must fall under refinement if it is discretisation error."""
    rows = []
    for n in levels:
        row = runCase(question, n, shrink)
        rows.append({"intervals": n,
            "rateIdentityMaxRelativeDeviation":
                row["rateIdentityMaxRelativeDeviation"],
            "cumulativeMaxAbsoluteResidualJPerM":
                row["cumulativeMaxAbsoluteResidualJPerM"]})
    return {"question": question, "rows": rows}

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    cases = [runCase("Q1", 200, False), runCase("Q23", 200, False),
        runCase("Q4", 200, True)]
    refinement = [refinementCheck("Q23", False), refinementCheck("Q4", True)]
    report = {
        "checkA_rateIdentity": (
            "d/dt sum_i 2 w_i B_i T_i R^2 = 2 h R (T_inf - T_s), verified by comparing the "
            "model's own right-hand side with the surface face at the same instant; no "
            "quadrature and no trajectory differencing are involved"),
        "checkB_cumulative": (
            "H0(t) = sum_i 2 w_i B_i(0) T_i(t) R(t)^2 with dE/dt = 2 h R (T_inf - T_s); "
            "exact for a capacity frozen in time, which is the only form available for this "
            "effective-capacity model"),
        "derivation": ("Summing the discrete temperature equations with weights 2 w_i R^2 makes "
            "every interior face cancel pairwise, leaving only the surface face, "
            "which in this code is -h R (T_s - T_inf)."),
        "whyIndependent": ("Neither check uses the mass bookkeeping variable or the moisture "
            "equation; both test only the thermal operator against its boundary "
            "face. The two conventions are reported separately and must not be "
            "conflated."),
        "declaredToleranceJPerM": DECLARED_TOLERANCE_J_PER_M,
        "generatedAtUtc": datetime.now(timezone.utc).isoformat(),
        "cases": cases,
        "refinement": refinement,
    }
    (OUT / "energy_balance.json").write_text(
        json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

E. 11 sensitivity_analysis.py: 修复后的D/k单参数扫描入口

文件: paper_output/code/verification/sensitivity_analysis.py; 共 292 行。采用范围: 仅采纳 dscale、kscale 分支及对应基准自检; N200, D 倍率 0.7–1.4、k 倍率 0.85–1.15。

版本说明与勘误: 完整保留 Morris/Sobol 函数, 不删源码。历史 Morris 的 D 仅缩放 properties 而未改变 Kirchhoff 通量, D 效应为零是失效注入; 后来的 water_internal_flux 修复及单参数扫描不使旧 Morris 或 Sobol 结果有效。默认无参数会进入 morris, 不是论文采纳的入口。

```
"""Global sensitivity analysis of the A-problem drying time.
```

```
Two stages, both on a coarse but validated surrogate grid:
```

```
stage 1 Morris elementary-effects screening (mu*, sigma) for all parameters
```

```
stage 2 Saltelli/Jansen variance-based Sobol indices for the influential subset
```

```
Why a coarse grid is legitimate here
```

```
The reported event time at N=200 differs from the production N=3200/N=6400
value by about 1e-3 h (Q23) and 4e-4 h (Q4), i.e. about 1e-5 relative. The
sensitivity ranking is therefore unaffected by the grid, and the surrogate
makes a few hundred model evaluations affordable. The grid-induced offset is
reported alongside the indices rather than hidden.
```

```
Uncertain inputs (all are closure/parameter uncertainties, not fitted quantities)
```

```
tailTemperatureC      plateau extension temperature
tailEquilibrium        plateau extension equilibrium moisture
h                     convective heat transfer coefficient
beta                  convective mass transfer coefficient
surfaceLatentFraction  fraction of the drainage flux charged as latent heat
equilibriumScale       scale on the whole air/material equilibrium mapping
dScale                 multiplicative uncertainty on the D empirical formula
kScale                 multiplicative uncertainty on the k empirical formula
```

```
Implementation note
```

```
dScale and kScale are not native settings. They are applied by wrapping
RadialModel.properties in this script only; the frozen solver file is not
modified. That makes the analytic Jacobian inconsistent, so every run here uses
the sparsity-coloured finite-difference Jacobian instead, which is consistent
with the wrapped right-hand side. A control run verifies that with all scales
at their baseline the wrapped model reproduces the unpatched result exactly.
```

```
Read-only with respect to given data, frozen results, model and settings.
```

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
import math
```

```
import os
```

```
import pathlib
```

```
import sys
```

```
import time
```

```
import uuid
```

```
from dataclasses import replace
```

```
ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
```

```
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
```

```
os.chdir(ROOT)
```

```
sys.dont_write_bytecode = True
```

```

sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import numpy as np # noqa: E402

import drying_core # noqa: E402
from drying_core import RadialModel, Settings, solve_case # noqa: E402
import q3_model # noqa: E402

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "sensitivity_v1"
INTERVALS = 200
SURROGATE_EVENT_H = {"Q23": 57.4727587858255, "Q4": 51.090973419826916}
PRODUCTION_EVENT_H = {"Q23": 57.47230195056044, "Q4": 51.09057478683054}

SCALES = {"d": 1.0, "k": 1.0}
_ORIGINAL_PROPERTIES = RadialModel.properties
_ORIGINAL_WATER_FLUX = RadialModel.water_internal_flux

def _scaled_properties(self, T, C):
    rho, cp, k, D = _ORIGINAL_PROPERTIES(self, T, C)
    return rho, cp, k * SCALES["k"], D * SCALES["d"]

def _scaled_water_flux(self, T, C, D):
    """Scale the internal water flux, which is the only place D enters the model.

    IMPORTANT: under face_scheme='kirchhoff' the solver does not use the D array
    returned by properties(); it evaluates the Kirchhoff primitive with a
    hard-coded per-question D0 (see drying_core.water_internal_flux, the line
    `return self.internal_faces * D0 * thermal_factor * difference / self.dx`).
    Scaling properties() alone therefore has NO effect on dScale, which is why an
    earlier Morris run reported mu* = 0 for dScale. The Kirchhoff flux is exactly
    linear in D0, so multiplying the returned flux by the scale factor is the exact
    realisation of a D pre-factor uncertainty. The surface Robin flux uses beta and
    is deliberately NOT scaled.
    """
    return _ORIGINAL_WATER_FLUX(self, T, C, D) * SCALES["d"]

RadialModel.properties = _scaled_properties
RadialModel.water_internal_flux = _scaled_water_flux

# name, unit, baseline, low, high
PARAMETERS = [
    ("tailTemperatureC", "degC", 50.0, 48.0, 52.0),
    ("tailEquilibrium", "kg/kg", 0.05, 0.04, 0.06),
    ("h", "W/(m2 K)", 25.0, 20.0, 30.0),
    ("beta", "m/s", 8e-7, 6.4e-7, 9.6e-7),
    ("surfaceLatentFraction", "-", 0.0, 0.0, 1.0),
    ("equilibriumScale", "-", 1.0, 0.8, 1.2),
    ("dScale", "-", 1.0, 0.7, 1.4),
    ("kScale", "-", 1.0, 0.85, 1.15),
]
NAMES = [p[0] for p in PARAMETERS]

def settings_for(question, values):
    shrink = question == "Q4"
    kwargs = {"tail_temperature_C": values["tailTemperatureC"],
              "tail_equilibrium": values["tailEquilibrium"],
              "h": values["h"], "beta": values["beta"],
              "surface_latent_fraction": values["surfaceLatentFraction"],
              "equilibrium_scale": values["equilibriumScale"]}
    return Settings(question=question, intervals=INTERVALS, shrink=shrink,
                    rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
                    early_max_step_s=2.0, max_step_s=120.0,

```

```

        face_scheme="kirchhoff", jacobian_mode="finite_difference",
        **kwargs)

FAILURES = []

def evaluate(question, unitPoint):
    """unitPoint: dict name -> value in [0, 1]; returns drying time in hours (or nan)."""
    values = {}
    for name, _unit, low, high in ((p[0], p[1], p[3], p[4]) for p in PARAMETERS):
        values[name] = low + unitPoint[name] * (high - low)
    SCALES["d"] = values["dScale"]
    SCALES["k"] = values["kScale"]
    try:
        run = solve_case(settings_for(question, values))
    except Exception as error: # a failed evaluation is recorded, never silently dropped
        FAILURES.append({"question": question, "values": values,
            "error": f"{type(error).__name__}: {error}"})
        return float("nan")
    try:
        return run.event_s / 3600.0
    finally:
        run.close()

def baseline_point():
    return {name: (base - low) / (high - low)
        for name, _unit, base, low, high in PARAMETERS}

def full_point(subset, row):
    """Expand a subset sample to a full unit point using baseline values elsewhere."""
    point = baseline_point()
    for name, value in zip(subset, row):
        point[name] = float(value)
    return point

def control_check():
    """All scales at baseline must reproduce the unpatched surrogate exactly."""
    SCALES["d"] = SCALES["k"] = 1.0
    base = Settings(question="Q23", intervals=INTERVALS, rtol=1e-10,
        atol_temperature=1e-10, atol_moisture=1e-12,
        early_max_step_s=2.0, max_step_s=120.0,
        face_scheme="kirchhoff", jacobian_mode="analytic")
    run = solve_case(base)
    reference = run.event_s / 3600.0
    run.close()
    patched = evaluate("Q23", baseline_point())
    return {"unpatchedAnalyticJacobianH": reference,
        "patchedFiniteDifferenceH": patched,
        "differenceSeconds": abs(reference - patched) * 3600.0,
        "referenceProductionN3200H": PRODUCTION_EVENT_H["Q23"],
        "surrogateOffsetVsProductionSeconds":
            abs(reference - PRODUCTION_EVENT_H["Q23"]) * 3600.0}

def morris(question, trajectories, levels=4):
    delta = levels / (2.0 * (levels - 1))
    grid = np.linspace(0.0, 1.0 - delta, levels)
    d = len(PARAMETERS)
    effects = {name: [] for name in NAMES}
    rng = np.random.default_rng(20260911)
    started = time.perf_counter()
    for t in range(trajectories):

```

```

base = rng.choice(grid, size=d)
point = {name: float(base[i]) for i, name in enumerate(NAMES)}
y0 = evaluate(question, point)
order = rng.permutation(d)
for index in order:
    name = NAMES[index]
    nxt = dict(point)
    nxt[name] = min(1.0, point[name] + delta)
    y1 = evaluate(question, nxt)
    effects[name].append((y1 - y0) / delta)
    point, y0 = nxt, y1
if (t + 1) % 5 == 0:
    print(f" morris {question} trajectory {t+1}/{trajectories} "
          f"({time.perf_counter()-started:.0f}s)", flush=True)
summary = []
for name in NAMES:
    arr = np.asarray(effects[name], dtype=float)
    summary.append({"parameter": name,
                    "muStar": float(np.mean(np.abs(arr))),
                    "mu": float(np.mean(arr)),
                    "sigma": float(np.std(arr, ddof=1)),
                    "min": float(arr.min()), "max": float(arr.max()),
                    "elementaryEffects": arr.tolist()})
summary.sort(key=lambda row: row["muStar"], reverse=True)
return {"question": question, "trajectories": trajectories, "levels": levels,
        "delta": delta, "runs": trajectories * (d + 1),
        "elapsedSeconds": time.perf_counter() - started, "ranking": summary}

def sobol(question, subset, baseSamples, seed=7):
    """Saltelli/Jansen first-order and total-order indices on a parameter subset."""
    d = len(subset)
    rng = np.random.default_rng(seed)
    A = rng.random((baseSamples, d))
    B = rng.random((baseSamples, d))
    fA = np.array([evaluate(question, full_point(subset, row)) for row in A])
    fB = np.array([evaluate(question, full_point(subset, row)) for row in B])
    var_y = float(np.var(np.concatenate([fA, fB]), ddof=1))
    rows = []
    started = time.perf_counter()
    for j, name in enumerate(subset):
        AB = A.copy()
        AB[:, j] = B[:, j]
        fAB = np.array([evaluate(question, full_point(subset, row)) for row in AB])
        # Jansen estimators
        s1 = float(np.mean(fB * (fAB - fA)) / var_y) if var_y > 0 else float("nan")
        st = float(np.mean((fA - fAB) ** 2) / (2.0 * var_y)) if var_y > 0 else float("nan")
        rows.append({"parameter": name, "firstOrder": s1, "totalOrder": st,
                     "interaction": st - s1})
    print(f" sobol {question} {name}: S1={s1:.4f} ST={st:.4f} "
          f"({time.perf_counter()-started:.0f}s)", flush=True)
    return {"question": question, "subset": subset, "baseSamples": baseSamples,
            "runs": baseSamples * (d + 2), "outputVariance": var_y,
            "indices": rows, "elapsedSeconds": time.perf_counter() - started}

def scan(stageName, parameter, values, questions=("Q23", "Q4")):
    """One-parameter scan used to repair the dScale entry and to validate kScale."""
    rows = []
    for question in questions:
        for value in values:
            point = baseline_point()
            low = next(p[3] for p in PARAMETERS if p[0] == parameter)
            high = next(p[4] for p in PARAMETERS if p[0] == parameter)
            point[parameter] = (value - low) / (high - low)
            y = evaluate(question, point)

```

```

        rows.append({"question": question, "parameter": parameter,
                    "value": value, "eventH": y})
        print(f" scan {question} {parameter}={value:g} -> {y:.6f} h", flush=True)
    byQ = {}
    for question in questions:
        ys = [r["eventH"] for r in rows if r["question"] == question]
        lo, hi = min(ys), max(ys)
        base = next((r["eventH"] for r in rows
                    if r["question"] == question and abs(r["value"] - 1.0) < 1e-12), None)
        byQ[question] = {"minH": lo, "maxH": hi, "spanH": hi - lo,
                        "baselineH": base,
                        "relativeSpanPercent": None if not base else 100.0 * (hi - lo) / base}
    return {"stage": stageName, "parameter": parameter, "rows": rows, "summary": byQ}

def main():
    stage = sys.argv[1] if len(sys.argv) > 1 else "morris"
    OUT.mkdir(parents=True, exist_ok=True)
    if stage == "control":
        result = {"control": control_check(),
                "surrogateEventH": SURROGATE_EVENT_H,
                "productionEventH": PRODUCTION_EVENT_H}
    elif stage == "morris":
        trajectories = int(sys.argv[2]) if len(sys.argv) > 2 else 20
        result = {"control": control_check(),
                "morris": [morris(q, trajectories) for q in ("Q23", "Q4")]}
    elif stage == "dscale":
        result = {"control": control_check(),
                "scan": scan("dscale", "dScale", [0.7, 0.875, 1.0, 1.05, 1.225, 1.4])}
    elif stage == "kscale":
        result = {"scan": scan("kscale", "kScale", [0.85, 0.925, 1.0, 1.075, 1.15])}
    elif stage == "sobol":
        subset = sys.argv[2].split(",")
        base = int(sys.argv[3]) if len(sys.argv) > 3 else 64
        result = {"sobol": [sobol(q, subset, base) for q in ("Q23", "Q4")]}
    else:
        raise SystemExit(f"unknown stage {stage}")
    path = OUT / f"{stage}.json"
    path.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(result.get("scan", {}).get("summary", {}), ensure_ascii=False),
          flush=True)
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

E. 12 isotherm_activity_closure.py: 经验边界阻力 p 情景入口

文件: paper_output/code/verification/isotherm_activity_closure.py; 共 494 行。采用范围: 仅采纳 N800、 $p=1/2/4$ 、latent=0 的事件结果与实际 partitionFactor/ActivityModel 运算。

版本说明与勘误: 完整保留原注释和分支, 但文件前必须勘误: 数据未强制材料平衡点; awRef 不进入 partitionFactor; anchor 零差并非与计算 RH 相等; $p<1$ 必负、基线真实下界等旧说明不成立。该批仅到事件, 非生产严格报告。

```
"""Scenario family: sorption-isotherm closure of the surface mass-transfer drive.
```

```
What the given data actually fix
```

```
Attachment 1 gives the chamber humidity ratio  $Y_{\text{inf}}(t)$  and temperature  $T_{\text{inf}}(t)$ .  
Converting to a vapour pressure and comparing with the saturation pressure gives  
a chamber relative humidity that settles to 0.60-0.62 once the chamber reaches  
its plateau (mean over the 4 h window: 0.623).
```

```
The statement supplies no sorption isotherm, but it does fix an equilibrium  
point of the material:  $C_{\text{eq}} = 0.05$  kg/kg at the plateau. A material that is  
genuinely at equilibrium there can neither gain nor lose water, so its water  
activity at that state is forced:
```

$$a_w(C = 0.05, T_{\text{inf}} = 50.3 \text{ C}) = RH_{\text{inf}} = \sim 0.60.$$

```
This is not an assumption imported from literature; it follows from the two  
numbers the statement itself provides. It is the anchor of the family below.
```

```
What this changes about the frozen baseline
```

```
Exact algebra (not an approximation) shows the baseline surface flux
```

```
 $j = \beta \rho_d [C_s - C_{\text{eq}}]$  with  $C_{\text{eq}} = Y_{\text{inf}}$   
is identical to a vapour-pressure drive in which the material surface is taken  
to be at water activity one:
```

$$j = \rho_d \beta [Y_{\text{sat}}(T_{\text{inf}}) - Y_{\text{inf}}] \text{ when } C_s \rightarrow C_{\text{eq}}.$$

```
So the baseline implicitly assumes  $a_w(C_s = 0.05) = 1$ , whereas the data force  
0.60. The baseline therefore over-states the late-stage driving force, and the  
reported drying times are a LOWER BOUND within every closure of this family.
```

```
The family
```

```
Monotone, anchored at the forced point, one shape parameter  $p$ :
```

```
 $a_w(C) = 1 - (1 - a_{w\text{Ref}}) (C_{\text{ref}} / C)^{(1/p)}$ ,  $C \geq C_{\text{ref}}$   
 $p = 1$  is the mildest member that still satisfies the anchor; larger  $p$  drives  
 $a_w$  down faster for a given  $C$ , i.e. a more strongly water-binding material.  
Every member is a legitimate scenario; none of them is claimed to be the  
material's true isotherm, which the statement does not provide.
```

```
Controls built into this script
```

```
* anchor check:  $a_w(C_{\text{ref}})$  must equal the chamber plateau relative humidity;  
* baseline-limit check:  $a_w = 1$  with  $T_s = T_{\text{inf}}$  reproduces the frozen baseline  
drive coefficient to machine precision;  
* trajectories are recorded even when no drying event fires, so "not dry within  
the horizon" is a measured statement and never a blank.
```

```
Read-only with respect to given data, frozen model, settings and results.
```

```
Run from the contest root:
```

```
C:\Python314\python.exe -B paper_output/code/verification/isotherm_activity_closure.py [N]  
[--long-horizon]
```

```
from __future__ import annotations
```

```

import json
import os
import pathlib
import sys
import time
import traceback
from datetime import datetime, timezone

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
CL = ROOT / "paper_output" / "data_cleaned"
OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "isotherm_closure_v1"

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import io # noqa: E402

import numpy as np # noqa: E402
import scipy.integrate._ivp.common # noqa: E402
from scipy.optimize._numdiff import approx_derivative # noqa: E402

import drying_core as core # noqa: E402

# ----- constants
KAPPA = 0.621945 # M_water / M_air
P_ATM = 101325.0 # Pa
C_REF = 0.05 # kg/kg, the equilibrium moisture stated in the problem
AW_REF_DEFAULT = 0.60 # chamber plateau relative humidity (computed below)
LATENT_J_KG = 2.4e6

def pSat(TC):
    """Magnus saturation vapour pressure over water, Pa; TC in degrees Celsius."""
    return 610.94 * np.exp(17.625 * TC / (TC + 243.04))

def chamberRelativeHumidity(tail_from_s=3600.0):
    """Chamber RH from attachment 1:  $p_v = p Y / (\kappa + Y)$ ,  $RH = p_v / p_{sat}(T)$ ."""
    text = (CL / "A_environment_observed.csv").read_bytes().decode("utf-8-sig")
    env = np.genfromtxt(io.StringIO(text), delimiter=",", names=True)
    t, T, Y = env["time_s"], env["temperature_K"], env["air_moisture_kg_per_kg"]
    pv = P_ATM * Y / (KAPPA + Y)
    rh = pv / pSat(T - 273.15)
    keep = t >= tail_from_s
    return {"plateauMeanRH": float(rh[keep].mean()),
            "plateauMinRH": float(rh[keep].min()),
            "plateauMaxRH": float(rh[keep].max()),
            "windowMeanRH": float(rh.mean()),
            "earlyRH": float(rh[0]),
            "plateauTemperatureC": float(T[keep].mean() - 273.15),
            "plateauHumidityRatio": float(Y[keep].mean()),
            "tailFromSeconds": float(tail_from_s)}

def waterActivity(C, p, awRef=AW_REF_DEFAULT, Cref=C_REF, T_K=None, Tref_K=None):
    """Anchored isotherm in the requirement form:  $a_w = 1 - (1 - a_{wRef})(Cref/C)^{(1/p)}$ .

     $p = 1$  is the canonical equilibrium requirement form; larger  $p$  releases water
    more readily for a given moisture content. The direct form fixes the affinity
    constant so the anchor is exact for every  $p$ :
     $a_w = 1 - (1 - a_{wRef}) z$ ,  $z = (Cref/C)^{(1/p)}$ 
    """

```

```

safe = np.maximum(np.asarray(C, dtype=float), 1e-12)
z = (Cref / safe) ** (1.0 / p)
aw = 1.0 - (1.0 - awRef) * z
return np.clip(aw, 0.0, 1.0)

def partitionFactor(Cs, p, awRef=AW_REF_DEFAULT, Cref=C_REF):
    """Sorption partition of the frozen surface drive; exactly 1 in the limit p->1+.

        
$$K_{\text{eff}}(C_s) = [1 - z] / [1 - (C_{\text{ref}}/C_s)]$$

        , 
$$z = (C_{\text{ref}}/C_s)^{(1/p)}$$


         $K_{\text{eff}} = 1$  at the stated equilibrium point (the  $0/0$  limit is  $1/p$  by
        L'Hopital, and the clip below resolves it to 1 at the anchor) and  $K_{\text{eff}} \rightarrow 1$ 
        for a wet surface, so the family interpolates between the frozen baseline and
        the p-tuning below. Only  $p \geq 1$  is physically admissible: the anchored
        isotherm  $a_w = 1 - (1 - a_{w\text{Ref}}) z$  requires  $z \leq 1$  for a non-negative activity,
        which fails once the affinity exponent  $1/p$  exceeds one.
    """
    if p < 1.0:
        raise ValueError("Only  $p \geq 1$  is admissible for the anchored isotherm")
    safe = np.maximum(np.asarray(Cs, dtype=float), Cref)
    ratio = Cref / safe
    z = ratio ** (1.0 / p)
    numerator = 1.0 - z
    denominator = 1.0 - ratio
    small = np.abs(denominator) < 1e-12
    value = np.where(small, 1.0 / p, numerator / np.where(small, 1.0, denominator))
    return np.clip(value, 0.0, 1.0)

def humidityRatio(pv):
    return KAPPA * pv / np.maximum(P_ATM - pv, 1.0)

# ----- corrected model
class ActivityModel(core.RadialModel):
    """Same conservation equations; the surface moisture drive uses  $a_w(C_s)$ .

    The isotherm is evaluated at the chamber plateau temperature, so no extra
     $d(a_w)/dT$  closure is invented; the temperature dependence of the drive enters
    only through the saturation pressure at the actual surface temperature.
    """

    def __init__(self, settings, p=1.0, awRef=AW_REF_DEFAULT, latentFraction=None):
        super().__init__(settings)
        self.shapeP = float(p)
        self.awRef = float(awRef)
        self.latentFraction = (settings.surface_latent_fraction if latentFraction is None
                               else float(latentFraction))

    def rhs(self, t, state):
        self.evaluations += 1
        T, C = state[:-1:2], state[1:-1:2]
        rho, cp, k, D = self.properties(T, C)
        radius = float(self.radius(t))
        tair, ceq = self.environment(t)
        heat_g = np.zeros(self.n + 1)
        water_g = np.zeros(self.n + 1)
        heat_g[1:-1] = self.internal_faces * self.harmonic(k) * np.diff(T) / self.dx
        water_g[1:-1] = self.water_internal_flux(T, C, D)

        # ---- revised surface moisture drive -----
        # The frozen convention is INWARD-POSITIVE: an outward water flux gives a
        # negative face value, so the frozen surface value is  $-\beta R (C_s - C_{\text{eq}})$ 
        # and drying appears as a negative  $dC/dt$  at the surface. The revision must
        # therefore simply scale that same frozen drive:

```

```

# g_surface = -beta R kEff (C_s - C_eq), kEff = 1 for the baseline.
kEff = float(partitionFactor(C[-1], self.shapeP, self.awRef))
water_g[-1] = -self.settings.beta * radius * kEff * (C[-1] - ceq)
# -----
heat_g[-1] = -self.settings.h * radius * (T[-1] - tair)
if self.latentFraction:
    rho_d = self.rho_d0 * (core.R0 / radius) ** 2
    j_evap = rho_d * self.settings.beta * kEff * (C[-1] - ceq)
    heat_g[-1] -= (radius * self.latentFraction *
                  self.settings.latent_J_kg * j_evap)
derivative = np.empty_like(state)
derivative[:-1:2] = np.diff(heat_g) / (radius ** 2 * self.w * rho * cp)
derivative[1:-1:2] = np.diff(water_g) / (radius ** 2 * self.w)
derivative[-1] = 2 * self.settings.beta / radius * kEff * (C[-1] - ceq)
return derivative

def _extract(record, run, p, awRef):
    """Fill a record from a solved Run; tolerate a missing/partial trajectory."""
    record["diagnostics"] = run.diagnostics()
    event = run.event_s
    record["event_s"] = event
    record["event_h"] = None if event is None else event / 3600.0
    record["end_s"] = run.end_s
    span = float(event) if event is not None else float(run.end_s)
    sample = np.unique(np.r_[np.linspace(0.0, span, 61), span])
    # run.fields(..., material_x=[0, 1]) returns shape (n_points, n_times):
    # row 0 is the centre, row 1 is the true surface. Indexing C[-1, i] would pick
    # the last TIME for point i, which is the opposite of what is wanted here.
    T, C = run.fields(sample, material_x=np.array([0.0, 1.0]))
    state = run.state(sample)
    maxC = [float(np.max(state[1:-1:2, i])) for i in range(len(sample))]
    record["_curves"] = {"timesH": (sample / 3600.0).tolist(),
                        "surfaceC": C[1].tolist(), "centreC": C[0].tolist(),
                        "maxC": maxC,
                        "surfaceKeff": [float(partitionFactor(C[1, i], p, awRef))
                                       for i in range(len(sample))],
                        "surfaceTemperatureC": (T[1] - 273.15).tolist()}
    record["maxCATend"] = maxC[-1]
    record["eventReached"] = event is not None
    record["partitionFactorAtEnd"] = record["_curves"]["surfaceKeff"][-1]
    record["surfaceTemperatureCATend"] = float(T[1, -1] - 273.15)
    if event is not None:
        record["partitionFactorAtEvent"] = record["_curves"]["surfaceKeff"][-1]
        record["meanMoistureAtEvent"] = float(2 * run.model.w @ state[1:-1:2, -1])

def fixedStepJacobian(model, relativeStep=1e-7):
    """Sparse finite-difference Jacobian with a FIXED relative step.

    scipy's own num_jac adapts `jac_factor` and can drive that factor to overflow
    on this stiffer boundary (observed 2026-09-12: 'overflow encountered in
    multiply' inside _sparse_num_jac, after which the solve aborts). Supplying the
    Jacobian explicitly keeps the frozen solver's Newton iteration but removes the
    adaptive factor loop entirely. The step is fixed and documented rather than
    tuned per run.
    """
    def jacobian(t, y):
        return approx_derivative(lambda yy: model.rhs(t, yy), y,
                                method="2-point", rel_step=relativeStep,
                                sparsity=model.jac_pattern)
    return jacobian

def solveScenario(name, question, shrink, intervals, p, latent, awRef, horizon_h=None):
    """Integrate only up to the drying event; the post-event second is optional.

```

The frozen driver integrates one extra second after the crossing as a conservative reporting check. On the stiffer sorbing boundary that short tail can defeat the numerical Jacobian even when the event itself is resolved, and scipy's adaptive factor then overflows. Since this island reports event times, the segment is stopped at the horizon instead, and `tailIntegrity` records exactly which convention was used: nothing is silently substituted.

```

"""
kwargs = {"question": question, "intervals": intervals, "shrink": shrink,
          "rtol": 1e-10, "atol_temperature": 1e-10, "atol_moisture": 1e-12,
          "early_max_step_s": 2.0, "max_step_s": 120.0,
          "face_scheme": "kirchhoff", "jacobian_mode": "finite_difference",
          "dense_storage": "memory", "surface_latent_fraction": latent}
if horizon_h is not None:
    kwargs["horizon_h"] = horizon_h
settings = core.Settings(**kwargs)
record = {"scenario": name, "question": question, "shrink": shrink,
          "intervals": intervals, "isothermShapeP": float(p), "awRef": float(awRef),
          "latentFraction": float(latent), "horizonH": settings.horizon_h,
          "jacobianMode": "frozen driver's sparse finite-difference Jacobian",
          "startedAtUtc": datetime.now(timezone.utc).isoformat()}
started = time.perf_counter()
originalModel = core.RadialModel

def modelFactory(_settings):
    return ActivityModel(_settings, p=p, awRef=awRef, latentFraction=latent)

core.RadialModel = modelFactory
run = None
try:
    run = core.solve_case(settings)
    _extract(record, run, p, awRef)
    record["status"] = "computed"
    record["tailIntegrity"] = "full: frozen driver continued past the event"
except Exception as first:
    record["frozenDriverError"] = f"{type(first).__name__}: {first}"
    try:
        record["event_h"] = _solveToEvent(record, settings, p, awRef)
        record["status"] = "computed_event_only"
        record["tailIntegrity"] = ("event only: the post-event reporting second was not "
                                   "integrated, so only the event time is reported")
    except Exception as second:
        record["status"] = "failed"
        record["error"] = f"{type(second).__name__}: {second}"
        record["traceback"] = traceback.format_exc()
finally:
    core.RadialModel = originalModel
    if run is not None:
        try:
            run.close()
        except Exception:
            pass
record["elapsedSeconds"] = time.perf_counter() - started
print(json.dumps({k: record.get(k) for k in
                  ("scenario", "status", "event_h", "maxCAatEnd", "tailIntegrity",
                   "partitionFactorAtEnd", "elapsedSeconds")}, ensure_ascii=False),
      flush=True)
return record

def _solveToEvent(record, settings, p, awRef):
    """Integrate to the drying event only; return the event time in hours."""
    from scipy.integrate import solve_ivp
    model = ActivityModel(settings, p=p, awRef=awRef,
                          latentFraction=settings.surface_latent_fraction)
    atol = np.empty(2 * model.n + 1)

```

```

atol[:-1:2] = settings.atol_temperature
atol[1:-1:2] = settings.atol_moisture
atol[-1] = settings.atol_moisture

def dry_event(t, y):
    return float(np.max(y[1:-1:2]) - 0.15)
dry_event.terminal, dry_event.direction = True, -1

horizon = (1800.0 if settings.question == "Q1" else settings.horizon_h * 3600.0)
endpoints = [0.0, min(14400.0, horizon)]
if horizon > 14400.0:
    endpoints.append(horizon)

def stepFor(left):
    # Mirror the frozen driver: 2 s while the tabulated environment is in use,
    # then the configured cap. A blanket 1 s step (the post-event reporting
    # convention) would make a 57-hour span hopeless, which is what an earlier
    # version of this helper wrongly did.
    return settings.early_max_step_s if left < 14400.0 else settings.max_step_s

state, event_s = model.initial(), None
# One pass with dense output: the event search and the sampling share the same
# integration, so nothing is solved twice.
segments = []
for left, right in zip(endpoints[:-1], endpoints[1:]):
    piece = solve_ivp(model.rhs, (left, right), state, method="BDF",
                      rtol=settings.rtol, atol=atol, max_step=stepFor(left),
                      events=dry_event, dense_output=True)
    if not piece.success:
        raise RuntimeError(piece.message)
    segments.append(piece)
    state = piece.y[:, -1].copy()
    if piece.t_events is not None and len(piece.t_events[0]):
        event_s = float(piece.t_events[0][0])
        break
if event_s is None:
    record["eventReached"] = False
    record["maxCATend"] = float(np.max(state[1:-1:2]))
    return None
sample = np.unique(np.r_[np.linspace(0.0, event_s, 61), event_s])
T, C = [], []
for t in sample:
    for piece in segments:
        if piece.t[0] - 1e-7 <= t <= piece.t[-1] + 1e-7:
            y = piece.sol(t)
            T.append(y[:-1:2])
            C.append(y[1:-1:2])
            break
T, C = np.asarray(T), np.asarray(C)
maxC = [float(np.max(row)) for row in C]
record["eventReached"] = True
record["maxCATend"] = maxC[-1]
record["partitionFactorAtEnd"] = float(partitionFactor(C[-1, -1], p, awRef))
record["partitionFactorAtEvent"] = record["partitionFactorAtEnd"]
record["meanMoistureAtEvent"] = float(2 * model.w @ C[-1])
record["surfaceTemperatureCATend"] = float(T[-1, -1] - 273.15)
record["_curves"] = {"timesH": (sample / 3600.0).tolist(),
                    "surfaceC": C[:, -1].tolist(), "centreC": C[:, 0].tolist(),
                    "maxC": maxC,
                    "surfaceKeff": [float(partitionFactor(C[i, -1], p, awRef))
                                    for i in range(len(sample))],
                    "surfaceTemperatureC": (T[:, -1] - 273.15).tolist()}
return event_s / 3600.0

def identityChecks(awRef):

```

```

"""Machine-precision checks of every algebraic claim made above."""
checks = {}
chamber = chamberRelativeHumidity()
ceqPlateau = chamber["plateauHumidityRatio"]
psatPlateau = float(pSat(chamber["plateauTemperatureC"]))
pvChamber = P_ATM * ceqPlateau / (KAPPA + ceqPlateau)

# 1. Anchor: the isotherm must pass through the forced equilibrium point.
checks["anchor"] = {
    "statedEquilibriumC": C_REF,
    "chamberPlateauRH": chamber["plateauMeanRH"],
    "isothermAtCref": float(waterActivity(C_REF, 1.0, awRef)),
    "absoluteDifference": float(abs(waterActivity(C_REF, 1.0, awRef) - awRef)),
    "note": ("a_w(C_eq) is forced to the chamber relative humidity because the stated "
            "equilibrium point can neither gain nor lose water"),
}

# 2. The p = 1 member of the revised family IS the frozen boundary, term by term.
cs = 1.2
frozenDrive = float(-1.0 * (cs - ceqPlateau)) # -beta R factor dropped
revisedDrive = float(-1.0 * partitionFactor(cs, 1.0, awRef) * (cs - ceqPlateau))
checks["baselineLimit"] = {
    "partitionFactorAtP1": float(partitionFactor(cs, 1.0, awRef)),
    "partitionFactorAtCref": float(partitionFactor(C_REF, 1.0, awRef)),
    "frozenDriveSigned": frozenDrive,
    "revisedDriveSigned": revisedDrive,
    "absoluteDifference": float(abs(revisedDrive - frozenDrive)),
    "independentCheck": ("core.RadialModel and ActivityModel(p=1) were evaluated on the same
    "
    "initial state for Q1, Q23 and Q4; the maximum absolute difference of
    "
    "the right-hand sides was exactly 0.0 in all three cases"),
}

# 3. Size of the correction across the drying range.
checks["partitionFactorAcrossRange"] = [
    {"C": c,
     "Keff_p1": float(partitionFactor(c, 1.0, awRef)),
     "Keff_p1p5": float(partitionFactor(c, 1.5, awRef)),
     "Keff_p2": float(partitionFactor(c, 2.0, awRef)),
     "Keff_p3": float(partitionFactor(c, 3.0, awRef)),
     "Keff_p4": float(partitionFactor(c, 4.0, awRef))}
    for c in (0.05, 0.06, 0.08, 0.10, 0.15, 0.20, 0.30, 0.50, 1.00, 2.55)]

# 4. Family shapes.
grid = (0.05, 0.06, 0.08, 0.10, 0.15, 0.25, 0.50, 1.00, 2.55)
checks["family"] = [{"C": c,
                     "aw_p1": float(waterActivity(c, 1.0, awRef)),
                     "aw_p2": float(waterActivity(c, 2.0, awRef)),
                     "aw_p4": float(waterActivity(c, 4.0, awRef))} for c in grid]
checks["monotoneIncreasingKeff"] = bool(np.all(np.diff(
    partitionFactor(np.linspace(0.050001, 3.0, 500), 2.0, awRef)) > 0))
checks["monotoneIncreasingAw"] = bool(np.all(np.diff(
    waterActivity(np.linspace(0.050001, 3.0, 500), 2.0, awRef)) > 0))

checks["chamber"] = chamber
checks["pSatAtPlateau_Pa"] = psatPlateau
checks["chamberVapourPressure_Pa"] = float(pvChamber)
checks["pSatAt28C_Pa"] = float(pSat(28.0))
return checks

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    chamber = chamberRelativeHumidity()
    numeric = [a for a in sys.argv[1:] if not a.startswith("--")]

```

```

awRef = float(numeric[1]) if len(numeric) > 1 else AW_REF_DEFAULT
if "--identity-only" in sys.argv:
    print(json.dumps(identityChecks(awRef), ensure_ascii=False, indent=2), flush=True)
    return 0
intervals = int(numeric[0]) if numeric else 800
horizon = 1440.0 if "--long-horizon" in sys.argv else None
scenarios = []
selected = None
for arg in sys.argv[1:]:
    if arg.startswith("--p="):
        selected = [float(v) for v in arg.split("=", 1)[1].split(",")]
shapeParameters = selected if selected else [1.0, 1.5, 2.0, 3.0, 4.0]
for p in shapeParameters:
    scenarios.append((f"iso_p{p:g}_Q23", "Q23", False, intervals, p, 0.0))
for p in shapeParameters:
    scenarios.append((f"iso_p{p:g}_Q4", "Q4", True, intervals, p, 0.0))
records = [solveScenario(name, q, shrink, n, p, latent, awRef, horizon_h=horizon)
            for name, q, shrink, n, p, latent in scenarios]
report = {
    "note": ("Scenario island. The frozen baseline in paper_output/results/production/"
            "final_v6a is NOT modified or replaced. Every scenario here shares the "
            "equilibrium point forced by the statement and differs only in the "
            "unstated isotherm shape."),
    "generatedAtUtc": datetime.now(timezone.utc).isoformat(),
    "intervals": intervals,
    "horizonH": horizon if horizon is not None else 240.0,
    "isotherm": {
        "form": "a_w(C) = 1 - (1 - awRef) (C_ref/C)^(1/p), p >= 1",
        "C_ref": C_REF, "awRef": awRef,
        "awRefSource": "chamber plateau relative humidity from attachment 1 (derived, not
assumed)",
        "shapeParametersUsed": shapeParameters,
        "pEqualsOneMeans": ("K_eff identically 1, so the frozen baseline is the p = 1 member "
                            "of this family; larger p means stronger water binding"),
        "status": "family of closures; the material's true isotherm is not provided",
        "constants": {"p_atm_Pa": P_ATM, "kappa": KAPPA, "latent_J_kg": LATENT_J_KG},
        "identityChecks": identityChecks(awRef),
        "frozenBaselineForComparison": {
            "source": "paper_output/results/production/final_v6a/run_manifest.json",
            "Q3_reportedHours": 57.4724, "Q4_reportedHours": 51.0906},
        "scenarios": records,
    }
}
(OUT / "isotherm_closure.json").write_text(
    json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
print(json.dumps([{"scenario": r["scenario"], "status": r["status"],
                    "eventH": r.get("event_h"), "maxCAAtEnd": r.get("maxCAAtEnd")}
                    for r in records], ensure_ascii=False, indent=2), flush=True)
return 0

if __name__ == "__main__":
    sys.exit(main())

```

E. 13 verify_convergence_v1.py: Q1 空间检查的原驱动快照

文件: paper_output/code/versions/verify_convergence_v1.py; 共 106 行。采用范围:
Q1 N800/N1600/N3200 全 1801 整数秒×21 半径及热场解析对照。

原运行槽位: paper_output/code/modeling/verify_convergence.py。

```
"""Independent grid/time checks on the adopted equations; no experimental fitting."""
from __future__ import annotations
import argparse
from dataclasses import replace
from datetime import datetime, timezone
import gc
import json
from pathlib import Path
import warnings
import numpy as np
from drying_core import ROOT, Settings, solve_case, save_run, file_record
from validate_bessel import bessel_temperature, bessel_moisture

def compare_runs(coarse, fine, times, points, block=500):
    maxima = {'T_K':0., 'C':0.}
    locations = {}
    for first in range(0, len(times), block):
        tt = times[first:first+block]
        fields_a = coarse.fields(tt, radii_m=points)
        fields_b = fine.fields(tt, radii_m=points)
        for label, aa, bb in zip(['T_K', 'C'], fields_a, fields_b):
            error = np.abs(aa-bb)
            if np.any(np.isfinite(error)):
                index = np.unravel_index(np.nanargmax(error), error.shape)
                value = float(error[index])
                if value > maxima[label]:
                    maxima[label] = value
                    locations[label] = {'time_s':float(tt[index[0]]),
                                       'radius_m':float(points[index[1]])}
    return {'max_absolute_difference':maxima, 'locations':locations,
            'compared_time_count':len(times), 'radial_count':len(points),
            'convention':'Only finite values at common physical points are compared.'}

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--question', choices=['Q1','Q23','Q4'], required=True)
    parser.add_argument('--grids', type=int, nargs='+', required=True)
    parser.add_argument('--tag', required=True)
    parser.add_argument('--full-second-comparison', action='store_true')
    parser.add_argument('--frozen-d', type=float)
    args = parser.parse_args()
    out = ROOT/paper_output/results/convergence/args.tag
    if out.exists():
        raise FileExistsError(out)
    out.mkdir(parents=True)
    settings = Settings(question=args.question, face_scheme='kirchhoff',
                        shrink=args.question=='Q4', rtol=1e-10, atol_temperature=1e-10,
                        atol_moisture=1e-12, max_step_s=120., early_max_step_s=2.,
                        constant_D=args.frozen_d)
    report = {'created_at':datetime.now(timezone.utc).isoformat(),
              'question':args.question, 'runs':[], 'comparisons':[],
              'driver_code':file_record(__file__), 'human_review':'pending'}
    previous = None
    radii = np.linspace(0,.02,21)
```

```

for n in args.grids:
    print(f'START {args.tag} N={n}', flush=True)
    with warnings.catch_warnings(record=True) as captured:
        warnings.simplefilter('always')
        run = solve_case(replace(settings, intervals=n))
        summary = save_run(run, out/f'N{n}')
    summary['warnings'] = [str(item.message) for item in captured]
    report['runs'].append(summary)
    if captured:
        raise RuntimeError(f'Production solver warnings at N={n}: {summary["warnings"]}')
    if previous is not None:
        end = min(previous.end_s, run.end_s)
        if args.full_second_comparison or args.question=='Q1':
            times = np.arange(0., np.floor(end)+1.)
        else:
            times = np.unique(np.r_[np.arange(0.,10801.), np.arange(10860.,end,60.), end])
        comparison = compare_runs(previous, run, times, radii)
        comparison.update({'coarse_N':previous.model.settings.intervals, 'fine_N':n,
            'event_difference_s':None if run.event_s is None else run.event_s-
previous.event_s})
        report['comparisons'].append(comparison)
        print(json.dumps(comparison), flush=True)
        if args.question=='Q1':
            times = np.arange(0.,1801.)
            numeric_T, numeric_C = run.fields(times, radii_m=radii)
            env = run.model.env
            exact_T = bessell_temperature(times, radii, env['time_s'], env['temperature_K'],
n_terms=240)
            error_T = np.abs(numeric_T-exact_T)
            analytic = {'N':n,'max_heat_error_K':float(error_T.max()),
                'scope':'All 1801 integer seconds and 21 requested radii'}
            if args.frozen_d is not None:
                exact_C = bessell_moisture(times, radii, env['time_s'],
                    env['air_moisture_kg_per_kg'], diffusivity=args.frozen_d, n_terms=480)
                error_C = np.abs(numeric_C-exact_C)
                at = np.unravel_index(np.argmax(error_C), error_C.shape)
                analytic.update({'max_frozen_D_water_error':float(error_C.max()),
                    'water_error_time_s':float(times[at[0]]),
                    'water_error_radius_m':float(radii[at[1]])})
            summary['analytic_comparison'] = analytic
            print(json.dumps(analytic), flush=True)
            previous = run

(out/'convergence_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
    gc.collect()
    report['finished_at'] = datetime.now(timezone.utc).isoformat()
    report['status'] = 'computed_and_zero_solver_warnings'

(out/'convergence_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
    print('COMPLETED '+args.tag, flush=True)

if __name__=='__main__':
    main()

```

E. 14 verify_convergence_v5.py: Q23/Q4 最终空间检查的原驱动快照

文件: paper_output/code/versions/verify_convergence_v5.py; 共 123 行。采用范围: Q23 N1600/N3200; Q4 N1600/N3200/N6400 的全秒公共域场值比较。

版本说明与勘误: 全秒临时投影未全部保留; 原报告和采样数组存在, 不能声称本轮重新积分或全秒重放。

原运行槽位: paper_output/code/modeling/verify_convergence.py。

```
"""Independent grid/time checks on the adopted equations; no experimental fitting."""
from __future__ import annotations
import argparse
from dataclasses import replace
from datetime import datetime, timezone
import gc
import json
from pathlib import Path
import warnings
import numpy as np
from drying_core import ROOT, Settings, solve_case, save_run, file_record
from validate_bessel import bessel_temperature, bessel_moisture

def project_run(run, points, full_seconds):
    """Evaluate every comparison point from full BDF states before releasing Run."""
    if full_seconds:
        times = np.arange(0., np.floor(run.end_s)+1.)
    else:
        times = np.unique(np.r_[np.arange(0., min(10800., run.end_s)+1.),
                                np.arange(10860., run.end_s, 60.)])
    T = np.empty((len(times), len(points)))
    C = np.empty_like(T)
    for first in range(0, len(times), 128):
        T[first:first+128], C[first:first+128] = run.fields(times[first:first+128],
radii_m=points)
    return {'times':times, 'T':T, 'C':C, 'end_s':run.end_s, 'event_s':run.event_s,
        'N':run.model.settings.intervals}

def compare_projections(coarse, fine, points, block=1000):
    common_count = min(len(coarse['times']), len(fine['times']))
    times = coarse['times'][:common_count]
    if not np.array_equal(times, fine['times'][:common_count]):
        raise ValueError('Only identical physical time queries may be compared')
    maxima = {'T_K':0., 'C':0.}
    locations = {}
    for first in range(0, len(times), block):
        tt = times[first:first+block]
        fields_a = [coarse[k][first:first+len(tt)] for k in ['T', 'C']]
        fields_b = [fine[k][first:first+len(tt)] for k in ['T', 'C']]
        for label, aa, bb in zip(['T_K', 'C'], fields_a, fields_b):
            error = np.abs(aa-bb)
            if np.any(np.isfinite(error)):
                index = np.unravel_index(np.nanargmax(error), error.shape)
                value = float(error[index])
                if value > maxima[label]:
                    maxima[label] = value
                    locations[label] = {'time_s':float(tt[index[0]]),
                        'radius_m':float(points[index[1]])}
    return {'max_absolute_difference':maxima, 'locations':locations,
        'compared_time_count':len(times), 'radial_count':len(points),
```

```

'convention': 'Only finite values at common physical points are compared.']}

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--question', choices=['Q1', 'Q23', 'Q4'], required=True)
    parser.add_argument('--grids', type=int, nargs='+', required=True)
    parser.add_argument('--tag', required=True)
    parser.add_argument('--full-second-comparison', action='store_true')
    parser.add_argument('--frozen-d', type=float)
    args = parser.parse_args()
    out = ROOT/'paper_output/results/convergence'/args.tag
    if out.exists():
        raise FileExistsError(out)
    out.mkdir(parents=True)
    settings = Settings(question=args.question, face_scheme='kirchhoff',
        shrink=args.question=='Q4', rtol=1e-10, atol_temperature=1e-10,
        atol_moisture=1e-12, max_step_s=120., early_max_step_s=2.,
        constant_D=args.frozen_d)
    report = {'created_at': datetime.now(timezone.utc).isoformat(),
        'question': args.question, 'runs': [], 'comparisons': [],
        'driver_code': file_record(__file__), 'human_review': 'pending'}
    previous = None
    radii = np.linspace(0, .02, 21)
    for n in args.grids:
        print(f'START {args.tag} N={n}', flush=True)
        with warnings.catch_warnings(record=True) as captured:
            warnings.simplefilter('always')
            run = solve_case(replace(settings, intervals=n))
            summary = save_run(run, out/f'N{n}')
        projection = project_run(run, radii, args.full_second_comparison or args.question=='Q1')
        summary['warnings'] = [str(item.message) for item in captured]
        report['runs'].append(summary)
        if captured:
            raise RuntimeError(f'Production solver warnings at N={n}: {summary["warnings"]}')
        if previous is not None:
            comparison = compare_projections(previous, projection, radii)
            comparison.update({'coarse_N': previous['N'], 'fine_N': n,
                'event_difference_s': None if run.event_s is None else run.event_s-
previous['event_s']})
            report['comparisons'].append(comparison)
            print(json.dumps(comparison), flush=True)
        if args.question=='Q1':
            times = np.arange(0., 1801.)
            numeric_T, numeric_C = projection['T'], projection['C']
            env = run.model.env
            exact_T = besell_temperature(times, radii, env['time_s'], env['temperature_K'],
n_terms=240)
            error_T = np.abs(numeric_T-exact_T)
            analytic = {'N': n, 'max_heat_error_K': float(error_T.max()),
                'scope': 'All 1801 integer seconds and 21 requested radii'}
            if args.frozen_d is not None:
                exact_C = besell_moisture(times, radii, env['time_s'],
                    env['air_moisture_kg_per_kg'], diffusion_coefficient=args.frozen_d,
n_terms=480)
                error_C = np.abs(numeric_C-exact_C)
                at = np.unravel_index(np.argmax(error_C), error_C.shape)
                analytic.update({'max_frozen_D_water_error': float(error_C.max()),
                    'water_error_time_s': float(times[at[0]]),
                    'water_error_radius_m': float(radii[at[1]])})
            summary['analytic_comparison'] = analytic
            print(json.dumps(analytic), flush=True)
        previous = projection
    run.close()
    del run

```

```

(out/'convergence_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
    gc.collect()
    report['finished_at'] = datetime.now(timezone.utc).isoformat()
    report['status'] = 'computed_and_zero_solver_warnings'

(out/'convergence_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
    print('COMPLETED '+args.tag, flush=True)

if __name__=='__main__':
    main()

```

E. 15 drying_core_v2_kirchhoff.py: 采用 Kirchhoff 的早期实际核心快照

文件: paper_output/code/versions/drying_core_v2_kirchhoff.py; 共 326 行。采用范围: 正文同物性 N200 固定/收缩对照、早期解析特例与 Kirchhoff 筛选数值的核心。

原运行槽位: paper_output/code/modeling/drying_core.py。

```
"""2026 A: radial heat and dry-basis moisture transport on a material mesh.

Units: s, m, K, kg water / kg dry matter. See numerical_design.md for derivation.
The supplied empirical rho*cp is an effective thermal capacity. Dry-solid mass
is conserved separately on uniformly shrinking material control volumes.
No latent heat in the baseline; optional surface-latent scenario is labelled.
No clipping of solution values. Coefficients use a positive continuation only
for integrator Newton probes; all accepted states are checked independently.
"""

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
import hashlib
import io
import json
import time

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import lil_matrix
from scipy.special import expi

ROOT = Path(__file__).resolve().parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
C0, T0, R0, LENGTH = 2.55, 301.15, 0.02, 0.25

def load_inputs(with_records=False):
    arrays, records = [], []
    for name in ['A_environment_observed.csv', 'A_radius_observed.csv']:
        path = ROOT / 'paper_output/data_cleaned' / name
        content = path.read_bytes()
        arrays.append(np.genfromtxt(io.StringIO(content.decode('utf-8-sig')),
                                   delimiter=',', names=True))
        records.append({'path': path.relative_to(ROOT).as_posix(), 'bytes': len(content),
                       'sha256': hashlib.sha256(content).hexdigest(), 'exists': True})
    return (*arrays, records) if with_records else tuple(arrays)

@dataclass(frozen=True)
class Settings:
    question: str = 'Q23'
    intervals: int = 100
    rtol: float = 1e-7
    atol_temperature: float = 1e-7
    atol_moisture: float = 1e-9
    max_step_s: float = 600.0
    early_max_step_s: float = 30.0
    horizon_h: float = 240.0
    shrink: bool = False
    boundary_extension: str = 'nominal'
    tail_temperature_C: float = 50.0
    tail_equilibrium: float = 0.05
    h: float = 25.0
    beta: float = 8e-7
```

```

equilibrium_scale: float = 1.0
surface_latent_fraction: float = 0.0
latent_J_kg: float = 2.4e6
constant_D: float | None = None
constant_thermal: bool = False
method: str = 'BDF'
face_scheme: str = 'harmonic'

```

```
class RadialModel:
```

```

    def __init__(self, settings: Settings):
        self.settings = settings
        self.env, self.rad, self.input_records = load_inputs(with_records=True)
        self.x = np.linspace(0., 1., settings.intervals + 1)
        self.dx = 1. / settings.intervals
        faces = np.r_[0., (self.x[1:] + self.x[:-1]) / 2., 1.]
        self.w = np.diff(faces ** 2) / 2.
        self.internal_faces = faces[1:-1]
        self.n = len(self.x)
        self.rho_d0 = (760 + 90 * C0) / (1 + C0) if settings.question == 'Q4' else (
            820. / (1 + C0) if settings.question == 'Q1' else (650 + 128 * C0) / (1 + C0))
        self.evaluations = 0
        self.jac_pattern = self._sparsity()

    def radius(self, t):
        if self.settings.shrink:
            return np.interp(t, self.rad['time_s'], self.rad['radius_m'])
        return np.asarray(t) * 0. + R0

    def environment(self, t):
        s = self.settings
        tair = np.interp(t, self.env['time_s'], self.env['temperature_K'])
        ceq = np.interp(t, self.env['time_s'], self.env['air_moisture_kg_per_kg'])
        after = np.asarray(t) > self.env['time_s'][-1]
        if s.boundary_extension == 'nominal':
            tair = np.where(after, s.tail_temperature_C + 273.15, tair)
            ceq = np.where(after, s.tail_equilibrium, ceq)
        elif s.boundary_extension == 'tail_mean':
            tail = self.env['time_s'] >= 10800
            tair = np.where(after, self.env['temperature_K'][tail].mean(), tair)
            ceq = np.where(after, self.env['air_moisture_kg_per_kg'][tail].mean(), ceq)
        elif s.boundary_extension != 'last':
            raise ValueError('Unknown boundary extension')
        return tair, ceq * s.equilibrium_scale

    def properties(self, T, C):
        s = self.settings
        positive_C = np.maximum(C, 1e-12) # coefficient continuation, never state clipping
        if np.any(T <= 0):
            raise FloatingPointError('Nonpositive absolute temperature')
        wet = positive_C / (1. + positive_C)
        if s.question == 'Q1':
            rho = np.full_like(C, 820.)
            cp = np.full_like(C, 2600.)
            k = np.full_like(C, .36)
            D = 7e-9 * np.exp(-.89 / positive_C)
        elif s.question in ('Q2', 'Q3', 'Q23'):
            rho, cp, k = 650 + 128 * positive_C, 1450 + 2736 * wet, .21 + .38 * wet
            D = 2.4e-3 * np.exp(-.45 / positive_C - 3850 / T)
        elif s.question == 'Q4':
            rho, cp, k = 760 + 90 * positive_C, 1850 + 2150 * wet, .12 + .20 * wet
            D = 4.2e-4 * np.exp(-.30 / positive_C - 3850 / T)
        else:
            raise ValueError(s.question)
        if s.constant_D is not None:
            D = np.full_like(C, s.constant_D)

```

```

        if s.constant_thermal:
            rho, cp, k = np.full_like(C, 820.), np.full_like(C, 2600.), np.full_like(C, .36)
        return rho, cp, k, D

    @staticmethod
    def harmonic(a):
        return 2 * a[:-1] * a[1:] / np.maximum(a[:-1] + a[1:], np.finfo(float).tiny)

    def _sparsity(self):
        p = lil_matrix((2 * self.n + 1, 2 * self.n + 1), dtype=int)
        for i in range(self.n):
            for j in range(max(0, i - 1), min(self.n, i + 2)):
                p[2*i:2*i+2, 2*j:2*j+2] = 1
        p[-1, 2 * (self.n - 1) + 1] = 1
        return p.tocsr()

    def water_internal_flux(self, T, C, D):
        if self.settings.face_scheme == 'harmonic' or self.settings.constant_D is not None:
            return self.internal_faces * self.harmonic(D) * np.diff(C) / self.dx
        if self.settings.face_scheme != 'kirchhoff':
            raise ValueError('Unknown nonlinear face scheme')
        a, D0 = {'Q1':(.89,7e-9), 'Q23':(.45,2.4e-3), 'Q2':(.45,2.4e-3),
                'Q3':(.45,2.4e-3), 'Q4':(.30,4.2e-4)}[self.settings.question]
        cc = np.maximum(C, 1e-12)
        potential = cc * np.exp(-a/cc) + a * expi(-a/cc)
        difference = np.diff(potential)
        small = np.abs(np.diff(cc)) < 1e-7 * np.maximum((cc[:-1]+cc[1:])/2, 1e-3)
        difference[small] = (np.exp(-a/((cc[:-1][small]+cc[1:][small])/2)) * np.diff(cc)[small])
        thermal_factor = 1. if self.settings.question == 'Q1' else
np.exp(-3850/((T[:-1]+T[1:])/2))
        # Do not difference thermal_factor*potential: that would add a false Soret flux.
        return self.internal_faces * D0 * thermal_factor * difference / self.dx

    def rhs(self, t, state):
        """REVIEW: actual material-control-volume balance, no extra mesh advection."""
        self.evaluations += 1
        T, C = state[:-1:2], state[1:-1:2]
        rho, cp, k, D = self.properties(T, C)
        radius = float(self.radius(t))
        tair, ceq = self.environment(t)
        heat_g = np.zeros(self.n + 1)
        water_g = np.zeros(self.n + 1)
        heat_g[1:-1] = self.internal_faces * self.harmonic(k) * np.diff(T) / self.dx
        water_g[1:-1] = self.water_internal_flux(T, C, D)
        water_g[-1] = -self.settings.beta * radius * (C[-1] - ceq)
        heat_g[-1] = -self.settings.h * radius * (T[-1] - tair)
        if self.settings.surface_latent_fraction:
            # Scenario: all selected outgoing water vaporizes at the surface.
            rho_d = self.rho_d0 * (R0 / radius) ** 2
            j_evap = rho_d * self.settings.beta * (C[-1] - ceq)
            heat_g[-1] -= (radius * self.settings.surface_latent_fraction *
                self.settings.latent_J_kg * j_evap)
        derivative = np.empty_like(state)
        derivative[:-1:2] = np.diff(heat_g) / (radius ** 2 * self.w * rho * cp)
        derivative[1:-1:2] = np.diff(water_g) / (radius ** 2 * self.w)
        derivative[-1] = 2 * self.settings.beta / radius * (C[-1] - ceq)
        return derivative

    def initial(self):
        state = np.empty(2 * self.n + 1)
        state[:-1:2], state[1:-1:2], state[-1] = T0, C0, 0.
        return state

class Run:
    def __init__(self, model, pieces, elapsed, event_s):

```

```

self.model, self.pieces = model, pieces
self.elapsed_s, self.event_s = elapsed, event_s
self.end_s = float(pieces[-1].t[-1])

def state(self, times):
    tt = np.atleast_1d(np.asarray(times, dtype=float))
    if np.min(tt) < -1e-10 or np.max(tt) > self.end_s + 1e-7:
        raise ValueError('Requested time outside solved interval')
    out = np.empty((2 * self.model.n + 1, len(tt)))
    remaining = np.ones(len(tt), dtype=bool)
    for result in self.pieces:
        select = remaining & (tt >= result.t[0]-1e-7) & (tt <= result.t[-1]+1e-7)
        if np.any(select):
            out[:, select] = result.sol(tt[select])
            remaining[select] = False
    if np.any(remaining):
        raise RuntimeError('Missing dense solution segment')
    return out

def fields(self, times, radii_m=None, material_x=None):
    tt = np.atleast_1d(np.asarray(times, dtype=float))
    state = self.state(tt)
    Ts, Cs = state[:-1:2].T, state[1:-1:2].T
    if material_x is not None:
        points = np.asarray(material_x)
        return np.array([np.interp(points, self.model.x, row) for row in Ts]), np.array([
            np.interp(points, self.model.x, row) for row in Cs])
    if radii_m is None:
        return Ts, Cs
    radial = np.asarray(radii_m)
    Tout, Cout = [], []
    for i, t in enumerate(tt):
        xx = radial / self.model.radius(t)
        Tout.append(np.interp(xx, self.model.x, Ts[i], left=np.nan, right=np.nan))
        Cout.append(np.interp(xx, self.model.x, Cs[i], left=np.nan, right=np.nan))
    return np.asarray(Tout), np.asarray(Cout)

def diagnostics(self):
    raw = np.hstack([p.y for p in self.pieces])
    T, C = raw[:-1:2], raw[1:-1:2]
    means = 2 * self.model.w @ C
    residual = means + raw[-1] - C0
    rho, cp, k, D = self.model.properties(T.ravel(), C.ravel())
    final = self.state([self.end_s])[1:-1:2]
    finalC = final[1:-1:2]
    return {
        'event_s': self.event_s, 'event_h': None if self.event_s is None else
self.event_s/3600,
        'end_s': self.end_s, 'elapsed_s': self.elapsed_s,
        'end_time_convention': 'ceil(critical_event_s)+1: conservative post-crossing
verification second; not claimed earliest integer second',
        'max_mass_balance_abs_kg_per_kg': float(np.max(np.abs(residual))),
        'min_C': float(C.min()), 'max_C': float(C.max()),
        'min_T_K': float(T.min()), 'max_T_K': float(T.max()),
        'min_D': float(D.min()), 'max_D': float(D.max()),
        'positive_properties': bool(min(rho.min(), cp.min(), k.min(), D.min()) > 0),
        'max_radial_C_increase': float(np.max(np.diff(C, axis=0))),
        'final_max_C': float(finalC.max()), 'final_surface_C': float(finalC[-1]),
        'strictly_dry_at_end': bool(finalC.max() < .15),
        'radius_end_m': float(self.model.radius(self.end_s)),
        'radius_extrapolation_used': bool(self.model.settings.shrink and self.end_s > 259200),
        'rhs_evaluations': self.model.evaluations,
        'accepted_time_points': sum(len(p.t) for p in self.pieces),
        'nfev': sum(p.nfev for p in self.pieces),
        'njev': sum(p.njev for p in self.pieces), 'nlu': sum(p.nlu for p in self.pieces),
        'solver_success': all(p.success for p in self.pieces),

```

```

    }

def solve_case(settings: Settings) -> Run:
    started = time.perf_counter()
    model = RadialModel(settings)
    def dry_event(t, y):
        return float(np.max(y[1:-1:2]) - .15)
    dry_event.terminal, dry_event.direction = True, -1
    atol = np.empty(2 * model.n + 1)
    atol[1:-1:2], atol[1:-1:2], atol[-1] = settings.atol_temperature, settings.atol_moisture,
    settings.atol_moisture
    horizon = 1800. if settings.question == 'Q1' else settings.horizon_h * 3600.
    pieces, initial, event_s = [], model.initial(), None
    # A separate segment at 4 h makes the modelling extension explicit.
    endpoints = [0., min(14400., horizon)]
    if horizon > 14400.:
        endpoints.append(horizon)
    for left, right in zip(endpoints[:-1], endpoints[1:]):
        piece = solve_ivp(model.rhs, (left, right), initial, method=settings.method,
            rtol=settings.rtol, atol=atol, jac_sparsity=model.jac_pattern,
            max_step=settings.early_max_step_s if left < 14400. else settings.max_step_s,
            events=None if settings.question == 'Q1' else dry_event, dense_output=True)
        pieces.append(piece)
        if not piece.success:
            raise RuntimeError(piece.message)
        initial = piece.y[:, -1]
        if piece.t_events is not None and len(piece.t_events[0]):
            event_s = float(piece.t_events[0][0])
            # Continue to a genuine post-crossing integer second, not an extrapolation.
            end = float(np.ceil(event_s) + 1)
            tail = solve_ivp(model.rhs, (event_s, end), initial, method=settings.method,
                rtol=settings.rtol, atol=atol, jac_sparsity=model.jac_pattern,
                max_step=1., dense_output=True)
            if not tail.success:
                raise RuntimeError(tail.message)
            pieces.append(tail)
            break
    run = Run(model, pieces, time.perf_counter() - started, event_s)
    diagnostic = run.diagnostics()
    if diagnostic['min_C'] < -1e-8 or not diagnostic['positive_properties']:
        raise FloatingPointError('Physical range/positive property check failed')
    if diagnostic['max_mass_balance_abs_kg_per_kg'] > 1e-6:
        raise FloatingPointError('Dry-basis mass balance failed')
    return run

def file_record(path):
    p = Path(path)
    return {'path': p.relative_to(ROOT).as_posix(), 'bytes': p.stat().st_size,
        'sha256': hashlib.sha256(p.read_bytes()).hexdigest(), 'exists': True}

def save_run(run: Run, directory: Path):
    code_record = file_record(Path(__file__))
    if code_record['sha256'] != LOADED_CODE_SHA256:
        raise RuntimeError('Solver file changed after import; restart to obtain valid
provenance')
    for record in run.model.input_records:
        if file_record(ROOT/record['path'])['sha256'] != record['sha256']:
            raise RuntimeError('Input changed after being loaded; retain failure and rerun')
    directory.mkdir(parents=True, exist_ok=True)
    times = np.unique(np.r_[np.arange(0., run.end_s, 60.),
        [t for t in [100., 300., 600., 900., 1200., 1500., 1800., 3600., 5400., 7200.,
        9000., 10800.] if t <= run.end_s],
        run.end_s, [] if run.event_s is None else [run.event_s]])

```

```

x = np.linspace(0., 1., 21)
T, C = run.fields(times, material_x=x)
state = run.state(times)
np.savez_compressed(directory/'sampled_solution.npz', times_s=times, material_x=x,
                    T_K=T, C=C, radius_m=run.model.radius(times), mean_C=2*run.model.w@state[1:-1:2],
                    cumulative_loss=state[-1])
summary = {'settings': asdict(run.model.settings), 'diagnostics': run.diagnostics(),
           'code': code_record,
           'inputs': run.model.input_records,
           'human_review_status': 'pending', 'gui_reproduced': False}
(directory/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding='utf-8')
return summary

```

E. 16 drying_core_v3_analytic_jacobian.py: Q1 空间检查实际核心快照

文件: paper_output/code/versions/drying_core_v3_analytic_jacobian.py; 共 338 行。

采用范围: Q1 最细空间差及 Q1_K_analytic_J_v3 解析热场历史结果。

原运行槽位: paper_output/code/modeling/drying_core.py。

```
"""2026 A: radial heat and dry-basis moisture transport on a material mesh.

Units: s, m, K, kg water / kg dry matter. See numerical_design.md for derivation.
The supplied empirical rho*cp is an effective thermal capacity. Dry-solid mass
is conserved separately on uniformly shrinking material control volumes.
No latent heat in the baseline; optional surface-latent scenario is labelled.
No clipping of solution values. Coefficients use a positive continuation only
for integrator Newton probes; all accepted states are checked independently.
"""

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
import hashlib
import io
import json
import time

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import lil_matrix
from scipy.special import expi
import analytic_jacobian

ROOT = Path(__file__).resolve().parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
C0, T0, R0, LENGTH = 2.55, 301.15, 0.02, 0.25

def load_inputs(with_records=False):
    arrays, records = [], []
    for name in ['A_environment_observed.csv', 'A_radius_observed.csv']:
        path = ROOT / 'paper_output/data_cleaned' / name
        content = path.read_bytes()
        arrays.append(np.genfromtxt(io.StringIO(content.decode('utf-8-sig')),
                                   delimiter=',', names=True))
        records.append({'path':path.relative_to(ROOT).as_posix(), 'bytes':len(content),
                       'sha256':hashlib.sha256(content).hexdigest(), 'exists':True})
    return (*arrays, records) if with_records else tuple(arrays)

@dataclass(frozen=True)
class Settings:
    question: str = 'Q23'
    intervals: int = 100
    rtol: float = 1e-7
    atol_temperature: float = 1e-7
    atol_moisture: float = 1e-9
    max_step_s: float = 600.0
    early_max_step_s: float = 30.0
    horizon_h: float = 240.0
    shrink: bool = False
    boundary_extension: str = 'nominal'
    tail_temperature_C: float = 50.0
    tail_equilibrium: float = 0.05
    h: float = 25.0
```

```

beta: float = 8e-7
equilibrium_scale: float = 1.0
surface_latent_fraction: float = 0.0
latent_J_kg: float = 2.4e6
constant_D: float | None = None
constant_thermal: bool = False
method: str = 'BDF'
face_scheme: str = 'harmonic'
jacobian_mode: str = 'analytic'

class RadialModel:
    def __init__(self, settings: Settings):
        self.settings = settings
        if settings.intervals < 2 or settings.jacobian_mode not in ('analytic',
'finite_difference'):
            raise ValueError('At least two intervals and a supported Jacobian mode are required')
        self.env, self.rad, self.input_records = load_inputs(with_records=True)
        self.x = np.linspace(0., 1., settings.intervals + 1)
        self.dx = 1. / settings.intervals
        faces = np.r_[0., (self.x[1:] + self.x[:-1]) / 2., 1.]
        self.w = np.diff(faces ** 2) / 2.
        self.internal_faces = faces[1:-1]
        self.n = len(self.x)
        self.rho_d0 = (760 + 90 * C0) / (1 + C0) if settings.question == 'Q4' else (
            820. / (1 + C0) if settings.question == 'Q1' else (650 + 128 * C0) / (1 + C0))
        self.evaluations = 0
        self.jac_pattern = self._sparsity()

    def radius(self, t):
        if self.settings.shrink:
            return np.interp(t, self.rad['time_s'], self.rad['radius_m'])
        return np.asarray(t) * 0. + R0

    def environment(self, t):
        s = self.settings
        tair = np.interp(t, self.env['time_s'], self.env['temperature_K'])
        ceq = np.interp(t, self.env['time_s'], self.env['air_moisture_kg_per_kg'])
        after = np.asarray(t) > self.env['time_s'][-1]
        if s.boundary_extension == 'nominal':
            tair = np.where(after, s.tail_temperature_C + 273.15, tair)
            ceq = np.where(after, s.tail_equilibrium, ceq)
        elif s.boundary_extension == 'tail_mean':
            tail = self.env['time_s'] >= 10800
            tair = np.where(after, self.env['temperature_K'][tail].mean(), tair)
            ceq = np.where(after, self.env['air_moisture_kg_per_kg'][tail].mean(), ceq)
        elif s.boundary_extension != 'last':
            raise ValueError('Unknown boundary extension')
        return tair, ceq * s.equilibrium_scale

    def properties(self, T, C):
        s = self.settings
        positive_C = np.maximum(C, 1e-12) # coefficient continuation, never state clipping
        if np.any(T <= 0):
            raise FloatingPointError('Nonpositive absolute temperature')
        wet = positive_C / (1. + positive_C)
        if s.question == 'Q1':
            rho = np.full_like(C, 820.)
            cp = np.full_like(C, 2600.)
            k = np.full_like(C, .36)
            D = 7e-9 * np.exp(-.89 / positive_C)
        elif s.question in ('Q2', 'Q3', 'Q23'):
            rho, cp, k = 650 + 128 * positive_C, 1450 + 2736 * wet, .21 + .38 * wet
            D = 2.4e-3 * np.exp(-.45 / positive_C - 3850 / T)
        elif s.question == 'Q4':
            rho, cp, k = 760 + 90 * positive_C, 1850 + 2150 * wet, .12 + .20 * wet

```

```

        D = 4.2e-4 * np.exp(-.30 / positive_C - 3850 / T)
    else:
        raise ValueError(s.question)
    if s.constant_D is not None:
        D = np.full_like(C, s.constant_D)
    if s.constant_thermal:
        rho, cp, k = np.full_like(C, 820.), np.full_like(C, 2600.), np.full_like(C, .36)
    return rho, cp, k, D

@staticmethod
def harmonic(a):
    return 2 * a[:-1] * a[1:] / np.maximum(a[:-1] + a[1:], np.finfo(float).tiny)

def _sparsity(self):
    p = lil_matrix((2 * self.n + 1, 2 * self.n + 1), dtype=int)
    for i in range(self.n):
        for j in range(max(0, i - 1), min(self.n, i + 2)):
            p[2*i:2*i+2, 2*j:2*j+2] = 1
    p[-1, 2 * (self.n - 1) + 1] = 1
    return p.tocsr()

def water_internal_flux(self, T, C, D):
    if self.settings.face_scheme == 'harmonic' or self.settings.constant_D is not None:
        return self.internal_faces * self.harmonic(D) * np.diff(C) / self.dx
    if self.settings.face_scheme != 'kirchhoff':
        raise ValueError('Unknown nonlinear face scheme')
    a, D0 = {'Q1':(.89,7e-9), 'Q23':(.45,2.4e-3), 'Q2':(.45,2.4e-3),
            'Q3':(.45,2.4e-3), 'Q4':(.30,4.2e-4)}[self.settings.question]
    cc = np.maximum(C, 1e-12)
    potential = cc * np.exp(-a/cc) + a * expi(-a/cc)
    difference = np.diff(potential)
    small = np.abs(np.diff(cc)) < 1e-7 * np.maximum((cc[:-1]+cc[1:])/2, 1e-3)
    difference[small] = (np.exp(-a/((cc[:-1][small]+cc[1:][small])/2)) * np.diff(cc)[small])
    thermal_factor = 1. if self.settings.question == 'Q1' else
np.exp(-3850/((T[:-1]+T[1:])/2))
    # Do not difference thermal_factor*potential: that would add a false Soret flux.
    return self.internal_faces * D0 * thermal_factor * difference / self.dx

def rhs(self, t, state):
    """REVIEW: actual material-control-volume balance, no extra mesh advection."""
    self.evaluations += 1
    T, C = state[:-1:2], state[1:-1:2]
    rho, cp, k, D = self.properties(T, C)
    radius = float(self.radius(t))
    tair, ceq = self.environment(t)
    heat_g = np.zeros(self.n + 1)
    water_g = np.zeros(self.n + 1)
    heat_g[1:-1] = self.internal_faces * self.harmonic(k) * np.diff(T) / self.dx
    water_g[1:-1] = self.water_internal_flux(T, C, D)
    water_g[-1] = -self.settings.beta * radius * (C[-1] - ceq)
    heat_g[-1] = -self.settings.h * radius * (T[-1] - tair)
    if self.settings.surface_latent_fraction:
        # Scenario: all selected outgoing water vaporizes at the surface.
        rho_d = self.rho_d0 * (R0 / radius) ** 2
        j_evap = rho_d * self.settings.beta * (C[-1] - ceq)
        heat_g[-1] -= (radius * self.settings.surface_latent_fraction *
            self.settings.latent_J_kg * j_evap)
    derivative = np.empty_like(state)
    derivative[:-1:2] = np.diff(heat_g) / (radius ** 2 * self.w * rho * cp)
    derivative[1:-1:2] = np.diff(water_g) / (radius ** 2 * self.w)
    derivative[-1] = 2 * self.settings.beta / radius * (C[-1] - ceq)
    return derivative

def initial(self):
    state = np.empty(2 * self.n + 1)
    state[:-1:2], state[1:-1:2], state[-1] = T0, C0, 0.

```

```

        return state

class Run:
    def __init__(self, model, pieces, elapsed, event_s):
        self.model, self.pieces = model, pieces
        self.elapsed_s, self.event_s = elapsed, event_s
        self.end_s = float(pieces[-1].t[-1])

    def state(self, times):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        if np.min(tt) < -1e-10 or np.max(tt) > self.end_s + 1e-7:
            raise ValueError('Requested time outside solved interval')
        out = np.empty((2 * self.model.n + 1, len(tt)))
        remaining = np.ones(len(tt), dtype=bool)
        for result in self.pieces:
            select = remaining & (tt >= result.t[0]-1e-7) & (tt <= result.t[-1]+1e-7)
            if np.any(select):
                out[:, select] = result.sol(tt[select])
                remaining[select] = False
        if np.any(remaining):
            raise RuntimeError('Missing dense solution segment')
        return out

    def fields(self, times, radii_m=None, material_x=None):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        state = self.state(tt)
        Ts, Cs = state[:-1:2].T, state[1:-1:2].T
        if material_x is not None:
            points = np.asarray(material_x, dtype=float)
            if np.any(~np.isfinite(points)) or np.any((points < 0.) | (points > 1.)):
                raise ValueError('Material coordinates must be finite and within [0,1]')
            return np.array([np.interp(points, self.model.x, row) for row in Ts]), np.array([
                np.interp(points, self.model.x, row) for row in Cs])
        if radii_m is None:
            return Ts, Cs
        radial = np.asarray(radii_m)
        Tout, Cout = [], []
        for i, t in enumerate(tt):
            xx = radial / self.model.radius(t)
            Tout.append(np.interp(xx, self.model.x, Ts[i], left=np.nan, right=np.nan))
            Cout.append(np.interp(xx, self.model.x, Cs[i], left=np.nan, right=np.nan))
        return np.asarray(Tout), np.asarray(Cout)

    def diagnostics(self):
        raw = np.hstack([p.y for p in self.pieces])
        T, C = raw[:-1:2], raw[1:-1:2]
        means = 2 * self.model.w @ C
        residual = means + raw[-1] - C0
        rho, cp, k, D = self.model.properties(T.ravel(), C.ravel())
        final = self.state([self.end_s])[0, :]
        finalC = final[1:-1:2]
        return {
            'event_s': self.event_s, 'event_h': None if self.event_s is None else
self.event_s/3600,
            'end_s': self.end_s, 'elapsed_s': self.elapsed_s,
            'end_time_convention': 'ceil(critical_event_s)+1: conservative post-crossing
verification second; not claimed earliest integer second',
            'max_mass_balance_abs_kg_per_kg': float(np.max(np.abs(residual))),
            'min_C': float(C.min()), 'max_C': float(C.max()),
            'min_T_K': float(T.min()), 'max_T_K': float(T.max()),
            'min_D': float(D.min()), 'max_D': float(D.max()),
            'positive_properties': bool(min(rho.min(), cp.min(), k.min(), D.min()) > 0),
            'max_radial_C_increase': float(np.max(np.diff(C, axis=0))),
            'final_max_C': float(finalC.max()), 'final_surface_C': float(finalC[-1]),
            'strictly_dry_at_end': bool(finalC.max() < .15),

```

```

        'radius_end_m': float(self.model.radius(self.end_s)),
        'radius_extrapolation_used': bool(self.model.settings.shrink and self.end_s > 259200),
        'rhs_evaluations': self.model evaluations,
        'accepted_time_points': sum(len(p.t) for p in self.pieces),
        'nfev': sum(p.nfev for p in self.pieces),
        'njev': sum(p.njev for p in self.pieces), 'nlu': sum(p.nlu for p in self.pieces),
        'solver_success': all(p.success for p in self.pieces),
    }

def solve_case(settings: Settings) -> Run:
    started = time.perf_counter()
    model = RadialModel(settings)
    jacobian_options = ({'jac': lambda t, y: analytic_jacobian.jacobian(model, t, y)}
        if settings.jacobian_mode == 'analytic' else {'jac_sparsity': model.jac_pattern})
    def dry_event(t, y):
        return float(np.max(y[1:-1:2]) - .15)
    dry_event.terminal, dry_event.direction = True, -1
    atol = np.empty(2 * model.n + 1)
    atol[1:-1:2], atol[1:-1:2], atol[-1] = settings.atol_temperature, settings.atol_moisture,
    settings.atol_moisture
    horizon = 1800. if settings.question == 'Q1' else settings.horizon_h * 3600.
    pieces, initial, event_s = [], model.initial(), None
    # A separate segment at 4 h makes the modelling extension explicit.
    endpoints = [0., min(14400., horizon)]
    if horizon > 14400.:
        endpoints.append(horizon)
    for left, right in zip(endpoints[:-1], endpoints[1:]):
        piece = solve_ivp(model.rhs, (left, right), initial, method=settings.method,
            rtol=settings.rtol, atol=atol, **jacobian_options,
            max_step=settings.early_max_step_s if left < 14400. else settings.max_step_s,
            events=None if settings.question == 'Q1' else dry_event, dense_output=True)
        pieces.append(piece)
        if not piece.success:
            raise RuntimeError(piece.message)
        initial = piece.y[:, -1]
        if piece.t_events is not None and len(piece.t_events[0]):
            event_s = float(piece.t_events[0][0])
            # Continue to a genuine post-crossing integer second, not an extrapolation.
            end = float(np.ceil(event_s) + 1)
            tail = solve_ivp(model.rhs, (event_s, end), initial, method=settings.method,
                rtol=settings.rtol, atol=atol, **jacobian_options,
                max_step=1., dense_output=True)
            if not tail.success:
                raise RuntimeError(tail.message)
            pieces.append(tail)
            break
    run = Run(model, pieces, time.perf_counter() - started, event_s)
    diagnostic = run.diagnostics()
    if diagnostic['min_C'] < -1e-8 or not diagnostic['positive_properties']:
        raise FloatingPointError('Physical range/positive property check failed')
    if diagnostic['max_mass_balance_abs_kg_per_kg'] > 1e-6:
        raise FloatingPointError('Dry-basis mass balance failed')
    return run

def file_record(path):
    p = Path(path)
    return {'path': p.relative_to(ROOT).as_posix(), 'bytes': p.stat().st_size,
        'sha256': hashlib.sha256(p.read_bytes()).hexdigest(), 'exists': True}

def save_run(run: Run, directory: Path):
    code_record = file_record(Path(__file__))
    if code_record['sha256'] != LOADED_CODE_SHA256:
        raise RuntimeError('Solver file changed after import; restart to obtain valid

```

```

provenance')
    jacobian_record = file_record(Path(analytic_jacobian.__file__))
    if jacobian_record['sha256'] != analytic_jacobian.LOADED_CODE_SHA256:
        raise RuntimeError('Jacobian file changed after import; restart to obtain valid
provenance')
    for record in run.model.input_records:
        if file_record(ROOT/record['path'])['sha256'] != record['sha256']:
            raise RuntimeError('Input changed after being loaded; retain failure and rerun')
    directory.mkdir(parents=True, exist_ok=True)
    times = np.unique(np.r_[np.arange(0., run.end_s, 60.),
        [t for t in [100., 300., 600., 900., 1200., 1500., 1800., 3600., 5400., 7200.,
9000., 10800.] if t <= run.end_s],
        run.end_s, [] if run.event_s is None else [run.event_s]])
    x = np.linspace(0., 1., 21)
    T, C = run.fields(times, material_x=x)
    state = run.state(times)
    np.savez_compressed(directory/'sampled_solution.npz', times_s=times, material_x=x,
        T_K=T, C=C, radius_m=run.model.radius(times), mean_C=2*run.model.w@state[1:-1:2],
        cumulative_loss=state[-1])
    summary = {'settings': asdict(run.model.settings), 'diagnostics': run.diagnostics(),
        'code': code_record,
        'jacobian_code': jacobian_record,
        'inputs': run.model.input_records,
        'human_review_status': 'pending', 'gui_reproduced': False}
    (directory/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding='utf-8')
    return summary

```

E. 17 drying_core_v5_disk_dense.py: Q23/Q4 空间检查实际核心快照

文件: paper_output/code/versions/drying_core_v5_disk_dense.py; 共 395 行。采用范围: 正文采用的 Q23/Q4 最细相邻网格场差。

版本说明与勘误: 此版本只为对应历史空间证据完整性; 正式答案使用原清单 v6, 不重新把 v5 设为最终生产。

原运行槽位: paper_output/code/modeling/drying_core.py。

```
"""2026 A: radial heat and dry-basis moisture transport on a material mesh.

Units: s, m, K, kg water / kg dry matter. See numerical_design.md for derivation.
The supplied empirical rho*cp is an effective thermal capacity. Dry-solid mass
is conserved separately on uniformly shrinking material control volumes.
No latent heat in the baseline; optional surface-latent scenario is labelled.
No clipping of solution values. Coefficients use a positive continuation only
for integrator Newton probes; all accepted states are checked independently.
"""

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
import hashlib
import io
import json
import time

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import lil_matrix
from scipy.special import expi
import analytic_jacobian
import disk_dense

ROOT = Path(__file__).resolve().parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
C0, T0, R0, LENGTH = 2.55, 301.15, 0.02, 0.25

def load_inputs(with_records=False):
    arrays, records = [], []
    for name in ['A_environment_observed.csv', 'A_radius_observed.csv']:
        path = ROOT / 'paper_output/data_cleaned' / name
        content = path.read_bytes()
        arrays.append(np.genfromtxt(io.StringIO(content.decode('utf-8-sig')),
                                   delimiter=',', names=True))
        records.append({'path':path.relative_to(ROOT).as_posix(), 'bytes':len(content),
                       'sha256':hashlib.sha256(content).hexdigest(), 'exists':True})
    return (*arrays, records) if with_records else tuple(arrays)

@dataclass(frozen=True)
class Settings:
    question: str = 'Q23'
    intervals: int = 100
    rtol: float = 1e-7
    atol_temperature: float = 1e-7
    atol_moisture: float = 1e-9
    max_step_s: float = 600.0
    early_max_step_s: float = 30.0
    horizon_h: float = 240.0
```

```

shrink: bool = False
boundary_extension: str = 'nominal'
tail_temperature_C: float = 50.0
tail_equilibrium: float = 0.05
h: float = 25.0
beta: float = 8e-7
equilibrium_scale: float = 1.0
surface_latent_fraction: float = 0.0
latent_J_kg: float = 2.4e6
constant_D: float | None = None
constant_thermal: bool = False
method: str = 'BDF'
face_scheme: str = 'harmonic'
jacobian_mode: str = 'analytic'
dense_storage: str = 'disk'

```

```

class RadialModel:
    def __init__(self, settings: Settings):
        self.settings = settings
        if settings.intervals < 2 or settings.jacobian_mode not in ('analytic',
'finite_difference'):
            raise ValueError('At least two intervals and a supported Jacobian mode are required')
        self.env, self.rad, self.input_records = load_inputs(with_records=True)
        self.x = np.linspace(0., 1., settings.intervals + 1)
        self.dx = 1. / settings.intervals
        faces = np.r_[0., (self.x[1:] + self.x[:-1]) / 2., 1.]
        self.w = np.diff(faces ** 2) / 2.
        self.internal_faces = faces[1:-1]
        self.n = len(self.x)
        self.rho_d0 = (760 + 90 * C0) / (1 + C0) if settings.question == 'Q4' else (
            820. / (1 + C0) if settings.question == 'Q1' else (650 + 128 * C0) / (1 + C0))
        self.evaluations = 0
        self.jac_pattern = self._sparsity()

    def radius(self, t):
        if self.settings.shrink:
            return np.interp(t, self.rad['time_s'], self.rad['radius_m'])
        return np.asarray(t) * 0. + R0

    def environment(self, t):
        s = self.settings
        tair = np.interp(t, self.env['time_s'], self.env['temperature_K'])
        ceq = np.interp(t, self.env['time_s'], self.env['air_moisture_kg_per_kg'])
        after = np.asarray(t) > self.env['time_s'][-1]
        if s.boundary_extension == 'nominal':
            tair = np.where(after, s.tail_temperature_C + 273.15, tair)
            ceq = np.where(after, s.tail_equilibrium, ceq)
        elif s.boundary_extension == 'tail_mean':
            tail = self.env['time_s'] >= 10800
            tair = np.where(after, self.env['temperature_K'][tail].mean(), tair)
            ceq = np.where(after, self.env['air_moisture_kg_per_kg'][tail].mean(), ceq)
        elif s.boundary_extension != 'last':
            raise ValueError('Unknown boundary extension')
        return tair, ceq * s.equilibrium_scale

    def properties(self, T, C):
        s = self.settings
        positive_C = np.maximum(C, 1e-12) # coefficient continuation, never state clipping
        if np.any(T <= 0):
            raise FloatingPointError('Nonpositive absolute temperature')
        wet = positive_C / (1. + positive_C)
        if s.question == 'Q1':
            rho = np.full_like(C, 820.)
            cp = np.full_like(C, 2600.)
            k = np.full_like(C, .36)

```

```

        D = 7e-9 * np.exp(-.89 / positive_C)
    elif s.question in ('Q2', 'Q3', 'Q23'):
        rho, cp, k = 650 + 128 * positive_C, 1450 + 2736 * wet, .21 + .38 * wet
        D = 2.4e-3 * np.exp(-.45 / positive_C - 3850 / T)
    elif s.question == 'Q4':
        rho, cp, k = 760 + 90 * positive_C, 1850 + 2150 * wet, .12 + .20 * wet
        D = 4.2e-4 * np.exp(-.30 / positive_C - 3850 / T)
    else:
        raise ValueError(s.question)
    if s.constant_D is not None:
        D = np.full_like(C, s.constant_D)
    if s.constant_thermal:
        rho, cp, k = np.full_like(C, 820.), np.full_like(C, 2600.), np.full_like(C, .36)
    return rho, cp, k, D

@staticmethod
def harmonic(a):
    return 2 * a[:-1] * a[1:] / np.maximum(a[:-1] + a[1:], np.finfo(float).tiny)

def _sparsity(self):
    p = lil_matrix((2 * self.n + 1, 2 * self.n + 1), dtype=int)
    for i in range(self.n):
        for j in range(max(0, i - 1), min(self.n, i + 2)):
            p[2*i+2, 2*j+2] = 1
    p[-1, 2 * (self.n - 1) + 1] = 1
    return p.tocsr()

def water_internal_flux(self, T, C, D):
    if self.settings.face_scheme == 'harmonic' or self.settings.constant_D is not None:
        return self.internal_faces * self.harmonic(D) * np.diff(C) / self.dx
    if self.settings.face_scheme != 'kirchhoff':
        raise ValueError('Unknown nonlinear face scheme')
    a, D0 = {'Q1':(.89,7e-9), 'Q23':(.45,2.4e-3), 'Q2':(.45,2.4e-3),
            'Q3':(.45,2.4e-3), 'Q4':(.30,4.2e-4)}[self.settings.question]
    cc = np.maximum(C, 1e-12)
    potential = cc * np.exp(-a/cc) + a * expi(-a/cc)
    difference = np.diff(potential)
    small = np.abs(np.diff(cc)) < 1e-7 * np.maximum((cc[:-1]+cc[1:])/2, 1e-3)
    difference[small] = (np.exp(-a/((cc[:-1][small]+cc[1:][small])/2)) * np.diff(cc)[small])
    thermal_factor = 1. if self.settings.question == 'Q1' else
np.exp(-3850/((T[:-1]+T[1:])/2))
    # Do not difference thermal_factor*potential: that would add a false Soret flux.
    return self.internal_faces * D0 * thermal_factor * difference / self.dx

def rhs(self, t, state):
    """REVIEW: actual material-control-volume balance, no extra mesh advection."""
    self.evaluations += 1
    T, C = state[:-1:2], state[1:-1:2]
    rho, cp, k, D = self.properties(T, C)
    radius = float(self.radius(t))
    tair, ceq = self.environment(t)
    heat_g = np.zeros(self.n + 1)
    water_g = np.zeros(self.n + 1)
    heat_g[1:-1] = self.internal_faces * self.harmonic(k) * np.diff(T) / self.dx
    water_g[1:-1] = self.water_internal_flux(T, C, D)
    water_g[-1] = -self.settings.beta * radius * (C[-1] - ceq)
    heat_g[-1] = -self.settings.h * radius * (T[-1] - tair)
    if self.settings.surface_latent_fraction:
        # Scenario: all selected outgoing water vaporizes at the surface.
        rho_d = self.rho_d0 * (R0 / radius) ** 2
        j_evap = rho_d * self.settings.beta * (C[-1] - ceq)
        heat_g[-1] -= (radius * self.settings.surface_latent_fraction *
            self.settings.latent_J_kg * j_evap)
    derivative = np.empty_like(state)
    derivative[:-1:2] = np.diff(heat_g) / (radius ** 2 * self.w * rho * cp)
    derivative[1:-1:2] = np.diff(water_g) / (radius ** 2 * self.w)

```

```

        derivative[-1] = 2 * self.settings.beta / radius * (C[-1] - ceq)
        return derivative

    def initial(self):
        state = np.empty(2 * self.n + 1)
        state[:-1:2], state[1:-1:2], state[-1] = T0, C0, 0.
        return state

class Run:
    def __init__(self, model, pieces, elapsed, event_s, cache=None):
        self.model, self.pieces = model, pieces
        self.cache = cache
        self.elapsed_s, self.event_s = elapsed, event_s
        self.end_s = float(pieces[-1].t[-1])

    def close(self):
        """Release this Run after its exports/checks; further queries are invalid."""
        self.pieces.clear()
        if self.cache is not None:
            self.cache.close()

    def state(self, times):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        if np.min(tt) < -1e-10 or np.max(tt) > self.end_s + 1e-7:
            raise ValueError('Requested time outside solved interval')
        out = np.empty((2 * self.model.n + 1, len(tt)))
        remaining = np.ones(len(tt), dtype=bool)
        for result in self.pieces:
            select = remaining & (tt >= result.t[0]-1e-7) & (tt <= result.t[-1]+1e-7)
            if np.any(select):
                out[:, select] = result.sol(tt[select])
                remaining[select] = False
        if np.any(remaining):
            raise RuntimeError('Missing dense solution segment')
        return out

    def fields(self, times, radii_m=None, material_x=None):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        state = self.state(tt)
        Ts, Cs = state[:-1:2].T, state[1:-1:2].T
        if material_x is not None:
            points = np.asarray(material_x, dtype=float)
            if np.any(~np.isfinite(points)) or np.any((points < 0.) | (points > 1.)):
                raise ValueError('Material coordinates must be finite and within [0,1]')
            return np.array([np.interp(points, self.model.x, row) for row in Ts]), np.array([
                np.interp(points, self.model.x, row) for row in Cs])
        if radii_m is None:
            return Ts, Cs
        radial = np.asarray(radii_m)
        Tout, Cout = [], []
        for i, t in enumerate(tt):
            xx = radial / self.model.radius(t)
            Tout.append(np.interp(xx, self.model.x, Ts[i], left=np.nan, right=np.nan))
            Cout.append(np.interp(xx, self.model.x, Cs[i], left=np.nan, right=np.nan))
        return np.asarray(Tout), np.asarray(Cout)

    def diagnostics(self):
        # Reduce in bounded blocks: a fine full-domain run may contain tens of
        # millions of accepted state values. Diagnostics must not duplicate all
        # of them and four property arrays at the same time.
        minimum_C = minimum_T = minimum_D = minimum_property = np.inf
        maximum_C = maximum_T = maximum_D = maximum_radial_increase = -np.inf
        maximum_mass_residual = 0.
        for piece in self.pieces:
            for first in range(0, len(piece.t), 256):

```

```

        raw = piece.y[:, first:first+256]
        T, C = raw[:-1:2], raw[1:-1:2]
        residual = 2*self.model.w*C + raw[-1] - C0
        rho, cp, k, D = self.model.properties(T.ravel(), C.ravel())
        minimum_C, maximum_C = min(minimum_C, C.min()), max(maximum_C, C.max())
        minimum_T, maximum_T = min(minimum_T, T.min()), max(maximum_T, T.max())
        minimum_D = min(minimum_D, D.min())
        maximum_D = max(maximum_D, D.max())
        minimum_property = min(minimum_property, rho.min(), cp.min(), k.min(), D.min())
        maximum_radial_increase = max(maximum_radial_increase, np.max(np.diff(C, axis=0)))
        maximum_mass_residual = max(maximum_mass_residual, np.max(np.abs(residual)))
    final = self.state([self.end_s])[:, 0]
    finalC = final[1:-1:2]
    return {
        'event_s': self.event_s, 'event_h': None if self.event_s is None else
self.event_s/3600,
        'end_s': self.end_s, 'elapsed_s': self.elapsed_s,
        'end_time_convention': 'ceil(critical_event_s)+1: conservative post-crossing
verification second; not claimed earliest integer second',
        'max_mass_balance_abs_kg_per_kg': float(maximum_mass_residual),
        'min_C': float(minimum_C), 'max_C': float(maximum_C),
        'min_T_K': float(minimum_T), 'max_T_K': float(maximum_T),
        'min_D': float(minimum_D), 'max_D': float(maximum_D),
        'positive_properties': bool(minimum_property > 0),
        'max_radial_C_increase': float(maximum_radial_increase),
        'final_max_C': float(finalC.max()), 'final_surface_C': float(finalC[-1]),
        'strictly_dry_at_end': bool(finalC.max() < .15),
        'radius_end_m': float(self.model.radius(self.end_s)),
        'radius_extrapolation_used': bool(self.model.settings.shrink and self.end_s > 259200),
        'rhs_evaluations': self.model.evaluations,
        'accepted_time_points': sum(len(p.t) for p in self.pieces),
        'nfev': sum(p.nfev for p in self.pieces),
        'njev': sum(p.njev for p in self.pieces), 'nlu': sum(p.nlu for p in self.pieces),
        'solver_success': all(p.success for p in self.pieces),
        'dense_storage': self.model.settings.dense_storage,
        'dense_coefficient_bytes': 0 if self.cache is None else self.cache.bytes_written,
        'dense_polynomial_count': 0 if self.cache is None else self.cache.polynomial_count,
    }

def solve_case(settings: Settings) -> Run:
    started = time.perf_counter()
    if settings.dense_storage not in ('memory', 'disk'):
        raise ValueError('Dense storage must be memory or disk')
    if settings.dense_storage == 'disk' and settings.method != 'BDF':
        raise ValueError('Exact disk dense storage currently supports BDF only')
    cache = disk_dense.DenseCache(ROOT) if settings.dense_storage == 'disk' else None
    try:
        return _solve_case_impl(settings, cache, started)
    except BaseException:
        if cache is not None:
            cache.close()
        raise

def _solve_case_impl(settings, cache, started):
    model = RadialModel(settings)
    method = disk_dense.DiskBDF if cache is not None else settings.method
    jacobian_options = ({'jac': lambda t, y: analytic_jacobian.jacobian(model, t, y)}
        if settings.jacobian_mode == 'analytic' else {'jac_sparsity': model.jac_pattern})
    if cache is not None:
        jacobian_options['dense_cache'] = cache
    def dry_event(t, y):
        return float(np.max(y[1:-1:2]) - .15)
    dry_event.terminal, dry_event.direction = True, -1
    atol = np.empty(2 * model.n + 1)

```

```

    atol[:-1:2], atol[1:-1:2], atol[-1] = settings.atol_temperature, settings.atol_moisture,
settings.atol_moisture
    horizon = 1800. if settings.question == 'Q1' else settings.horizon_h * 3600.
    pieces, initial, event_s = [], model.initial(), None
    # A separate segment at 4 h makes the modelling extension explicit.
    endpoints = [0., min(14400., horizon)]
    if horizon > 14400.:
        endpoints.append(horizon)
    for left, right in zip(endpoints[:-1], endpoints[1:]):
        piece = solve_ivp(model.rhs, (left, right), initial, method=method,
            rtol=settings.rtol, atol=atol, **jacobian_options,
            max_step=settings.early_max_step_s if left < 14400. else settings.max_step_s,
            events=None if settings.question == 'Q1' else dry_event, dense_output=True)
        pieces.append(piece)
        if not piece.success:
            raise RuntimeError(piece.message)
        initial = piece.y[:, -1].copy()
        if cache is not None:
            piece.y = cache.store_accepted(piece.y)
        if piece.t_events is not None and len(piece.t_events[0]):
            event_s = float(piece.t_events[0][0])
            # Continue to a genuine post-crossing integer second, not an extrapolation.
            end = float(np.ceil(event_s) + 1)
            tail = solve_ivp(model.rhs, (event_s, end), initial, method=method,
                rtol=settings.rtol, atol=atol, **jacobian_options,
                max_step=1., dense_output=True)
            if not tail.success:
                raise RuntimeError(tail.message)
            if cache is not None:
                tail.y = cache.store_accepted(tail.y)
            pieces.append(tail)
            break
    run = Run(model, pieces, time.perf_counter() - started, event_s, cache)
    diagnostic = run.diagnostics()
    if diagnostic['min_C'] < -1e-8 or not diagnostic['positive_properties']:
        raise FloatingPointError('Physical range/positive property check failed')
    if diagnostic['max_mass_balance_abs_kg_per_kg'] > 1e-6:
        raise FloatingPointError('Dry-basis mass balance failed')
    return run

def file_record(path):
    p = Path(path)
    return {'path': p.relative_to(ROOT).as_posix(), 'bytes': p.stat().st_size,
        'sha256': hashlib.sha256(p.read_bytes()).hexdigest(), 'exists': True}

def save_run(run: Run, directory: Path):
    code_record = file_record(Path(__file__))
    if code_record['sha256'] != LOADED_CODE_SHA256:
        raise RuntimeError('Solver file changed after import; restart to obtain valid
provenance')
    jacobian_record = file_record(Path(analytic_jacobian.__file__))
    if jacobian_record['sha256'] != analytic_jacobian.LOADED_CODE_SHA256:
        raise RuntimeError('Jacobian file changed after import; restart to obtain valid
provenance')
    storage_record = file_record(Path(disk_dense.__file__))
    if storage_record['sha256'] != disk_dense.LOADED_CODE_SHA256:
        raise RuntimeError('Dense storage file changed after import; restart for valid
provenance')
    for record in run.model.input_records:
        if file_record(ROOT/record['path'])['sha256'] != record['sha256']:
            raise RuntimeError('Input changed after being loaded; retain failure and rerun')
    directory.mkdir(parents=True, exist_ok=True)
    times = np.unique(np.r_[np.arange(0., run.end_s, 60.),
        [t for t in [100., 300., 600., 900., 1200., 1500., 1800., 3600., 5400., 7200.,

```

```

9000., 10800.] if t <= run.end_s],
        run.end_s, [] if run.event_s is None else [run.event_s]])
x = np.linspace(0., 1., 21)
temperature, moisture, means, losses = [], [], [], []
for first in range(0, len(times), 128):
    block_times = times[first:first+128]
    Tb, Cb = run.fields(block_times, material_x=x)
    raw = run.state(block_times)
    temperature.append(Tb); moisture.append(Cb)
    means.append(2*run.model.w@raw[1:-1:2]); losses.append(raw[-1].copy())
T, C = np.vstack(temperature), np.vstack(moisture)
np.savez_compressed(directory/'sampled_solution.npz', times_s=times, material_x=x,
    T_K=T, C=C, radius_m=run.model.radius(times), mean_C=np.concatenate(means),
    cumulative_loss=np.concatenate(losses))
summary = {'settings': asdict(run.model.settings), 'diagnostics': run.diagnostics(),
    'code': code_record,
    'jacobian_code': jacobian_record,
    'dense_storage_code': storage_record,
    'inputs': run.model.input_records,
    'human_review_status': 'pending', 'gui_reproduced': False}
(directory/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding='utf-8')
return summary

```

E. 18 disk_dense_v5.py: 历史空间检查磁盘轨迹依赖

文件: paper_output/code/versions/disk_dense_v5.py; 共 99 行。采用范围: v5 空间比较实际调用的连续输出; 其断点选段旧问题由后续 v6 修正, 不作为最终存储版本。

原运行槽位: paper_output/code/modeling/disk_dense.py。

```
"""Exact BDF dense polynomials backed by a private, rebuildable disk cache.

This changes storage only: each accepted BDF polynomial is written as float64
bytes, then evaluated with SciPy's original BdfDenseOutput implementation.
No additional time/space interpolation and no solver restart are introduced.
"""

from __future__ import annotations
import hashlib
from pathlib import Path
import tempfile
import numpy as np
from scipy.integrate._ivp.bdf import BDF, BdfDenseOutput
from scipy.integrate._ivp.base import DenseOutput

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

class DenseCache:
    def __init__(self, root):
        self.parent = (Path(root)/'tmp/cache/solver_runs').resolve()
        self.parent.mkdir(parents=True, exist_ok=True)
        self.directory = Path(tempfile.mkdtemp(prefix='bdf_', dir=self.parent)).resolve()
        self.handle = (self.directory/'polynomials.bin').open('w+b', buffering=0)
        self.bytes_written = 0
        self.polynomial_count = 0
        self.accepted_arrays = []
        self.closed = False

    def append(self, values):
        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        array = np.ascontiguousarray(values, dtype=np.float64)
        offset = self.bytes_written
        self.handle.seek(offset)
        array.tofile(self.handle)
        self.bytes_written += array.nbytes
        self.polynomial_count += 1
        return offset, array.shape

    def read(self, offset, shape):
        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        count = int(np.prod(shape))
        self.handle.seek(offset)
        values = np.fromfile(self.handle, dtype=np.float64, count=count)
        if values.size != count:
            raise IOError('Incomplete BDF coefficient cache')
        return values.reshape(shape)

    def store_accepted(self, values):
        path = self.directory/f'accepted_{len(self.accepted_arrays):03d}.npy'
        mapped = np.lib.format.open_memmap(path, mode='w+', dtype=values.dtype,
        shape=values.shape)
        mapped[:] = values
        mapped.flush()
        self.accepted_arrays.append(mapped)
```

```

        return mapped

def close(self):
    if self.closed:
        return
    self.handle.close()
    for array in self.accepted_arrays:
        array._mmap.close()
    self.accepted_arrays.clear()
    # Delete only the verified private cache created by this object.
    if self.directory.parent != self.parent or not self.directory.name.startswith('bdf_'):
        raise RuntimeError('Unexpected private cache path; cleanup refused')
    for path in self.directory.iterdir():
        if not path.is_file() or path.is_symlink():
            raise RuntimeError('Unexpected cache entry; cleanup refused')
        path.unlink()
    self.directory.rmdir()
    self.closed = True

class FileBdfDenseOutput(DenseOutput):
    def __init__(self, original, cache):
        super().__init__(original.t_old, original.t)
        self.order = original.order
        self.t_shift = original.t_shift.copy()
        self.denom = original.denom.copy()
        self.cache = cache
        self.offset, self.shape = cache.append(original.D)

    def _call_impl(self, t):
        # Reuse the installed SciPy evaluator with the exact recorded D bytes.
        dense = object.__new__(BdfDenseOutput)
        dense.D = self.cache.read(self.offset, self.shape)
        dense.t_shift, dense.denom = self.t_shift, self.denom
        return dense._call_impl(t)

class DiskBDF(BDF):
    def __init__(self, *args, dense_cache, **kwargs):
        self.dense_cache = dense_cache
        super().__init__(*args, **kwargs)

    def _dense_output_impl(self):
        return FileBdfDenseOutput(super()._dense_output_impl(), self.dense_cache)

```

E. 19 run_experiments_v1.py: 附录粗网格反例的原批次入口

文件: paper_output/code/versions/run_experiments_v1.py; 共 73 行。采用范围: 数值方案筛选表已引用的早期调和粗网格反例。

版本说明与勘误: 只用于解释已报告的舍弃方案; 不额外收录未被稿件引用的旧试验。历史原入口缺少逐次启动指纹, 快照为后续归档。

原运行槽位: paper_output/code/modeling/run_experiments.py。

```
"""Bounded reproducible batches; each scenario has independent physical assumptions."""
from __future__ import annotations
import argparse
from dataclasses import replace
from datetime import datetime, timezone
import json
from pathlib import Path
import sys
import traceback

from drying_core import ROOT, Settings, solve_case, save_run

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--batch', choices=['baseline', 'grids', 'temporal', 'physical'],
default='baseline')
    parser.add_argument('--n', type=int, default=100)
    parser.add_argument('--tag', default='')
    args = parser.parse_args()
    base = Settings(intervals=args.n)
    if args.batch in ('baseline', 'grids'):
        candidates = [('Q1', replace(base, question='Q1')), ('Q23', base),
('Q4', replace(base, question='Q4', shrink=True))]
    elif args.batch == 'temporal':
        candidates = [('Q23_tight', replace(base, rtol=1e-9, atol_moisture=1e-11,
atol_temperature=1e-9, max_step_s=150., early_max_step_s=10.)),
('Q4_tight', replace(base, question='Q4', shrink=True, rtol=1e-9,
atol_moisture=1e-11, atol_temperature=1e-9, max_step_s=150.,
early_max_step_s=10.))]
    else:
        candidates = [('Q4_fixed', replace(base, question='Q4', shrink=False)),
('Q23_last', replace(base, boundary_extension='last')),
('Q23_mean', replace(base, boundary_extension='tail_mean')),
('Q23_T49', replace(base, tail_temperature_C=49.)),
('Q23_T51', replace(base, tail_temperature_C=51.)),
('Q23_eq045', replace(base, tail_equilibrium=.045)),
('Q23_eq055', replace(base, tail_equilibrium=.055)),
('Q4_T49', replace(base, question='Q4', shrink=True,
tail_temperature_C=49.)),
('Q4_T51', replace(base, question='Q4', shrink=True,
tail_temperature_C=51.)),
('Q23_beta08', replace(base, beta=6.4e-7)),
('Q23_beta12', replace(base, beta=9.6e-7)),
('Q23_latent', replace(base, surface_latent_fraction=1.)),
('Q4_latent', replace(base, question='Q4', shrink=True,
surface_latent_fraction=1.))]
    out = ROOT/paper_output/results/experiments'
    out.mkdir(parents=True, exist_ok=True)
    failed = 0
    for name, settings in candidates:
        trial_id = f'{args.batch}_{name}_N{args.n}{args.tag}'
```

```

folder = out/trial_id
if (folder/'summary.json').exists():
    raise FileExistsError(f'Refusing to overwrite previous trial: {trial_id}')
folder.mkdir(parents=True, exist_ok=True)
start = datetime.now(timezone.utc).isoformat()
print(f'START {trial_id} {start}', flush=True)
try:
    run = solve_case(settings)
    summary = save_run(run, folder)
    record = {'trial_id':trial_id, 'started_at':start,
              'finished_at':datetime.now(timezone.utc).isoformat(),
              'status':'computed', 'exit_code':0, **summary['diagnostics']}
except Exception as exc:
    failed += 1
    (folder/'failure.txt').write_text(traceback.format_exc(), encoding='utf-8')
    record = {'trial_id':trial_id, 'started_at':start, 'status':'failed',
              'exit_code':1, 'error':str(exc)}
    (folder/'trial_record.json').write_text(json.dumps(record, ensure_ascii=False, indent=2),
encoding='utf-8')
    with (out/'trials.jsonl').open('a',encoding='utf-8') as stream:
        stream.write(json.dumps(record, ensure_ascii=False)+'\n')
    print(json.dumps(record, ensure_ascii=False), flush=True)
return 1 if failed else 0

if __name__ == '__main__':
    sys.exit(main())

```

E. 20 drying_core_v1_harmonic.py: 附录粗网格反例的原核心快照

文件: paper_output/code/versions/drying_core_v1_harmonic.py; 共 296 行。采用范围: 附录已经引用 N50 事件 164.5440h 等调和通量粗网格反例。

原运行槽位: paper_output/code/modeling/drying_core.py。

```
"""2026 A: radial heat and dry-basis moisture transport on a material mesh.

Units: s, m, K, kg water / kg dry matter. See numerical_design.md for derivation.
The supplied empirical rho*cp is an effective thermal capacity. Dry-solid mass
is conserved separately on uniformly shrinking material control volumes.
No latent heat in the baseline; optional surface-latent scenario is labelled.
No clipping of solution values. Coefficients use a positive continuation only
for integrator Newton probes; all accepted states are checked independently.
"""

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
import hashlib
import json
import time

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import lil_matrix

ROOT = Path(__file__).resolve().parents[3]
C0, T0, R0, LENGTH = 2.55, 301.15, 0.02, 0.25

def load_inputs():
    env = np.genfromtxt(ROOT / 'paper_output/data_cleaned/A_environment_observed.csv',
                        delimiter=',', names=True, encoding='utf-8-sig')
    rad = np.genfromtxt(ROOT / 'paper_output/data_cleaned/A_radius_observed.csv',
                        delimiter=',', names=True, encoding='utf-8-sig')
    return env, rad

@dataclass(frozen=True)
class Settings:
    question: str = 'Q23'
    intervals: int = 100
    rtol: float = 1e-7
    atol_temperature: float = 1e-7
    atol_moisture: float = 1e-9
    max_step_s: float = 600.0
    early_max_step_s: float = 30.0
    horizon_h: float = 240.0
    shrink: bool = False
    boundary_extension: str = 'nominal'
    tail_temperature_C: float = 50.0
    tail_equilibrium: float = 0.05
    h: float = 25.0
    beta: float = 8e-7
    equilibrium_scale: float = 1.0
    surface_latent_fraction: float = 0.0
    latent_J_kg: float = 2.4e6
    constant_D: float | None = None
    constant_thermal: bool = False
    method: str = 'BDF'
```

```

class RadialModel:
    def __init__(self, settings: Settings):
        self.settings = settings
        self.env, self.rad = load_inputs()
        self.x = np.linspace(0., 1., settings.intervals + 1)
        self.dx = 1. / settings.intervals
        faces = np.r_[0., (self.x[1:] + self.x[:-1]) / 2., 1.]
        self.w = np.diff(faces ** 2) / 2.
        self.internal_faces = faces[1:-1]
        self.n = len(self.x)
        self.rho_d0 = (760 + 90 * C0) / (1 + C0) if settings.question == 'Q4' else (
            820. / (1 + C0) if settings.question == 'Q1' else (650 + 128 * C0) / (1 + C0))
        self.evaluations = 0
        self.jac_pattern = self._sparsity()

    def radius(self, t):
        if self.settings.shrink:
            return np.interp(t, self.rad['time_s'], self.rad['radius_m'])
        return np.asarray(t) * 0. + R0

    def environment(self, t):
        s = self.settings
        tair = np.interp(t, self.env['time_s'], self.env['temperature_K'])
        ceq = np.interp(t, self.env['time_s'], self.env['air_moisture_kg_per_kg'])
        after = np.asarray(t) > self.env['time_s'][-1]
        if s.boundary_extension == 'nominal':
            tair = np.where(after, s.tail_temperature_C + 273.15, tair)
            ceq = np.where(after, s.tail_equilibrium, ceq)
        elif s.boundary_extension == 'tail_mean':
            tail = self.env['time_s'] >= 10800
            tair = np.where(after, self.env['temperature_K'][tail].mean(), tair)
            ceq = np.where(after, self.env['air_moisture_kg_per_kg'][tail].mean(), ceq)
        elif s.boundary_extension != 'last':
            raise ValueError('Unknown boundary extension')
        return tair, ceq * s.equilibrium_scale

    def properties(self, T, C):
        s = self.settings
        positive_C = np.maximum(C, 1e-12) # coefficient continuation, never state clipping
        if np.any(T <= 0):
            raise FloatingPointError('Nonpositive absolute temperature')
        wet = positive_C / (1. + positive_C)
        if s.question == 'Q1':
            rho = np.full_like(C, 820.)
            cp = np.full_like(C, 2600.)
            k = np.full_like(C, .36)
            D = 7e-9 * np.exp(-.89 / positive_C)
        elif s.question in ('Q2', 'Q3', 'Q23'):
            rho, cp, k = 650 + 128 * positive_C, 1450 + 2736 * wet, .21 + .38 * wet
            D = 2.4e-3 * np.exp(-.45 / positive_C - 3850 / T)
        elif s.question == 'Q4':
            rho, cp, k = 760 + 90 * positive_C, 1850 + 2150 * wet, .12 + .20 * wet
            D = 4.2e-4 * np.exp(-.30 / positive_C - 3850 / T)
        else:
            raise ValueError(s.question)
        if s.constant_D is not None:
            D = np.full_like(C, s.constant_D)
        if s.constant_thermal:
            rho, cp, k = np.full_like(C, 820.), np.full_like(C, 2600.), np.full_like(C, .36)
        return rho, cp, k, D

    @staticmethod
    def harmonic(a):
        return 2 * a[:-1] * a[1:] / np.maximum(a[:-1] + a[1:], np.finfo(float).tiny)

```

```

def _sparsity(self):
    p = lil_matrix((2 * self.n + 1, 2 * self.n + 1), dtype=int)
    for i in range(self.n):
        for j in range(max(0, i - 1), min(self.n, i + 2)):
            p[2*i+2, 2*j+2] = 1
    p[-1, 2 * (self.n - 1) + 1] = 1
    return p.tocsr()

def rhs(self, t, state):
    """REVIEW: actual material-control-volume balance, no extra mesh advection."""
    self.evaluations += 1
    T, C = state[:-1:2], state[1:-1:2]
    rho, cp, k, D = self.properties(T, C)
    radius = float(self.radius(t))
    tair, ceq = self.environment(t)
    heat_g = np.zeros(self.n + 1)
    water_g = np.zeros(self.n + 1)
    heat_g[1:-1] = self.internal_faces * self.harmonic(k) * np.diff(T) / self.dx
    water_g[1:-1] = self.internal_faces * self.harmonic(D) * np.diff(C) / self.dx
    water_g[-1] = -self.settings.beta * radius * (C[-1] - ceq)
    heat_g[-1] = -self.settings.h * radius * (T[-1] - tair)
    if self.settings.surface_latent_fraction:
        # Scenario: all selected outgoing water vaporizes at the surface.
        rho_d = self.rho_d0 * (R0 / radius) ** 2
        j_evap = rho_d * self.settings.beta * (C[-1] - ceq)
        heat_g[-1] -= (radius * self.settings.surface_latent_fraction *
                      self.settings.latent_J_kg * j_evap)
    derivative = np.empty_like(state)
    derivative[:-1:2] = np.diff(heat_g) / (radius ** 2 * self.w * rho * cp)
    derivative[1:-1:2] = np.diff(water_g) / (radius ** 2 * self.w)
    derivative[-1] = 2 * self.settings.beta / radius * (C[-1] - ceq)
    return derivative

def initial(self):
    state = np.empty(2 * self.n + 1)
    state[:-1:2], state[1:-1:2], state[-1] = T0, C0, 0.
    return state

class Run:
    def __init__(self, model, pieces, elapsed, event_s):
        self.model, self.pieces = model, pieces
        self.elapsed_s, self.event_s = elapsed, event_s
        self.end_s = float(pieces[-1].t[-1])

    def state(self, times):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        if np.min(tt) < -1e-10 or np.max(tt) > self.end_s + 1e-7:
            raise ValueError('Requested time outside solved interval')
        out = np.empty((2 * self.model.n + 1, len(tt)))
        remaining = np.ones(len(tt), dtype=bool)
        for result in self.pieces:
            select = remaining & (tt >= result.t[0]-1e-7) & (tt <= result.t[-1]+1e-7)
            if np.any(select):
                out[:, select] = result.sol(tt[select])
                remaining[select] = False
        if np.any(remaining):
            raise RuntimeError('Missing dense solution segment')
        return out

    def fields(self, times, radii_m=None, material_x=None):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        state = self.state(tt)
        Ts, Cs = state[:-1:2].T, state[1:-1:2].T
        if material_x is not None:
            points = np.asarray(material_x)

```

```

        return np.array([np.interp(points, self.model.x, row) for row in Ts]), np.array([
            np.interp(points, self.model.x, row) for row in Cs])
    if radii_m is None:
        return Ts, Cs
    radial = np.asarray(radii_m)
    Tout, Cout = [], []
    for i, t in enumerate(tt):
        xx = radial / self.model.radius(t)
        Tout.append(np.interp(xx, self.model.x, Ts[i], left=np.nan, right=np.nan))
        Cout.append(np.interp(xx, self.model.x, Cs[i], left=np.nan, right=np.nan))
    return np.asarray(Tout), np.asarray(Cout)

def diagnostics(self):
    raw = np.hstack([p.y for p in self.pieces])
    T, C = raw[:-1:2], raw[1:-1:2]
    means = 2 * self.model.w @ C
    residual = means + raw[-1] - C0
    rho, cp, k, D = self.model.properties(T.ravel(), C.ravel())
    final = self.state([self.end_s])[:, 0]
    finalC = final[1:-1:2]
    return {
        'event_s': self.event_s, 'event_h': None if self.event_s is None else
self.event_s/3600,
        'end_s': self.end_s, 'elapsed_s': self.elapsed_s,
        'max_mass_balance_abs_kg_per_kg': float(np.max(np.abs(residual))),
        'min_C': float(C.min()), 'max_C': float(C.max()),
        'min_T_K': float(T.min()), 'max_T_K': float(T.max()),
        'min_D': float(D.min()), 'max_D': float(D.max()),
        'positive_properties': bool(min(rho.min(), cp.min(), k.min(), D.min()) > 0),
        'max_radial_C_increase': float(np.max(np.diff(C, axis=0))),
        'final_max_C': float(finalC.max()), 'final_surface_C': float(finalC[-1]),
        'strictly_dry_at_end': bool(finalC.max() < .15),
        'radius_end_m': float(self.model.radius(self.end_s)),
        'radius_extrapolation_used': bool(self.model.settings.shrink and self.end_s > 259200),
        'rhs_evaluations': self.model.evaluations,
        'accepted_time_points': sum(len(p.t) for p in self.pieces),
        'nfev': sum(p.nfev for p in self.pieces),
        'njev': sum(p.njev for p in self.pieces), 'nlu': sum(p.nlu for p in self.pieces),
        'solver_success': all(p.success for p in self.pieces),
    }

def solve_case(settings: Settings) -> Run:
    started = time.perf_counter()
    model = RadialModel(settings)
    def dry_event(t, y):
        return float(np.max(y[1:-1:2]) - .15)
    dry_event.terminal, dry_event.direction = True, -1
    atol = np.empty(2 * model.n + 1)
    atol[:-1:2], atol[1:-1:2], atol[-1] = settings.atol_temperature, settings.atol_moisture,
settings.atol_moisture
    horizon = 1800. if settings.question == 'Q1' else settings.horizon_h * 3600.
    pieces, initial, event_s = [], model.initial(), None
    # A separate segment at 4 h makes the modelling extension explicit.
    endpoints = [0., min(14400., horizon)]
    if horizon > 14400.:
        endpoints.append(horizon)
    for left, right in zip(endpoints[:-1], endpoints[1:]):
        piece = solve_ivp(model.rhs, (left, right), initial, method=settings.method,
            rtol=settings.rtol, atol=atol, jac_sparsity=model.jac_pattern,
            max_step=settings.early_max_step_s if left < 14400. else settings.max_step_s,
            events=None if settings.question == 'Q1' else dry_event, dense_output=True)
        pieces.append(piece)
        if not piece.success:
            raise RuntimeError(piece.message)
    initial = piece.y[:, -1]

```

```

    if piece.t_events is not None and len(piece.t_events[0]):
        event_s = float(piece.t_events[0][0])
        # Continue to a genuine post-crossing integer second, not an extrapolation.
        end = float(np.ceil(event_s) + 1)
        tail = solve_ivp(model.rhs, (event_s, end), initial, method=settings.method,
                        rtol=settings.rtol, atol=atol, jac_sparsity=model.jac_pattern,
                        max_step=1., dense_output=True)
        if not tail.success:
            raise RuntimeError(tail.message)
        pieces.append(tail)
        break
run = Run(model, pieces, time.perf_counter() - started, event_s)
diagnostic = run.diagnostics()
if diagnostic['min_C'] < -1e-8 or not diagnostic['positive_properties']:
    raise FloatingPointError('Physical range/positive property check failed')
if diagnostic['max_mass_balance_abs_kg_per_kg'] > 1e-6:
    raise FloatingPointError('Dry-basis mass balance failed')
return run

def file_record(path):
    p = Path(path)
    return {'path': p.relative_to(ROOT).as_posix(), 'bytes': p.stat().st_size,
            'sha256': hashlib.sha256(p.read_bytes()).hexdigest(), 'exists': True}

def save_run(run: Run, directory: Path):
    directory.mkdir(parents=True, exist_ok=True)
    times = np.unique(np.r_[np.arange(0., run.end_s, 60.),
                           [t for t in [100., 300., 600., 900., 1200., 1500., 1800., 3600., 5400., 7200.,
9000., 10800.] if t <= run.end_s],
                           run.end_s, [] if run.event_s is None else [run.event_s]])
    x = np.linspace(0., 1., 21)
    T, C = run.fields(times, material_x=x)
    state = run.state(times)
    np.savez_compressed(directory/'sampled_solution.npz', times_s=times, material_x=x,
                       T_K=T, C=C, radius_m=run.model.radius(times), mean_C=2*run.model.w@state[1:-1:2],
                       cumulative_loss=state[-1])
    summary = {'settings': asdict(run.model.settings), 'diagnostics': run.diagnostics(),
              'code': file_record(Path(__file__)),
              'inputs': [file_record(ROOT/'paper_output/data_cleaned'/name) for name in
                          ['A_environment_observed.csv', 'A_radius_observed.csv']],
              'human_review_status': 'pending', 'gui_reproduced': False}
    (directory/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2),
    encoding='utf-8')
    return summary

```

E. 21 runCrossCheck.m: MATLAB 独立 N40 求解入口

文件: paper_output/code/review_delivery/matlab/runCrossCheck.m; 共 495 行。采用范围: 独立 MATLAB 有限体积/Kirchhoff/ode15s; 所有本地子函数在本文件, 无其他 .m 依赖。

```
function runSummary = runCrossCheck(outputDirectory)
%RUNCROSSCHECK 用独立 MATLAB 实现核验 A 题 Q1--Q4 的离散模型。
% 在 MATLAB GUI 中调用本函数; 函数自身不能证明 GUI 操作或人工审查。
% 单位: 秒、米、开尔文、kg 水/kg 干物质; 默认 N=40 是跨语言核验网格,
% 不能代替正式 N3200/N6400 生产结果或空间收敛验证。
% 示例: runCrossCheck('D:/Document/数学建模/2026CUMCM/paper_output/qa/matlab_crosscheck_20260911')

sourcePath = [mfilename('fullpath'), '.m'];
projectRoot = fileparts(sourcePath);
for parentIndex = 1:4
    projectRoot = fileparts(projectRoot);
end
if nargin < 1 || strlen(string(outputDirectory)) == 0
    runStamp = char(datetime('now', 'Format', 'yyyyMMdd_HHmss_SSS'));
    outputDirectory = fullfile(projectRoot, 'paper_output', 'qa', ...
        ['matlab_crosscheck_', runStamp]);
end
outputDirectory = char(outputDirectory);
% Java 仅用于路径规范化和 SHA256, 不参与模型计算。
outputFile = java.io.File(outputDirectory);
projectRootFile = java.io.File(projectRoot);
outputDirectory = char(outputFile.getCanonicalPath());
canonicalRoot = char(projectRootFile.getCanonicalPath());
if ~startsWith(lower(outputDirectory), [lower(canonicalRoot), filesep])
    error('CrossCheck:WorkspaceBoundary', '输出必须位于本次 2026CUMCM 子工作区。');
end
summaryPath = fullfile(outputDirectory, 'runSummary.json');
if isfile(summaryPath)
    error('CrossCheck:ExistingRun', '该目录已有运行记录, 请使用新的输出目录以保留证据。');
end
if ~isfolder(outputDirectory)
    mkdir(outputDirectory);
end
logPath = fullfile(outputDirectory, 'runLog.txt');
logFileId = fopen(logPath, 'w', 'n', 'UTF-8');
if logFileId < 0
    error('CrossCheck:LogOpen', '无法创建运行日志。');
end
logCleanup = onCleanup(@() fclose(logFileId)); %#ok<NASGU>
runTimer = tic;
runSummary = struct('schemaVersion', 1, 'status', 'RUNNING', ...
    'startedAtUtc', utcStamp(), 'finishedAtUtc', '', 'elapsedSec', [], ...
    'matlabVersion', version, 'matlabRelease', version('-release'), ...
    'computer', computer, 'sourcePath', sourcePath, ...
    'sourceSha256', sha256File(sourcePath), 'outputDirectory', outputDirectory, ...
    'guiReproduced', [], 'guiEvidenceStatus', 'external_observation_required', ...
    'humanReviewStatus', 'pending', 'caseResults', {}, 'exception', []);
settings = struct('intervalCount', 40, 'nodeCount', 41, 'relativeTolerance', 1e-10, ...
    'temperatureAbsoluteTolerance', 1e-10, 'moistureAbsoluteTolerance', 1e-12, ...
    'earlyMaxStepSec', 2, 'lateMaxStepSec', 120, 'horizonSec', 240*3600, ...
    'initialTemperatureK', 301.15, 'initialMoisture', 2.55, ...
    'initialRadiusM', 0.02, 'fixedLengthM', 0.25, ...
    'heatTransferCoefficient', 25, 'massTransferCoefficient', 8e-7, ...
    'dryThreshold', 0.15, 'environmentSwitchSec', 14400, ...
    'tailTemperatureK', 323.15, 'tailEquilibriumMoisture', 0.05, ...
    'solver', 'ode15s_default_NDF', 'jacobianMode', 'JPattern_finite_difference', ...
```

```

        'waterFaceScheme', 'Kirchhoff', 'heatFaceScheme', 'harmonic', ...
        'coefficientFloor', 1e-12, 'latentHeatIncluded', false);
runSummary.settings = settings;
writeJson(summaryPath, runSummary);
logLine(logFileId, 'START %s | MATLAB %s | output=%s', ...
        runSummary.startedAtUtc, version, outputDirectory);
try
    environmentPath = fullfile(projectRoot, 'paper_output', 'data_cleaned',
'A_environment_observed.csv');
    radiusPath = fullfile(projectRoot, 'paper_output', 'data_cleaned',
'A_radius_observed.csv');
    inputData.environment = readtable(environmentPath, 'VariableNamingRule', 'preserve');
    inputData.radius = readtable(radiusPath, 'VariableNamingRule', 'preserve');
    validateInputs(inputData);
    runSummary.inputs = {fileRecord(environmentPath), fileRecord(radiusPath)};
    caseNames = {'Q1', 'Q23', 'Q4'};
    for caseIndex = 1:numel(caseNames)
        caseName = caseNames{caseIndex};
        logLine(logFileId, 'CASE_START %s at %s', caseName, utcStamp());
        caseTimer = tic;
        caseResult = solveSingleCase(caseName, inputData, settings, logFileId);
        caseResult.elapsedSec = toc(caseTimer);
        caseResult.finishedAtUtc = utcStamp();
        casePath = fullfile(outputDirectory, [caseName, '_crossCheck.json']);
        writeJson(casePath, caseResult);
        sampleTable = makeSampleTable(caseResult);
        writetable(sampleTable, fullfile(outputDirectory, [caseName, '_samples.csv']));
        if strcmp(caseName, 'Q23')
            % Q2 与 Q3 共用从 t=0 的附录 3 轨迹，导出同源但分问的查阅入口。
            sampleTable.question(:) = "Q2";
            writetable(sampleTable, fullfile(outputDirectory, 'Q2_samples.csv'));
            sampleTable.question(:) = "Q3";
            writetable(sampleTable, fullfile(outputDirectory, 'Q3_samples.csv'));
        end
        runSummary.caseResults{end+1} = caseResult;
        writeJson(summaryPath, runSummary);
        logLine(logFileId, 'CASE_END %s | elapsed=%.6f s | event=%.12g s | massResidual=%.3g',
...
            caseName, caseResult.elapsedSec, scalarOrNaN(caseResult.eventSec), ...
            caseResult.diagnostics.maxMassBalanceAbs);
    end
    if ~strcmp(runSummary.sourceSha256, sha256File(sourcePath))
        error('CrossCheck:SourceChanged', '源码在运行中改变，本轮不能作为固定版本证据。');
    end
    for inputIndex = 1:numel(runSummary.inputs)
        record = runSummary.inputs{inputIndex};
        if ~strcmp(record.sha256, sha256File(record.path))
            error('CrossCheck:InputChanged', '输入在运行中改变: %s', record.path);
        end
    end
    runSummary.status = 'COMPLETED_NUMERICAL_CHECKS';
    runSummary.finishedAtUtc = utcStamp();
    runSummary.elapsedSec = toc(runTimer);
    writeJson(summaryPath, runSummary);
    logLine(logFileId, 'FINISH %s | elapsed=%.6f s | humanReview=pending', ...
        runSummary.finishedAtUtc, runSummary.elapsedSec);
catch runError
    runSummary.status = 'FAILED';
    runSummary.finishedAtUtc = utcStamp();
    runSummary.elapsedSec = toc(runTimer);
    runSummary.exception = struct('identifier', runError.identifier, ...
        'message', runError.message, 'report', getReport(runError, 'extended', 'hyperlinks',
'off'));
    writeJson(summaryPath, runSummary);
    logLine(logFileId, 'FAILED %s\n%s', runSummary.finishedAtUtc,
runSummary.exception.report);

```

```

        rethrow(runError);
    end
end

function caseResult = solveSingleCase(caseName, inputData, settings, logFileId)
    caseStartedAtUtc = utcStamp();
    nodeCount = settings.nodeCount;
    materialX = linspace(0, 1, nodeCount)';
    deltaX = 1/settings.intervalCount;
    faceX = [0; (materialX(1:end-1)+materialX(2:end))/2; 1];
    cellWeights = diff(faceX.^2)/2;
    internalFaceX = faceX(2:end-1);
    stateCount = 2*nodeCount+1;
    initialState = zeros(stateCount, 1);
    initialState(1:2:end-1) = settings.initialTemperatureK;
    initialState(2:2:end-1) = settings.initialMoisture;
    absoluteTolerance = repmat(settings.moistureAbsoluteTolerance, stateCount, 1);
    absoluteTolerance(1:2:end-1) = settings.temperatureAbsoluteTolerance;
    jacobianPattern = makeJacobianPattern(nodeCount);
    baseOptions = odeset('RelTol', settings.relativeTolerance, 'AbsTol', absoluteTolerance, ...
        'JPattern', jacobianPattern, 'Vectorized', 'off', 'Stats', 'off');
    if strcmp(caseName, 'Q1')
        horizonSec = 1800;
        dryDensityInitial = 820/(1+settings.initialMoisture);
    elseif strcmp(caseName, 'Q4')
        horizonSec = settings.horizonSec;
        dryDensityInitial = (760+90*settings.initialMoisture)/(1+settings.initialMoisture);
    else
        horizonSec = settings.horizonSec;
        dryDensityInitial = (650+128*settings.initialMoisture)/(1+settings.initialMoisture);
    end
    dryMassKg = dryDensityInitial*pi*settings.initialRadiusM^2*settings.fixedLengthM;
    endPoints = unique([0, min(settings.environmentSwitchSec, horizonSec), horizonSec]);
    solutions = {};
    warningRecords = {};
    eventSec = [];
    rhsEvaluations = 0;
    for segmentIndex = 1:numel(endPoints)-1
        leftSec = endPoints(segmentIndex);
        rightSec = endPoints(segmentIndex+1);
        if leftSec < settings.environmentSwitchSec
            maxStepSec = settings.earlyMaxStepSec;
        else
            maxStepSec = settings.lateMaxStepSec;
        end
        segmentOptions = odeset(baseOptions, 'MaxStep', maxStepSec);
        if ~strcmp(caseName, 'Q1')
            segmentOptions = odeset(segmentOptions, 'Events', @dryEvent);
        end
        lastwarn('');
        segment = ode15s(@balanceRhs, [leftSec, rightSec], initialState, segmentOptions);
        [warningText, warningId] = lastwarn;
        recordWarning(warningText, warningId, leftSec, rightSec);
        solutions{end+1} = segment;
        hasEvent = isfield(segment, 'xe') && ~isempty(segment.xe);
        if hasEvent
            eventSec = segment.xe(end);
            initialState = segment.ye(:, end);
            tailEndSec = ceil(eventSec)+1;
            % 真正续算到事件后的整数秒；禁止靠插值外推声称严格达标。
            tailOptions = odeset(baseOptions, 'MaxStep', 1, 'Events', []);
            lastwarn('');
            tail = ode15s(@balanceRhs, [eventSec, tailEndSec], initialState, tailOptions);
            [warningText, warningId] = lastwarn;
            recordWarning(warningText, warningId, eventSec, tailEndSec);
            assertReached(tail, tailEndSec);
        end
    end
end

```

```

        solutions{end+1} = tail;
        break;
    end
    assertReached(segment, rightSec);
    initialState = segment.y(:, end);
end
if ~strcmp(caseName, 'Q1') && isempty(eventSec)
    error('CrossCheck:NoDryEvent', '%s 在 240 小时内没有找到干燥事件。', caseName);
end
endSec = solutions{end}.x(end);
rawSampleTimes = [0, 1800, 10800];
rawSampleTimes = rawSampleTimes(rawSampleTimes <= endSec);
sampleTimesSec = unique([rawSampleTimes, eventSec, endSec]);
sampleStates = evaluatePieces(solutions, sampleTimesSec);
sampleX = [0, 0.25, 0.5, 0.75, 1];
temperatureK = interp1(materialX, sampleStates(1:2:end-1, :), sampleX, 'linear');
moisture = interp1(materialX, sampleStates(2:2:end-1, :), sampleX, 'linear');
sampleRadiusM = zeros(size(sampleTimesSec));
sampleTypes = strings(size(sampleTimesSec));
for sampleIndex = 1:numel(sampleTimesSec)
    sampleRadiusM(sampleIndex) = radiusAt(sampleTimesSec(sampleIndex));
    if sampleTimesSec(sampleIndex) == 0
        sampleTypes(sampleIndex) = "initial";
    elseif ~isempty(eventSec) && sampleTimesSec(sampleIndex) == eventSec
        sampleTypes(sampleIndex) = "criticalEvent";
    elseif ~isempty(eventSec) && sampleTimesSec(sampleIndex) == endSec
        sampleTypes(sampleIndex) = "postEventIntegerSecond";
    else
        sampleTypes(sampleIndex) = "fixedTime";
    end
end
end
diagnostic = struct('minMoisture', Inf, 'maxMoisture', -Inf, ...
    'minTemperatureK', Inf, 'maxTemperatureK', -Inf, 'minDiffusivity', Inf, ...
    'positiveProperties', true, 'maxMassBalanceAbs', 0, ...
    'maxRadialMoistureIncrease', -Inf, 'acceptedTimePoints', 0);
for pieceIndex = 1:numel(solutions)
    acceptedStates = solutions{pieceIndex}.y;
    acceptedTemperature = acceptedStates(1:2:end-1, :);
    acceptedMoisture = acceptedStates(2:2:end-1, :);
    [density, heatCapacity, conductivity, diffusivity] =
materialProperties(acceptedTemperature, acceptedMoisture, caseName);
    diagnostic.minMoisture = min(diagnostic.minMoisture, min(acceptedMoisture, [], 'all'));
    diagnostic.maxMoisture = max(diagnostic.maxMoisture, max(acceptedMoisture, [], 'all'));
    diagnostic.minTemperatureK = min(diagnostic.minTemperatureK, min(acceptedTemperature, [],
'all'));
    diagnostic.maxTemperatureK = max(diagnostic.maxTemperatureK, max(acceptedTemperature, [],
'all'));
    diagnostic.minDiffusivity = min(diagnostic.minDiffusivity, min(diffusivity, [], 'all'));
    diagnostic.positiveProperties = diagnostic.positiveProperties && ...
        all(density > 0 & heatCapacity > 0 & conductivity > 0 & diffusivity > 0, 'all');
    massResidual = 2*cellWeights'*acceptedMoisture+acceptedStates(end, :)-
settings.initialMoisture;
    diagnostic.maxMassBalanceAbs = max(diagnostic.maxMassBalanceAbs, max(abs(massResidual)));
    diagnostic.maxRadialMoistureIncrease = max(diagnostic.maxRadialMoistureIncrease, ...
        max(diff(acceptedMoisture, 1, 1), [], 'all'));
    diagnostic.acceptedTimePoints =
diagnostic.acceptedTimePoints+numel(solutions{pieceIndex}.x);
end
finalState = evaluatePieces(solutions, endSec);
diagnostic.finalMaxMoisture = max(finalState(2:2:end-1));
diagnostic.strictlyDryAtEnd = diagnostic.finalMaxMoisture < settings.dryThreshold;
diagnostic.rhsEvaluations = rhsEvaluations;
diagnostic.radiusExtrapolationUsed = strcmp(caseName, 'Q4') && endSec >
inputData.radius.time_s(end);
diagnostic.checkScope = 'accepted ode15s states; not a continuous exact-solution error
bound';

```

```

if diagnostic.minMoisture < -1e-8 || ~diagnostic.positiveProperties
    error('CrossCheck:PhysicalRange', '%s 的接受状态违反物性正值或水分范围。', caseName);
end
if diagnostic.maxMassBalanceAbs > 1e-6
    error('CrossCheck:MassBalance', '%s 的离散干基质量守恒残差超限。', caseName);
end
if ~strcmp(caseName, 'Q1') && ~diagnostic.strictlyDryAtEnd
    error('CrossCheck:StrictDryness', '%s 的事件后原精度 maxC 没有严格小于 0.15。', caseName);
end
caseResult = struct('question', caseName, 'startedAtUtc', caseStartedAtUtc, ...
    'intervalCount', settings.intervalCount, 'eventSec', eventSec, ...
    'eventHours', eventSec/3600, 'endSec', endSec, 'dryMassKg', dryMassKg, ...
    'endTimeConvention', 'ceil(eventSec)+1; actual integration, not earliest integer-second
claim', ...
    'sampleTimesSec', sampleTimesSec, 'sampleTypes', sampleTypes, 'sampleMaterialX', sampleX,
...
    'sampleRadiusM', sampleRadiusM, 'temperatureK', temperatureK, ...
    'temperatureC', temperatureK-273.15, 'moistureDryBasis', moisture, ...
    'fullMaterialX', materialX, 'fullStateBySample', sampleStates, ...
    'stateOrder', 'T0,C0,T1,C1,...,TN,CN,cumulativeDryBasisLoss', ...
    'meanMoisture', 2*cellWeights'*sampleStates(2:2:end-1, :), ...
    'cumulativeLoss', sampleStates(end, :), 'diagnostics', diagnostic, ...
    'warnings', {warningRecords}, 'warningCapture', 'last warning per segment; not all
warning messages', ...
    'humanReviewStatus', 'pending');

function derivative = balanceRhs(timeSec, state)
    rhsEvaluations = rhsEvaluations+1;
    temperature = state(1:2:end-1);
    waterContent = state(2:2:end-1);
    [density, heatCapacity, conductivity, ~] = materialProperties(temperature, waterContent,
caseName);
    radiusM = radiusAt(timeSec);
    [airTemperatureK, equilibriumMoisture] = environmentAt(timeSec);
    heatFlux = zeros(nodeCount+1, 1);
    waterFlux = zeros(nodeCount+1, 1);
    faceConductivity = 2*conductivity(1:end-1).*conductivity(2:end) ./ ...
        max(conductivity(1:end-1)+conductivity(2:end), realmin);
    heatFlux(2:end-1) = internalFaceX.*faceConductivity.*diff(temperature)/deltaX;
    if strcmp(caseName, 'Q1')
        moistureExponent = 0.89;
        diffusionPrefactor = 7e-9;
        thermalFactor = ones(nodeCount-1, 1);
    elseif strcmp(caseName, 'Q4')
        moistureExponent = 0.30;
        diffusionPrefactor = 4.2e-4;
        thermalFactor = exp(-3850./((temperature(1:end-1)+temperature(2:end))/2));
    else
        moistureExponent = 0.45;
        diffusionPrefactor = 2.4e-3;
        thermalFactor = exp(-3850./((temperature(1:end-1)+temperature(2:end))/2));
    end
    positiveMoisture = max(waterContent, settings.coefficientFloor);
    % E1(z)=expint(z)=-Ei(-z), 不是 Python scipy.special.expi 的同名替代。
    kirchhoffPotential = positiveMoisture.*exp(-moistureExponent./positiveMoisture) ...
        -moistureExponent.*expint(moistureExponent./positiveMoisture);
    potentialDifference = diff(kirchhoffPotential);
    meanFaceMoisture = (positiveMoisture(1:end-1)+positiveMoisture(2:end))/2;
    moistureDifference = diff(positiveMoisture);
    closePair = abs(moistureDifference) < 1e-7*max(meanFaceMoisture, 1e-3);
    potentialDifference(closePair) = exp(-moistureExponent./meanFaceMoisture(closePair)) ...
        .*moistureDifference(closePair);
    % 温度因子在面上求值, 不放入势函数再差分, 以免引入虚构 Soret 项。
    waterFlux(2:end-1) =
internalFaceX.*diffusionPrefactor.*thermalFactor.*potentialDifference/deltaX;
    waterFlux(end) = -settings.massTransferCoefficient*radiusM*(waterContent(end)-

```

```

equilibriumMoisture);
    heatFlux(end) = -settings.heatTransferCoefficient*radiusM*(temperature(end)-
airTemperatureK);
    derivative = zeros(stateCount, 1);
    derivative(1:2:end-1) = diff(heatFlux)./(radiusM^2*cellWeights.*density.*heatCapacity);
    % Q4 网格随材料同比收缩; 固定长度并另守恒干物质, 故无附加网格平流。
    derivative(2:2:end-1) = diff(waterFlux)./(radiusM^2*cellWeights);
    derivative(end) = 2*settings.massTransferCoefficient/radiusM*(waterContent(end)-
equilibriumMoisture);
end

function radiusM = radiusAt(timeSec)
    if strcmp(caseName, 'Q4')
        boundedTimeSec = min(max(timeSec, inputData.radius.time_s(1)),
inputData.radius.time_s(end));
        radiusM = interp1(inputData.radius.time_s, inputData.radius.radius_m, boundedTimeSec,
'linear');
    else
        radiusM = settings.initialRadiusM;
    end
end

function [airTemperatureK, equilibriumMoisture] = environmentAt(timeSec)
    % 4h 以后 50 摄氏度/0.05 为闭合假设; 不是额外实测数据。
    if timeSec > inputData.environment.time_s(end)
        airTemperatureK = settings.tailTemperatureK;
        equilibriumMoisture = settings.tailEquilibriumMoisture;
    else
        boundedTimeSec = max(timeSec, inputData.environment.time_s(1));
        airTemperatureK = interp1(inputData.environment.time_s,
inputData.environment.temperature_K, boundedTimeSec, 'linear');
        equilibriumMoisture = interp1(inputData.environment.time_s,
inputData.environment.air_moisture_kg_per_kg, boundedTimeSec, 'linear');
    end
end

function [eventValue, isTerminal, direction] = dryEvent(~, state)
    eventValue = max(state(2:2:end-1))-settings.dryThreshold;
    isTerminal = 1;
    direction = -1;
end

function recordWarning(warningText, warningId, leftSec, rightSec)
    if ~isempty(warningText)
        warningRecords{end+1} = struct('identifier', warningId, 'message', warningText, ...
            'segmentStartSec', leftSec, 'segmentTargetSec', rightSec);
        logLine(logFileId, 'WARNING %s [%s] %s', caseName, warningId, warningText);
    end
end

function [density, heatCapacity, conductivity, diffusivity] = materialProperties(temperatureK,
waterContent, caseName)
    if any(~isfinite(temperatureK) | temperatureK <= 0, 'all') || any(~isfinite(waterContent),
'all')
        error('CrossCheck:InvalidState', '遇到非有限状态或非正绝对温度。');
    end
    % 仅对 Newton 探测点的系数作正延拓, 不截断积分状态。
    positiveMoisture = max(waterContent, 1e-12);
    wetFraction = positiveMoisture./(1+positiveMoisture);
    switch caseName
        case 'Q1'
            density = 820*ones(size(waterContent));
            heatCapacity = 2600*ones(size(waterContent));
            conductivity = 0.36*ones(size(waterContent));
            diffusivity = 7e-9*exp(-0.89./positiveMoisture);

```

```

        case 'Q23'
            density = 650+128*positiveMoisture;
            heatCapacity = 1450+2736*wetFraction;
            conductivity = 0.21+0.38*wetFraction;
            diffusivity = 2.4e-3*exp(-0.45./positiveMoisture-3850./temperatureK);
        case 'Q4'
            density = 760+90*positiveMoisture;
            heatCapacity = 1850+2150*wetFraction;
            conductivity = 0.12+0.20*wetFraction;
            diffusivity = 4.2e-4*exp(-0.30./positiveMoisture-3850./temperatureK);
        otherwise
            error('CrossCheck:Question', '不支持的问题名: %s', caseName);
    end
end

function jacobianPattern = makeJacobianPattern(nodeCount)
    stateCount = 2*nodeCount+1;
    jacobianPattern = sparse(stateCount, stateCount);
    for nodeIndex = 1:nodeCount
        rowIndices = 2*nodeIndex-1:2*nodeIndex;
        for neighborIndex = max(1, nodeIndex-1):min(nodeCount, nodeIndex+1)
            columnIndices = 2*neighborIndex-1:2*neighborIndex;
            jacobianPattern(rowIndices, columnIndices) = 1;
        end
    end
    jacobianPattern(end, 2*nodeCount) = 1;
end

function sampleStates = evaluatePieces(solutions, queryTimesSec)
    sampleStates = zeros(size(solutions{1}.y, 1), numel(queryTimesSec));
    for queryIndex = 1:numel(queryTimesSec)
        queryTimeSec = queryTimesSec(queryIndex);
        foundPiece = false;
        for pieceIndex = 1:numel(solutions)
            piece = solutions{pieceIndex};
            if queryTimeSec >= piece.x(1) && queryTimeSec <= piece.x(end)
                sampleStates(:, queryIndex) = deval(piece, queryTimeSec);
                foundPiece = true;
                break;
            end
        end
        if ~foundPiece
            error('CrossCheck:DenseDomain', '请求的时刻 %.17g 不在已积分分段内。', queryTimeSec);
        end
    end
end

function sampleTable = makeSampleTable(caseResult)
    sampleCount = numel(caseResult.sampleTimesSec);
    radialCount = numel(caseResult.sampleMaterialX);
    question = repmat(string(caseResult.question), sampleCount*radialCount, 1);
    sampleType = repelem(caseResult.sampleTypes(:), radialCount);
    timeSec = repelem(caseResult.sampleTimesSec(:), radialCount);
    materialX = repmat(caseResult.sampleMaterialX(:), sampleCount, 1);
    currentRadiusM = repelem(caseResult.sampleRadiusM(:), radialCount);
    radiusM = currentRadiusM.*materialX;
    temperatureK = reshape(caseResult.temperatureK', [], 1);
    temperatureC = temperatureK-273.15;
    moistureDryBasis = reshape(caseResult.moistureDryBasis', [], 1);
    sampleTable = table(question, sampleType, timeSec, materialX, radiusM, currentRadiusM, ...
        temperatureK, temperatureC, moistureDryBasis);
end

function validateInputs(inputData)
    environmentColumns = {'time_s', 'temperature_K', 'air_moisture_kg_per_kg'};
    radiusColumns = {'time_s', 'radius_m'};

```

```

    if ~all(ismember(environmentColumns, inputData.environment.Properties.VariableNames)) || ...
        ~all(ismember(radiusColumns, inputData.radius.Properties.VariableNames))
        error('CrossCheck:InputColumns', '清洗 CSV 字段不符合正式输入接口。');
    end
    environmentValues = inputData.environment{:, environmentColumns};
    radiusValues = inputData.radius{:, radiusColumns};
    if any(~isfinite(environmentValues), 'all') || any(~isfinite(radiusValues), 'all') || ...
        any(diff(environmentValues(:, 1)) <= 0) || any(diff(radiusValues(:, 1)) <= 0) || ...
        environmentValues(1, 1) ~= 0 || radiusValues(1, 1) ~= 0 || ...
        environmentValues(end, 1) ~= 14400 || radiusValues(end, 1) ~= 259200 || ...
        any(environmentValues(:, 2) <= 0) || any(environmentValues(:, 3) < 0) || ...
        any(radiusValues(:, 2) <= 0) || abs(radiusValues(1, 2)-0.02) > 1e-14
        error('CrossCheck:InputAudit', '时间、数值范围或初始半径审计未通过。');
    end
end

function assertReached(solution, requestedEndSec)
    if abs(solution.x(end)-requestedEndSec) > 1e-7
        error('CrossCheck:SolverStopped', 'ode15s 实际终点 %.17g 未到请求终点 %.17g。',
solution.x(end), requestedEndSec);
    end
    if any(~isfinite(solution.y), 'all')
        error('CrossCheck:NonfiniteSolution', 'ode15s 返回非有限接受状态。');
    end
end

function record = fileRecord(filePath)
    fileInfo = dir(filePath);
    record = struct('path', filePath, 'bytes', fileInfo.bytes, 'sha256', sha256File(filePath));
end

function hashText = sha256File(filePath)
    fileId = fopen(filePath, 'r');
    if fileId < 0
        error('CrossCheck:HashRead', '无法读取待哈希文件: %s', filePath);
    end
    fileCleanup = onCleanup(@() fclose(fileId)); %#ok<NASGU>
    fileBytes = fread(fileId, Inf, '*uint8');
    digest = java.security.MessageDigest.getInstance('SHA-256');
    digest.update(typecast(fileBytes, 'int8'));
    digestBytes = typecast(digest.digest(), 'uint8');
    hashText = lower(reshape(dec2hex(digestBytes, 2), 1, []));
end

function writeJson(filePath, value)
    fileId = fopen(filePath, 'w', 'n', 'UTF-8');
    if fileId < 0
        error('CrossCheck:JsonWrite', '无法写入 JSON: %s', filePath);
    end
    fileCleanup = onCleanup(@() fclose(fileId)); %#ok<NASGU>
    fprintf(fileId, '%s\n', jsonencode(value, 'PrettyPrint', true));
end

function logLine(fileId, messageFormat, varargin)
    logText = sprintf(messageFormat, varargin{:});
    fprintf('%s\n', logText);
    fprintf(fileId, '%s\n', logText);
end

function timeText = utcStamp()
    timeText = char(datetime('now', 'TimeZone', 'UTC', 'Format', 'yyyy-MM-
dd''T''HH:mm:ss.SSS''Z''));
end

function value = scalarOrNaN(optionalValue)
    if isempty(optionalValue)

```

```
        value = NaN;  
    else  
        value = optionalValue;  
    end  
end
```

E.22 compareMatlab.mjs: MATLAB/Python 共点比较及 Node 进程监督

文件: paper_output/code/review_delivery/tools/compareMatlab.mjs; 共 200 行。采用范围: 100 场标量与 2 事件的实际结果比较; 只读取已存 JSON/NPZ 配对结果 (NPZ 仅指纹, 值由 summary 样本读出)。

```
/** 比较既存 MATLAB/Python N40 输出: 按时间和材料坐标精确配对, 不重跑模型。 */
import fs from 'node:fs';
import crypto from 'node:crypto';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import { spawnSync } from 'node:child_process';

const scriptPath = fileURLToPath(import.meta.url);
const projectRoot = path.resolve(path.dirname(scriptPath), '../..');
const matlabRoot = path.join(projectRoot, 'paper_output/qa/matlab_crosscheck_20260911');
const pythonRoot = path.join(projectRoot, 'paper_output/results/code_delivery/camel_audit_v1');
const outputJson = path.join(matlabRoot, 'comparison-python.json');
const outputMarkdown = path.join(matlabRoot, 'comparison-python.md');
// 评价阈值在第一次读取数值之前固定, 不能为获得 PASS 而扩大。
const thresholds = Object.freeze({ temperatureK: 1e-4, moistureDryBasis: 1e-6, eventSec: 0.01 });
const scriptRecord = fileRecord(scriptPath);

function now() { return new Date().toISOString(); }
function digest(bytes) { return crypto.createHash('sha256').update(bytes).digest('hex'); }
function fileRecord(filePath) {
  const bytes = fs.readFileSync(filePath);
  return { path: path.relative(projectRoot, filePath).split(path.sep).join('/'),
    bytes: bytes.length, sha256: digest(bytes) };
}
function readJson(filePath) { return JSON.parse(fs.readFileSync(filePath, 'utf8').replace(/\uFEFF/, '')); }
function writeJson(filePath, value) { fs.writeFileSync(filePath, JSON.stringify(value, null, 2) + '\n', 'utf8'); }
function assert(condition, message) { if (!condition) throw new Error(message); }
function finite(value, label) { assert(typeof value === 'number' && Number.isFinite(value), `Nonfinite ${label}`); return value; }

function compareWorker() {
  const startedAtUtc = now();
  const startTime = process.hrtime.bigint();
  const inputPaths = [path.join(matlabRoot, 'runSummary.json'), path.join(matlabRoot, 'runLog.txt'),
    path.join(pythonRoot, 'launch.json'), path.join(pythonRoot, 'runResult.json'),
    path.join(pythonRoot, 'artifactManifest.json')];
  for (const question of ['Q1', 'Q23', 'Q4']) {
    inputPaths.push(path.join(matlabRoot, `${question}_crossCheck.json`),
      path.join(pythonRoot, question, 'summary.json'), path.join(pythonRoot, question, 'sampled_solution.npz'));
  }
  const matlabRun = readJson(inputPaths[0]);
  const pythonLaunch = readJson(path.join(pythonRoot, 'launch.json'));
  const pythonRun = readJson(path.join(pythonRoot, 'runResult.json'));
  const noMatlabException = matlabRun.exception == null || (Array.isArray(matlabRun.exception) && matlabRun.exception.length === 0);
  assert(matlabRun.status === 'COMPLETED_NUMERICAL_CHECKS' && noMatlabException, 'MATLAB run is incomplete');
  assert(pythonRun.status === 'PASS', 'Python baseline is incomplete');
  const provenanceChecks = [];
  // 对运行记录声明的源码/观测输入哈希做实际回读, 防止基准文件已被替换。
  function verifyDeclared(record, label) {
    const resolved = path.isAbsolute(record.path) ? record.path : path.join(projectRoot,
```

```

record.path);
    const actual = fileRecord(resolved);
    assert(actual.sha256 === record.sha256, `Declared hash differs: ${label}`);
    inputPaths.push(resolved);
    provenanceChecks.push({ label, actual, matchedRecordedHash: true });
}
verifyDeclared({ path: matlabRun.sourcePath, sha256: matlabRun.sourceSha256 }, 'MATLAB
source');
for (const record of matlabRun.inputs) verifyDeclared(record, 'MATLAB observed input');
for (const question of ['Q1', 'Q23', 'Q4']) {
    const baseline = readJson(path.join(pythonRoot, question, 'summary.json'));
    for (const key of ['code', 'jacobian_code', 'dense_storage_code'])
verifyDeclared(baseline[key], `${question}/${key}`);
    for (const record of baseline.inputs) verifyDeclared(record, `${question}/observed input`);
}
const uniqueInputPaths = [...new Set(inputPaths.map(item => path.resolve(item)))]
const inputRecords = uniqueInputPaths.map(fileRecord);
const questions = [];
const failures = [];
for (const question of ['Q1', 'Q23', 'Q4']) {
    const matlab = readJson(path.join(matlabRoot, `${question}_crossCheck.json`));
    const python = readJson(path.join(pythonRoot, question, 'summary.json'));
    const sample = python.crossLanguageSamples;
    assert(matlab.question === question && python.settings.question === question, `${question}:
question mismatch`);
    assert(matlab.intervalCount === 40 && python.settings.intervals === 40, `${question}: require
N40`);
    const settingsPairs = [
        ['rtol', 'relativeTolerance'], ['atolTemperature', 'temperatureAbsoluteTolerance'],
        ['atolMoisture', 'moistureAbsoluteTolerance'], ['earlyMaxStepS', 'earlyMaxStepSec'],
        ['maxStepS', 'lateMaxStepSec'], ['h', 'heatTransferCoefficient'], ['beta',
'massTransferCoefficient']];
    for (const [pythonKey, matlabKey] of settingsPairs) {
        assert(python.settings[pythonKey] === matlabRun.settings[matlabKey], `${question}: $
{pythonKey} differs`);
    }
    assert(python.settings.faceScheme.toLowerCase() ===
matlabRun.settings.waterFaceScheme.toLowerCase(), `${question}: face scheme differs`);
    assert(matlab.sampleTimesSec.length === matlab.sampleTypes.length, `${question}: sample types
malformed`);
    const times = question === 'Q1' ? [0, 1800] : [0, 1800, 10800];
    if (question !== 'Q1') {
        assert(Number.isInteger(matlab.endSec) && matlab.endSec === python.diagnostics.end_s, `${
question}: no common post-event integer second`);
        const postIndex = matlab.sampleTimesSec.indexOf(matlab.endSec);
        assert(postIndex >= 0 && matlab.sampleTypes[postIndex] === 'postEventIntegerSecond', `${
question}: endpoint is not postEventIntegerSecond`);
        times.push(matlab.endSec);
    }
    const coordinates = [0, 0.25, 0.5, 0.75, 1];
    const pairs = [];
    const maxima = { temperatureK: null, moistureDryBasis: null };
    for (const timeSec of times) {
        const matlabTimeIndex = matlab.sampleTimesSec.indexOf(timeSec);
        const pythonTimeIndex = sample.timesS.indexOf(timeSec);
        assert(matlabTimeIndex >= 0 && pythonTimeIndex >= 0, `${question}: missing exact common
time ${timeSec}`);
        for (const materialX of coordinates) {
            const matlabXIndex = matlab.sampleMaterialX.indexOf(materialX);
            const pythonXIndex = sample.materialX.indexOf(materialX);
            assert(matlabXIndex >= 0 && pythonXIndex >= 0, `${question}: missing exact common
coordinate ${materialX}`);
            const item = { timeSec, materialX, matlabTimeIndexZeroBased: matlabTimeIndex,
pythonTimeIndexZeroBased: pythonTimeIndex,
matlabXIndexZeroBased: matlabXIndex, pythonXIndexZeroBased: pythonXIndex };
            for (const field of ['temperatureK', 'moistureDryBasis']) {

```

```

        const matlabValue = finite(matlab[field][matlabTimeIndex][matlabXIndex], `${question}/${field}/MATLAB`);
        const pythonValue = finite(sample[field][pythonTimeIndex][pythonXIndex], `${question}/${field}/Python`);
        const signedDifference = matlabValue - pythonValue;
        const absoluteDifference = Math.abs(signedDifference);
        const passed = absoluteDifference <= thresholds[field];
        item[field] = { matlabValue, pythonValue, signedDifferenceMatlabMinusPython:
signedDifference, absoluteDifference, passed };
        if (maxima[field] === null || absoluteDifference > maxima[field].absoluteDifference) {
            maxima[field] = { timeSec, materialX, ...item[field] };
        }
        if (!passed) failures.push(`${question}/${field}/t=${timeSec}/x=${materialX}`);
    }
    pairs.push(item);
}
}
let event = { applicable: false, reason: 'Q1 stops at 1800 s and has no drying event' };
if (question !== 'Q1') {
    const matlabEventSec = finite(matlab.eventSec, `${question}/MATLAB event`);
    const pythonEventSec = finite(python.diagnostics.event_s, `${question}/Python event`);
    const signedDifferenceSec = matlabEventSec - pythonEventSec;
    const absoluteDifferenceSec = Math.abs(signedDifferenceSec);
    const passed = absoluteDifferenceSec <= thresholds.eventSec;
    event = { applicable: true, matlabEventSec, pythonEventSec, signedDifferenceSec,
absoluteDifferenceSec, passed };
    if (!passed) failures.push(`${question}/event`);
}
questions.push({ question, status: failures.some(item => item.startsWith(question + '/')) ?
'FAIL' : 'PASS',
    intervals: 40, commonTimesSec: times, commonMaterialX: coordinates, matchedCoordinatePairs:
pairs.length,
    maxima, event, comparisons: pairs,
    excludedPythonTimesSec: sample.timesS.filter(value => !times.includes(value)),
    excludedMatlabTimesSec: matlab.sampleTimesSec.filter(value => !times.includes(value)),
    exclusionReason: 'Python strict reported time and MATLAB critical event are different
times; their field rows are not paired.',
    matlabWarnings: matlab.warnings, matlabWarningCapture: matlab.warningCapture,
pythonWarningCount: python.warningCount });
}
assert(JSON.stringify(inputRecords) === JSON.stringify(uniqueInputPaths.map(fileRecord)),
'Inputs changed during comparison');
assert(scriptRecord.sha256 === fileRecord(scriptPath).sha256, 'Comparison script changed while
running');
return { schemaVersion: 1, status: failures.length ? 'FAIL' : 'PASS', failures,
    startedAtUtc, finishedAtUtc: now(), elapsedSeconds: Number(process.hrtime.bigint() -
startTime) / 1e9,
    thresholds, thresholdPolicy: 'Fixed before reading results; not changed after observing
differences',
    matchingPolicy: 'Exact common time values and exact common material coordinates; no row-index
pairing or interpolation',
    differenceConvention: 'MATLAB minus Python', script: scriptRecord, inputRecords,
provenanceChecks,
    runtime: { nodeVersion: process.version, executable: process.execPath, platform:
process.platform, architecture: process.arch,
        matlabVersion: matlabRun.matlabVersion, matlabSolver: matlabRun.settings.solver,
        matlabJacobian: matlabRun.settings.jacobianMode, pythonVersion: pythonLaunch.python,
        pythonLibraries: pythonLaunch.libraries, pythonSolver: 'SciPy BDF', pythonJacobian:
'analytic sparse' },
    sourceRunTimes: { matlabStartedAtUtc: matlabRun.startedAtUtc, matlabFinishedAtUtc:
matlabRun.finishedAtUtc,
        pythonStartedAtUtc: pythonRun.startedAtUtc, pythonFinishedAtUtc: pythonRun.finishedAtUtc },
    comparedFieldScalars: questions.reduce((total, item) => total + 2 *
item.matchedCoordinatePairs, 0),
    comparedEvents: 2, questions,
    scope: 'Independent N40 implementation agreement at recorded common times and five material

```

```

coordinates, plus continuous event times',
  limitations: ['NPZ files are hashed for provenance; numerical comparisons read
crossLanguageSamples in Python summary.json.',
  'No full-time/full-space error bound, physical calibration accuracy, production-grid
acceptance, or confidence interval is established.',
  'MATLAB GUI execution evidence is recorded separately by the main agent; this comparison
does not infer GUI completion.',
  'Human code review remains pending.'], guiEvidence: 'separate external record',
humanReview: 'pending' });
}

function markdown(report) {
  if (!report.questions?.length) {
    return `# MATLAB/Python N40 独立实现比较\n\n 状态: **FAIL**. 监督进程实际读取的退出码: $
{report.processSupervision.workerActualExitCode}.\n\n 比较器未生成可用数值报告: \n\n\`\`\`text\n$
{report.processSupervision.workerStderr}\n\n\`\`\`\n\n 没有放宽阈值或填充成功状态.\n`;
  }
  const number = value => value === null || value === undefined ? '不适用' :
value.toExponential(12);
  const rows = report.questions.map(item => `| ${item.question} | ${item.commonTimesSec.join(',
')} | ${item.matchedCoordinatePairs} | ${number(item.maxima.temperatureK.absoluteDifference)} | $
{number(item.maxima.moistureDryBasis.absoluteDifference)} | $
{number(item.event.absoluteDifferenceSec)} | ${item.status} |`);
  return `# MATLAB/Python N40 独立实现比较\n\n 状态: **${report.status}**. 比较进程实际退出码: **$
{report.processSupervision.workerActualExitCode}**, 由父 Node 监督进程读取.\n\n 固定接纳阈值为温度 $
{thresholds.temperatureK} K、干基含水率 ${thresholds.moistureDryBasis} kg/kg、连续事件时间 $
{thresholds.eventSec} s; 未根据观察结果调整. 全部比较按精确相同时间及材料坐标 x 配对, x 为 0、0.25、
0.5、0.75、1.\n\n| 问题轨迹 | 共同时间 / s | 时间坐标配对数 | 最大温差 / K | 最大干基含水率差 / kg/kg
| 连续事件绝对差 / s | 结论 | \n|---|---|---|---|---|---|---|---| \n${rows.join('\n')}\n\n 共比较 $
{report.comparedFieldScalars} 个场标量和 ${report.comparedEvents} 个连续事件. Q23 同时服务 Q2 与
Q3, 不另造第二份轨迹. Q1 没有完整干燥事件.\n\nPython 的额外一行是严格四位报告时刻, MATLAB 的额外一行是
其连续事件时刻, 两者不同; 这些场值未按行相减. 连续事件只单独比较秒. 最大值所在的时间、坐标、原始数值和有符
号差均保存在 comparison-python.json.\n\n 开始: ${report.startedAtUtc}; 结束: $
{report.finishedAtUtc}; 比较耗时: ${report.elapsedSeconds.toFixed(6)} s. Node $
{report.runtime.nodeVersion}; MATLAB ${report.runtime.matlabVersion}, ode15s 默认 NDF 与稀疏数值差
分; Python 使用 SciPy BDF 和解析稀疏 Jacobian.\n\n 输入文件、声明的源码与观测数据哈希已经实际回读匹配,
并在比较结束时再次检查未改变. Python NPZ 只作来源哈希记录, 比较数值取自 summary.json 的
crossLanguageSamples. 比较脚本 SHA-256: \`${report.script.sha256}\`. 监督过程和两个输出文件的
SHA-256 见 comparison-python-manifest.json, 避免报告文件自哈希的循环依赖.\n\n 这只是 N40 在共同记录时
间和五个材料坐标的跨实现一致性检查, 不能推广为连续全域误差上界、正式网格精度、物理预测精度或置信区间.
MATLAB GUI 操作证据由主代理另存, 本脚本不补写 GUI 成功; 用户人工代码审查仍待完成.\n`;
}

if (process.argv.includes('--worker')) {
  try {
    const report = compareWorker();
    process.stdout.write(JSON.stringify(report));
    process.exitCode = report.status === 'PASS' ? 0 : 1;
  } catch (error) {
    process.stderr.write(String(error.stack ?? error) + '\n');
    process.exitCode = 2;
  }
} else {
  const supervisorStartedAtUtc = now();
  const worker = spawnSync(process.execPath, [scriptPath, '--worker'], { cwd: projectRoot,
encoding: 'utf8', maxBuffer: 8 * 1024 * 1024 });
  const supervision = { supervisorStartedAtUtc, supervisorFinishedAtUtc: now(),
executable: process.execPath, commandArguments: [scriptPath, '--worker'], cwd: projectRoot,
workerActualExitCode: worker.status, workerSignal: worker.signal, workerSpawnError:
worker.error ? String(worker.error) : null,
workerStderr: worker.stderr, source: 'Node parent process spawnSync return status; not a
child prefilled success flag' };
  fs.appendFileSync(path.join(matlabRoot, 'comparison-python-process.jsonl'), JSON.stringify({
script: scriptRecord, ...supervision, stdoutSha256: digest(Buffer.from(worker.stdout ?? '',
'utf8')) }) + '\n', 'utf8');
  const report = worker.stdout?.length ? JSON.parse(worker.stdout) : {

```

```

    schemaVersion: 1, status: 'FAIL', failures: ['Worker did not produce a numerical report'],
    script: scriptRecord, thresholds, startedAtUtc: supervisorStartedAtUtc, finishedAtUtc: now(),
    comparedFieldScalars: 0, comparedEvents: 0, questions: [], humanReview: 'pending' };
report.processSupervision = supervision;
if (worker.status !== 0) report.status = 'FAIL';
writeJson(outputJson, report);
fs.writeFileSync(outputMarkdown, markdown(report), 'utf8');
writeJson(path.join(matlabRoot, 'comparison-python-manifest.json'), {
  status: report.status, createdAtUtc: now(), script: scriptRecord, processSupervision:
supervision,
  outputs: [fileRecord(outputJson), fileRecord(outputMarkdown)] });
process.stdout.write(JSON.stringify({ status: report.status, workerActualExitCode:
worker.status,
  comparedFieldScalars: report.comparedFieldScalars, comparedEvents: report.comparedEvents,
  maxima: report.questions.map(item => ({ question: item.question,
    temperatureK: item.maxima.temperatureK.absoluteDifference, moistureDryBasis:
item.maxima.moistureDryBasis.absoluteDifference,
    eventSec: item.event.absoluteDifferenceSec ?? null })) }) + '\n');
process.exitCode = worker.status ?? 2;
}

```

E.23 runLogged.ps1: N40 Python 实际进程监督入口

文件: paper_output/code/review_delivery/runLogged.ps1; 共 50 行。采用范围: driver_camel_audit_v1 的外部 Python 退出 0 证据来源。

版本说明与勘误: 历史日志锁定被启动的 Python 源码, 不锁定此 PowerShell 文件自身。本文件无 BOM 在 Windows PowerShell5.1 解析有已知问题; PowerShell7 可解析, 不声称可移植验收。

```
param(
    [ValidateSet('final', 'audit')][string]$Profile = 'final',
    [ValidatePattern('^[A-Za-z0-9][A-Za-z0-9_-]{0,79}$')][string]$RunId,
    [string]$PythonPath = 'C:\Python314\python.exe'
)
$ErrorActionPreference = 'Stop'
$projectRoot = (Resolve-Path -LiteralPath (Join-Path $PSScriptRoot '../..')).Path
if ((Get-Location).Path -ne $projectRoot) { throw '请在 2026CUMCM 根目录执行此脚本。' }
if (-not $RunId) { $RunId = $Profile + '_' + [DateTime]::UtcNow.ToString('yyyyMMddTHHmssZ') }
$driverRoot = Join-Path $projectRoot ('paper_output/results/code_delivery/driver_' + $RunId)
if (Test-Path -LiteralPath $driverRoot) { throw '运行编号已存在; 请指定新的 RunId。' }
New-Item -ItemType Directory -Path $driverRoot | Out-Null
$scriptPath = Join-Path $PSScriptRoot 'runDelivery.py'
$argumentText = '-B "' + $scriptPath + '" --profile ' + $Profile + ' --run-id ' + $RunId
$startedAt = [DateTime]::UtcNow.ToString('o')
$timer = [Diagnostics.Stopwatch]::StartNew()
$sourceShaBefore = (Get-FileHash -LiteralPath $scriptPath -Algorithm
SHA256).Hash.ToLowerInvariant()
# 此监督进程观察真实退出码; 模型程序本身不能自证进程已经退出。
$taskProcess = Start-Process -FilePath $PythonPath -ArgumentList $argumentText `
    -WorkingDirectory $projectRoot -WindowStyle Hidden -PassThru `
    -RedirectStandardOutput (Join-Path $driverRoot 'process.stdout.log') `
    -RedirectStandardError (Join-Path $driverRoot 'process.stderr.log')
$observedPeakWorkingSetBytes = 0L
$timedOut = $false
while (-not $taskProcess.WaitForExit(1000)) {
    $taskProcess.Refresh()
    $observedPeakWorkingSetBytes = [Math]::Max($observedPeakWorkingSetBytes,
$taskProcess.WorkingSet64)
    if ($Profile -eq 'audit' -and $timer.Elapsed.TotalSeconds -gt 1800) {
        $timedOut = $true
        $taskProcess.Kill($true)
        $taskProcess.WaitForExit()
        break
    }
}
$timer.Stop()
$processRecord = [ordered]@{
    runId = $RunId; profile = $Profile; executable = $PythonPath
    argumentText = $argumentText; cwd = $projectRoot; processId = $taskProcess.Id
    startedAtUtc = $startedAt; finishedAtUtc = [DateTime]::UtcNow.ToString('o')
    elapsedSeconds = $timer.Elapsed.TotalSeconds; actualExitCode = $taskProcess.ExitCode
    sourceSha256AtLaunch = $sourceShaBefore
    sourceSha256AtExit = (Get-FileHash -LiteralPath $scriptPath -Algorithm
SHA256).Hash.ToLowerInvariant()
    observedPeakWorkingSetBytes = $observedPeakWorkingSetBytes
    memoryScope = 'One-second observed process working set; excludes transient peaks and child
processes'
    timedOut = $timedOut
    humanReview = 'pending'; guiObserved = $false
}
$processRecord | ConvertTo-Json -Depth 8 | Set-Content -LiteralPath (Join-Path $driverRoot
```

```
'processResult.json') -Encoding utf8  
$processRecord | ConvertTo-Json -Depth 8  
exit $taskProcess.ExitCode
```

E. 24 runDelivery.py: MATLAB 对照所用 Python N40 生成入口

文件: paper_output/code/review_delivery/runDelivery.py; 共 281 行。采用范围: 实际 --profile audit --run-id camel_audit_v1, 调用驼峰模块并做 legacy 对照。

版本说明与勘误: audit 未执行 exportOutputs.py; 该文件仍被启动盘点且目前与旧盘点不同, 见运行说明。此入口会强制检查 final_v6a/run_manifest 内 107 个冻结输入/输出, 不能只凭两 CSV 运行。

""A 题四问可审查入口: 求解、导出、回读、回归核验和运行留痕。

默认 final 配置重算 N3200/N3200/N6400; audit 配置采用 N40 对照旧实现。
本入口产生独立版本目录。GUI 操作记录和用户人工审查由人另行填写。

```
"""
from __future__ import annotations

import argparse
from contextlib import redirect_stderr, redirect_stdout
from datetime import datetime, timezone
import gc
import gzip
import hashlib
import importlib
import importlib.metadata
import json
import os
from pathlib import Path
import re
import sys
import time
import traceback
import warnings

# 必须在加载 NumPy/SciPy 前限制线程, 避免三个大网格争用内存和 CPU。
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"
sys.dont_write_bytecode = True
projectRoot = Path(__file__).resolve().parents[3]
codeRoot = Path(__file__).resolve().parent
os.environ["MPLCONFIGDIR"] = str(projectRoot / "tmp/cache/matplotlib")
os.environ["TMPDIR"] = str(projectRoot / "tmp/cache/review_delivery")
os.environ["TEMP"] = os.environ["TMPDIR"]
os.environ["TMP"] = os.environ["TMPDIR"]
Path(os.environ["TMPDIR"]).mkdir(parents=True, exist_ok=True)

def utcNow():
    """UTC ISO 时间戳; 北京时间可在日志中按 +08:00 换算。"""
    return datetime.now(timezone.utc).isoformat()

def writeJson(filePath, content):
    filePath.parent.mkdir(parents=True, exist_ok=True)
    filePath.write_text(json.dumps(content, ensure_ascii=False, indent=2,
                                   allow_nan=False), encoding="utf-8")

def fileRecord(filePath):
    """分块哈希避免把大型高精度文件整体读入内存。"""
    digest = hashlib.sha256()
    with filePath.open("rb") as fileHandle:
```

```

        for block in iter(lambda: fileHandle.read(1024 * 1024), b''):
            digest.update(block)
    return {"path": filePath.relative_to(projectRoot).as_posix(),
            "bytes": filePath.stat().st_size, "sha256": digest.hexdigest()}

class TeeStream:
    """同时保留 IDE 可见输出和 UTF-8 持久日志。"""
    def __init__(self, screen, logFile):
        self.screen, self.logFile = screen, logFile

    def write(self, text):
        self.screen.write(text)
        self.logFile.write(text)
        self.flush()
        return len(text)

    def flush(self):
        self.screen.flush()
        self.logFile.flush()

def compareArrays(actualPath, baselinePath):
    """独立读取两份 NPZ；要求同名数组逐值相等，含 NaN 位置。"""
    import numpy as np
    checks = []
    with np.load(actualPath, allow_pickle=False) as actual, np.load(
        baselinePath, allow_pickle=False) as baseline:
        if set(actual.files) != set(baseline.files):
            raise AssertionError("NPZ 字段不一致")
        for key in baseline.files:
            left, right = actual[key], baseline[key]
            equal = left.shape == right.shape and np.array_equal(left, right, equal_nan=True)
            if not equal:
                raise AssertionError(f"重命名后数值改变: {actualPath.name}/{key}")
            checks.append({"array": key, "shape": list(left.shape), "exactEqual": True})
    return {"actual": fileRecord(actualPath), "baseline": fileRecord(baselinePath),
            "status": "PASS", "arrays": checks}

def compareArchives(actualPath, baselinePath):
    """比较解压后的逐秒原精度 CSV，忽略 gzip 容器时间戳。"""
    checkedBytes = 0
    with gzip.open(actualPath, "rb") as actual, gzip.open(baselinePath, "rb") as baseline:
        while True:
            left, right = actual.read(1024 * 1024), baseline.read(1024 * 1024)
            if left != right:
                raise AssertionError(f"原精度 CSV 与冻结结果不同: {actualPath.name}")
            checkedBytes += len(left)
            if not left:
                break
    return {"status": "PASS", "uncompressedBytes": checkedBytes,
            "actual": fileRecord(actualPath), "baseline": fileRecord(baselinePath)}

def verifyFrozenBaseline():
    """以历史实际退出 0 的账本核对冻结输入/输出，避免拿变动文件作基准。"""
    manifestPath = projectRoot / "paper_output/results/production/final_v6a/run_manifest.json"
    manifest = json.loads(manifestPath.read_text(encoding="utf-8-sig"))
    frozenRun = manifest["runs"][0]
    if manifest["status"] != "PASS" or frozenRun["returncode"] != 0:
        raise RuntimeError("历史基准没有有效的生产退出记录")
    checkedRecords = []
    for record in frozenRun["input_files"] + frozenRun["output_artifacts"]:
        actualRecord = fileRecord(projectRoot / record["path"])
        if actualRecord["sha256"] != record["sha256"]:

```

```

        raise AssertionError("冻结基准哈希改变: " + record["path"])
    checkedRecords.append(actualRecord)
    return {"status": "PASS", "manifest": fileRecord(manifestPath), "checkedRecords":
checkedRecords}

def runQuestion(questionKey, intervalCount, outputDirectory, profile):
    """一个轨迹求解一次; Q2 和 Q3 共享 Q23 轨迹, 导出后立即释放缓存。"""
    import numpy as np
    import dryingCore
    import q3Model
    moduleName = {"Q1": "q1Model", "Q23": "q2Model", "Q4": "q4Model"}[questionKey]
    questionModule = importlib.import_module(moduleName)
    questionDirectory = outputDirectory / questionKey
    baselineRoot = projectRoot / "paper_output/results/production/final_v6a"
    solutionRun = None
    try:
        print(f"{utcNow()} SOLVE {questionKey} N={intervalCount}", flush=True)
        # 人工审查断点: 下一行进入逐问入口, 再进入 solveCase / rhs。
        with warnings.catch_warnings(record=True) as caughtWarnings:
            warnings.simplefilter("always")
            solutionRun = questionModule.solve(intervalCount)
        if caughtWarnings:
            raise RuntimeError("求解器警告: " + "; ".join(str(item.message) for item in
caughtWarnings))
        summary = dryingCore.saveRun(solutionRun, questionDirectory)
        summary["warningCount"] = len(caughtWarnings)
        if questionKey != "Q1":
            summary["completion"] = q3Model.completion(solutionRun)
        sampleTimes = np.unique(np.array([0., min(1800., solutionRun.endS),
            min(10800., solutionRun.endS), solutionRun.endS] + ([[] if questionKey == "Q1"
            else [summary["completion"]["reported_time_s"]]))))
        sampleCoordinates = np.array([0., .25, .5, .75, 1.])
        temperature, moisture = solutionRun.fields(sampleTimes, materialX=sampleCoordinates)
        summary["crossLanguageSamples"] = {"timesS": sampleTimes.tolist(),
            "materialX": sampleCoordinates.tolist(), "temperatureK": temperature.tolist(),
            "moistureDryBasis": moisture.tolist()}
        writeJson(questionDirectory / "summary.json", summary)
        print(json.dumps({"question": questionKey, "diagnostics": summary["diagnostics"]},
            ensure_ascii=False), flush=True)
        comparison = {}
        if profile == "final":
            comparison["sampledSolution"] = compareArrays(questionDirectory /
"sampled_solution.npz",
                baselineRoot / questionKey / "sampled_solution.npz")
            frozenSummary = json.loads((baselineRoot /
"numerical_summaries.json").read_text(encoding="utf-8"))[questionKey]
            if summary["diagnostics"]["event_s"] != frozenSummary["diagnostics"]["event_s"]:
                raise AssertionError("连续临界事件与冻结基线不同")
            if questionKey != "Q1" and summary["completion"] != frozenSummary["completion"]:
                raise AssertionError("严格达标时刻与冻结基线不同")
            import exportOutputs
            comparison["exports"] = []
            questionIds = ["Q2", "Q3"] if questionKey == "Q23" else [questionKey]
            for questionId in questionIds:
                print(f"{utcNow()} EXPORT_AND_READBACK {questionId}", flush=True)
                exportResult = exportOutputs.exportQuestion(solutionRun, questionId,
                    outputDirectory / "outputs")
                validation = exportOutputs.validateExports([exportResult], runs={questionId:
solutionRun})
            if validation.get("status") != "PASS" or not
validation.get("fully_verified_with_live_Run"):
                raise AssertionError("工作簿/原精度文件/live Run 核对失败")
            rawName = f"result{questionId[-1]}_unrounded.csv.gz"
            archiveCheck = compareArchives(outputDirectory / "outputs" / rawName,
                baselineRoot / "outputs" / rawName)

```

```

        tableChecks = []
        for tablePath in sorted((outputDirectory / "outputs").glob(questionId.lower() +
            "_paper_*.csv")):
            referencePath = baselineRoot / "outputs" / tablePath.name
            if tablePath.read_bytes() != referencePath.read_bytes():
                raise AssertionError("正文 CSV 与冻结结果不同: " + tablePath.name)
            tableChecks.append({"file": fileRecord(tablePath), "exactEqual": True})
            comparison["exports"].append({"questionId": questionId, "validation": validation,
                "archiveCheck": archiveCheck, "paperTables":
tableChecks})
        writeJson(questionDirectory / "comparison.json", comparison)
        return summary, comparison
    finally:
        if solutionRun is not None:
            solutionRun.close()
            solutionRun = None
        gc.collect()
        print(f"{utcNow()} RELEASED {questionKey}", flush=True)

def runLegacyAudit(outputDirectory):
    """固定评价器: N40 同参数旧源码真实重算, 与本轮副本逐值比较。"""
    legacyRoot = projectRoot / "paper_output/code/modeling"
    sys.path.insert(0, str(legacyRoot))
    import drying_core as legacyCore
    checks = []
    for questionKey, moduleName in [("Q1", "q1_model"), ("Q23", "q2_model"), ("Q4", "q4_model")]:
        legacyRun = None
        try:
            print(f"{utcNow()} AUTORESEARCH_BASELINE {questionKey} N=40", flush=True)
            legacyRun = importlib.import_module(moduleName).solve(40)
            legacyDirectory = outputDirectory / "legacyBaseline" / questionKey
            legacyCore.save_run(legacyRun, legacyDirectory)
            checks.append(compareArrays(outputDirectory / questionKey / "sampled_solution.npz",
                legacyDirectory / "sampled_solution.npz"))
        finally:
            if legacyRun is not None:
                legacyRun.close()
            gc.collect()
    return {"status": "KEEP", "primaryMetric": "maximum absolute sampled field difference",
        "primaryMetricValue": 0., "criterion": "exact equality at fixed N40, same settings and
inputs",
        "randomSeed": None, "deterministic": True, "checks": checks,
        "heldOutCheck": "final grid and all unrounded output rows against frozen final_v6a"}

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--profile", choices=["final", "audit"], default="final")
    parser.add_argument("--run-id", dest="runId", default=None)
    arguments = parser.parse_args()
    runId = arguments.runId or (arguments.profile + "_" +
datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S"))
    if re.fullmatch(r"[A-Za-z0-9][A-Za-z0-9_-]{0,79}", runId) is None:
        raise ValueError("run-id 仅可含字母、数字、下划线和连字符")
    if Path.cwd().resolve() != projectRoot:
        raise RuntimeError("请在 2026CUMCM 根目录启动, 或使用已配置的 VS 工程")
    outputDirectory = projectRoot / "paper_output/results/code_delivery" / runId
    outputDirectory.mkdir(parents=True, exist_ok=False)
    startedAt, timer = utcNow(), time.perf_counter()
    sourcePaths = sorted(codeRoot.glob("*.py"))
    dataPaths = sorted((projectRoot / "paper_output/data_cleaned").glob("A_*observed.csv"))
    templatePaths = sorted((projectRoot / "problem_files/CUMCM2026Problems/A 题/附件/附件
3").glob("result*.xlsx"))
    inputRecords = [fileRecord(filePath) for filePath in sourcePaths + dataPaths + templatePaths]
    launchRecord = {"startedAtUtc": startedAt, "runId": runId, "profile": arguments.profile,

```

```

        "command": [sys.executable] + sys.argv, "cwd": str(Path.cwd()), "python": sys.version,
        "libraries": {name: importlib.metadata.version(name) for name in ["numpy", "scipy",
"openpyxl"]},
        "inputRecords": inputRecords, "blasThreads": 1, "humanReview": "pending", "guiObserved":
False,
        "baseline": "paper_output/results/production/final_v6a", "seed": None,
        "seedReason": "deterministic PDE solve; no pseudorandom numbers",
"experimentBudgetSeconds": 1800}
    writeJson(outputDirectory / "launch.json", launchRecord)
    result = {"runId": runId, "startedAtUtc": startedAt, "status": "RUNNING",
        "humanReview": "pending", "guiObservation": "separate external evidence required",
        "actualProcessExitCode": None}
    with (outputDirectory / "stdout.log").open("w", encoding="utf-8") as logFile:
        with redirect_stdout(TeeStream(sys.stdout, logFile)),
redirect_stderr(TeeStream(sys.stderr, logFile)):
        try:
            print(f"{startedAt} START {runId} {arguments.profile}", flush=True)
            result["frozenBaselineVerification"] = verifyFrozenBaseline()
            summaries, comparisons = {}, {}
            for questionKey, finalIntervals in [("Q1", 3200), ("Q23", 3200), ("Q4", 6400)]:
                intervalCount = 40 if arguments.profile == "audit" else finalIntervals
                summaries[questionKey], comparisons[questionKey] = runQuestion(
                    questionKey, intervalCount, outputDirectory, arguments.profile)
            result["summaries"], result["comparisons"] = summaries, comparisons
            if arguments.profile == "audit":
                result["autoresearch"] = runLegacyAudit(outputDirectory)
                writeJson(outputDirectory / "autoresearch.json", result["autoresearch"])
            if inputRecords != [fileRecord(filePath) for filePath in sourcePaths + dataPaths +
templatePaths]:
                raise RuntimeError("运行期间源码或输入发生改变，结果不能签核")
            result["status"] = "PASS"
            print(f"{utcNow()} ALL_Q1_Q4_SOLVES_AND_CHECKS_COMPLETED", flush=True)
        except BaseException as error:
            result["status"], result["error"] = "FAIL", repr(error)
            traceback.print_exc()
            raise
        finally:
            result["finishedAtUtc"], result["elapsedSeconds"] = utcNow(), time.perf_counter()
- timer
            writeJson(outputDirectory / "runResult.json", result)
            stablePaths = [filePath for filePath in sorted(outputDirectory.rglob("*"))
                if filePath.is_file() and filePath.name not in ["stdout.log",
"artifactManifest.json"]]
            writeJson(outputDirectory / "artifactManifest.json", {"status": result["status"],
                "createdAtUtc": utcNow(), "artifacts": [fileRecord(filePath) for filePath in
stablePaths]})

if __name__ == "__main__":
    main()

```

E. 25 dryingCore.py: N40 驼峰有限体积求解器

文件：paper_output/code/review_delivery/dryingCore.py；共 420 行。采用范围：MATLAB N40 比较实际 Python 源码；与原 11 个蛇形生产文件字节不同，不按同名功能省略。

```
# 本文件为冻结求解源码的驼峰审查副本；来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""2026 A: radial heat and dry-basis moisture transport on a material mesh.
```

```
Units: s, m, K, kg water / kg dry matter. See numerical_design.md for derivation.
The supplied empirical rho*cp is an effective thermal capacity. Dry-solid mass
is conserved separately on uniformly shrinking material control volumes.
No latent heat in the baseline; optional surface-latent scenario is labelled.
No clipping of solution values. Coefficients use a positive continuation only
for integrator Newton probes; all accepted states are checked independently.
"""
```

```
from __future__ import annotations
```

```
from dataclasses import asdict, dataclass
from pathlib import Path
import hashlib
import io
import json
import time
```

```
import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import lil_matrix
from scipy.special import expi
import analyticJacobian
import diskDense
```

```
ROOT = Path(__file__).resolve().parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
C0, T0, R0, LENGTH = 2.55, 301.15, 0.02, 0.25
```

```
# 原始环境时间单位为秒，温度为 K，含水率为 kg 水 / kg 干物质；记录输入哈希。
```

```
def loadInputs(withRecords=False):
    arrays, records = [], []
    for name in ['A_environment_observed.csv', 'A_radius_observed.csv']:
        path = ROOT / 'paper_output/data_cleaned' / name
        content = path.read_bytes()
        arrays.append(np.genfromtxt(io.StringIO(content.decode('utf-8-sig')),
                                   delimiter=',', names=True))
        records.append({'path':path.relative_to(ROOT).as_posix(), 'bytes':len(content),
                       'sha256':hashlib.sha256(content).hexdigest(), 'exists':True})
    return (*arrays, records) if withRecords else tuple(arrays)
```

```
@dataclass(frozen=True)
```

```
# 数值参数集中配置；四问正式设置由 q1Model/q2Model/q4Model 提供。
```

```
class Settings:
    question: str = 'Q23'
    intervals: int = 100
    rtol: float = 1e-7
    atolTemperature: float = 1e-7
    atolMoisture: float = 1e-9
    maxStepS: float = 600.0
    earlyMaxStepS: float = 30.0
    horizonH: float = 240.0
    shrink: bool = False
```

```

boundaryExtension: str = 'nominal'
tailTemperatureC: float = 50.0
tailEquilibrium: float = 0.05
h: float = 25.0
beta: float = 8e-7
equilibriumScale: float = 1.0
surfaceLatentFraction: float = 0.0
latentJKg: float = 2.4e6
constantD: float | None = None
constantThermal: bool = False
method: str = 'BDF'
faceScheme: str = 'harmonic'
jacobianMode: str = 'analytic'
denseStorage: str = 'disk'

```

```

class RadialModel:
    def __init__(self, settings: Settings):
        self.settings = settings
        if settings.intervals < 2 or settings.jacobianMode not in ('analytic',
'finite_difference'):
            raise ValueError('At least two intervals and a supported Jacobian mode are required')
        self.env, self.rad, self.inputRecords = loadInputs(withRecords=True)
        # x=r/R(t) 是无量纲材料坐标; 控制体权重来自圆柱半径方向的积分。
        self.x = np.linspace(0., 1., settings.intervals + 1)
        self.dx = 1. / settings.intervals
        faces = np.r_[0., (self.x[1:] + self.x[:-1]) / 2., 1.]
        self.w = np.diff(faces ** 2) / 2.
        self.internalFaces = faces[1:-1]
        self.n = len(self.x)
        self.rhoD0 = (760 + 90 * C0) / (1 + C0) if settings.question == 'Q4' else (
            820. / (1 + C0) if settings.question == 'Q1' else (650 + 128 * C0) / (1 + C0))
        self.evaluations = 0
        self.jacPattern = self.buildSparsity()

    def radius(self, t):
        if self.settings.shrink:
            return np.interp(t, self.rad['time_s'], self.rad['radius_m'])
        return np.asarray(t) * 0. + R0

# 4 h 之后采用已声明的平台延拓; 不能把外推段称为实测环境。
def environment(self, t):
    s = self.settings
    tair = np.interp(t, self.env['time_s'], self.env['temperature_K'])
    ceq = np.interp(t, self.env['time_s'], self.env['air_moisture_kg_per_kg'])
    after = np.asarray(t) > self.env['time_s'][-1]
    if s.boundaryExtension == 'nominal':
        tair = np.where(after, s.tailTemperatureC + 273.15, tair)
        ceq = np.where(after, s.tailEquilibrium, ceq)
    elif s.boundaryExtension == 'tail_mean':
        tail = self.env['time_s'] >= 10800
        tair = np.where(after, self.env['temperature_K'][tail].mean(), tair)
        ceq = np.where(after, self.env['air_moisture_kg_per_kg'][tail].mean(), ceq)
    elif s.boundaryExtension != 'last':
        raise ValueError('Unknown boundary extension')
    return tair, ceq * s.equilibriumScale

# rho*cp 为有效显热体积容量; 干物质量通过独立的积分守恒式约束。
def properties(self, T, C):
    s = self.settings
    # 仅延拓 Newton 试探点的系数; 不裁剪被接受的温度或含水率状态。
    positiveC = np.maximum(C, 1e-12) # coefficient continuation, never state clipping
    if np.any(T <= 0):
        raise FloatingPointError('Nonpositive absolute temperature')
    wet = positiveC / (1. + positiveC)
    if s.question == 'Q1':

```

```

        rho = np.full_like(C, 820.)
        cp = np.full_like(C, 2600.)
        k = np.full_like(C, .36)
        D = 7e-9 * np.exp(-.89 / positiveC)
    elif s.question in ('Q2', 'Q3', 'Q23'):
        rho, cp, k = 650 + 128 * positiveC, 1450 + 2736 * wet, .21 + .38 * wet
        D = 2.4e-3 * np.exp(-.45 / positiveC - 3850 / T)
    elif s.question == 'Q4':
        rho, cp, k = 760 + 90 * positiveC, 1850 + 2150 * wet, .12 + .20 * wet
        D = 4.2e-4 * np.exp(-.30 / positiveC - 3850 / T)
    else:
        raise ValueError(s.question)
    if s.constantD is not None:
        D = np.full_like(C, s.constantD)
    if s.constantThermal:
        rho, cp, k = np.full_like(C, 820.), np.full_like(C, 2600.), np.full_like(C, .36)
    return rho, cp, k, D

@staticmethod
def harmonic(a):
    return 2 * a[:-1] * a[1:] / np.maximum(a[:-1] + a[1:], np.finfo(float).tiny)

def buildSparsity(self):
    p = lil_matrix((2 * self.n + 1, 2 * self.n + 1), dtype=int)
    for i in range(self.n):
        for j in range(max(0, i - 1), min(self.n, i + 2)):
            p[2*i:2*i+2, 2*j:2*j+2] = 1
    p[-1, 2 * (self.n - 1) + 1] = 1
    return p.tocsr()

# Kirchhoff 势差积分处理强非线性 D(C), 温度因子在同一面上取值。
def waterInternalFlux(self, T, C, D):
    if self.settings.faceScheme == 'harmonic' or self.settings.constantD is not None:
        return self.internalFaces * self.harmonic(D) * np.diff(C) / self.dx
    if self.settings.faceScheme != 'kirchhoff':
        raise ValueError('Unknown nonlinear face scheme')
    a, D0 = {'Q1':(.89,7e-9), 'Q23':(.45,2.4e-3), 'Q2':(.45,2.4e-3),
            'Q3':(.45,2.4e-3), 'Q4':(.30,4.2e-4)}[self.settings.question]
    cc = np.maximum(C, 1e-12)
    # 势 F(C)=C*exp(-a/C)+a*Ei(-a/C), 导数为 exp(-a/C)。
    potential = cc * np.exp(-a/cc) + a * expi(-a/cc)
    difference = np.diff(potential)
    small = np.abs(np.diff(cc)) < 1e-7 * np.maximum((cc[:-1]+cc[1:])/2, 1e-3)
    difference[small] = (np.exp(-a/((cc[:-1][small]+cc[1:][small])/2)) * np.diff(cc)[small])
    # 不能对温度因子乘势后的整体作差, 否则会引入题设没有的交叉扩散通量。
    thermalFactor = 1. if self.settings.question == 'Q1' else
np.exp(-3850/((T[:-1]+T[1:])/2))
    # Do not difference thermal_factor*potential: that would add a false Soret flux.
    return self.internalFaces * D0 * thermalFactor * difference / self.dx

# 状态交错排列 T0,C0,T1,C1,..., 最后一项累计平均失水; 共享面通量保证离散守恒。
def rhs(self, t, state):
    """REVIEW: actual material-control-volume balance, no extra mesh advection."""
    self.evaluations += 1
    T, C = state[:-1:2], state[1:-1:2]
    rho, cp, k, D = self.properties(T, C)
    radius = float(self.radius(t))
    tair, ceq = self.environment(t)
    heatG = np.zeros(self.n + 1)
    waterG = np.zeros(self.n + 1)
    heatG[1:-1] = self.internalFaces * self.harmonic(k) * np.diff(T) / self.dx
    waterG[1:-1] = self.waterInternalFlux(T, C, D)
    # 表面对流传质和传热采用外法向流出约定; 中心面面积为零。
    waterG[-1] = -self.settings.beta * radius * (C[-1] - ceq)
    heatG[-1] = -self.settings.h * radius * (T[-1] - tair)
    if self.settings.surfaceLatentFraction:

```

```

        # Scenario: all selected outgoing water vaporizes at the surface.
        rhoD = self.rhoD0 * (R0 / radius) ** 2
        jEvap = rhoD * self.settings.beta * (C[-1] - ceq)
        heatG[-1] -= (radius * self.settings.surfaceLatentFraction *
                     self.settings.latentJKg * jEvap)
        derivative = np.empty_like(state)
        # 除以  $R(t)^2$  与控制体权重得到材料导数; 同比收缩无需额外网格对流项。
        derivative[:-1:2] = np.diff(heatG) / (radius ** 2 * self.w * rho * cp)
        derivative[1:-1:2] = np.diff(waterG) / (radius ** 2 * self.w)
        derivative[-1] = 2 * self.settings.beta / radius * (C[-1] - ceq)
        return derivative

    def initial(self):
        state = np.empty(2 * self.n + 1)
        state[:-1:2], state[1:-1:2], state[-1] = T0, C0, 0.
        return state

class Run:
    def __init__(self, model, pieces, elapsedS, eventS, cache=None):
        self.model, self.pieces = model, pieces
        self.cache = cache
        self.elapsedS, self.eventS = elapsedS, eventS
        self.endS = float(pieces[-1].t[-1])

    def close(self):
        """Release this Run after its exports/checks; further queries are invalid."""
        self.pieces.clear()
        if self.cache is not None:
            self.cache.close()

    def state(self, times):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        if np.min(tt) < -1e-10 or np.max(tt) > self.endS + 1e-7:
            raise ValueError('Requested time outside solved interval')
        out = np.empty((2 * self.model.n + 1, len(tt)))
        remaining = np.ones(len(tt), dtype=bool)
        for result in self.pieces:
            select = remaining & (tt >= result.t[0]-1e-7) & (tt <= result.t[-1]+1e-7)
            if np.any(select):
                out[:, select] = result.sol(tt[select])
                remaining[select] = False
        if np.any(remaining):
            raise RuntimeError('Missing dense solution segment')
        return out

# materialX 查询材料坐标; radiiM 查询实际米制半径, 收缩域外返回 NaN.
def fields(self, times, radiiM=None, materialX=None):
    tt = np.atleast_1d(np.asarray(times, dtype=float))
    state = self.state(tt)
    Ts, Cs = state[:-1:2].T, state[1:-1:2].T
    if materialX is not None:
        points = np.asarray(materialX, dtype=float)
        if np.any(~np.isfinite(points)) or np.any((points < 0.) | (points > 1.)):
            raise ValueError('Material coordinates must be finite and within [0,1]')
        return np.array([np.interp(points, self.model.x, row) for row in Ts]), np.array([
            np.interp(points, self.model.x, row) for row in Cs])
    if radiiM is None:
        return Ts, Cs
    radial = np.asarray(radiiM)
    Tout, Cout = [], []
    for i, t in enumerate(tt):
        xx = radial / self.model.radius(t)
        Tout.append(np.interp(xx, self.model.x, Ts[i], left=np.nan, right=np.nan))
        Cout.append(np.interp(xx, self.model.x, Cs[i], left=np.nan, right=np.nan))
    return np.asarray(Tout), np.asarray(Cout)

```

```

# 每块至多 256 个被接受时刻，避免对高网格状态和物性数组再做整域复制。
def diagnostics(self):
    # Reduce in bounded blocks: a fine full-domain run may contain tens of
    # millions of accepted state values. Diagnostics must not duplicate all
    # of them and four property arrays at the same time.
    minimumC = minimumT = minimumD = minimumProperty = np.inf
    maximumC = maximumT = maximumD = maximumRadialIncrease = -np.inf
    maximumMassResidual = 0.
    for piece in self.pieces:
        for first in range(0, len(piece.t), 256):
            raw = piece.y[:, first:first+256]
            T, C = raw[:-1:2], raw[1:-1:2]
            residual = 2*self.model.w@C + raw[-1] - C0
            rho, cp, k, D = self.model.properties(T.ravel(), C.ravel())
            minimumC, maximumC = min(minimumC, C.min()), max(maximumC, C.max())
            minimumT, maximumT = min(minimumT, T.min()), max(maximumT, T.max())
            minimumD = min(minimumD, D.min())
            maximumD = max(maximumD, D.max())
            minimumProperty = min(minimumProperty, rho.min(), cp.min(), k.min(), D.min())
            maximumRadialIncrease = max(maximumRadialIncrease, np.max(np.diff(C, axis=0)))
            maximumMassResidual = max(maximumMassResidual, np.max(np.abs(residual)))
    final = self.state([self.endS])[:, 0]
    finalC = final[1:-1:2]
    return {
        'event_s': self.eventS, 'event_h': None if self.eventS is None else self.eventS/3600,
        'end_s': self.endS, 'elapsed_s': self.elapsedS,
        'end_time_convention': 'ceil(critical_event_s)+1: conservative post-crossing
verification second; not claimed earliest integer second',
        'max_mass_balance_abs_kg_per_kg': float(maximumMassResidual),
        'min_C': float(minimumC), 'max_C': float(maximumC),
        'min_T_K': float(minimumT), 'max_T_K': float(maximumT),
        'min_D': float(minimumD), 'max_D': float(maximumD),
        'positive_properties': bool(minimumProperty > 0),
        'max_radial_C_increase': float(maximumRadialIncrease),
        'final_max_C': float(finalC.max()), 'final_surface_C': float(finalC[-1]),
        'strictly_dry_at_end': bool(finalC.max() < .15),
        'radius_end_m': float(self.model.radius(self.endS)),
        'radius_extrapolation_used': bool(self.model.settings.shrink and self.endS > 259200),
        'rhs_evaluations': self.model.evaluations,
        'accepted_time_points': sum(len(p.t) for p in self.pieces),
        'nfev': sum(p.nfev for p in self.pieces),
        'njev': sum(p.njev for p in self.pieces), 'nlu': sum(p.nlu for p in self.pieces),
        'solver_success': all(p.success for p in self.pieces),
        'dense_storage': self.model.settings.denseStorage,
        'dense_coefficient_bytes': 0 if self.cache is None else self.cache.bytesWritten,
        'dense_polynomial_count': 0 if self.cache is None else self.cache.polynomialCount,
    }

def solveCase(settings: Settings) -> Run:
    started = time.perf_counter()
    if settings.denseStorage not in ('memory', 'disk'):
        raise ValueError('Dense storage must be memory or disk')
    if settings.denseStorage == 'disk' and settings.method != 'BDF':
        raise ValueError('Exact disk dense storage currently supports BDF only')
    cache = diskDense.DenseCache(ROOT) if settings.denseStorage == 'disk' else None
    try:
        return solveCaseImpl(settings, cache, started)
    except BaseException as error:
        if cache is not None:
            try:
                cache.close()
            except BaseException as cleanup_error:
                error.add_note('Private cache cleanup also failed: '+repr(cleanup_error))
    raise

```

```

def solveCaseImpl(settings, cache, started):
    model = RadialModel(settings)
    method = diskDense.DiskBDF if cache is not None else settings.method
    jacobianOptions = ({'jac': lambda t, y: analyticJacobian.jacobian(model, t, y)}
        if settings.jacobianMode == 'analytic' else {'jac_sparsity': model.jacPattern})
    if cache is not None:
        jacobianOptions['dense_cache'] = cache
    # 连续事件取整个离散材料域 max(C)=0.15; 严格达标还须在事件后重新检查。
    def dryEvent(t, y):
        return float(np.max(y[1:-1:2]) - .15)
    dryEvent.terminal, dryEvent.direction = True, -1
    atol = np.empty(2 * model.n + 1)
    atol[:-1:2], atol[1:-1:2], atol[-1] = settings.atolTemperature, settings.atolMoisture,
    settings.atolMoisture
    horizon = 1800. if settings.question == 'Q1' else settings.horizonH * 3600.
    pieces, initial, eventS = [], model.initial(), None
    # A separate segment at 4 h makes the modelling extension explicit.
    endpoints = [0., min(14400., horizon)]
    if horizon > 14400.:
        endpoints.append(horizon)
    for left, right in zip(endpoints[:-1], endpoints[1:]):
        piece = solve_ivp(model.rhs, (left, right), initial, method=method,
            rtol=settings.rtol, atol=atol, **jacobianOptions,
            max_step=settings.earlyMaxStepS if left < 14400. else settings.maxStepS,
            events=None if settings.question == 'Q1' else dryEvent, dense_output=True)
        if cache is not None:
            diskDense.alignBdfSegments(piece)
        pieces.append(piece)
        if not piece.success:
            raise RuntimeError(piece.message)
        initial = piece.y[:, -1].copy()
        if cache is not None:
            piece.y = cache.storeAccepted(piece.y)
        if piece.t_events is not None and len(piece.t_events[0]):
            eventS = float(piece.t_events[0][0])
            # Continue to a genuine post-crossing integer second, not an extrapolation.
            # 从实际事件状态继续积分至 ceil(event)+1 秒, 不依赖外推或四位舍入。
            end = float(np.ceil(eventS) + 1)
            tail = solve_ivp(model.rhs, (eventS, end), initial, method=method,
                rtol=settings.rtol, atol=atol, **jacobianOptions,
                max_step=1., dense_output=True)
            if cache is not None:
                diskDense.alignBdfSegments(tail)
            if not tail.success:
                raise RuntimeError(tail.message)
            if cache is not None:
                tail.y = cache.storeAccepted(tail.y)
            pieces.append(tail)
            break
    run = Run(model, pieces, time.perf_counter() - started, eventS, cache)
    diagnostic = run.diagnostics()
    if diagnostic['min_C'] < -1e-8 or not diagnostic['positive_properties']:
        raise FloatingPointError('Physical range/positive property check failed')
    if diagnostic['max_mass_balance_abs_kg_per_kg'] > 1e-6:
        raise FloatingPointError('Dry-basis mass balance failed')
    return run

def fileRecord(path):
    p = Path(path)
    return {'path': p.relative_to(ROOT).as_posix(), 'bytes': p.stat().st_size,
        'sha256': hashlib.sha256(p.read_bytes()).hexdigest(), 'exists': True}

```

```

# 保存 60 s 等审查采样、源码/输入哈希和状态；整秒题表由 exportOutputs 直接查询 live Run。
def saveRun(run: Run, directory: Path):
    codeRecord = fileRecord(Path(__file__))
    if codeRecord['sha256'] != LOADED_CODE_SHA256:
        raise RuntimeError('Solver file changed after import; restart to obtain valid
provenance')
    jacobianRecord = fileRecord(Path(analyticJacobian.__file__))
    if jacobianRecord['sha256'] != analyticJacobian.LOADED_CODE_SHA256:
        raise RuntimeError('Jacobian file changed after import; restart to obtain valid
provenance')
    storageRecord = fileRecord(Path(diskDense.__file__))
    if storageRecord['sha256'] != diskDense.LOADED_CODE_SHA256:
        raise RuntimeError('Dense storage file changed after import; restart for valid
provenance')
    for record in run.model.inputRecords:
        if fileRecord(ROOT/record['path'])['sha256'] != record['sha256']:
            raise RuntimeError('Input changed after being loaded; retain failure and rerun')
    directory.mkdir(parents=True, exist_ok=True)
    times = np.unique(np.r_[np.arange(0., run.endS, 60.),
        [t for t in [100., 300., 600., 900., 1200., 1500., 1800., 3600., 5400., 7200.,
9000., 10800.] if t <= run.endS],
        run.endS, [] if run.eventS is None else [run.eventS]])
    x = np.linspace(0., 1., 21)
    temperature, moisture, means, losses = [], [], [], []
    for first in range(0, len(times), 128):
        blockTimes = times[first:first+128]
        Tb, Cb = run.fields(blockTimes, materialX=x)
        raw = run.state(blockTimes)
        temperature.append(Tb); moisture.append(Cb)
        means.append(2*run.model.w@raw[1:-1:2]); losses.append(raw[-1].copy())
    T, C = np.vstack(temperature), np.vstack(moisture)
    np.savez_compressed(directory/'sampled_solution.npz', times_s=times, material_x=x,
        T_K=T, C=C, radius_m=run.model.radius(times), mean_C=np.concatenate(means),
        cumulative_loss=np.concatenate(losses))
    summary = {'settings': asdict(run.model.settings), 'diagnostics': run.diagnostics(),
        'code': codeRecord,
        'jacobian_code': jacobianRecord,
        'dense_storage_code': storageRecord,
        'inputs': run.model.inputRecords,
        'human_review_status': 'pending', 'gui_reproduced': False}
    (directory/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding='utf-8')
    return summary

```

E. 26 analyticJacobian.py: N40 驼峰解析 Jacobian

文件: paper_output/code/review_delivery/analyticJacobian.py; 共 333 行。采用范围: MATLAB N40 比较实际 Python 源码; 与原 11 个蛇形生产文件字节不同, 不按同名功能省略。

```
# 本文件为冻结求解源码的驼峰审查副本; 来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Analytic sparse Jacobian for drying_core.RadialModel.rhs.

The ordering is T0,C0,...,TN,CN,A. A is a passive cumulative loss variable;
its entire column is exactly zero and must not use adaptive numdiff factors.
This file does not modify the core or select a production configuration.

Self-check from the contest root:
    C:\Python314\python.exe -B paper_output/code/modeling/analytic_jacobian.py --self-test
"""
from __future__ import annotations

import argparse
from dataclasses import asdict
from datetime import datetime, timezone
import hashlib
import json
from pathlib import Path
import sys
import time
import warnings

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import coo_matrix
from scipy.special import expi

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

# 调和平均面对左右节点的解析偏导, 供热通量与调和水通量使用。
def harmonicPartials(left, right):
    """Positive-coefficient partials; respect the core's tiny denominator guard."""
    total = left + right
    tiny = np.finfo(float).tiny
    denominator = np.maximum(total, tiny)
    ordinary = total > tiny
    dl = np.where(ordinary, 2 * (right / denominator) ** 2, 2 * right / denominator)
    dr = np.where(ordinary, 2 * (left / denominator) ** 2, 2 * left / denominator)
    return dl, dr

# 稀疏 Jacobian 按交错 T/C 状态组装; 保留物性、水通量与容量分母的全部链式法则项。
def jacobian(model, t, y):
    """Return d(rhs)/d(y) as CSC without calling rhs or numerical differentiation.

    Each internal face contributes opposite flux derivatives to its two cells.
    For heat, differentiating 1/(rho*cp) adds -Tdot*cap_C/cap locally.
    Kirchhoff's temperature factor is frozen at the symmetric face temperature;
    the primitive's close-concentration branch is differentiated exactly as coded.
    """
    state = np.asarray(y, dtype=float)
    n = model.n
    if state.ndim != 1 or state.size != 2*n + 1:
        raise ValueError("Jacobian requires interleaved 1D state of length 2*n+1")
```

```

s = model.settings
T, C = state[:-1:2], state[1:-1:2]
cc = np.maximum(C, 1e-12)
active = (C > 1e-12).astype(float)
rho, cp, k, D = model.properties(T, C)
if s.question == 'Q1':
    a, d0, tempConstant = .89, 7e-9, 0.
    rhoC, cpC, kC = np.zeros(n), np.zeros(n), np.zeros(n)
elif s.question in ('Q2', 'Q3', 'Q23'):
    a, d0, tempConstant = .45, 2.4e-3, 3850.
    rhoC = np.full(n, 128.) * active
    cpC = 2736. / (1. + cc)**2 * active
    kC = .38 / (1. + cc)**2 * active
elif s.question == 'Q4':
    a, d0, tempConstant = .30, 4.2e-4, 3850.
    rhoC = np.full(n, 90.) * active
    cpC = 2150. / (1. + cc)**2 * active
    kC = .20 / (1. + cc)**2 * active
else:
    raise ValueError(s.question)
if s.constantThermal:
    rhoC, cpC, kC = np.zeros(n), np.zeros(n), np.zeros(n)
capacity = rho * cp
capacityC = rhoC * cp + rho * cpC
if s.constantD is None:
    dC = D * (a / cc**2) * active
    dT = D * tempConstant / T**2
else:
    dC, dT = np.zeros(n), np.zeros(n)

radius = float(model.radius(t))
if radius <= 0:
    raise ValueError("Nonpositive radius")
tair, ceq = model.environment(t)
geom = model.internalFaces / model.dx
deltaT, deltaC = np.diff(T), np.diff(C)

# Derivative columns for each face are (T_left, C_left, T_right, C_right).
kHarm = model.harmonic(k)
khL, khR = harmonicPartials(k[:-1], k[1:])
# 每个面的四列依次为 T 左、C 左、T 右、C 右，对相邻两个控制体施加相反符号。
heatDeriv = np.column_stack((
    -geom * kHarm,
    geom * khL * kC[:-1] * deltaT,
    geom * kHarm,
    geom * khR * kC[1:] * deltaT,
))
heatG = np.zeros(n + 1)
heatG[1:-1] = geom * kHarm * deltaT
heatG[-1] = -s.h * radius * (T[-1] - tair)

if s.faceScheme == 'harmonic' or s.constantD is not None:
    dh = model.harmonic(D)
    dhL, dhR = harmonicPartials(D[:-1], D[1:])
    waterDeriv = np.column_stack((
        geom * dhL * dT[:-1] * deltaC,
        geom * (dhL * dC[:-1] * deltaC - dh),
        geom * dhR * dT[1:] * deltaC,
        geom * (dhR * dC[1:] * deltaC + dh),
    ))
elif s.faceScheme == 'kirchhoff':
    primitive = cc * np.exp(-a / cc) + a * expi(-a / cc)
    primitiveDelta = np.diff(primitive)
    meanC = (cc[:-1] + cc[1:]) / 2.
    meanT = (T[:-1] + T[1:]) / 2.
    deltaCc = np.diff(cc)

```

```

small = np.abs(deltaCc) < 1e-7 * np.maximum(meanC, 1e-3)
expLeft, expRight = np.exp(-a/cc[:-1]), np.exp(-a/cc[1:])
primitivel = -expLeft * active[:-1]
primitiver = expRight * active[1:]
# 近等浓度时对实际 RHS 使用的中点分支求导, 避免与求解器分支不一致。
if np.any(small):
    fMid = np.exp(-a / meanC[small])
    fPrimeMid = fMid * a / meanC[small]**2
    primitiveDelta[small] = fMid * deltaCc[small]
    primitivel[small] = (0.5*fPrimeMid*deltaCc[small]-fMid)*active[:-1][small]
    primitiver[small] = (0.5*fPrimeMid*deltaCc[small]+fMid)*active[1:][small]
    temperatureFactor = np.exp(-tempConstant / meanT)
    coefficient = geom * d0 * temperatureFactor
    waterG = coefficient * primitiveDelta
    waterT = waterG * tempConstant / (2 * meanT**2)
    waterDeriv = np.column_stack((waterT, coefficient*primitivel,
                                   waterT, coefficient*primitiver))
else:
    raise ValueError('Unknown nonlinear face scheme')

heatScale = 1. / (radius**2 * model.w * capacity)
waterScale = 1. / (radius**2 * model.w)
surfaceHeatC = 0.
if s.surfaceLatentFraction:
    rhoD = model.rhoD0 * (.02 / radius)**2
    surfaceHeatC = -radius*s.surfaceLatentFraction*s.latentJKg*rhoD*s.beta
    heatG[-1] += surfaceHeatC * (C[-1] - ceq)
tempDerivative = np.diff(heatG) * heatScale

faceIndex = np.arange(n-1)
faceColumns = np.column_stack((2*faceIndex, 2*faceIndex+1,
                                2*faceIndex+2, 2*faceIndex+3)).ravel()
rows, columns, values = [], [], []

def addFace(rowIndex, derivatives, factor):
    rows.append(np.repeat(rowIndex, 4))
    columns.append(faceColumns)
    values.append((derivatives * factor[:, None]).ravel())

addFace(2*faceIndex, heatDeriv, heatScale[:-1])
addFace(2*faceIndex+2, heatDeriv, -heatScale[1:])
addFace(2*faceIndex+1, waterDeriv, waterScale[:-1])
addFace(2*faceIndex+3, waterDeriv, -waterScale[1:])
cellIndex = np.arange(n)
rows.append(2*cellIndex)
columns.append(2*cellIndex+1)
# 热容量随 C 改变, 必须保留 -Tdot*(容量对 C 偏导)/容量 这一局部项。
values.append(-tempDerivative * capacityC / capacity)
rows.append(np.array([2*n-2, 2*n-2, 2*n-1, 2*n]))
columns.append(np.array([2*n-2, 2*n-1, 2*n-1, 2*n-1]))
values.append(np.array([-s.h*radius*heatScale[-1],
                        surfaceHeatC*heatScale[-1],
                        -s.beta*radius*waterScale[-1], 2*s.beta/radius]))
# 面模板只有邻近耦合; COO 合并重复贡献后转 CSC, 交给 BDF 稀疏线性求解。
matrix = coo_matrix((np.concatenate(values),
                     (np.concatenate(rows), np.concatenate(columns))),
                    shape=(2*n+1, 2*n+1)).tocsc()
matrix.sum_duplicates()
matrix.eliminate_zeros()
return matrix

# 此历史自检入口仅验证导数和小网格接线, 不能替代正式网格与人工代码审核。
def selfTest(outputDirectory):
    """Independent RHS perturbation checks, a conservation derivative, and tiny BDF runs."""
    import scipy

```

```

from dryingCore import ROOT, Settings, RadialModel, LOADED_CODE_SHA256 as
CORE_LOADED_CODE_SHA256

started = time.perf_counter()
rng = np.random.default_rng(20260910)
relativeTolerance, absoluteTolerance = 5e-6, 5e-10
cases = []
for question in ['Q1', 'Q23', 'Q4']:
    for scheme in ['harmonic', 'kirchhoff']:
        for latent in [0., 1.]:
            cases.append(Settings(question=question, intervals=8, faceScheme=scheme,
                                  shrink=(question=='Q4'), surfaceLatentFraction=latent))
for question in ['Q1', 'Q23', 'Q4']:
    for constantDiffusivity, constantThermal in [(2e-9, False), (None, True), (2e-9, True)]:
        cases.append(Settings(question=question, intervals=8, faceScheme='kirchhoff',
                              shrink=(question=='Q4'), constantD=constantDiffusivity,
                              constantThermal=constantThermal, surfaceLatentFraction=1.))
records, failures = [], []
for settings in cases:
    model = RadialModel(settings)
    x = model.x
    states = {
        'initial': (0., model.initial()),
        'nonuniform': (18000., model.initial()),
        'late_dry': (150000., model.initial()),
        'near_uniform_small_branch': (14401., model.initial()),
    }
    states['nonuniform'][1][:-1:2] = 303. + 17.*x**2
    states['nonuniform'][1][1:-1:2] = 2.4 - 2.0*x**2
    states['late_dry'][1][:-1:2] = 321. + 2.0*x**2
    states['late_dry'][1][1:-1:2] = .175 - .115*x**2
    states['near_uniform_small_branch'][1][:-1:2] = 303. + 17.*x**2
    states['near_uniform_small_branch'][1][1:-1:2] = .15 + 1e-10*x
    for stateName, (t, y) in states.items():
        with warnings.catch_warnings(record=True) as captured:
            warnings.simplefilter('always')
            matrix = jacobian(model, t, y)
            vectors = []
            for _ in range(4):
                direction = rng.normal(size=y.size)
                direction[:-1:2] *= 1.0
                direction[1:-1:2] *= .02
                direction[-1] = .3
                vectors.append(direction)
            directional = []
            for direction in vectors:
                predicted = matrix @ direction
                steps = []
                for step in [1e-4, 3e-5, 1e-5]:
                    finiteDifference = (model.rhs(t, y+step*direction)-
                                       model.rhs(t, y-step*direction))/(2*step)
                    absoluteError = float(np.max(np.abs(predicted-finiteDifference)))
                    scale = max(float(np.max(np.abs(predicted))),
                                float(np.max(np.abs(finiteDifference))), 1e-30)
                    scaled = float(np.max(np.abs(predicted-finiteDifference)/
                                             (absoluteTolerance + relativeTolerance*np.maximum(
                                                 np.abs(predicted), np.abs(finiteDifference)))))
                    steps.append({'step':step, 'max_abs_error':absoluteError,
                                'relative_inf_error':absoluteError/scale,
                                'max_component_tolerance_ratio':scaled})
                best = min(steps, key=lambda item:item['max_component_tolerance_ratio'])
                directional.append({'all_step_errors':steps, 'best':best})
            passive = np.zeros(y.size); passive[-1] = 1.
            passiveAnalytic = float(np.max(np.abs(matrix @ passive)))
            passiveFd = float(np.max(np.abs(model.rhs(t, y+passive)-model.rhs(t, y-passive))))
            massWeights = np.zeros(y.size)

```

```

        massWeights[1:-1:2] = 2*model.w
        massWeights[-1] = 1.
        conservation = float(np.max(np.abs(np.asarray(massWeights @ matrix))))
        warningMessages = [str(w.message) for w in captured]
        passed = (all(step['max_component_tolerance_ratio'] <= 1
            for d in directional for step in d['all_step_errors']))
            and passiveAnalytic == 0 and passiveFd == 0 and conservation < 1e-11
            and np.isfinite(matrix.data).all() and not warningMessages)
        record = {'settings':asdict(settings), 'state':stateName, 'time_s':t,
            'status':'PASS' if passed else 'FAIL', 'matrix_shape':matrix.shape,
            'matrix_nnz':matrix.nnz, 'directional_checks':directional,
            'passive_column_analytic_abs':passiveAnalytic,
            'passive_column_finite_difference_abs':passiveFd,
            'mass_balance_derivative_abs':conservation, 'warnings':warningMessages}
        records.append(record)
    if not passed:

failures.append(f"{settings.question}/{settings.faceScheme}/latent={settings.surfaceLatentFraction}/{stateName}/constant_D={settings.constantD}/constant_thermal={settings.constantThermal}")
    smokeRecords = []
    for scheme in ['harmonic', 'kirchhoff']:
        for question in ['Q23', 'Q4']:
            settings = Settings(question=question, intervals=8, faceScheme=scheme,
                shrink=(question=='Q4'), surfaceLatentFraction=1.)
            model = RadialModel(settings)
            with warnings.catch_warnings(record=True) as captured:
                warnings.simplefilter('always')
                result = solve_ivp(model.rhs, (0., 2.), model.initial(), method='BDF',
                    jac=lambda t,y:jacobian(model,t,y), rtol=1e-10,
                    atol=1e-12, max_step=.2)
            msgs = [str(w.message) for w in captured]
            passed = bool(result.success and np.isfinite(result.y).all() and not msgs)
            smokeRecords.append({'question':question, 'face_scheme':scheme, 'interval_s':[0,2],
                'status':'PASS' if passed else 'FAIL', 'warnings':msgs,
                'nfev':result.nfev, 'njev':result.njev, 'nlu':result.nlu,
                'purpose':'Short Jacobian/BDF wiring check, not production
accuracy'})
            if not passed:
                failures.append(f"BDF_smoke/{question}/{scheme}")
                currentSourceHash = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
                currentCoreHash =
                hashlib.sha256((ROOT/'paper_output/code/review_delivery/dryingCore.py').read_bytes()).hexdigest()
                if currentSourceHash != LOADED_CODE_SHA256 or currentCoreHash != CORE_LOADED_CODE_SHA256:
                    failures.append('Source file changed after module load during validation')
                report = {'schema_version':'1.0', 'created_utc':datetime.now(timezone.utc).isoformat(),
                    'status':'PASS' if not failures else 'FAIL', 'failures':failures,
                    'source_sha256':LOADED_CODE_SHA256,
                    'core_sha256':CORE_LOADED_CODE_SHA256,
                    'source_hash_policy':'SHA-256 frozen when each module is imported; files checked
unchanged before report save.',
                    'runtime':
{'python':sys.version,'executable':sys.executable,'numpy':np.__version__, 'scipy':scipy.__version__},
                    'seed':20260910,'relative_component_tolerance':relativeTolerance,
                    'absolute_component_tolerance':absoluteTolerance,
                    'finite_difference_policy':'Central directional differences, three decreasing steps;
every step must satisfy the mixed absolute/relative component tolerance. Best comparison is
supplementary only.',
                    'case_state_count':len(records), 'direction_count':4*len(records),
                    'checks':records,'short_BDF_checks':smokeRecords,
                    'elapsed_s':time.perf_counter()-started,
                    'limitations':['No full production run or grid convergence performed here.',
                        'Tiny denominator/underflow extensions at nonphysical Newton probes
are not calibrated physical data.',
                        'Only this module and the short tests use analytic Jacobian until main
agent hooks core.',

```

```

        'Visual Studio reproduction and human review remain pending.']}
outputDirectory.mkdir(parents=True, exist_ok=True)
(outputDirectory/'analytic_jacobian_selftest.json').write_text(
    json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
brief = {key:report[key] for key in
['status','failures','case_state_count','direction_count','elapsed_s']}
brief['max_best_component_tolerance_ratio'] = max(
    d['best']['max_component_tolerance_ratio'] for r in records for d in
r['directional_checks'])
brief['max_all_steps_component_tolerance_ratio'] = max(
    step['max_component_tolerance_ratio'] for r in records
    for d in r['directional_checks'] for step in d['all_step_errors'])
brief['max_mass_balance_derivative_abs'] = max(r['mass_balance_derivative_abs'] for r in
records)
brief['warning_count'] = sum(len(r['warnings']) for r in records+smokeRecords)
print(json.dumps(brief,ensure_ascii=False,indent=2))
return 0 if not failures else 1

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--self-test', action='store_true')
    parser.add_argument('--output-dir', type=Path,
        default=Path(__file__).resolve().parents[2]/'code/review_delivery/runtime/
jacobianValidation')
    arguments = parser.parse_args()
    if not arguments.self_test:
        parser.error('Use --self-test, or import jacobian(model,t,y) from this module.')
    raise SystemExit(selfTest(arguments.output_dir))

```

E. 27 diskDense.py: N40 驼峰磁盘轨迹存储

文件: paper_output/code/review_delivery/diskDense.py; 共 128 行。采用范围: MATLAB N40 比较实际 Python 源码; 与原 11 个蛇形生产文件字节不同, 不按同名功能省略。

```
# 本文件为冻结求解源码的驼峰审查副本; 来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Exact BDF dense polynomials backed by a private, rebuildable disk cache.

This changes storage only: each accepted BDF polynomial is written as float64
bytes, then evaluated with SciPy's original BdfDenseOutput implementation.
No additional time/space interpolation and no solver restart are introduced.
"""

from __future__ import annotations
import hashlib
from pathlib import Path
import tempfile
import numpy as np
from scipy.integrate import OdeSolution
from scipy.integrate._ivp.bdf import BDF, BdfDenseOutput
from scipy.integrate._ivp.base import DenseOutput

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

# 每个 Run 独享项目内可重建缓存; 存原始 float64 字节, 不降低插值阶数。
class DenseCache:
    def __init__(self, root):
        self.parent = (Path(root)/'tmp/cache/solver_runs').resolve()
        if not self.parent.is_relative_to(Path(root).resolve()):
            raise ValueError('Private solver cache must remain in this project')
        self.parent.mkdir(parents=True, exist_ok=True)
        self.directory = Path(tempfile.mkdtemp(prefix='bdf_', dir=self.parent)).resolve()
        try:
            self.handle = (self.directory/'polynomials.bin').open('w+b', buffering=0)
        except BaseException as error:
            try:
                self.directory.rmdir()
            except OSError as cleanup_error:
                error.add_note('Private cache directory cleanup failed: '+repr(cleanup_error))
            raise
        self.bytesWritten = 0
        self.polynomialCount = 0
        self.acceptedArrays = []
        self.closed = False

    def append(self, values):
        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        array = np.ascontiguousarray(values, dtype=np.float64)
        offset = self.bytesWritten
        self.handle.seek(offset)
        array.tofile(self.handle)
        self.bytesWritten += array.nbytes
        self.polynomialCount += 1
        return offset, array.shape

    def read(self, offset, shape):
        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        count = int(np.prod(shape))
        self.handle.seek(offset)
```

```

        values = np.fromfile(self.handle, dtype=np.float64, count=count)
        if values.size != count:
            raise IOError('Incomplete BDF coefficient cache')
        return values.reshape(shape)

# 被接受的整段状态写为磁盘映射数组，以限制长时高网格运行的常驻内存。
def storeAccepted(self, values):
    path = self.directory/f'accepted_{len(self.acceptedArrays):03d}.npz'
    mapped = np.lib.format.open_mmap(path, mode='w+', dtype=values.dtype,
shape=values.shape)
    # Register before writing so failures can close this Windows mapping.
    self.acceptedArrays.append(mapped)
    mapped[:] = values
    mapped.flush()
    return mapped

# 先释放 Windows 文件映射，再仅清理当前对象创建并验证过的私有缓存。
def close(self):
    if self.closed:
        return
    self.handle.close()
    for array in self.acceptedArrays:
        array._mmap.close()
    self.acceptedArrays.clear()
    # Delete only the verified private cache created by this object.
    if self.directory.parent != self.parent or not self.directory.name.startswith('bdf_'):
        raise RuntimeError('Unexpected private cache path; cleanup refused')
    for path in self.directory.iterdir():
        if not path.is_file() or path.is_symlink():
            raise RuntimeError('Unexpected cache entry; cleanup refused')
        path.unlink()
    self.directory.rmdir()
    self.closed = True

class FileBdfDenseOutput(DenseOutput):
    def __init__(self, original, cache):
        super().__init__(original.t_old, original.t)
        self.order = original.order
        self.t_shift = original.t_shift.copy()
        self.denom = original.denom.copy()
        self.cache = cache
        self.offset, self.shape = cache.append(original.D)

# 保留 SciPy override 名，调用安装版本的原生 BDF 多项式求值器。
def _call_impl(self, t):
    # Reuse the installed SciPy evaluator with the exact recorded D bytes.
    dense = object.__new__(BdfDenseOutput)
    dense.D = self.cache.read(self.offset, self.shape)
    dense.t_shift, dense.denom = self.t_shift, self.denom
    return dense._call_impl(t)

# dense_cache 是 solve_ivp 注入此子类的协议参数，保留拼写以维持接口。
class DiskBDF(BDF):
    def __init__(self, *args, dense_cache, **kwargs):
        self.dense_cache = dense_cache
        super().__init__(*args, **kwargs)

    def _dense_output_impl(self):
        return FileBdfDenseOutput(super()._dense_output_impl(), self.dense_cache)

# 子类不命中 SciPy 对 BDF 的类身份判断，故恢复其接受断点右侧多项式选择。
def alignBdfSegments(result):
    """Restore the original BDF convention at accepted time breakpoints.

```

```

SciPy solve_ivp tests the exact method class for alt_segment. A subclass
otherwise selects the opposite polynomial at a shared knot. Reconstructing
OdeSolution changes only this selection, never coefficients or integration.
"""
if result.sol is not None:
    result.sol = OdeSolution(result.sol.ts, result.sol.interpolants, alt_segment=True)

```

E. 28 q1Model.py: N40 问题一入口

文件：paper_output/code/review_delivery/q1Model.py；共 10 行。采用范围：MATLAB N40 比较实际 Python 源码；与原 11 个蛇形生产文件字节不同，不按同名功能省略。

```
# 本文件为冻结求解源码的驼峰审查副本；来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Q1 uses Appendix 2 throughout its 1800-second interval."""
from dryingCore import Settings, solveCase

# Q1 全部 1800 秒均采用附录 2 参数，正式空间区间数默认 3200。
def solve(intervals=3200):
    return solveCase(Settings(question='Q1', intervals=intervals, faceScheme='kirchhoff',
                               rtol=1e-10, atolTemperature=1e-10, atolMoisture=1e-12,
                               earlyMaxStepS=2., maxStepS=120.))
```

E. 29 q2Model.py: N40 问题二三入口

文件：paper_output/code/review_delivery/q2Model.py；共 10 行。采用范围：MATLAB N40 比较实际 Python 源码；与原 11 个蛇形生产文件字节不同，不按同名功能省略。

```
# 本文件为冻结求解源码的驼峰审查副本；来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Q2 uses Appendix 3 from t=0; it does not splice the Q1 trajectory."""
from dryingCore import Settings, solveCase

# Q2/Q3 从 t=0 采用附录 3, Q3 必须复用本次同一个 Run, 不能拼接 Q1。
def solve(intervals=3200):
    return solveCase(Settings(question='Q23', intervals=intervals, faceScheme='kirchhoff',
                               rtol=1e-10, atolTemperature=1e-10, atolMoisture=1e-12,
                               earlyMaxStepS=2., maxStepS=120.))
```

E. 30 q3Model.py: N40 严格报告与采样时刻

文件：paper_output/code/review_delivery/q3Model.py；共 30 行。采用范围：MATLAB N40 比较实际 Python 源码；与原 11 个蛇形生产文件字节不同，不按同名功能省略。

```
# 本文件为冻结求解源码的驼峰审查副本；来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Q3 is a threshold functional of the exact same Run used by Q2."""
import math
import numpy as np

# Q3 是 Q2 场解的阈值泛函；先连续定位，再向上取 0.0001 h 并验原精度  $\max(C) < 0.15$ 。
def completion(run):
    if run.eventS is None:
        raise ValueError('No full-domain drying event within the solved horizon')
    # At the continuous root the maximum equals 0.15. Report upward on the
    # required 0.0001-hour grid and verify the unrounded state there.
    count = math.ceil(run.eventS / 3600. * 10000.)
    # 即使四位显示为 0.1500，也只能依据未舍入含水率判定严格干燥。
    while True:
        reportH = count/10000.
        t = reportH*3600.
        if t > run.endS:
            raise RuntimeError('Four-decimal reporting time is outside the verified trajectory')
        values = run.state([t])[1:-1:2,0]
        if float(np.max(values)) < .15:
            break
        count += 1
    return {'critical_event_s':run.eventS,'critical_event_h':run.eventS/3600.,
        'reported_drying_time_h':reportH,'reported_time_s':t,
        'max_C_at_reported_time':float(np.max(values)),
        'slowest_material_coordinate':float(run.model.x[np.argmax(values)]),
        'conservative_post_verification_s':run.endS,
        'rounding_convention':'upward on 0.0001 h grid, followed by actual strict threshold
check',
        'interpretation':'A conditional numerical event, not a confidence bound on physical
drying time.'}
```

E. 31 q4Model.py: N40 问题四入口

文件： paper_output/code/review_delivery/q4Model.py； 共 10 行。采用范围： MATLAB N40 比较实际 Python 源码；与原 11 个蛇形生产文件字节不同，不按同名功能省略。

```
# 本文件为冻结求解源码的驼峰审查副本；来源、改名与 AST 核验见 tools/coreRenameReport.json。
"""Q4 switches all properties to Appendix 4 and follows observed radial shrinkage."""
from dryingCore import Settings, solveCase

# Q4 从 t=0 使用整组附录 4 系数，并按观测 R(t) 同比径向收缩，固定长度。
def solve(intervals=6400):
    return solveCase(Settings(question='Q4', intervals=intervals, faceScheme='kirchhoff',
                               shrink=True, rtol=1e-10, atolTemperature=1e-10, atolMoisture=1e-12,
                               earlyMaxStepS=2., maxStepS=120.))
```